

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 23 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention-2split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMMA)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127 #include <cuda/ptx>
128
129 #if defined(NOTES_V2_ENABLE_CUDNN)
130 #pragma nv_diag_suppress 128
131 #include <cuda/cudnn_frontend.h>
132 #pragma nv_diag_default 128
133 namespace fe = cudnn_frontend;
134 #endif
135
136 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
137 #error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
138 #endif
139
140 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
141 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
142 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
143
144 static constexpr int kWarpSize = 32;
145
146 // =====
147 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
148 // =====
149
150 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
151 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
152 // 复杂度 O(logN), N=32 时只需 5 步
153 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
154 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
155 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
156 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
157 //
158 // 初始: 每个 lane 持有自己的值 v0..v7
159 // lane: 0 1 2 3 4 5 6 7
160 // val: v0 v1 v2 v3 v4 v5 v6 v7
161 //
162 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
163 //
164 //      ┌──────────┐
165 // 对: (0,4) (1,5) (2,6) (3,7)
166 //
167 // lane: 0 1 2 3 4 5 6 7
168 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
169 //
170 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
171 //
172 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
173 // 对: (0,2) (1,3) (4,6) (5,7)
174 //      ─── 前4个一组 ───      ─── 后4个一组 ───
175 //
176 // lane: 0 1 2 3 4 5 6 7 (每 lane 持有前一轮2个值的和再加本轮配对)
177 // val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}
178 //      = v0+v2+v4+v6 ... (逐步归约)
179 //
180 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
181 //
182 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
183 // 对: (0,1) (2,3) (4,5) (6,7)
184 //
185 // lane: 0 1 2 3 4 5 6 7
186 // val: Σall Σall Σall Σall Σall Σall Σall Σall ← 所有 lane 拥有全归约结果!
187 //
188 // mask=0: 循环终止, 归约完成。
189 //
190 // 关键性质:
191 // - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i

```

```

189 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
190 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
191 // - 0(log2 N) 步完成, 无需 shared memory, 纯寄存器操作
192 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
193 // 当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
194 template <const int kWarpWidth = kWarpSize, typename T = float>
195 __device__ __forceinline__ T warp_reduce_sum(T val) {
196 #pragma unroll
197     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
198         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
199     }
200     return val;
201 }
202
203 // Warp Reduce Max — generic
204 template <const int kWarpWidth = kWarpSize, typename T = float>
205 __device__ __forceinline__ T warp_reduce_max(T val) {
206 #pragma unroll
207     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
208         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
209     }
210     return val;
211 }
212
213 // =====
214 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
215 // =====
216
217 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
218 // 两级归约流程:
219 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
220 // 2. warp leader (lane=0) 写入 shared memory
221 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
222 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
223 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
224 template <const int kNumThreads = 256>
225 __device__ float block_reduce_sum(float val) {
226     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
227     int warp = threadIdx.x / kWarpSize;
228     int lane = threadIdx.x % kWarpSize;
229     __shared__ float shared[kNumWarps];
230
231     float value = warp_reduce_sum<kWarpSize>(val);
232     if (lane == 0)
233         shared[warp] = value;
234     __syncthreads();
235     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
236     value = warp_reduce_sum<kNumWarps>(value);
237     // 关键: broadcast 结果到所有线程, 后续用 result
238     // 做除法等操作时所有线程都能拿到
239     value = __shfl_sync(0xffffffff, value, 0, 32);
240     return value;
241 }
242
243 // Block Reduce Max — FP32 (增强版, 带 broadcast)
244 template <const int kNumThreads = 256>
245 __device__ float block_reduce_max(float val) {
246     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
247     int warp = threadIdx.x / kWarpSize;
248     int lane = threadIdx.x % kWarpSize;
249     __shared__ float shared[kNumWarps];
250
251     float value = warp_reduce_max<kWarpSize>(val);

```

```

252     if (lane == 0)
253         shared[warp] = value;
254     __syncthreads();
255     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
256     value = warp_reduce_max<kNumWarps>(value);
257     value = __shfl_sync(0xffffffff, value, 0, 32);
258     return value;
259 }
260
261 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
262 // 多 block 各自做 warp->smem->warp0 reduce, 然后 atomicAdd 到全局 y
263 // 跨 block 求和的常见模式, 适合 N 较大时使用;
264 // Grid: ((N + 255) / 256, 1, 1)
265 // Block: (256, 1, 1)
266 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
267 template <const int kNumThreads = 256>
268 __global__ void block_reduce_all(float *a, float *y, int N) {
269     int tid = threadIdx.x;
270     int idx = blockIdx.x * kNumThreads + tid;
271     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
272     __shared__ float shared[kNumWarps];
273
274     float val = (idx < N) ? a[idx] : 0.0f;
275     int warp = tid / kWarpSize;
276     int lane = tid % kWarpSize;
277
278     val = warp_reduce_sum<kWarpSize>(val);
279     if (lane == 0)
280         shared[warp] = val;
281     __syncthreads();
282
283     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
284     if (warp == 0)
285         val = warp_reduce_sum<kNumWarps>(val);
286     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
287         atomicAdd(y, val);
288 }
289
290 // ---- Dot Product: y = sum(a[i] * b[i]) ----
291 // 核心模式: elementwise 乘法 -> block reduce -> atomicAdd 全局累加
292 // Grid: ((N + 255) / 256, 1, 1)
293 // Block: (256, 1, 1)
294 // source: LeetCUDA/kernels/dot-product/dot_product.cu
295 template <const int kNumThreads = 256>
296 __global__ void dot(float *a, float *b, float *y, int N) {
297     int tid = threadIdx.x;
298     int idx = blockIdx.x * kNumThreads + tid;
299     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
300     __shared__ float shared[kNumWarps];
301
302     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
303     int warp = tid / kWarpSize;
304     int lane = tid % kWarpSize;
305
306     prod = warp_reduce_sum<kWarpSize>(prod);
307     if (lane == 0)
308         shared[warp] = prod;
309     __syncthreads();
310
311     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
312     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
313         prod = warp_reduce_sum<kNumWarps>(prod);
314     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加

```

```

315     atomicAdd(y, prod);
316 }
317
318 // Dot Product + float4
319 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
320 // Block: (64, 1, 1), 256/4=64
321 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
322 // source: LeetCUDA/kernels/dot-product/dot_product.cu
323 template <const int kNumThreads = 256 / 4>
324 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
325     int tid = threadIdx.x;
326     int idx = (blockIdx.x * kNumThreads + tid) * 4;
327     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
328     __shared__ float shared[kNumWarps];
329
330     float4 reg_a = FLOAT4(a[idx]);
331     float4 reg_b = FLOAT4(b[idx]);
332     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
333                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
334                       : 0.0f;
335     int warp = tid / kWarpSize;
336     int lane = tid % kWarpSize;
337
338     prod = warp_reduce_sum<kWarpSize>(prod);
339     if (lane == 0)
340         shared[warp] = prod;
341     __syncthreads();
342
343     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
344     if (warp == 0)
345         prod = warp_reduce_sum<kNumWarps>(prod);
346     if (tid == 0)
347         atomicAdd(y, prod);
348 }
349
350
351 // =====
352 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
353 // =====
354 // 面试要点:
355 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
356 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
357 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
358
359 // ---- ReLU: y = max(0, x) ----
360 // Grid: ((N + 255) / 256, 1, 1)
361 // Block: (256, 1, 1)
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu(float *x, float *y, int N) {
364     int idx = blockIdx.x * blockDim.x + threadIdx.x;
365     if (idx < N)
366         y[idx] = fmaxf(0.0f, x[idx]);
367 }
368
369 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
370 // block(64)x4(float4)=256 元素/block, 与基础版吞吐相同
371 // Grid: ((N + 255) / 256, 1, 1)
372 // Block: (64, 1, 1)
373 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
374 // source: LeetCUDA/kernels/relu/relu.cu
375 __global__ void relu_vec4(float *x, float *y, int N) {
376     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
377     if (idx < N) {

```

```

378     float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
379     float4 reg_y;
380     reg_y.x = fmaxf(0.0f, reg_x.x);
381     reg_y.y = fmaxf(0.0f, reg_x.y);
382     reg_y.z = fmaxf(0.0f, reg_x.z);
383     reg_y.w = fmaxf(0.0f, reg_x.w);
384     FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
385 }
386 }
387
388 // ---- Elementwise Add: c = a + b ----
389 // Grid: ((N + 255) / 256, 1, 1)
390 // Block: (256, 1, 1)
391 // source: LeetCUDA/kernels/elementwise/elementwise.cu
392 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
393     int idx = blockIdx.x * blockDim.x + threadIdx.x;
394     if (idx < N)
395         c[idx] = a[idx] + b[idx];
396 }
397
398 // Elementwise Add + float4 向量化
399 // Grid: ((N + 255) / 256, 1, 1)
400 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
401 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
402 // source: LeetCUDA/kernels/elementwise/elementwise.cu
403 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
404     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
405     if ((idx + 3) < N) {
406         float4 reg_a = FLOAT4(a[idx]);
407         float4 reg_b = FLOAT4(b[idx]);
408         float4 reg_c;
409         reg_c.x = reg_a.x + reg_b.x;
410         reg_c.y = reg_a.y + reg_b.y;
411         reg_c.z = reg_a.z + reg_b.z;
412         reg_c.w = reg_a.w + reg_b.w;
413         FLOAT4(c[idx]) = reg_c;
414     } else if (idx < N) {
415         for (int i = 0; (idx + i) < N; i++) {
416             c[idx + i] = a[idx + i] + b[idx + i];
417         }
418     }
419 }
420
421 // ---- Histogram: y[a[i]]++ ----
422 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
423 // Grid: ((N + 255) / 256, 1, 1)
424 // Block: (256, 1, 1)
425 // source: LeetCUDA/kernels/histogram/histogram.cu
426 __global__ void histogram(int *a, int *y, int N) {
427     int idx = blockIdx.x * blockDim.x + threadIdx.x;
428     if (idx < N)
429         atomicAdd(&(y[a[idx]]), 1);
430 }
431
432 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
433 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
434 //
435 // 面试要点:
436 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
437 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
438 //   - 数学公式 (5 步推导):
439 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
440 //         L_max = max(LSE_1, LSE_2)

```

```

441 // Step 2 — 还原指数权重 (un-normalized) :
442 //      w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
443 // Step 3 — 归一化为混合比例:
444 //      alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
445 //      满足 alpha + beta = 1
446 // Step 4 — 逐元素加权合并:
447 //      O = alpha * O_1 + beta * O_2
448 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention-2惯例一致,
449 //   lse[head_idx][token_idx])
450 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
451 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
452 // -  $-\infty$  LSE  $\rightarrow -\infty$ : 空 attention 段 (causal mask 等导致全部 score
453 //   为  $-\infty$ ) 的 LSE 可能为  $+\infty$ , 替换后  $\exp(-\infty - L_{\max}) = 0$ ,
454 //   该段权重退化为 0
455 // - 128-bit 向量化: uint4 = 4  $\times$  float, 每线程处理 4 个输出元素,
456 //   pack load  $\rightarrow$  FMA  $\rightarrow$  pack store 一条流水线
457 // - AI  $\approx$  3 FLOP / 20 bytes  $\approx$  0.15 FLOPS/Byte  $\rightarrow$  severely memory-bound
458 //
459 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
460 // kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
461 // threads_per_head = head_size / 4 = 2
462 // total_threads = num_tokens * num_heads * threads_per_head = 8
463 //
464 // global_idx:      0      1      2      3      4      5      6      7
465 // token_head_idx:  0      0      1      1      2      2      3      3
466 //   (第几个 (token,head) 对, 行优先)
467 // pack_idx:        0      1      0      1      0      1      0      1
468 //   (该 (token,head) 对内第几个 pack)
469 // token_idx:       0      0      0      0      1      1      1      1
470 //   (token_head_idx / num_heads, token 变化慢)
471 // head_idx:        0      0      1      1      0      0      1      1
472 //   (token_head_idx % num_heads, head 变化快)
473 // pack_offset:     0      4      0      4      0      4      0      4
474 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
475 // head_offset:     0      0      8      8      16     16     24     24
476 //   (token_idx * num_heads * head_size + head_idx * head_size,
477 //   该 (token,head) 对在 output 展平数组中的起始偏移)
478 //
479 // Grid: ((total_threads + 127) / 128, 1, 1)
480 // Block: (128, 1, 1)
481 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
482 __global__ void merge_attn_states(
483     float *output,           // [num_tokens, num_heads, head_size]
484     const float *prefix_output, // [num_tokens, num_heads, head_size]
485     const float *prefix_lse,   // [num_heads, num_tokens]
486     const float *suffix_output, // [num_tokens, num_heads, head_size]
487     const float *suffix_lse,   // [num_heads, num_tokens]
488     int num_tokens, int num_heads, int head_size) {
489
490     constexpr int kNumThreads = 128;
491     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
492     using pack_t = uint4;
493
494     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
495     const int threads_per_head = head_size / kPackSize;
496     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
497     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
498     const int total_threads = num_tokens * num_heads * threads_per_head;
499
500     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
501     if (global_idx >= total_threads)
502         return;
503 
```

```

504 // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
505 // token_head_idx: 第几个 (token, head) 对, 行优先展平
506 const int token_head_idx = global_idx / threads_per_head;
507 // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
508 const int pack_idx = global_idx % threads_per_head;
509
510 // token_head_idx 分解为 (token_idx, head_idx)
511 const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
512 const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
513
514 // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
515 const int pack_offset = pack_idx * kPackSize;
516 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
517 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
518
519 // 定位到当前 (token, head) 的输出段起始
520 const float *prefix_head = prefix_output + head_offset;
521 const float *suffix_head = suffix_output + head_offset;
522 float *output_head = output + head_offset;
523
524 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
525 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
526 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
527
528 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
529 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
530 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
531
532 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
533 const float max_lse = fmaxf(p_lse, s_lse);
534 p_lse -= max_lse;
535 s_lse -= max_lse;
536
537 // Step 2-3: 指数还原 → 混合比例 alpha, beta
538 const float p_se = expf(p_lse);
539 const float s_se = expf(s_lse);
540 const float out_se = p_se + s_se;
541 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
542 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
543
544 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
545 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
546 if (pack_offset < head_size) {
547     pack_t p_pack =
548         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
549     pack_t s_pack =
550         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
551     pack_t o_pack;
552
553     #pragma unroll
554     for (int i = 0; i < kPackSize; ++i) {
555         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
556         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
557         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
558         reinterpret_cast<float *>(&o_pack)[i] = o_v;
559     }
560
561     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
562 }
563 }
564
565 // =====
566 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm

```

```

567 // =====
568 // 面试要点:
569 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
570 //   online (1-pass, 增量更新)
571 //   - Online Softmax 是 FlashAttention-2的数学基础
572 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
573 //   + variance)
574 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
575
576 // =====
577 // Phase 3a: Softmax — 三级递进 (面试核心考点)
578 // =====
579 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
580 // 回答线索: naive(溢出) → safe → online
581
582 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
583 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
584 // 问题: x 值很大时 exp(x) 溢出为 inf
585 template <const int kNumThreads = 256>
586 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
587 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
588 // source: LeetCUDA/kernels/softmax/softmax.cu
589 __global__ void softmax_per_token(float *x, float *y, int N) {
590     const int tid = threadIdx.x;
591     const int idx = blockIdx.x * blockDim.x + tid;
592
593     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
594     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
595     if (idx < N)
596         y[idx] = exp_val / exp_sum;
597 }
598
599 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
600 // 面试重点: 为什么 Softmax 需要 Safe?
601 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
602 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
603 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
604 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
605 // Grid: (S, 1, 1)
606 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
607 // source: LeetCUDA/kernels/softmax/softmax.cu
608 template <const int kNumThreads = 256>
609 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
610     const int tid = threadIdx.x;
611     const int idx = blockIdx.x * blockDim.x + tid;
612
613     // Pass 1: block reduce max — 找最大值
614     float val = (idx < N) ? x[idx] : -FLT_MAX;
615     float max_val = block_reduce_max<kNumThreads>(val);
616
617     // Pass 2: exp(x - max) → block reduce sum
618     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
619     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
620
621     if (idx < N)
622         y[idx] = exp_val / exp_sum;
623 }
624
625 // ---- Level 3: Online Safe Softmax (FlashAttention-2的数学基础) ----
626 // 面试重点:
627 // 两种使用场景 (公式不同, 但结果等价) :
628 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
629 //       m_new = max(m_old, x_i)

```

```

630 //      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
631 //  2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
632 //      m = max(m1, m2) safe softmax用的max值
633 //      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
634 //      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
635 //  算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
636 //  Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
637 //
638 //  不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
639 //  warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
640 //  (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
641 //  (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
642 //
643 //  合并公式 (对称, 不依赖"新/旧"概念) :
644 //  m = max(m1, m2)
645 //  d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
646 //
647 //  该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
648 struct __align__(8) MD {
649     float m; // running max
650     float d; // running denominator (sum of exp(x - max))
651 };
652
653 template <const int kWarpWidth = kWarpSize>
654 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
655     #pragma unroll
656     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
657         MD md2;
658         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
659         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
660
661         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
662         MD s_m = (md1.m > md2.m) ? md2 : md1;
663
664         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
665         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
666         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
667         md1.m = b_m.m;
668     }
669     return md1;
670 }
671
672 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
673 // Grid: (S, 1, 1)
674 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
675 // source: LeetCUDA/kernels/softmax/softmax.cu
676 template <const int kNumThreads = 256>
677 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
678     int tid = threadIdx.x;
679     int idx = blockIdx.x * kNumThreads + threadIdx.x;
680     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
681     __shared__ MD shared[kNumWarps];
682
683     float val = (idx < N) ? x[idx] : -FLT_MAX;
684     int warp = tid / kWarpSize;
685     int lane = tid % kWarpSize;
686
687     // 初始化: 每个线程持有一个 (max, denom) 对
688     MD md;
689     md.m = idx < N ? val : -FLT_MAX;
690     md.d = idx < N ? 1.0f : 0.0f;
691
692     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d

```

```

693 md = warp_reduce_md<kWarpSize>(md);
694
695 if (lane == 0)
696     shared[warp] = md;
697 __syncthreads();
698
699 // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
700 md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
701 md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
702
703 // 用全局 max 和 denom 做最终 softmax
704 float d_inv = __fdivdef(1.0f, md.d);
705 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
706 if (idx < N) {
707     y[idx] = __expf(val - md.m) * d_inv;
708 }
709 }
710
711 // =====
712 // Phase 3b: RMS Normalization (1-pass reduce)
713 // =====
714 // 面试要点:
715 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
716 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
717 //   - Llama 系列使用 RMS Norm
718 //   - grid(N, K/K), block(K): 一行一个 block
719 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
720 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
721 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
722 template <const int kNumThreads = 128>
723 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
724     int tid = threadIdx.x;
725     int idx = blockIdx.x * blockDim.x + threadIdx.x;
726     const float epsilon = 1e-5f;
727
728     __shared__ float s_variance;
729     float value = (idx < N * K) ? x[idx] : 0.0f;
730     float variance = value * value;
731     variance = block_reduce_sum<kNumThreads>(variance);
732     if (tid == 0)
733         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
734     __syncthreads();
735     if (idx < N * K)
736         y[idx] = (value * s_variance) * g;
737 }
738
739 // RMS Norm + float4
740 // Grid: (N, 1, 1)
741 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
742 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
743 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
744 template <const int kNumThreads = 128 / 4>
745 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
746     int tid = threadIdx.x;
747     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
748     const float epsilon = 1e-5f;
749
750     __shared__ float s_variance;
751     float4 reg_x = FLOAT4(x[idx]);
752     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
753                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
754                                     : 0.0f;
755     variance = block_reduce_sum<kNumThreads>(variance);

```

```

756     if (tid == 0)
757         s_variance = rsqrtf(variance / (float)K + epsilon);
758     __syncthreads();
759     float4 reg_y;
760     reg_y.x = reg_x.x * s_variance * g;
761     reg_y.y = reg_x.y * s_variance * g;
762     reg_y.z = reg_x.z * s_variance * g;
763     reg_y.w = reg_x.w * s_variance * g;
764     if (idx < N * K)
765         FLOAT4(y[idx]) = reg_y;
766 }
767
768 // =====
769 // Phase 3c: Layer Normalization (2-pass reduce)
770 // =====
771 // 面试要点:
772 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
773 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
774 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
775 // Grid: (N, 1, 1), 一行一个 block
776 // Block: (128, 1, 1), kNumThreads=K=128
777 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
778 template <const int kNumThreads = 128>
779 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
780     int tid = threadIdx.x;
781     int idx = blockIdx.x * blockDim.x + threadIdx.x;
782     const float epsilon = 1e-5f;
783
784     __shared__ float s_mean;
785     __shared__ float s_variance;
786     float value = (idx < N * K) ? x[idx] : 0.0f;
787
788     // Pass 1: compute mean
789     float sum = block_reduce_sum<kNumThreads>(value);
790     if (tid == 0)
791         s_mean = sum / (float)K;
792     __syncthreads(); // 必须等待 s_mean 对所有线程可见
793
794     // Pass 2: compute variance = (x - mean)2
795     float variance = (value - s_mean) * (value - s_mean);
796     variance = block_reduce_sum<kNumThreads>(variance);
797     if (tid == 0)
798         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
799     __syncthreads(); // 必须等待 s_variance 对所有线程可见
800
801     if (idx < N * K)
802         y[idx] = ((value - s_mean) * s_variance) * g + b;
803 }
804
805 // Layer Norm + float4
806 // Grid: (N, 1, 1)
807 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
808 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
809 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
810 template <const int kNumThreads = 128 / 4>
811 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
812     int tid = threadIdx.x;
813     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
814     const float epsilon = 1e-5f;
815
816     __shared__ float s_mean;
817     __shared__ float s_variance;
818     float4 reg_x = FLOAT4(x[idx]);

```

```

819 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
820
821 float sum = block_reduce_sum<kNumThreads>(value);
822 if (tid == 0)
823     s_mean = sum / (float)K;
824 __syncthreads();
825
826 float4 reg_x_hat;
827 reg_x_hat.x = reg_x.x - s_mean;
828 reg_x_hat.y = reg_x.y - s_mean;
829 reg_x_hat.z = reg_x.z - s_mean;
830 reg_x_hat.w = reg_x.w - s_mean;
831 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
832                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
833 variance = block_reduce_sum<kNumThreads>(variance);
834 if (tid == 0)
835     s_variance = rsqrtf(variance / (float)K + epsilon);
836 __syncthreads();
837
838 float4 reg_y;
839 reg_y.x = reg_x_hat.x * s_variance * g + b;
840 reg_y.y = reg_x_hat.y * s_variance * g + b;
841 reg_y.z = reg_x_hat.z * s_variance * g + b;
842 reg_y.w = reg_x_hat.w * s_variance * g + b;
843 if (idx < N * K)
844     FLOAT4(y[idx]) = reg_y;
845 }
846
847 // =====
848 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
849 // =====
850 // 面试要点:
851 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
852 //     [x1']   [cos(θ)  -sin(θ)] [x1]
853 //     [x2'] = [sin(θ)   cos(θ)] [x2]
854 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
855 //   - token_pos = idx / N: token 在序列中的位置
856 //   - token_idx = idx % N: token 内的维度对索引
857 //   - 输入 [seq_len, hidden_size], 输出同形状
858
859 // Grid: ((seq_len * N + 255) / 256, 1, 1)
860 // Block: (256, 1, 1)
861 // source: LeetCUDA/kernels/rope/rope.cu
862 __global__ void rope(float *x, float *out, int seq_len, int N) {
863     int idx = blockIdx.x * blockDim.x + threadIdx.x;
864     float x1 = x[idx * 2];
865     float x2 = x[idx * 2 + 1];
866     int token_pos = idx / N; // 序列位置
867     int token_idx = idx % N; // 维度对索引
868
869     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
870     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
871     float sin_v = sinf(token_pos * exp_v);
872     float cos_v = cosf(token_pos * exp_v);
873
874     // 2D 旋转
875     float out1 = x1 * cos_v - x2 * sin_v;
876     float out2 = x1 * sin_v + x2 * cos_v;
877     out[idx * 2] = out1;
878     out[idx * 2 + 1] = out2;
879 }
880
881 // =====

```

```

882 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
883 // =====
884 // 面试要点 (Bank Conflict 专题) :
885 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
886 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
887 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
888 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
889 //
890 // 转置的四步演进:
891 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
892 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
893 //   BCF: smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
894 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
895 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
896
897 // 每个线程处理 1 个元素, block(16,16)
898 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
899 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
900 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
901 // Block: (16, 16, 1)
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
904     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
905     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
906     if (r < row && c < col) {
907         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
908         y[c * row + r] = x[r * col + c];
909     }
910 }
911
912 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
913 // Block: (16, 16, 1)
914 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
915 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
916 __global__ void mat_transpose_padded(
917     float *x, float *y, const int row, const int col) {
918     const int tx = threadIdx.x;
919     const int ty = threadIdx.y;
920
921     constexpr int TILE = 16;
922     constexpr int PAD = 1;
923     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
924     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
925
926     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
927     const int x_c = blockIdx.x * TILE + tx;
928     const int x_r = blockIdx.y * TILE + ty;
929
930     if (x_r * 4 < row && x_c < col) {
931         // Step 1: 4 rows × 1 col → smem (row-major);
932 #pragma unroll
933         for (int i = 0; i < 4; ++i) {
934             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
935         }
936         __syncthreads();
937
938         // Step 2: Transposed read from smem
939         float4 reg_trans;
940         reg_trans.x = tile[tx * 4 + 0][ty];
941         reg_trans.y = tile[tx * 4 + 1][ty];
942         reg_trans.z = tile[tx * 4 + 2][ty];
943         reg_trans.w = tile[tx * 4 + 3][ty];
944     }

```

```

945 // y 空间坐标: y_r = x 的列, y_c = x 的行
946 const int y_r = blockIdx.x * TILE + ty; // = x col
947 const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
948
949 // Coalesced write: y[y_r][y_c .. y_c+3]
950 FLOAT4(y[y_r * row + y_c]) = reg_trans;
951 }
952 }
953
954 // =====
955 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
956 // =====
957 // 面试要点:
958 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
959 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
960 // - 不同 K 值对应不同分块策略:
961 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
962 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
963 //   K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
964 //
965 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
966
967 // ---- SGEMV K32: 基础 warp-per-row ----
968 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
969 // grid(M/4), 每个 warp 负责一行
970 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
971 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
972 // Block: (32, 4, 1), 每行 1 warp
973 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
974 // source: LeetCode/kernels/sgemv/sgemv.cu
975 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
976     int tx = threadIdx.x; // 0~31
977     int ty = threadIdx.y; // 0~3
978     int lane = tx % kWarpSize; // 0~31
979     int m = blockIdx.x * blockDim.y + ty; // 全局行号
980     if (m < M) {
981         float sum = 0.0f;
982         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
983         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
984         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
985 #pragma unroll
986         for (int w = 0; w < NUM_ITERS; ++w) {
987             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
988             int k = w * kWarpSize + lane;
989             sum += a[m * K + k] * x[k];
990         }
991         sum = warp_reduce_sum<kWarpSize>(sum);
992         // 每个 warp 处理一行, lane 0 写回结果
993         if (lane == 0)
994             y[m] = sum;
995     }
996 }
997
998 // ---- SGEMV K128: float4 向量化 ----
999 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
1000 // Grid: ((M + 3) / 4, 1, 1)
1001 // Block: (32, 4, 1)
1002 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
1003 // source: LeetCode/kernels/sgemv/sgemv.cu
1004 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
1005     int tx = threadIdx.x; // 0~31
1006     int ty = threadIdx.y; // 0~3
1007     int lane = tx % kWarpSize; // 0~31

```

```

1008 int m = blockDim.y * blockIdx.x + ty;
1009
1010 if (m < M) {
1011     float sum = 0.0f;
1012     // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1013     const int NUM_ITERS = ((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1014 #pragma unroll
1015     for (int w = 0; w < NUM_ITERS; ++w) {
1016         int k = (w * kWarpSize + lane) * 4;
1017         float4 reg_x = FLOAT4(x[k]);
1018         float4 reg_a = FLOAT4(a[m * K + k]);
1019         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1020             reg_a.w * reg_x.w);
1021     }
1022     sum = warp_reduce_sum<kWarpSize>(sum);
1023     if (lane == 0)
1024         y[m] = sum;
1025 }
1026 }
1027
1028 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1029 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1030 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1031 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1032 // Block: (32, 4, 1)
1033 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1034 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1035 template <const int kRowPerWarp = 2>
1036 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1037     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1038     int tx = threadIdx.x;
1039     int ty = threadIdx.y;
1040     int lane = tx % kWarpSize;
1041     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1042     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1043     if (m < M) {
1044         float sum = A[m * K + k] * x[k];
1045         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1046         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1047         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1048         if (k == 0)
1049             y[m] = sum;
1050     }
1051 }
1052
1053 // =====
1054 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1055 // =====
1056 // 面试要点 (GEMM 优化五层金字塔):
1057 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1058 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1059 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1060 //   load/store 指令数 Level 4 — Tensor Core (MMA
1061 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1062 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1063 //
1064 // 计算密度递进:
1065 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1066 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1067 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1068
1069 // =====
1070 // Phase 7a: SGEMM (非 Tensor Core 路径)

```

```

1071 // =====
1072
1073 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
1074 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1075 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1076 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1077 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1078 // Block: (32, 32, 1), 1024 线程
1079 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1080 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1081     constexpr int BM = 32; // vec 版: 32x4 = 128
1082     constexpr int BN = 32; // vec 版: 32x4 = 128
1083     constexpr int BK = 32;
1084     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1085
1086     int bx = blockIdx.x;
1087     int by = blockIdx.y;
1088     int tx = threadIdx.x;
1089     int tid = threadIdx.y * blockDim.x + tx;
1090
1091     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1092     // 技巧: 一般来说 “/” 表示线程不是连续排布的, “%” 表示线程是连续排布的
1093     // 因此, 在需要考虑连续访问的维度使用 “%”, 比如, 连续的线程访问列方向连续的元素
1094     // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
1095     // 线程不连续访问 M 维度 → 用 /;
1096     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1097     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1098     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1099     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1100     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1101     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1102
1103     float sum = 0.f; // 遍历完整的K, slice K;
1104     // 这里不用pragma unroll, 因为K不是编译器常量, 编译器无法展开循环
1105     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1106         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1107         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1108         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1109         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1110         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1111         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1112         __syncthreads(); // 确保整个 smem tile 加载完毕
1113
1114     #pragma unroll
1115     for (int k = 0; k < BK; ++k) {
1116         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1117         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1118         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1119     }
1120     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1121 }
1122 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1123 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1124 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1125 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1126 }
1127
1128 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1129 // 在 Level 1 基础上引入两层优化:
1130 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1131 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1132 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1133 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major

```

```

1134 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4×4=16 个 C 元素
1135 // 1024 × 16 = 16384 = 128 × 128 ✓
1136 //
1137 // 线程到 4×4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1138 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1139 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1140 //
1141 // 加载映射 (每线程 4 个元素, float4):
1142 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1143 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1144 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1145 //
1146 // △ Bank Conflict 提示 (面试加分点):
1147 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1148 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1149 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1150 //
1151 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1152 // Block: (32, 32, 1), 1024 线程
1153 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1154 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1155 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1156     constexpr int BM = 128;
1157     constexpr int BN = 128;
1158     constexpr int BK = 32;
1159     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1160     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1161
1162     int bx = blockIdx.x;
1163     int by = blockIdx.y;
1164     int tx = threadIdx.x;
1165     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1166
1167     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1168     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1169     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1170     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1171     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1172     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1173
1174     int load_gmem_a_m = by * BM + load_smem_a_m;
1175     int load_gmem_b_n = bx * BN + load_smem_b_n;
1176
1177     // 4×4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1178     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1179     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1180     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1181     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1182     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1183
1184     float sum[4][4] = {0.f};
1185     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1186         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1187         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1188         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1189         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1190         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1191         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1192         __syncthreads();
1193     }
1194     #pragma unroll
1195     for (int k = 0; k < BK; ++k) {
1196         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4×4=16 次 FMA

```

```

1197     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1198                       s_a[comp_smem_a_m_base + 1][k],
1199                       s_a[comp_smem_a_m_base + 2][k],
1200                       s_a[comp_smem_a_m_base + 3][k]};
1201     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1202                       s_b[k][comp_smem_b_n_base + 1],
1203                       s_b[k][comp_smem_b_n_base + 2],
1204                       s_b[k][comp_smem_b_n_base + 3]};
1205 #pragma unroll
1206     for (int i = 0; i < 4; ++i) {
1207 #pragma unroll
1208         for (int j = 0; j < 4; ++j) {
1209             sum[i][j] += a_vals[i] * b_vals[j];
1210         }
1211     }
1212 }
1213 __syncthreads();
1214 }
1215
1216 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1217 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1218 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1219 #pragma unroll
1220 for (int i = 0; i < 4; ++i) {
1221     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1222     float4 reg_c;
1223     reg_c.x = sum[i][0];
1224     reg_c.y = sum[i][1];
1225     reg_c.z = sum[i][2];
1226     reg_c.w = sum[i][3];
1227     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1228 }
1229 }
1230
1231 // =====
1232 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1233 // =====
1234 // 面试要点:
1235 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1236 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1237 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1238 //   寄存器)
1239 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1240 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1241 //
1242 // ★ TN 布局详解 (面试高频考点):
1243 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1244 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1245 //   N = Normal (列优先, BLAS 原生格式)
1246 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1247 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1248 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1249 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1250 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1251 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1252 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1253 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1254 //
1255 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1256 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1257 //   N = col-major (列优先, BLAS native = normal)
1258 //
1259 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]

```

```

1260 // - A: row-major [M, K], B: row-major [K, N]
1261 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1262 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1263 //
1264 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1265 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1266 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1267 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1268 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1269 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1270 //
1271 // WGMMA (Warp Group MMA, Hopper+):
1272 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1273 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1274 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1275 // threads) 做计算
1276 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1277 //
1278 // =====
1279 // Phase 7b-1: MMA PTX 宏定义
1280 // =====
1281 //
1282 // ---- gmem → smem: cp.async ----
1283 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1284 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1285 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1286 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1287 // - wait_all: 等价于 commit_group + wait_group 0。
1288 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1289 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1290 #define CP_ASYNC_WAIT_GROUP(n) \
1291     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1292 //
1293 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1294 #define CP_ASYNC_CG(dst, src, bytes) \
1295     asm volatile( \
1296         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1297         "l"(src), "n"(bytes))
1298 //
1299 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1300 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1301 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1302 // aligned: 要求 128-bit 对齐, trans: 转置加载
1303 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1304     asm volatile( \
1305         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1306         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1307         : "r"(addr))
1308 //
1309 #define LDMATRIX_X2(R0, R1, addr) \
1310     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1311         : "=r"(R0), "=r"(R1) \
1312         : "r"(addr))
1313 //
1314 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1315 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1316 #define LDMATRIX_X2_T(R0, R1, addr) \
1317     asm volatile( \
1318         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1319         : "=r"(R0), "=r"(R1) \
1320         : "r"(addr))
1321 //
1322 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----

```

```

1323 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1324 // row.col: A 是 row-major, B 是 col-major
1325 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1326 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1327 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1328 #define MMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1329     asm volatile( \
1330         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1331         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1332         : "=r"(RD0), "=r"(RD1) \
1333         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1334         "r"(RC1)) \
1335
1336 // mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
1337 // f32 accumulator variant: 4 output registers (RD0-RD3), A/B still f16.
1338 // C matrix 16x8=128 elements, 32 threads x 4 f32 = 4 uint32 per thread.
1339 #define MMA16816F32(RD0, RD1, RD2, RD3, RA0, RA1, RA2, RA3, RB0, RB1, RC0, \
1340                     RC1, RC2, RC3) \
1341     asm volatile( \
1342         "mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 {%0, %1, %2, " \
1343         "%3}, {%4, %5, %6, %7}, {%8, %9}, {%10, %11, %12, %13};\n" \
1344         : "=r"(RD0), "=r"(RD1), "=r"(RD2), "=r"(RD3) \
1345         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1346         "r"(RC1), "r"(RC2), "r"(RC3)) \
1347
1348 // ---- MMA 辅助函数 ----
1349 // div_ceil: 整数除法向上取整
1350 #define HOST_DEVICE_INLINE __device__ __host__ inline
1351 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1352     return (a % b != 0) ? (a / b + 1) : (a / b);
1353 }
1354
1355 // =====
1356 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1357 // =====
1358 // 面试重点 — Tile Hierarchy:
1359 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1360 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1361 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1362 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1363 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1364 //
1365 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1366 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1367 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1368 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1369 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1370 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1371 //
1372 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1373 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1374 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1375 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1376 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1377 //
1378 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1379 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1380 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1381 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1382 //
1383 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1384 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1385 //

```

```

1386 //      N / bx → 0      1      2      3      ...
1387 //      M ↓
1388 //      by = 0      

|        |        |        |        |
|--------|--------|--------|--------|
| SM0    | SM1    | SM2    | SM3    |
| A 0,B0 | A 0,B1 | A 0,B2 | A 0,B3 |

      ...
1389 //
1390 //
1391 //
1392 // 同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1393 // 推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1394 // L2 更容易被其他 B tile 挤占。
1395 //
1396 // Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1397 // 下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:
1398 //
1399 //      N / local x → 0      1
1400 //      M ↓
1401 //      by = 0      

|     |     |
|-----|-----|
| SM0 | SM1 |
| SM2 | SM3 |

      → A 0; B^T 0, B^T 1
1402 //
1403 //      by = 1      

|     |     |
|-----|-----|
| SM2 | SM3 |
|-----|-----|

      → A 1; B^T 0, B^T 1
1404 //
1405 //
1406 // 对于这个 2x2 窗口: 同一行横向复用 A (SM0/SM1、SM2/SM3), 同一列纵向复用 B
1407 // (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2, 但缩小它会缩短再次访问相同 bx 的距离。
1408 // 此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会, 不保证 CTA 的 z 顺序、
1409 // 缓存驻留或 L2 hit。
1410 //
1411 // Launch (kBlockSwizzle=false, 当前默认路径):
1412 //   grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1413 // Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid):
1414 //   tiles_n = ceil(N / BN), S = ceil(N / 2048)
1415 //   grid = (ceil(tiles_n / S), ceil(M / BM), S)
1416 //   bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1417 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1418 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1419 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1420 // 对齐; ceil 只保证逻辑 tile 编号不遗漏, 并不使部分尾 tile 变为安全。
1421 // Block: (256, 1, 1), 8 warps
1422 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1423 // =====
1424 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1425           const int kMmaN = 8,           // MMA atom N dim
1426           const int kMmaK = 16,          // MMA atom K dim, also BK tile = kMmaK
1427           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1428           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1429           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1430           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1431           const int kStages = 3,         // cp.async pipeline depth
1432           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1433 __global__ void __launch_bounds__(256)
1434   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1435   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1436   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1437   // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1438   // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1439   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1440   const int by = blockIdx.y;
1441   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1442   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1443   constexpr int BK = kMmaK; // 16
1444
1445   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1446   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1447   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1448   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外

```

```

1449 // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1450 extern __shared__ half smem[];
1451 half *s_a = smem;
1452 half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1453 constexpr int s_a_stage_offset = BM * BK; // 128*16
1454 constexpr int s_b_stage_offset = BN * BK; // 128*16  $\Delta$  BN(128)×BK(16) = BT row-major
1455 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1456 const int warp_id = tid / kWarpSize; // 0~7
1457 const int lane_id = tid % kWarpSize; // 0~31
1458 // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1459 // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1460 //
1461 //           N direction (warp_n)
1462 //           0      1      2      3
1463 // M direction (warp_m) 0  MMA0  MMA2  MMA4  MMA6
1464 //                       1  MMA1  MMA3  MMA5  MMA7
1465 // MMA的排布方式不是唯一的, 现在这样排是因为结合MMA/VAL Tile之后, 刚好能覆盖整个
1466 // C Tile[128,128]。只要调整MMA/VAL Tile, 这里的排布方式也可以跟着调整。要注意的
1467 // 是, MMA0-7逻辑上是可以认为是并行执行的, 各自的计算结果累计加到对应的C Tile位置上。
1468 const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1469 const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1470
1471 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1472 // TN 布局关键: A[m*K+k] 是 row-major, BT[n*K+k] 是 row-major (内维连续的是 K)
1473 int load_smem_a_m = tid / 2; // 0~127
1474 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1475 int load_smem_b_n = tid / 2; // 0~127  $\rightarrow$  BT 的 N 方向 (row-major 的行)
1476 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8  $\rightarrow$  BT 的 K 方向 (row-major 的列)
1477 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1478 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1479 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1480     return;
1481
1482 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1483 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1484 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1485 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1486 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1487
1488 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1489 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1490 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1491
1492 // 预加载前 (kStages-1) 个 stage
1493 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), BT 用 n*K+k(row-major, 即 B col-major [K×N])
1494 #pragma unroll
1495 for (int k = 0; k < (kStages - 1); ++k) {
1496     int load_gmem_a_k = k * BK + load_smem_a_k;
1497     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1498     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1499         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1500     );
1501     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1502
1503     int load_gmem_b_k = k * BK + load_smem_b_k;
1504     // BT: [n][k] row-major (即 B[k][n] col-major)  $\Delta$ 
1505     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1506     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1507         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1508     );
1509     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1510
1511     CP_ASYNC_COMMIT_GROUP();
1512 }

```

```

1512 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1513 __syncthreads();
1514
1515 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1516 const int NUM_K_TILES = div_ceil(K, BK);
1517 // 此处不用pragma unroll, 因为 K 不是编译期常量, 因此NUM_K_TILES 也不是编译期常量
1518 for (int k = 0; k < NUM_K_TILES; ++k) {
1519     int smem_sel = k % kStages; // 计算 tile k 的 stage
1520     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1521
1522     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
1523     // 加载了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1524     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1525     // (row-major, 内维连续的是 K)
1526     if (k + kStages - 1 < NUM_K_TILES) {
1527         int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1528         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1529         int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1530         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1531
1532         uint32_t load_smem_a_ptr =
1533             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1534                 load_smem_a_m * BK + load_smem_a_k) *
1535                 sizeof(half));
1536         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1537
1538         uint32_t load_smem_b_ptr =
1539             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1540                 load_smem_b_n * BK + load_smem_b_k) *
1541                 sizeof(half));
1542         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1543         CP_ASYNC.COMMIT_GROUP();
1544     }
1545
1546     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1547     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1548     // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1549     uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1550     uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1551
1552     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1553 #pragma unroll
1554     for (int i = 0; i < kValTileM; ++i) {
1555         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1556         // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1557         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1558         // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1559         // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1560         int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1561         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1562         uint32_t lane_smem_a_ptr =
1563             (smem_a_base_ptr +
1564                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1565                 sizeof(half));
1566         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1567     }
1568
1569     // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1570     // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1571     // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1572 #pragma unroll
1573     for (int j = 0; j < kValTileN; ++j) {

```

```

1575 // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1576 // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1577 // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1578 int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1579 // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1580 // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1581 int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1582 int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1583 uint32_t lane_smem_b_ptr =
1584     (smem_b_base_ptr +
1585      (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1586       sizeof(half));
1587 LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1588 }
1589
1590 // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次, 在N方向重复kValTileN(4)次
1591 #pragma unroll
1592 for (int i = 0; i < kValTileM; ++i) {
1593 #pragma unroll
1594     for (int j = 0; j < kValTileN; ++j) {
1595         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1596                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1597                 RB[j][0], RB[j][1], // B fragment
1598                 RC[i][j][0], RC[i][j][1]);
1599     }
1600 }
1601
1602 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1603 if (k + kStages - 1 < NUM_K_TILES) {
1604     CP_ASYNC_WAIT_GROUP(kStages - 2);
1605 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1606     CP_ASYNC_WAIT_GROUP(0);
1607 }
1608 __syncthreads();
1609 }
1610
1611 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1612 {
1613     for (int i = 0; i < kValTileM; ++i) {
1614         // RC[kValTileM][kValTileN][2] 中 RC[...][...][2] 中保存了2个 uint32
1615         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...][...][0]
1616         // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1617         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1618         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1619         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1620         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1621         // =====
1622         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1623         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1624         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1625         //
1626         //      cols: c0-1   c2-3   c4-5   c6-7
1627         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1628         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1629         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1630         //  ...                                     |
1631         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1632         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1633         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1634         // r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1635         //  ...                                     |
1636         // r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1637         //

```

```

1638 // Within a 4-lane group (e.g. T0-T3 for row 0):
1639 //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1640 //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1641 // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1642 // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1643 // =====
1644 #pragma unroll
1645 for (int j = 0; j < kValTileN; ++j) {
1646     RC0[j][0] = RC[i][j][0];
1647     RC1[j][0] = RC[i][j][1];
1648     RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1649     RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1650     RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1651     RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1652     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1653     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1654 }
1655 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1656 // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1657 //
1658 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1659 // |-----|-----|-----|-----|
1660 // | 0~3    | 0        | row 0      | row 8      |
1661 // | 4~7    | 1        | row 1      | row 9      |
1662 // | 8~11   | 2        | row 2      | row 10     |
1663 // | 12~15  | 3        | row 3      | row 11     |
1664 // | 16~19  | 4        | row 4      | row 12     |
1665 // | 20~23  | 5        | row 5      | row 13     |
1666 // | 24~27  | 6        | row 6      | row 14     |
1667 // | 28~31  | 7        | row 7      | row 15     |
1668 //
1669 if (lane_id % 4 == 0) {
1670     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1671     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
1672     // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}, 对应 RC0/RC1 (+8) 的行号, 表示2个8x8矩阵的行号
1673     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
1674 #pragma unroll
1675     for (int j = 0; j < kValTileN; ++j) {
1676         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1677         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
1678         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
1679         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
1680         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
1681         // 128-bit store: 一次写入 8 个 half
1682         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1683             *reinterpret_cast<float4*>(&RC0[j][0]);
1684         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1685             *reinterpret_cast<float4*>(&RC1[j][0]);
1686     }
1687 }
1688 }
1689 }
1690 }
1691
1692 // =====
1693 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1694 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1695 // =====
1696 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1697 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1698 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1699 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1700 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:

```

```

1701 //      rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1702 //      rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1703 //      rows 8~11: repeat rows 0~3 pattern
1704 //      rows 12~15: repeat rows 4~7 pattern
1705 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1706 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1707 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1708 //      对 MMA m16n8k16 的 BK=16 而言刚好满足。
1709 //
1710 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1711 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1712
1713 // ---- Swizzle 辅助函数 ----
1714
1715 // i: row index; j: col index.
1716 // Returns the chunk-level (8 fp16 = 16 B per chunk) column swizzle, i.e.
1717 // 0 / 8 / 16 / ... that the caller adds to the chunk-aligned base column.
1718 // The within-chunk offset (`j & 7`) is preserved by the caller.
1719 //
1720 // kColStride matches the equivalent CuTe ``Swizzle<B, M, S>`` swizzle
1721 // hardware mode used by TMA bulk-tensor descriptors:
1722 // * kColStride = 16 fp16 = 32 B/row -> Swizzle<1, 4, 3> = SWIZZLE_32B
1723 // * kColStride = 32 fp16 = 64 B/row -> Swizzle<2, 4, 3> = SWIZZLE_64B
1724 // * kColStride = 64 fp16 = 128 B/row -> Swizzle<3, 4, 3> = SWIZZLE_128B
1725 // The XOR mask is derived from the byte-address bit positions used by the
1726 // underlying CuTe swizzle pattern: bits {4..(4+B-1)} XOR bits
1727 // {7..(7+B-1)} of the absolute byte address. Translating to (i, j) with
1728 // row stride ``kColStride * 2`` bytes:
1729 // * 32B (B=1) : (j>>3) ^ (i>>2) [chunk in {0, 1}]
1730 // * 64B (B=2) : chunk(2b) ^ {(i>>1)&1, (i>>2)&1} [{0..3}]
1731 // * 128B (B=3) : chunk(3b) ^ {i&1, (i>>1)&1, (i>>2)&1} [{0..7}]
1732 // source: LeetCUDA/ffpa-attn/cuffpa/swizzle.cuh
1733 //
1734 // v1/v2 实现 (由编译宏 NOTES_V2_ENABLE_SWIZZLE_V2 门控):
1735 // swizzle_v1_impl : 原手写 XOR 分支 (permuted<kColStride,8>), 按列宽展开。
1736 // swizzle_v2_impl : 用本地 SwizzleBMS<B,M,S> 镜像 cute::Swizzle<B,M,S>::apply
1737 // 位级公式统一表达 (16/32/64); kColStride=8 回退 v1。
1738 // swizzle : 公开发版器, 宏门控, 接口与原 swizzle<kColStride> 一致。
1739 // v2 不是新算法, 而是把 v1 的手写展开重新表达为 cute 的统一位置换公式,
1740 // 便于脱离 cute 框架理解 swizzle 本质。v1/v2 bit-exact 等价。
1741 template <const int kColStride = 16, const int kStep = 8>
1742 static __host__ __device__ __forceinline__ int permuted(int i, int j) {
1743     static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64 ||
1744         kColStride == 8,
1745         "kColStride must be one of {8, 16, 32, 64} (matches "
1746         "SWIZZLE_32B/64B/128B + the "
1747         "kStep=4 narrow legacy case).");
1748     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1749     static_assert(kColStride % kStep == 0,
1750         "kColStride must be multiple of kStep.");
1751     if constexpr (kStep == 4) {
1752         static_assert(kColStride <= 16, "kStep=4 only supports (kColStride <= 16).");
1753         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1754     } else if constexpr (kColStride == 16) {
1755         return (((j >> 3) ^ (i >> 2)) & 1) << 3; // SWIZZLE_32B
1756     } else if constexpr (kColStride == 32) {
1757         const int chunk = (j >> 3) & 3;
1758         const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
1759         return (chunk ^ xor_mask) << 3; // SWIZZLE_64B
1760     } else { // kColStride == 64
1761         const int chunk = (j >> 3) & 7;
1762         const int xor_mask =
1763             (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);

```

```

1764     return (chunk ^ xor_mask) << 3; // SWIZZLE_128B
1765 }
1766 }
1767
1768 // Manually SMEM swizzling for bank conflict free.
1769 // -----
1770 // [INFO] Assert smem store layout col_stride <= 16, prefer 16. |
1771 // [INFO] For logical_col_stride > 16, we have to permute the |
1772 // [INFO] smem store layout using col major ZigZag method: |
1773 // [INFO] e.g, --> Q smem logical layout [Br][64]. |
1774 // [INFO] --> col major ZigZag permuted --> |
1775 // [INFO] --> Q smem store layout [4][Br][16]. |
1776 // -----
1777 // -----
1778 // -----swizzle layout-----
1779 // -----logical col 0~64, step 8-----
1780 // -----smem col 0~16, step 8-----
1781 // -----
1782 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1783 // |row 0 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1784 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1785 // |row 1 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1786 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1787 // |row 2 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1788 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1789 // |row 3 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1790 // -----
1791 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1792 // |row 4 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1793 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1794 // |row 5 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1795 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1796 // |row 6 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1797 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1798 // |row 7 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1799 // -----
1800 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1801 // |row 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1802 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1803 // |row 9 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1804 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1805 // |row 10| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1806 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1807 // |row 11| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1808 // -----
1809 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1810 // |row 12| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1811 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1812 // |row 13| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1813 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1814 // |row 14| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1815 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1816 // |row 15| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1817 // -----
1818 template <const int kColStride = 16>
1819 static __host__ __device__ __forceinline__ int swizzle_v1_impl(int i, int j) {
1820     return permuted<kColStride, 8>(i, j);
1821 }
1822
1823 // Swizzle v2: cute::Swizzle<B,M,S> 位级镜像（脱离 cute 框架独立理解）
1824 //
1825 // v2 双重目的：
1826 // (1) 探索优化 v1 的可能 -- v1 按 kColStride 手写 XOR 分支展开，已被 nvcc

```

```

1827 //      strength-reduce 到极简位运算; v2 用 cute 的统一位置换公式重新表达,
1828 //      预期中性, 但统一表达式可能更利于合并到地址算术 (bonus).
1829 //      (2) 把 cute Swizzle 的核心位逻辑从 cute 框架提取出来独立理解 -- 让 notes-v2
1830 //      不依赖 cute 头即可展示 Swizzle<B,M,S> 的位级语义, 便于读者脱离 cute
1831 //      模板体系直接掌握 swizzle 原理.
1832 //
1833 // 注释叙述参考: 上方 hgemm_mma_stages_tn_cute 处的 Swizzle<B,M,S> 原理解释
1834 // (二维逻辑空间 + 列置换), 此处适配 v2 的 (B,M,S)=(1/2/3, 4, 3) 语境.
1835 //
1836 // Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 byte offset
1837 // 重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理
1838 // offset:
1839 // 1. 连续的 2^M 个元素组成二维空间中的一个基本元素 (元素宽度);
1840 // 2. 连续的 2^S 个基本元素组成一行 (列数);
1841 // 3. 二维空间包含 2^B 行 (行数, 这些行的 chunk 互相置换);
1842 // 4. 对二维坐标做列置换: icol' = irow ^ icol;
1843 // 5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset.
1844 //
1845 // 位级实现 (与 cutlass/include/cute/swizzle.hpp 一致):
1846 // bit 布局: 0bxxxxxxxxxxxxxxxxYYxxxxxxxxZZxxxx
1847 //           ^--^ MBase: 保留不参与置换的低位数
1848 //           ^-^   ^-^ BBits: XOR 掩码位数
1849 //           ^-----^ SShift: YYY 相对 ZZZ 的位移
1850 // apply(off) = off ^ shiftr(off & yyy_msk, S)
1851 // yyy_msk    = ((1<<B)-1) << (M + max(0,S)) // YYY 掩码位置
1852 // zzz_msk    = ((1<<B)-1) << (M - min(0,S)) // ZZZ 掩码位置 (文档用)
1853 //
1854 // 模板参数物理/语义含义:
1855 // B (BBits) : 二维逻辑空间的行数位 = 2^B 行, 即 swizzle 周期覆盖 2^B 个 chunk
1856 //            (这些 chunk 互相置换); 同时是 TMA 硬件 swizzle 模式
1857 //            SWIZZLE_{32,64,128}B 的位数 (B=1/2/3).
1858 // M (MBase) : 基本元素宽度位 = 2^M 个连续元素组成一个基本元素. 对 fp16 +
1859 //            8-element chunk, 基本元素 = 16B = 2^4, 故 M=4; 地址 bit[0..M-1]
1860 //            在 apply 中保持不变 (保留).
1861 // S (SShift) : 二维空间每行列数位 = 2^S 个基本元素组成一行; 同时是 YYY 掩码
1862 //            相对 ZZZ 掩码的位移. S>0 表示 YYY 在更高位, 向右移 S 位到 ZZZ
1863 //            位置后 XOR. 本 notes-v2 三模式 S 恒为 3 (YYY 在 bit[7..7+B-1],
1864 //            即行索引 i 的低 B 位; ZZZ 在 bit[4..4+B-1], 即列 chunk 索引).
1865 //
1866 // 完整 swizzle 周期覆盖 2^(M+S+B) 个元素 (fp16 下 2^(M+S+B+1) 字节):
1867 // (B,M,S)=(1,4,3): 256 elem / 512B (SWIZZLE_32B)
1868 // (B,M,S)=(2,4,3): 512 elem / 1024B (SWIZZLE_64B)
1869 // (B,M,S)=(3,4,3): 1024 elem / 2048B (SWIZZLE_128B)
1870 template <int B, int M, int S>
1871 struct SwizzleBMS {
1872     static_assert(M >= 0, "MBase must be non-negative.");
1873     static_assert(B > 0, "BBits must be positive.");
1874     static_assert((S > 0 ? S : -S) >= B,
1875         "abs(SShift) must be >= BBits (cute::Swizzle constraint).");
1876
1877     static constexpr int bit_msk = (1 << B) - 1;
1878     static constexpr int yyy_msk = bit_msk << (M + (S > 0 ? S : 0));
1879     static constexpr int zzz_msk = bit_msk << (M - (S < 0 ? S : 0));
1880
1881     // apply: 对 byte offset 做 XOR 位置换, ZZZ ^= (YYY >> S).
1882     static __host__ __device__ __forceinline__ int apply(int offset) {
1883         if constexpr (S >= 0) {
1884             return offset ^ ((offset & yyy_msk) >> S);
1885         } else {
1886             return offset ^ ((offset & yyy_msk) << (-S));
1887         }
1888     }
1889 };

```

```

1890
1891 // swizzle_v2_impl: 用 SwizzleBMS<B,M,S>::apply 计算 chunk 级列 swizzle.
1892 //
1893 // 流程: (i,j) -> byte offset off=(i*kColStride+j)*sizeof(half) -> SwizzleBMS
1894 //      位级置换 -> 取 [M, M+B) 位作为物理 chunk 索引 -> 乘 kStep 还原为列偏移.
1895 //
1896 // kColStride -> (B,M,S) 映射表 (对应 TMA 硬件 swizzle 模式):
1897 //   kColStride=16 fp16=32B/row -> (B,M,S)=(1,4,3) = SWIZZLE_32B
1898 //   kColStride=32 fp16=64B/row -> (B,M,S)=(2,4,3) = SWIZZLE_64B
1899 //   kColStride=64 fp16=128B/row -> (B,M,S)=(3,4,3) = SWIZZLE_128B
1900 //
1901 // kColStride=8 (kStep=4 legacy): 无对应 cute 标准 TMA swizzle 模式, 回退到
1902 //   swizzle_v1_impl. 该路径当前无任何 kernel 使用, 回退保证接口契约零变更.
1903 //
1904 // 等价性: 对 (16,32,64), v2 输出与 v1 bit-exact 相同 (见 test_swizzle_equiv).
1905 //   证明: off 的 bit[4..4+B-1] 恰是 (j>>3) 低 B 位 (因 i*kColStride 在这些位
1906 //   为 0); bit[7..7+B-1] 恰是 i 低 B 位 (因 j<kColStride 不贡献高位). apply
1907 //   把 YYY(bit[7..]) XOR 到 ZZZ(bit[4..]), 取 [M,M+B)=bit[4..4+B-1] 即得
1908 //   (j>>3)^i 的低 B 位 = v1 的 chunk^xor_mask.
1909 template <const int kColStride = 16>
1910 static __host__ __device__ __forceinline__ int swizzle_v2_impl(int i, int j) {
1911     if constexpr (kColStride == 8) {
1912         return swizzle_v1_impl<kColStride>(i, j);
1913     } else {
1914         static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64,
1915             "v2 supports kColStride in {16, 32, 64} (cute TMA swizzle); "
1916             "8 falls back to v1.");
1917         constexpr int B = (kColStride == 16) ? 1 : (kColStride == 32) ? 2 : 3;
1918         constexpr int M = 4;
1919         constexpr int S = 3;
1920         constexpr int kStep = 8;
1921         const int off = (i * kColStride + j) * (int)sizeof(half);
1922         const int sw = SwizzleBMS<B, M, S>::apply(off);
1923         return ((sw >> M) & ((1 << B) - 1)) * kStep;
1924     }
1925 }
1926
1927 // swizzle: 公开派发器, 由编译宏 NOTES_V2_ENABLE_SWIZZLE_V2 门控 v1/v2.
1928 //   未定义宏 (默认) -> swizzle_v1_impl (原手写 XOR 分支, 历史默认路径).
1929 //   定义宏 -> swizzle_v2_impl (cute Swizzle<B,M,S> 位级镜像).
1930 // 接口与原 swizzle<kColStride>(i,j) 完全一致, 所有 kernel 调用点无需改动.
1931 // v1/v2 等价性由 host 端 test_swizzle_equiv (--swizzle-eq-check) 验证.
1932 template <const int kColStride = 16>
1933 static __device__ __forceinline__ int swizzle(int i, int j) {
1934     #if defined(NOTES_V2_ENABLE_SWIZZLE_V2)
1935         return swizzle_v2_impl<kColStride>(i, j);
1936     #else
1937         return swizzle_v1_impl<kColStride>(i, j);
1938     #endif
1939 }
1940
1941 // =====
1942 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1943 //      + Register Double Buffering (generic kValTileK, default BK=64)
1944 // =====
1945 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1946 //   - kValTileK: 每个 BK tile 包含 kValTileK 个 kMmaK slice (BK = kMmaK * kValTileK)
1947 //   - 统一 BK tile: 所有 k_step slice 连续存放在同一个 BK 宽的 smem tile 中
1948 //   - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换 (与 kValTileK 无关)
1949 //   - ldmatrix 与 MMA 计算重叠: 加载 k_step+1 的同时用另一组寄存器做 k_step 计算
1950 //   - 支持任意 kValTileK >= 2; 默认 kValTileK=4 (BK=64), kStages=2 推荐 64KB smem
1951 //
1952 // ★ smem 布局:

```

```

1953 // s_a: [stage0][stage1]...[stage(kStages-1)], 每个 stage = BM × BK = 128 × BK
1954 // s_b: 紧接 s_a 之后
1955 // 每个 stage 内 k_step slice 列偏移 = k_step * kMmaK
1956 //
1957 // ★ 寄存器双缓冲时间线 (每 k 迭代内) :
1958 // Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1959 // Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1960 // Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1961 // Step 4: adaptive wait + __syncthreads
1962 // Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1963 //
1964 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1965 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1966 // Block: (256, 1, 1), 8 warps
1967 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1968           const int kMmaN = 8,           // MMA atom N dim
1969           const int kMmaK = 16,          // MMA atom K dim
1970           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1971           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1972           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1973           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1974           const int kValTileK = 4,       // MMA_K slices per BK tile, BK = kMmaK * kValTileK
1975           const int kStages = 2,         // cp.async pipeline depth (2 for kValTileK>=4 to fit smem)
1976           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1977 __global__ void __launch_bounds__(256)
1978   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1979   static_assert(kValTileK >= 2, "Register double buffering requires kValTileK >= 2");
1980   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1981   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1982   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1983   const int bx = ((int)kBlockSwizzle) * blockDim.z * gridDim.x + blockIdx.x;
1984   const int by = blockIdx.y;
1985   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1986   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1987   constexpr int BK = kMmaK * kValTileK;           // 16 * kValTileK
1988
1989   // kStages stages, each with BM×BK for A, BN×BK for B
1990   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK 宽 tile 中
1991   extern __shared__ half smem[];
1992   half *s_a = smem;
1993   half *s_b = smem + kStages * BM * BK;
1994   constexpr int s_a_stage_offset = BM * BK;
1995   constexpr int s_b_stage_offset = BN * BK;
1996
1997   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1998   const int warp_id = tid / kWarpSize;
1999   const int lane_id = tid % kWarpSize;
2000   const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
2001   const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
2002
2003   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
2004   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
2005   // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 在后续的 kValTileK 循环中
2006   // 会加上 k_step*kMmaK, 最终覆盖全部 BK 列
2007   int load_smem_a_m = tid / 2;           // 0~127
2008   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
2009   int load_smem_b_n = tid / 2;           // 0~127 → B^T 的 N 方向 (row-major 的行)
2010   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
2011   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
2012   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
2013   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
2014     return;
2015

```

```

2016 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
2017 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
2018 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
2019 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
2020 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
2021
2022 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
2023 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
2024
2025 // Prefetch (kStages-1) stages: load all k_step slices for each early stage
2026 #pragma unroll
2027 for (int k = 0; k < (kStages - 1); ++k) {
2028 #pragma unroll
2029 for (int k_step = 0; k_step < kValTileK; ++k_step) {
2030     int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
2031     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
2032     int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
2033     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
2034
2035     uint32_t load_smem_a_ptr =
2036         (smem_a_base_ptr +
2037          (k * s_a_stage_offset + load_smem_a_m * BK +
2038           k_step * kMmaK +
2039           swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
2040           sizeof(half));
2041     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
2042
2043     uint32_t load_smem_b_ptr =
2044         (smem_b_base_ptr +
2045          (k * s_b_stage_offset + load_smem_b_n * BK +
2046           k_step * kMmaK +
2047           swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
2048           sizeof(half));
2049     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
2050 }
2051 CP_ASYNC.COMMIT_GROUP();
2052 }
2053
2054 CP_ASYNC.WAIT_GROUP(kStages - 2);
2055 __syncthreads();
2056
2057 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
2058 uint32_t RA[2][kValTileM][4];
2059 uint32_t RB[2][kValTileN][2];
2060 int reg_st_idx = 0; // write target for ldmatrix
2061 int reg_ld_idx = 1; // read source for MMA
2062
2063 // Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) -> reg[0]
2064 // 此时, reg_st_idx = 0, 对0位置的寄存器buffer做初始化。
2065 #pragma unroll
2066 for (int i = 0; i < kValTileM; ++i) {
2067     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2068     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2069     int lane_smem_a_k = (lane_id / 16) * 8;
2070     uint32_t lane_smem_a_ptr =
2071         (smem_a_base_ptr +
2072          (0 * s_a_stage_offset + lane_smem_a_m * BK +
2073           swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2074           sizeof(half));
2075     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2076                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2077                 lane_smem_a_ptr);
2078 }

```

```

2079 #pragma unroll
2080 for (int j = 0; j < kValTileN; ++j) {
2081     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2082     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2083     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2084     uint32_t lane_smem_b_ptr =
2085         (smem_b_base_ptr +
2086          (0 * s_b_stage_offset + lane_smem_b_n * BK +
2087           swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2088           sizeof(half));
2089     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2090                lane_smem_b_ptr);
2091 }
2092
2093 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
2094 // BK = kMmaK * kValTileK, 每个 k 迭代覆盖 BK 个 K 元素
2095 const int NUM_K_TILES = div_ceil(K, BK);
2096 for (int k = 0; k < NUM_K_TILES; ++k) {
2097     int smem_sel = k % kStages;
2098     int smem_sel_next = (k + kStages - 1) % kStages;
2099
2100     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
2101     if (k + kStages - 1 < NUM_K_TILES) {
2102 #pragma unroll
2103         for (int k_step = 0; k_step < kValTileK; ++k_step) {
2104             int load_gmem_a_k =
2105                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
2106             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
2107             int load_gmem_b_k =
2108                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
2109             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
2110
2111             uint32_t load_smem_a_ptr =
2112                 (smem_a_base_ptr +
2113                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
2114                   k_step * kMmaK +
2115                   swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
2116                   sizeof(half));
2117             CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
2118
2119             uint32_t load_smem_b_ptr =
2120                 (smem_b_base_ptr +
2121                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
2122                   k_step * kMmaK +
2123                   swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
2124                   sizeof(half));
2125             CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
2126         }
2127         CP_ASYNC.COMMIT_GROUP();
2128     }
2129
2130     // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
2131     // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
2132     // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
2133
2134 #pragma unroll
2135     for (int k_step = 0; k_step < kValTileK; ++k_step) {
2136         reg_st_idx ^= 1; // 0 → 1; 1 → 0
2137         reg_ld_idx ^= 1; // 1 → 0; 0 → 1
2138
2139         if (k_step + 1 < kValTileK) {
2140             int smem_k_offset = (k_step + 1) * kMmaK;
2141 #pragma unroll

```

```

2142     for (int i = 0; i < kValTileM; ++i) {
2143         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2144         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2145         int lane_smem_a_k = (lane_id / 16) * 8;
2146         uint32_t lane_smem_a_ptr =
2147             (smem_a_base_ptr +
2148              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2149               smem_k_offset +
2150               swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2151              sizeof(half));
2152         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2153                     RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2154                     lane_smem_a_ptr);
2155     }
2156     #pragma unroll
2157     for (int j = 0; j < kValTileN; ++j) {
2158         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2159         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2160         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2161         uint32_t lane_smem_b_ptr =
2162             (smem_b_base_ptr +
2163              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2164               smem_k_offset +
2165               swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2166              sizeof(half));
2167         LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2168                     lane_smem_b_ptr);
2169     }
2170 }
2171
2172 #pragma unroll
2173 for (int i = 0; i < kValTileM; ++i) {
2174     #pragma unroll
2175     for (int j = 0; j < kValTileN; ++j) {
2176         HMMMA16816(RC[i][j][0], RC[i][j][1],
2177                    RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
2178                    RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
2179                    RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
2180                    RC[i][j][0], RC[i][j][1]);
2181     }
2182 }
2183 }
2184
2185 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
2186 if (k + kStages - 1 < NUM_K_TILES) {
2187     CP_ASYNC_WAIT_GROUP(kStages - 2);
2188 } else {
2189     CP_ASYNC_WAIT_GROUP(0);
2190 }
2191 __syncthreads();
2192
2193 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
2194 if (k + 1 < NUM_K_TILES) {
2195     smem_sel = (k + 1) % kStages; // old smem_sel + 1
2196     #pragma unroll
2197     for (int i = 0; i < kValTileM; ++i) {
2198         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2199         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2200         int lane_smem_a_k = (lane_id / 16) * 8;
2201         uint32_t lane_smem_a_ptr =
2202             (smem_a_base_ptr +
2203              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2204               swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *

```

```

2205         sizeof(half));
2206     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2207         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2208         lane_smem_a_ptr);
2209 }
2210 #pragma unroll
2211 for (int j = 0; j < kValTileN; ++j) {
2212     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2213     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2214     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2215     uint32_t lane_smem_b_ptr =
2216         (smem_b_base_ptr +
2217         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2218         swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2219         sizeof(half));
2220     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2221         lane_smem_b_ptr);
2222 }
2223 }
2224 }
2225
2226 // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
2227 // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间, RA[2][4][4]正好可以复用
2228 for (int i = 0; i < kValTileM; ++i) {
2229 #pragma unroll
2230 for (int j = 0; j < kValTileN; ++j) {
2231     RA[0][j][0] = RC[i][j][0];
2232     RA[1][j][0] = RC[i][j][1];
2233     RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2234     RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2235     RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2236     RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2237     RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2238     RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2239 }
2240 if (lane_id % 4 == 0) {
2241     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2242     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2243 #pragma unroll
2244 for (int j = 0; j < kValTileN; ++j) {
2245     int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2246     int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2247     int store_gmem_c_addr_0 =
2248         store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2249     int store_gmem_c_addr_1 =
2250         (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2251     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2252         *reinterpret_cast<float4*>(&RA[0][j][0]);
2253     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2254         *reinterpret_cast<float4*>(&RA[1][j][0]);
2255 }
2256 }
2257 }
2258 }
2259
2260 // =====
2261 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2262 // =====
2263 // 面试要点 (CuTe vs 手写 MMA PTX) :
2264 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2265 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2266 // - 与手写 MMA PTX (Phase 7b) 的对比:
2267 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle

```

```

2268 // - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2269 //      cute::copy / cute::gemm 自动展开为高效指令序列
2270 // - 核心抽象 ("CuTe 五要素") :
2271 //      1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2272 //      2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2273 //      3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2274 //      4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2275 //      5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2276 //
2277 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2278 //      MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2279 //      → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2280 //      → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2281 //      → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2282 //
2283 // 调参口诀 (面试速记) :
2284 //      BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2285 //      Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2286 //      重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2287 //      1. 连续的 2^M 个元素组成二维空间中的一个基本元素; (元素宽度)
2288 //      2. 连续的 2^S 个基本元素组成一行; (列数)
2289 //      3. 二维空间包含 2^B 行; (行数)
2290 //      4. 对二维坐标做列置换: icol' = irow ^ icol;
2291 //      5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2292 //      因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2293 //      CuTe 的位级实现等价于:
2294 //      offset' = offset ^ ((offset & YYY) >> S),
2295 //      YYY = ((1 << B) - 1) << (M + S)。
2296 //      对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2297 //      二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2298 //      一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2299 //      kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2300 //      EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2301 //      与逻辑 tile 组织; MMA atom越多, 需要的线程数就越多
2302 //
2303 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2304 //      - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2305 //      - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2306 //      - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2307 //
2308 // ★ 本实现的 Tile 配置:
2309 //      - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2310 //      - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2311 //      - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2312 //      BN=256 不是由 "8x2x16" 直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2313 //      - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2314 //      的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2315 //      - Threads: size(MMA{ }) = 128 (4 warps × 2x2 EU repeat)。
2316 //      - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2317 //
2318 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2319 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2320 // =====
2321
2322 #if defined(NOTES_V2_ENABLE_CUTE)
2323
2324 #include <cute/tensor.hpp>
2325 #include <cutlass/arch/barrier.h>
2326 #include <cutlass/device_kernel.h>
2327 #include <cutlass/numeric_conversion.h>
2328
2329 // =====
2330 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline

```

```

2331 // =====
2332 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
2333 //   -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价 B col-major  $[K \times N]$ )
2334 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2335 //
2336 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链):
2337 //   1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
2338 //   2. 主循环 over K tiles:
2339 //       a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2340 //       b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2341 //       c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2342 //       d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2343 //   3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store)。
2344 //
2345 // 面试对比 — 手写 MMA vs CuTe 代码量:
2346 //   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2347 //   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2348 template <typename T,                                // element type (half)
2349           int BM, int BN, int BK,                    // block tile: BM=128, BN=256, BK=32
2350           int kStage,                                // cp.async pipeline depth (2)
2351           typename TiledMMA,                          // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
2352           typename G2SCopyA,                          // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2353           typename G2SCopyB,                          // G→S B: same as G2SCopyA
2354           typename SmemLayoutA,                       // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2355           typename SmemLayoutB,                       // smem B: same atom → (BN,BK,kStage)
2356           typename SmemLayoutC,                       // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2357           typename S2RCopyAtomA,                      // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2358           typename S2RCopyAtomB,                      // S→R B: same as S2RCopyAtomA
2359           typename R2SCopyAtomC,                      // R→S C: UniversalCopy<int> (32-bit store)
2360           typename S2GCopyAtomC,                      // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2361           typename S2GCopyC,                          // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2362           const int BlockSwizzle = 0>                // 1 enables 3D grid swizzle for L2 locality
2363 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2364                                           int n, int k) {
2365     using namespace cute;
2366     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2367     static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");
2368
2369     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2370     extern __shared__ T shm_data[];
2371     T *Ashm = shm_data;
2372     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2373
2374     int idx = threadIdx.x;
2375     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2376     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2377     int iy = blockIdx.y; // M 方向 block 索引
2378     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2379     // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2380     if (iy * BM >= m || ix * BN >= n) return;
2381
2382     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2383     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2384     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2385                             make_stride(k, Int<1>{}));
2386     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2387                             make_stride(k, Int<1>{}));
2388     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2389                             make_stride(n, Int<1>{}));
2390
2391     // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2392     // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2393     //   gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。

```

```

2394 Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2395 Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2396 Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2397
2398 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2399 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2400 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2401
2402 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2403 TiledMMA tiled_mma;
2404 auto thr_mma = tiled_mma.get_slice(idx);
2405 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2406 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2407 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2408 cclear(tCrD); // 累加器清零
2409
2410 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2411 G2SCopyA g2s_tiled_copy_a;
2412 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2413 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2414 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2415
2416 G2SCopyB g2s_tiled_copy_b;
2417 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2418 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2419 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2420
2421 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2422 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2423 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2424 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2425 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2426 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2427
2428 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2429 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2430 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2431 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2432
2433 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2434 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2435 #pragma unroll
2436 for (int istage = 0; istage < kStage - 1; ++istage) {
2437     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2438               tAsA_copy(_, _, _, istage));
2439     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2440               tBsB_copy(_, _, _, istage));
2441     cp_async_fence();
2442     ++itile_to_read;
2443     ++ismem_write;
2444 }
2445 cp_async_wait<kStage - 2>();
2446 __syncthreads();
2447
2448 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2449 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2450 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2451 int num_k_tiles = k / BK;
2452 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2453 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2454 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2455
2456 #pragma unroll 1

```

```

2457 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2458     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode) 。
2459     //
2460     // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2461     //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2462     //   - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix) ,
2463     //     所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2464     //   - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2465     //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2466     //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2467     //     无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2468     //         for (i=0; i<kValTileM; ++i)
2469     //             for (j=0; j<kValTileN; ++j)
2470     //                 HMMA16816(RC[i][j][0], RC[i][j][1], ...);
2471     //
2472     // CUTLASS 推导链 (mma_atom.hpp) :
2473     //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMNK
2474     //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2475     //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2476     //     做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2477     //     K 被分为 BK/kMmaPK 个 perm-K slice
2478     //     随后按 AtomShape_MNK<2> (MMA_K_atom) 做 zipped_divide
2479     //     每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2480     //   → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2481     // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1) , BK=32 → num_k_steps=2
2482     int num_k_steps = size<2>(tCrA);
2483
2484     #pragma unroll
2485     for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2486         int k_step_next = (k_step + 1) % num_k_steps;
2487
2488         if (k_step == num_k_steps - 1) {
2489             // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2490             cp_async_wait<kStage - 2>();
2491             __syncthreads();
2492             ismem_read = (ismem_read + 1) % kStage;
2493         }
2494
2495         // S→R: 加载下一组 A/B fragment 到寄存器。
2496         //
2497         // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2498         //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2499         // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**:
2500         //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2501         //   让 S→R 延迟与 MMA 计算重叠。
2502         // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2503         cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2504                   tCrA_view(_, _, k_step_next));
2505         cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2506                   tCrB_view(_, _, k_step_next));
2507
2508         if (k_step == 0) {
2509             // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2510             if (itile_to_read < num_k_tiles) {
2511                 cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2512                           tAsA_copy(_, _, _, ismem_write));
2513                 cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2514                           tBsB_copy(_, _, _, ismem_write));
2515                 ++itile_to_read;
2516                 ismem_write = (ismem_write + 1) % kStage;
2517             }
2518             cp_async_fence();
2519         }

```

```

2520 // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2521 // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2522 cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2523 }
2524 }
2525 }
2526
2527 // Epilogue: D 寄存器 → shared memory → global memory。
2528 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2529 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2530 // sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2531 // 只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2532 // 解释这同一段地址。当前 wrapper 的固定配置下:
2533 // 一个 A stage: 128 x 32 = 4096 个 half;
2534 // C scratchpad: 32 x 32 x 4 = 4096 个 half。
2535 // 两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2536 // 是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2537 // C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2538 // 检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2539 //
2540 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2541 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2542 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2543 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2544
2545 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2546 // R2SCopyAtomC = CopyAtom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2547 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2548 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2549 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2550 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2551 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2552 //
2553 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2554 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2555 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2556 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2557 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2558 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2559 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2560 // 角色, 而不是另一次 memory transfer。
2561 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2562 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2563 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2564 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2565 // MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2566 // → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2567 // → make_tiled_copy_C 的 tiler = (32, 32)
2568 // → SmemLayoutC 的逻辑 MxN = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2569 // → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2570 // → 结果 shape 的 M/N remainder = _1, _1
2571 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2572 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2573 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2574 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2575
2576 // S2G TiledCopy: shared memory → global memory (128-bit store)
2577 // S2GCopyAtomC(CopyAtom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2578 S2GCopyC s2g_tiled_copy_c;
2579 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2580 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2581 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2582 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,

```

```

2583 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2584 // 的逻辑 mode，不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2585 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2586 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2587
2588 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2589 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2590 //   -> Tensor(CPY, (CPY_M, CPY_N))
2591 // 它只重组 Tensor 的 layout，不分配内存、不移动 data()，也不改变元素访问的物理地址。
2592 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系，只是现在可以用一个坐标
2593 // i+j 访问这个组合 mode；因此这里的 group_modes 更准确地说是“合并 mode”，
2594 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2595 // 从 type/shape 结构看，结果确实是 Tensor(CPY, (CPY_M, CPY_N))；但 grouped
2596 // mode 的 cardinality 是两个子 mode 的乘积，因此 size<1>(…) 可作为一个标量
2597 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2598 // 的线性 logical coordinate，不是在 C++ 中对二维 tuple 做加法，更不是 raw
2599 // pointer offset；最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2600 //
2601 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2602 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment，原始形状为 (CPY, CPY_M, CPY_N)；
2603 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐；
2604 //   - 两者使用相同的 [1,3) 合并后，源/目的 Tensor 仍保持相同的坐标空间，
2605 //     cute::copy 可按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2606 //
2607 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode，因为它们还需要显式保留
2608 // shared-memory C scratchpad 的 pipeline 维度；step=size<3>(tCsC_r2s) 表示
2609 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2610 // 按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 sC(_,0,0,j)，
2611 // 再从同一个 slot 写回 global memory。
2612 // 当前 kSmemLayoutCBatch=4，因此 step=4。代码未对 i+j 做尾部 predication，
2613 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应将
2614 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2615 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2616 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2617
2618 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数（由 kSmemLayoutCBatch=4 指定）
2619 // 双层循环的语义（注意 step 与 CPY_N 无关）：
2620 //   group_modes<1,3> 之后，tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2621 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N，即该线程需处理的 C fragment 总数。
2622 //   外循环以 step=4 为步长，内循环 j=0..3 每次处理 1 个 fragment，将其写入
2623 //   第 j 号 C scratchpad pipe slot，再读出写回 global。
2624 //
2625 // 这里 step=4 是 scratchpad pipeline 深度，不是 CPY_N。CPY_N 的值取决于
2626 // TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上，
2627 // 通常 ≠ 4。循环之所以成立，是因为 grouped mode 用线性坐标 i+j 遍历整个
2628 // CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向，也不要求 CPY_N == step。
2629 // 唯一前提是 CPY_M * CPY_N 能被 step 整除（当前静态 layout 编译期保证）。
2630 //
2631 // 第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2632 // 各自推导；因为二者都从同一个 sC (SmemLayoutC) 出发，pipe mode 保持一致，
2633 // 写入和读取命中同一段 shared memory。
2634 #pragma unroll
2635 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2636     // 每轮处理 step 个 fragment，内层 j 选 pipe slot。
2637     #pragma unroll
2638     for (int j = 0; j < step; ++j) {
2639         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2640         // 的 register Tensor；普通 cute::copy 将 accumulator fragment materialize
2641         // 成输出元素类型 T 的临时 payload。
2642         //
2643         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2644         //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2645         // 因此当 accumulator 与 T 类型不同时，它自动完成 dtype 转换；当二者相同

```

```

2646 // (如本 kernel 的 f16 accumulator + T=half) , cast 退化为 no-op.
2647 //
2648 // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2649 // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2650 // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2651 // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2652 // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2653 // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2654 // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2655 // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2656 //
2657 // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2658 // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2659 // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2660 // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2661 // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2662 // 保持同源实现的通用 staging 写法。
2663 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2664 cute::copy(tCrC_r2sx(_, i + j), t);
2665 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2666 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2667 //
2668 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2669 //   r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>,T>
2670 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2671 //   → src(t) 与 dst(tCsC_r2s(_,0,0,j)) 都是 rank-1 → 直接 copy_atom.call
2672 //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count) ,
2673 //   走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2674 //   → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2675 //
2676 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2677 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2678 //   - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2679 //   → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2680 //   - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2681 //   → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2682 //   计算了当前线程每个 val 在 smem 中的物理 offset
2683 //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2684 //   因此逐元素 copy 天然把正确的数据写到了正确的地址。
2685 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2686 }
2687 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2688 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2689 __syncthreads();
2690
2691 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2692 #pragma unroll
2693 for (int j = 0; j < step; ++j) {
2694   // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2695   // (32-bit) 不同, 它一次搬运 128-bit (8 个 half) , 映射为一条 wide store
2696   // (如 st.global.v4.f32 或 st.global.b128) 。
2697   //
2698   // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2699   //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>,T>
2700   //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2701   //   → src=tCsC_s2g(_,0,0,j) 与 dst=tCgC_s2gx(_,i+j) 都是 rank-1
2702   //   → 直接 copy_atom.call(src, dst)
2703   //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2704   //   → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2705   //   → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2706   //
2707   // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2708   //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S

```

```

2709 // 用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2710 // 在 sC (shared memory) 中的物理 offset。
2711 // S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2712 // = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2713 // 因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2714 // M/N remainder。
2715 // - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2716 // → slice. gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2717 // 因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2718 // repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2719 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2720 // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2721 cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2722 }
2723 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2724 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2725 __syncthreads();
2726 }
2727
2728 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2729 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2730 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2731 // CuTe 版没有把这些 lane 映射写在 kernel 源码中: retile_S 表达 accumulator
2732 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2733 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2734 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2735 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2736 }
2737
2738 // =====
2739 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2740 // =====
2741 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2742 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2743 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2744 //
2745 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2746 //
2747 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2748 // 答: 手写需要:
2749 // 1) 手动计算每线程的 smem 地址偏移
2750 // 2) 手动计算 ldmatrix 的 lane 映射
2751 // 3) 手动插入 swizzle XOR 到每个地址计算点
2752 // 4) 手动做 epilogue 的 warp shuffle 编排
2753 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2754 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2755 // =====
2756 template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
2757 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2758     using namespace cute;
2759
2760     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2761     //
2762     // kernel 用法对照:
2763     // BM=128: local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2764     //          local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2765     // BN=256: local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2766     //          local_tile(D, ...) → gD 的列方向
2767     // BK=32: num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2768     // kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2769     // kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2770     auto BM = Int<128>{};
2771     auto BN = Int<256>{};

```

```

2772 auto BK = Int<32>{};
2773 auto KStage = Int<Stages>{};
2774 auto kSmemLayoutCBatch = Int<4>{};
2775
2776 // — SmemLayoutA / SmemLayoutB —
2777 // kernel 用法:
2778 // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2779 // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2780 // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2781 //
2782 // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2783 //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2784 //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2785 //   - B=3: 共有 2^3=8 行。
2786 // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2787 // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2788 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2789 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2790 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2791 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2792 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2793 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2794 // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2795 using SmemLayoutAtom = decltype(composition(
2796     Swizzle<3, 3, 3>{},
2797     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2798         make_stride(Int<BK>{}, Int<1>{}))));
2799 using SmemLayoutA = decltype(tile_to_shape(
2800     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2801 using SmemLayoutB = decltype(tile_to_shape(
2802     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2803
2804 // — MMA (TiledMMA) —
2805 // kernel 用法:
2806 // TiledMMA tiled_mma;
2807 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2808 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2809 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2810 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2811 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2812 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2813 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2814 //
2815 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2816 //   → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2817 //   → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2818 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2819 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2820 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2821 using mma_traits = MMA_Traits<mma_op>;
2822 using mma_atom = MMA_Atom<mma_traits>;
2823 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2824
2825 static constexpr int kMmaEURepeatM = 2;
2826 static constexpr int kMmaEURepeatN = 2;
2827 static constexpr int kMmaEURepeatK = 1;
2828 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2829 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2830 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2831 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2832
2833 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2834     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));

```

```

2835 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2836 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2837
2838 // — G2SCopyA / G2SCopyB —
2839 // kernel 用法:
2840 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2841 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2842 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2843 // G→S: 128-bit cp.async。ThrLayout{32,4} × ValLayout{1,8}:
2844 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2845 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2846 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2847 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2848 using G2SCopyA = decltype(make_tiled_copy(
2849     g2s_copy_atom{},
2850     make_layout(make_shape(Int<32>{}, Int<4>{}),
2851         make_stride(Int<4>{}, Int<1>{}))),
2852     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2853 using G2SCopyB = G2SCopyA;
2854
2855 // — S2RCopyAtomA / S2RCopyAtomB —
2856 // kernel 用法:
2857 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2858 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2859 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2860 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2861 // S→R: ldmatrix (SM75_U32x4_LDSTM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2862 // 与 MMA fragment 对齐的 shared-register 映射, 无需手动指定 ThrLayout/ValLayout。
2863 using s2r_copy_op = SM75_U32x4_LDSTM_N;
2864 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2865 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2866 using S2RCopyAtomA = s2r_copy_atom;
2867 using S2RCopyAtomB = s2r_copy_atom;
2868
2869 // — SmemLayoutC —
2870 // kernel 用法:
2871 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2872 // 复用当前 A stage 的空间作为 C scratchpad
2873 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2874 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2875 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2876 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等于主循环
2877 // 的 kStage K-tile pipeline。
2878 using SmemLayoutAtomC = decltype(composition(
2879     Swizzle<3, 3, 3>{},
2880     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2881         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2882 using SmemLayoutC = decltype(tile_to_shape(
2883     SmemLayoutAtomC{},
2884     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2885
2886 // — R2SCopyAtomC —
2887 // kernel 用法:
2888 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2889 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2890 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2891 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2892 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2893 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2894
2895 // — S2GCopyC —
2896 // kernel 用法:
2897 // S2GCopyC s2g_tiled_copy_c;

```

```

2898 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2899 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared-global wide store.
2900 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2901 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2902 using S2GCopyAtomC = CopyAtom<UniversalCopy<cute::uint128_t>, T>;
2903 using S2GCopyC = decltype(make_tiled_copy(
2904     S2GCopyAtomC{},
2905     make_layout(make_shape(Int<32>{}, Int<4>{}),
2906         make_stride(Int<4>{}, Int<1>{})),
2907     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2908
2909 // — Grid/Block 配置 —
2910 // kernel 用法:
2911 // int idx = threadIdx.x;          // 0..127
2912 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2913 // int iy = blockIdx.y;           // M 方向 CTA 坐标
2914 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2915 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2916 // M % BM == 0、N % BN == 0、K % BK == 0。
2917 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2918 constexpr int kSwizzleStride = 2048;
2919 int BX = (N + BN - 1) / BN;
2920 int BY = (M + BM - 1) / BM;
2921 int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2922 BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2923 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2924 dim3 grid(BX, BY, BZ);
2925
2926 // — Dynamic Shared Memory 大小 —
2927 // kernel 用法:
2928 // extern __shared__ T shm_data[];
2929 // T *Ashm = shm_data;
2930 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2931 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2932 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2933 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2934 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2935 // 能放进一个 A stage (128*32 = 4096)。
2936 static constexpr int shm_size_AB =
2937     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2938 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2939 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2940     "C shared memory must fit within one A pipe");
2941 static constexpr int kShmSize =
2942     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2943
2944 cudaFuncSetAttribute(
2945     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2946         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2947         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2948         S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
2949     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2950
2951 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2952     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2953     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2954     BlockSwizzle>
2955     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2956 }
2957
2958 #endif /* NOTES_V2_ENABLE_CUTE */
2959
2960 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)

```

```

2961 // TMA/mbarrier helpers.
2962 //
2963 // Two implementations gated by NOTES_V2_FORCE_INLINE_ASYNC_PROXY:
2964 // - Defined (default): raw `asm volatile` PTX. ptxas keeps setmaxnreg
2965 //   because raw asm carries no source annotation that ptxas would treat as
2966 //   an "extern call" boundary (warning C7506 otherwise drops the hint).
2967 // - Undef'd: cuda::ptx:: / cuda::device:: C++ wrappers. Cleaner API but
2968 //   ptxas drops setmaxnreg with C7506 even though the wrappers inline to
2969 //   identical PTX; useful for comparing behavior / debugging.
2970 //
2971 // cuda::barrier (init/arrive/wait) is shared by both paths: it inlines to raw
2972 // mbarrier PTX and does not trigger C7506, so it needs no gating.
2973 //
2974 // Mirrors the raw-PTX style of tmp/LeetGPU/CUDA/22_GEMM/sm90_wgmma_tma_ws_pingpong.cu.
2975 #if defined(NOTES_V2_FORCE_INLINE_ASYNC_PROXY)
2976
2977 static __device__ __forceinline__ uint32_t
2978 cast_smem_ptr_to_uint(void const *ptr) {
2979     return static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2980 }
2981
2982 static __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
2983     asm volatile("fence.proxy.async.shared::cta;\n" ::: "memory");
2984 }
2985
2986 // cp.async.bulk.tensor.2d: 2D TMA copy from global to shared::cluster.
2987 // [dst] = shared memory destination (smem addr, 32-bit)
2988 // [desc, {c0, c1}] = CUTensorMap + (minor_coord, major_coord) coords
2989 // [mbar] = mbarrier in shared memory (smem addr, 32-bit)
2990 // mbarrier::complete_tx::bytes: barrier flips phase when TMA bytes land.
2991 // L2::cache_hint omitted (EVICT_NORMAL); matches the cuda::ptx default.
2992 static __device__ __forceinline__ void tma_load_2d(
2993     void *dst, const CUTensorMap *tensor_map, int minor_coord, int major_coord,
2994     cuda::barrier<cuda::thread_scope_block> &barrier) {
2995     uint64_t gmem_int_desc = reinterpret_cast<uint64_t>(tensor_map);
2996     uint32_t smem_int_ptr = cast_smem_ptr_to_uint(dst);
2997     uint32_t smem_int_mbar =
2998         cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
2999     asm volatile(
3000         "cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes"
3001         " [%0], [%1, {%3, %4}], [%2];"
3002         :
3003         : "r"(smem_int_ptr), "l"(gmem_int_desc), "r"(smem_int_mbar),
3004           "r"(minor_coord), "r"(major_coord)
3005         : "memory");
3006 }
3007
3008 // mbarrier.arrive.expect_tx: register one arrival on the mbarrier and declare
3009 // that `bytes` of async copy traffic is expected before the phase flips.
3010 // Equivalent to cuda::ptx::mbarrier_arrive_expect_tx(release, cta, shared).
3011 static __device__ __forceinline__ void tma_arrive_expect_tx(
3012     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
3013     uint32_t smem_int_mbar =
3014         cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
3015     asm volatile(
3016         "mbarrier.arrive.expect_tx.shared::cta.b64 __, [%0], %1;\n"
3017         :
3018         : "r"(smem_int_mbar), "r"(bytes)
3019         : "memory");
3020 }
3021
3022 #else // !NOTES_V2_FORCE_INLINE_ASYNC_PROXY: use cuda::ptx / cuda::device wrappers
3023

```

```

3024 static __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
3025     #if CUDART_VERSION >= 13020
3026         cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
3027     #else
3028         cuda::device::experimental::fence_proxy_async_shared_cta();
3029     #endif
3030 }
3031
3032 static __device__ __forceinline__ void tma_load_2d(
3033     void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
3034     cuda::barrier<cuda::thread_scope_block> &barrier) {
3035     #if CUDART_VERSION >= 13020
3036         const int32_t coords[]{minor_coord, major_coord};
3037         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
3038         cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
3039                                         cuda::ptx::space_global, dst, tensor_map,
3040                                         coords, barrier_handle);
3041     #else
3042         cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3043             dst, tensor_map, minor_coord, major_coord, barrier);
3044     #endif
3045 }
3046
3047 static __device__ __forceinline__ void tma_arrive_expect_tx(
3048     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
3049     #if CUDART_VERSION >= 13020
3050         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
3051         [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
3052             cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
3053             barrier_handle, bytes);
3054     #else
3055         [[maybe_unused]] auto token =
3056             cuda::device::barrier_arrive_tx(barrier, 1, bytes);
3057     #endif
3058 }
3059
3060 #endif // NOTES_V2_FORCE_INLINE_ASYNC_PROXY
3061
3062 // Warpgroup-level register rebalancing for warp specialization.
3063 // setmaxnreg.dec releases registers (producer TMA path needs few),
3064 // setmaxnreg.inc requests registers (consumer MMA path needs many).
3065 // Both require all warps in a warpgroup to execute the same instruction,
3066 // and require __launch_bounds__(N, 1) so the compiler permits up to 256
3067 // regs/warp (otherwise the hint may have no effect). Supported on sm_90a/sm_120a.
3068 // __forceinline__ is mandatory: without it ptxas drops setmaxnreg with
3069 // warning C7506 "ignored to maintain compatibility into 'extern' call",
3070 // because a non-inlined call boundary forces a fixed register convention.
3071 //
3072 // On sm_120a (Blackwell, CUDA 13.2) ptxas drops setmaxnreg with C7506 even
3073 // when the PTX is fully inlined (no call.uni), because ptxas treats
3074 // cp.async.bulk.tensor (TMA) usage as an implicit extern-call boundary.
3075 // sm_90a (Hopper) is unaffected. Gate the *call sites* with
3076 // NOTES_V2_ENABLE_SETMAXNREGS so sm_120a builds stay warning-free and avoid
3077 // the register-allocation side effects of __launch_bounds__(N,1) until ptxas
3078 // is fixed. The function templates themselves are always defined.
3079 #if defined(NOTES_V2_ENABLE_SETMAXNREGS)
3080     #define NOTES_V2_REG_DEALLOC(N) warpgroup_reg_dealloc<N>()
3081     #define NOTES_V2_REG_ALLOC(N) warpgroup_reg_alloc<N>()
3082 #else
3083     #define NOTES_V2_REG_DEALLOC(N) ((void)0)
3084     #define NOTES_V2_REG_ALLOC(N) ((void)0)
3085 #endif
3086

```

```

3087 template <uint32_t kNumRegs>
3088 __device__ __forceinline__ void warpgroup_reg_dealloc()
3089 {
3090     asm volatile("setmaxnreg.dec.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
3091 }
3092
3093 template <uint32_t kNumRegs>
3094 __device__ __forceinline__ void warpgroup_reg_alloc()
3095 {
3096     asm volatile("setmaxnreg.inc.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
3097 }
3098 #endif
3099
3100 #if defined(NOTES_V2_ENABLE_WGMMA)
3101 // =====
3102 // Phase 7d: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
3103 // =====
3104 // 面试要点 (WGMMA vs MMA 对比) :
3105 //   - MMA: warp 级 (32 threads) , 同步执行
3106 //   - WGMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
3107 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
3108 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
3109 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMA, 通过 barrier 同步
3110 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
3111
3112 // ---- WGMMA 辅助函数 ----
3113 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
3114 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
3115 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
3116 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。
3117 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
3118 //     都是 "wait until only N or fewer groups are pending"。
3119 #define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
3120 #define WGMMA_COMMIT_GROUP() \
3121     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
3122 #define WGMMA_WAIT_GROUP(n) \
3123     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
3124
3125 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
3126 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
3127 //   encoded = (x & 0x3FFFF) >> 4
3128 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
3129 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4) , 低 4 bit 恒为 0,
3130 // 可省略以扩大可表示范围。
3131 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
3132 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
3133
3134 // make_smem_desc: 构造 WGMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
3135 // 硬件 WGMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
3136 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
3137 //
3138 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
3139 // -----
3140 // | 位域          | 范围      | 大小 | 含义
3141 // |-----|-----|-----|-----
3142 // | start_address  | [0, 14)   | 14   | smem 基址编码
3143 // | (unused)       | [14, 16)  | 2    | -
3144 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
3145 // | (unused)       | [30, 32)  | 2    | -
3146 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
3147 // | (unused)       | [46, 49)  | 3    | -
3148 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
3149 // | (unused)       | [52, 62) | 10   | -

```

```

3150 // | layout_type          | [62, 64) | 2 | swizzle 模式
3151 // -----
3152 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
3153 //
3154 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
3155 // - 128B swizzle atom 在 128-bit 单位下为 8×8
3156 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
3157 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
3158 // - 这是 stride_byte_offset=1024 的来源
3159 //
3160 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
3161 //   此处填 16 仅为占位, 编码后为 1。
3162 //
3163 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
3164 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
3165 //
3166 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
3167 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WmmaSMem 注释。
3168 device inline uint64_t make_smem_desc(half *ptr) {
3169 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
3170 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
3171 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
3172
3173 // 从全零开始: 所有位域初始化为 0。
3174 uint64_t desc = 0x0000000000000000;
3175
3176 // — bits [0, 14): start_address —
3177 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
3178 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
3179 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
3180 desc |= SMEM_DESC_ENCODE(addr);
3181
3182 // — bits [16, 30): leading_byte_offset —
3183 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
3184 // << 16 将编码值放到 bits [16, 30)。
3185 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
3186 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
3187 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
3188 //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
3189 //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
3190 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
3191
3192 // — bits [32, 46): stride_byte_offset —
3193 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
3194 // << 32 将编码值放到 bits [32, 46)。
3195 // K-Major 下定义为“从首 8 行到次 8 行的字节偏移” (PTX ISA §9.7.15.5.1.2.1.2) 。
3196 // 1024 的来源 (K-Major + 128B swizzle + half) :
3197 //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
3198 //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
3199 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
3200 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
3201
3202 // — bits [62, 64): layout_type (swizzle mode) —
3203 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
3204 // 1 = 128B swizzle mode。
3205 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
3206 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
3207 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
3208 desc |= 1llu << 62;
3209
3210 return desc;
3211 }
3212

```

```

3213 // ---- WGMMA PTX 指令宏 ----
3214 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
3215 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
3216 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
3217 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
3218 // 每线程 64 half = 32 uint32。
3219 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
3220 // Scaled: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
3221 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
3222 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
3223 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
3224 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
3225                                     TransB) \
3226 { \
3227     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
3228     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
3229     asm volatile( \
3230         "{\n" \
3231         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
3232         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
3233         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
3234         " %16, %17, %18, %19, %20, %21, %22, %23, " \
3235         " %24, %25, %26, %27, %28, %29, %30, %31}, " \
3236         " %32, " \
3237         " %33, " \
3238         " %34, %35, %36, %37, %38;\n" \
3239         "}\n" \
3240         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
3241         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
3242         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
3243         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
3244         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
3245         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
3246         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
3247         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
3248         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
3249         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
3250         "n"(int32_t(TransB))); \
3251 }
3252
3253 // ---- TMA Shared Memory Layout ----
3254 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
3255 //
3256 // 每个 stage 包含两块 smem:
3257 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
3258 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
3259 //
3260 // ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
3261 //
3262 //   上层数据: 源矩阵 BT [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
3263 //   物理存储: TMA 将 BT 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
3264 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
3265 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
3266 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
3267 //   逻辑视图: WGMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
3268 //               通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
3269 //               实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
3270 //               swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
3271 //
3272 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
3273 //       128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
3274 //       stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
3275 //                               = 8 rows × (BK=64) halves × 2 bytes = 1024。

```

```

3276 // 这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
3277 //
3278 // 与 MMA(TN) 的对比:
3279 // MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
3280 // WGMMA: smem 物理存 [BN, BK] (TMA 直写), WGMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
3281 // 简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
3282 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
3283     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
3284     // B tile 标记:
3285     // - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
3286     // - 逻辑视图: WGMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
3287     // 以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
3288     // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
3289     // 但写 BN×BK 更能体现物理存储形态
3290     alignas(128) half B[BN * BK * QSIZE];
3291 };
3292
3293 // ---- WGMMA Kernel: Warp Specialization + TMA ----
3294 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
3295 //
3296 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
3297 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMA 支持将工作拆分为两个
3298 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
3299 //
3300 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
3301 // 将 A/B tile 从 HBM 搬到 shared memory。
3302 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
3303 //
3304 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
3305 // - full[stage]: Producer 发信号表示 stage stage 的数据已就绪, 可被使用
3306 // - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
3307 //
3308 // 本节重点理解:
3309 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
3310 // 即可搬运整个 2D tile, 无需所有线程参与。
3311 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
3312 // 3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
3313 // Producer 可以搬运 stage (stage+1), 隐藏 HBM→SMEM 延迟。
3314 //
3315 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
3316 // WGMMA Atom: m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
3317 // K Tile: BK=64, 每个 K tile 包含 BK/kWgmmaK=4 个 WGMMA atom
3318 // M Tile: BM=128, 每个 M tile 包含 BM/kWgmmaM=2 个 WGMMA atom
3319 // N Tile: BN=128, 单个 kWgmmaN=128 即可覆盖, 无需在 N 方向分块
3320 // Block Tile: C[128,128] = A[128,64] × B[64,128]
3321 // Threads: 256 = 2 warpgroups × 128 threads/warpgroup
3322 // Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
3323 // Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMA 和写回)
3324 //
3325 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
3326 // 的延迟被计算完全掩盖。
3327 //
3328 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
3329 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
3330 // Block: (256, 1, 1), 2 warpgroups
3331 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
3332 template <const int kWgmmaM = 64, // WGMMA atom M dim (m64n128k16)
3333         const int kWgmmaN = 128, // WGMMA atom N dim
3334         const int kWgmmaK = 16, // WGMMA atom K dim
3335         const int BM = 128, // block tile M, 2 × kWgmmaM
3336         const int BN = 128, // block tile N, 1 × kWgmmaN
3337         const int BK = 64, // block tile K, 4 × kWgmmaK
3338         const int kNumThreads = 256, // 2 warpgroups × 128 threads

```

```

3339         const int kStages = 3,           // TMA pipeline depth (full/empty barriers)
3340         const int kBlockSwizzle = 0>     // 1 enables 3D grid swizzle for L2 locality
3341 __global__ void __launch_bounds__(kNumThreads, 1)
3342     hgemm_wgmma_stages_tn(
3343         int M, int N, int K, half *C,
3344         const CUtensorMap *__restrict__ tensorMapA,
3345         const CUtensorMap *__restrict__ tensorMapB) {
3346     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3347     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3348     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3349     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3350     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3351     // 不展开宿主侧 create_tensor_map 细节。
3352
3353     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3354     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3355     const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3356     const int by = blockIdx.y;
3357     constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
3358     // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3359     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
3360     constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
3361     // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
3362     // 当只有一个 consumer 时, 它负责全部 BM=128 行
3363     constexpr int kWarpgroupM = BM / kNumConsumers; // 128
3364
3365     // 边界检查: 确保当前 block 不超出 M/N 范围
3366     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3367         return;
3368
3369     // ---- Shared Memory 分配 ----
3370     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3371     // TMA + WGMMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
3372     // make_smem_desc() 构造 WGMMMA descriptor 时, start_address 字段
3373     // 完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
3374     // 硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
3375     // descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
3376     // 1024B 边界, WGMMMA 硬件也能正确读取 TMA 写入的数据。
3377     // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle<64>()
3378     // , 该函数无法感知 smem 绝对地址, 硬编码了 phase=0 的假设, 因此必须
3379     // __align__(1024) 来保证 phase 确实为 0。从健壮性角度看本 kernel 也
3380     // 应该用 __align__(1024); 这里使用 128 也可以正常运行。
3381     extern __shared__ __align__(1024) uint8_t smem_tma_wgmma_ws[];
3382     WgmmaSMem<BM, BN, BK, kStages> &s =
3383         *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages>*>(smem_tma_wgmma_ws);
3384     half *s_a = s.A;
3385     half *s_b = s.B;
3386
3387     // ---- cuda::barrier 初始化 ----
3388     //
3389     // ★ Barrier 机制速查 (arrive / wait 核心语义):
3390     //
3391     // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3392     //
3393     // init(bar, N): 设置 arrive_count = N。
3394     // 含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3395     //
3396     // arrive(): 线程声明"我已到达此 barrier 点"。
3397     // - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3398     // - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3399     //
3400     // wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3401     // - 即: 等待 phase 翻转。

```

```

3402 // - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3403 // (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3404 //
3405 // 典型用法: b.wait(b.arrive())
3406 // - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3407 // - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3408 // - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3409 // - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3410 //
3411 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129):
3412 //
3413 // Producer (1 thread) Consumer (128 threads)
3414 // -----
3415 // C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3416 // ┌ for each k_tile: ─┐ ┌ for each k_tile: ─┐
3417 // | P1: arrive+wait(empty[q]) | C1: arrive+wait(full[q])
3418 // |   ↓ 等 stage q 被消费完 |   ↓ 等 TMA 数据就绪
3419 // | P2: TMA(A[q]) + TMA(B[q]) | C2: WGMMMA 计算
3420 // |   ↓ 异步拷贝 | C3: wait WGMMMA 完成
3421 // | P3: arrive_tx(full[q]) | C4: arrive(empty[q])
3422 // |   ↑ 通知: stage q 数据就绪 |   ↑ 通知: stage q 已消费完
3423 // └──────────────────┘ └──────────────────┘
3424 //
3425 // 关键不变式 (invariant):
3426 // - Producer 不会覆盖 Consumer 正在读的 stage
3427 // - Consumer 不会读 Producer 还没写完的 stage
3428 // - 通过 full/empty 两个 barrier 的 phase 交替来保证
3429 //
3430 // 每个 stage 有两个 barrier:
3431 // full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3432 // empty[stage]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3433 //
3434 // arrive_count = 128 (consumer) + 1 (producer) = 129:
3435 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3436 #pragma nv_diag_suppress static_var_with_dynamic_init
3437 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3438 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3439 #pragma nv_diag_default static_var_with_dynamic_init
3440
3441 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3442 const int NUM_K_TILES = div_ceil(K, BK);
3443 const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
3444 const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup
3445
3446 // 初始化 barriers (仅 thread 0 执行)
3447 if (threadIdx.x == 0) {
3448     for (int i = 0; i < kStages; ++i) {
3449         // arrive_count = 129: 128 consumer + 1 producer
3450         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3451         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3452         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3453         init(&full[i], kConsumerThreads + 1); // 128 consumer + 1 producer
3454         init(&empty[i], kConsumerThreads + 1); // same
3455     }
3456     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3457     // 对 async proxy (TMA/WGMMMA) 可见。
3458     tma_fence_proxy_async_shared_cta();
3459 }
3460 __syncthreads();
3461
3462 // =====
3463 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3464 // =====

```

```

3465 //
3466 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
3467 //   - WG0 (threadIdx.x 0~127)： 全部走 if 分支，做 Producer。
3468 //   - WG1 (threadIdx.x 128~255)：全部走 else 分支，做 Consumer。
3469 //
3470 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
3471 //   Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 } ← 自己的循环
3472 //   Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
3473 //
3474 // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
3475 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
3476 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
3477 //
3478 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
3479 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3480 // =====
3481 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3482 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
3483 // 只有 wg_tid==0 执行实际拷贝提交，其余 127 个线程空闲。
3484 // =====
3485 if (wg_idx == 0) {
3486     NOTES_V2_REG_DEALLOC(40);
3487     if (wg_tid == 0) {
3488         // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3489         int stage = 0;
3490         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3491             if (stage == kStages)
3492                 stage = 0;
3493
3494             // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)
3495             //
3496             // 模式 empty[stage].wait(empty[stage].arrive()):
3497             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
3498             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3499             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
3500             //     → phase 立即翻转。
3501             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3502             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
3503             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3504             //
3505             // 语义：Producer 说"我准备好写 stage stage 了，Consumer 用完没有？"
3506             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3507             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3508             empty[stage].wait(empty[stage].arrive());
3509
3510             // Step P2: 提交 TMA 2D 拷贝指令
3511             // cp_async_bulk_tensor_2d_global_to_shared:
3512             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3513             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3514             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3515             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3516             //
3517             // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
3518             // 无需线程逐元素搬运，零寄存器开销。
3519             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3520             tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
3521                 full[stage]);
3522
3523             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3524             tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
3525                 full[stage]);
3526
3527             // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪

```

```

3528 //
3529 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3530 // - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3531 // - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3532 // - Phase 翻转条件:
3533 // (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3534 // (b) 所有声明的 async 字节已写入 smem
3535 // 两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3536 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3537 tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
3538 }
3539 }
3540 }
3541 // =====
3542 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3543 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3544 // 所有 128 个线程 (4 warps) 全部参与。
3545 // =====
3546 else {
3547 // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3548 //
3549 // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3550 // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[stage].wait()
3551 // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3552 //
3553 // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3554 // Producer 后续的 empty[stage].arrive() 作为第 129 次, 触发 phase 翻转。
3555 NOTES_V2_REG_ALLOC(232);
3556
3557 for (int i = 0; i < kStages; ++i) {
3558 [[maybe_unused]] auto token = empty[i].arrive();
3559 }
3560
3561 // 累加器寄存器声明
3562 // d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4]:
3563 // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3564 // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3565 // - d[*][g][*]: N 方向第 g 组 16 列
3566 // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3567 // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3568 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3569 uint32_t d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4] = {};
3570
3571 int stage = 0;
3572 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3573 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3574 if (stage == kStages)
3575 stage = 0;
3576
3577 // Step C1: 等待 TMA 数据就绪 (full 信号)
3578 //
3579 // 模式 full[stage].wait(full[stage].arrive()):
3580 // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3581 // - 当前 phase 累积: 128/129, phase 尚未翻转。
3582 // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3583 // 贡献第 129 次到达 + TMA 字节全部写完 -> phase 翻转 -> wait 返回。
3584 //
3585 // 语义: Consumer 说"我准备好读 stage stage 了, 数据到了没有?"
3586 full[stage].wait(full[stage].arrive());
3587
3588 // Step C2: 发射 WGMMMA 指令序列
3589 //
3590 // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。

```

```

3591 // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3592 //
3593 // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3594 //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3595 //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3596 //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3597 //
3598 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3599 //   D[64×128] += A[64×16] × B[16×128]
3600 //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3601 //   ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都使用 ScaleD=1。
3602 WGMMMA_FENCE();
3603
3604 // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3605 // 每个 atom 处理 64 行 M 方向数据。
3606 #pragma unroll
3607 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3608     // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3609     // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3610     half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;
3611
3612     // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3613     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3614 #pragma unroll
3615     for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
3616         // 第 k_step 次 WGMMMA:
3617         //   A: wgmma_sA + k_step * kWgmmaK (A 的 K 维起始位置)
3618         //   B: s_b + stage * BK * BN + k_step * kWgmmaK (B 的 K 维起始位置)
3619         //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3620         //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3621         //   ScaleD=1: 累加 (K 维迭代需要累积结果)
3622         WGMMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
3623                                     s_b + stage * BK * BN + k_step * kWgmmaK,
3624                                     1, // ScaleD=1: accumulate (不清零)
3625                                     1, 1, 0, 0);
3626     }
3627 }
3628
3629 // Step C3: 提交并等待 WGMMMA 完成
3630 //
3631 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3632 //   将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3633 //   类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3634 //
3635 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3636 //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3637 //   ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3638 //   两者都是 "wait until only N or fewer of the most recent groups are pending"。
3639 //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3640 //   必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3641 WGMMMA_COMMIT_GROUP();
3642 WGMMMA_WAIT_GROUP(0);
3643
3644 // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
3645 //
3646 // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
3647 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3648 //
3649 // 语义: Consumer 说 "stage stage 我已经读完了, 你可以放心覆盖了。"
3650 [[maybe_unused]] auto token = empty[stage].arrive();
3651 }
3652
3653 // =====

```

```

3654 // Epilogue: 将寄存器中的累加结果写回 global memory C
3655 // =====
3656 //
3657 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3658 //
3659 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3660 // 这些 half 分布在 d[2][8][4] 数组中:
3661 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3662 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3663 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3664 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3665 //
3666 // 其中:
3667 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3668 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3669 //
3670 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3671 // +-----+-----+
3672 // | reg[0] | reg[2] | <- rows [row, row+8)
3673 // |(col,+2)|(col+8)|
3674 // +-----+-----+
3675 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3676 // |(col,+2)|(col+8)|
3677 // +-----+-----+
3678 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3679 //
3680 // 三维遍历:
3681 // m_it=0: rows 0~63, m_it=1: rows 64~127
3682 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3683 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3684 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3685
3686 const int lane = wg_tid % 32;
3687 const int warp = wg_tid / 32;
3688 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3689 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3690 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3691 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3692 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3693 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3694 const int row = warp * 16 + lane / 4;
3695 // block_C: 当前 C tile 在 global memory 中的起始地址
3696 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3697 half *block_C = C + by * BM * N + bx * BN;
3698
3699 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3700 // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
3701 #pragma unroll
3702 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3703     int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
3704     #pragma unroll
3705     for (int g = 0; g < kWgmmaN / 16; ++g) {
3706         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3707         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3708         // 左上象限: (row, col)
3709         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) = d[m][g][0];
3710         // 左下象限: (row+8, col)
3711         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
3712         // 右上象限: (row, col+8)
3713         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
3714         // 右下象限: (row+8, col+8)
3715         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
3716     }

```

```

3717     }
3718 }
3719 }
3720
3721 #endif /* NOTES_V2_ENABLE_WGMMA */
3722
3723 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3724 // SM120 不支持 WGMMA, 但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3725 // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
3726 template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
3727     static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
3728     half A[BM * BK * QSIZE];
3729     half B[BN * BK * QSIZE];
3730 };
3731
3732 // 消费者 MMA 层级映射参考 hgemm_mma_stages_tn_swizzle。与那个
3733 // 八 warp kernel 不同, 本 warp-specialized 消费者仅拥有四个 warp。
3734 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
3735           const int kMmaN = 8,           // MMA atom N dim
3736           const int kMmaK = 16,          // MMA atom K dim
3737           const int kMmaTileM = 2,       // consumer warps along M, warp tile M = 32
3738           const int kMmaTileN = 2,       // consumer warps along N, warp tile N = 16
3739           const int kValTileM = 4,        // value-repeat along M, BM = 16*2*4 = 128
3740           const int kValTileN = 8,        // value-repeat along N, BN = 8*2*8 = 128
3741           const int kValTileK = 4,        // MMA_K slices per BK tile, BK = 16*4 = 64
3742           const int kStages = 2,          // TMA full/empty pipeline depth
3743           const int kNumThreads = 256,    // 128 producer + 128 consumer threads
3744           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
3745 __global__ void __launch_bounds__(kNumThreads)
3746     hgemm_tma_mma_ws_tn(
3747         int M, int N, int K, half *C,
3748         const CUtensorMap *__restrict__ tensorMapA,
3749         const CUtensorMap *__restrict__ tensorMapB) {
3750
3751     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3752     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3753     constexpr int BK = kMmaK * kValTileK;
3754     static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3755     static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3756     static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3757     static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3758     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3759
3760     const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3761     const int by = blockIdx.y;
3762     constexpr int kConsumerThreads = kNumThreads / 2;
3763     static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3764         "Consumer threads must cover the MMA warp grid");
3765
3766     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3767         return;
3768
3769     // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐,
3770     // 以确保硬件 swizzle phase 从零开始。消费者 swizzle<64>()
3771     // 假设零 phase; 其他对齐 (如 128) 会导致 phase 偏移, 产生不匹配的
3772     // 物理地址, 破坏整个输出。
3773     //
3774     // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128), 而本 kernel
3775     // 必须 __align__(1024)? 核心区别在于消费者如何感知 swizzle phase:
3776     //
3777     // WGMMA 消费者: 通过 make_smem_desc() 构造 64-bit WGMMA descriptor,
3778     // 其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3779     // 1024B 边界的偏移量 ((addr >> 7) & 0x7)。硬件在读取 smem 时会

```

```

3780 // 用这个 base_offset 自动补偿 swizzle phase, 因此 WGMMMA 路径不
3781 // 需要 1024 字节对齐也能得到正确数据。
3782 //
3783 // 本 kernel 消费者: 使用 ldmatrix + 手写 swizzle<64>() 来计算
3784 // smem 物理地址。这个函数是纯软件公式, 没有任何 base_offset
3785 // 补偿机制。它假设 smem 基址恰好落在 1024B 边界上 (phase=0)。
3786 // 如果基址只满足 128B 对齐, TMA 硬件会以非零 phase 写入数据,
3787 // 而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3788 //
3789 // 简言之: WGMMMA 用硬件 descriptor base_offset 自动解决 phase 偏移;
3790 // ldmatrix 路径没有等价机制, 必须靠 1024B 对齐来保证 phase=0。
3791 extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3792 TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3793     *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
3794 half *s_a = s.A;
3795 half *s_b = s.B;
3796
3797 #pragma nv_diag_suppress static_var_with_dynamic_init
3798 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3799 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3800 #pragma nv_diag_default static_var_with_dynamic_init
3801
3802 const int NUM_K_TILES = div_ceil(K, BK);
3803 const int wg_idx = threadIdx.x / kConsumerThreads;
3804 const int wg_tid = threadIdx.x % kConsumerThreads;
3805
3806 if (threadIdx.x == 0) {
3807     for (int stage = 0; stage < kStages; ++stage) {
3808         init(&full[stage], kConsumerThreads + 1);
3809         init(&empty[stage], kConsumerThreads + 1);
3810     }
3811     tma_fence_proxy_async_shared_cta();
3812 }
3813 __syncthreads();
3814
3815 if (wg_idx == 0) {
3816     // 生产者 Warpgroup (wg_idx==0)
3817     NOTES_V2_REG_DEALLOC(40);
3818     if (wg_tid == 0) {
3819         int stage = 0;
3820         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3821             if (stage == kStages)
3822                 stage = 0;
3823
3824             empty[stage].wait(empty[stage].arrive());
3825
3826             tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
3827                 full[stage]);
3828             tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
3829                 full[stage]);
3830             tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
3831         }
3832     }
3833 } else {
3834     // 消费者 Warpgroup (wg_idx==1)
3835     // 初始化: 标记所有 stage 为"空" (可被 Producer 写入)
3836     NOTES_V2_REG_ALLOC(232);
3837
3838     for (int stage = 0; stage < kStages; ++stage) {
3839         [[maybe_unused]] auto token = empty[stage].arrive();
3840     }
3841
3842     const int warp_id = wg_tid / kWarpSize;

```

```

3843 const int lane_id = wg_tid % kWarpSize;
3844 // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
3845 const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
3846 const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
3847 uint32_t RA[kValTileM][4];
3848 uint32_t RB[kValTileN][2];
3849 uint32_t RC[kValTileM][kValTileN][2] = {0};
3850
3851 const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
3852 const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
3853 int stage = 0;
3854 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3855     if (stage == kStages)
3856         stage = 0;
3857
3858     full[stage].wait(full[stage].arrive());
3859     tma_fence_proxy_async_shared_cta();
3860
3861     // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
3862     // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
3863     // 同步机制与 warp specialization 的线程分工:
3864     //
3865     // 1. 生产者-消费者解耦: TMA 拷贝由 producer (wg_idx==0 的单个线程)
3866     //    通过 tma_load_2d + tma_arrive_expect_tx 提交, 而非所有 256 个线程
3867     //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
3868     //    拷贝都完成后进入计算; 而 TMA 路径用 cuda::barrier 的 full/empty
3869     //    协议完成 CTA 级同步——full[stage].wait() 返回时, producer 已通过
3870     //    expect_tx 声明了完整的传输字节数, 且硬件保证这些字节已全部写入 smem。
3871     //
3872     // 2. async proxy fence 替代 __syncthreads: fence_proxy_async(space_shared)
3873     //    在 async proxy (TMA 写入通道) 和后续 smem load (ldmatrix 读取通道)
3874     //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作 (TMA)
3875     //    对 fence 之后的所有 smem 读操作 (ldmatrix) 可见。这是一个轻量级
3876     //    proxy 间 ordering, 不需要阻塞线程, 也不会让 warp 等待其他 warp。
3877     //
3878     // 3. 消费者内部 warp 间无依赖: WG1 内的 4 个 warp (2x2 grid) 各自独立
3879     //    读取 smem 中互不重叠的 A/B 区域 (warp_m 和 warp_n 分别索引不同的
3880     //    行/列)。它们之间不需要读写共享数据, 因此不需要 __syncthreads 来
3881     //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令, 保证同一
3882     //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。
3883
3884     // 注意: 这里已经没有 NUM_K_TILES 循环了, 因为 K loop load 已经被 WG0 生产者处理了
3885 #pragma unroll
3886     for (int k_step = 0; k_step < kValTileK; ++k_step) {
3887
3888 #pragma unroll
3889         for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
3890             // kMmaM * kValTileM = 16*4 = 64, 每个消费者 warp 在 M 方向覆盖 64 行。
3891             const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3892             const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
3893             // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
3894             // lane_smem_a_k 中传给 swizzle<64>(), 而非像 swizzle kernel
3895             // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
3896             //   swizzle<64> 作用于完整 BK=64, swizzle 周期覆盖全部 64 列,
3897             //   k_step=0 和 k_step=1 的 chunk 会被 XOR 交叉混合 → k_step
3898             //   偏移必须在 swizzle 内部参与 chunk 计算。
3899             //   swizzle<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
3900             //   swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
3901             const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
3902             const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
3903                 (stage * BM * BK + lane_smem_a_m * BK +
3904                  swizzle<64>(lane_smem_a_m, lane_smem_a_k)) *
3905                 sizeof(half);

```

```

3906     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
3907 }
3908
3909 #pragma unroll
3910 for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
3911     // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
3912     const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3913     const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
3914     const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
3915     const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
3916         (stage * BN * BK + lane_smem_b_n * BK +
3917          swizzle<64>(lane_smem_b_n, lane_smem_b_k)) *
3918         sizeof(half);
3919     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
3920 }
3921
3922 #pragma unroll
3923 for (int i = 0; i < kValTileM; ++i) {
3924     #pragma unroll
3925     for (int j = 0; j < kValTileN; ++j) {
3926         HMMMA16816(RC[i][j][0], RC[i][j][1],
3927                    RA[i][0], RA[i][1], RA[i][2], RA[i][3],
3928                    RB[j][0], RB[j][1],
3929                    RC[i][j][0], RC[i][j][1]);
3930     }
3931 }
3932 }
3933 [[maybe_unused]] auto token = empty[stage].arrive();
3934 }
3935
3936 // Epilogue: 复用 RA[0] 和 RA[1] (已在寄存器中) 作为 shuffle 缓冲,
3937 // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
3938 // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4,
3939 // 由 lane 0 发出对齐的 128-bit store。
3940 #pragma unroll
3941 for (int i = 0; i < kValTileM; ++i) {
3942     const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3943     const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
3944     #pragma unroll
3945     for (int j = 0; j < kValTileN; ++j) {
3946         const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3947         RA[0][0] = RC[i][j][0];
3948         RA[1][0] = RC[i][j][1];
3949         RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
3950         RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
3951         RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
3952         RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
3953         RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
3954         RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
3955         if (lane_id % 4 == 0) {
3956             const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
3957             const int store_gmem_c_addr_0 =
3958                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
3959             const int store_gmem_c_addr_1 =
3960                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
3961             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
3962                 *reinterpret_cast<float4*>(&RA[0][0]);
3963             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
3964                 *reinterpret_cast<float4*>(&RA[1][0]);
3965         }
3966     }
3967 }
3968 }

```

```

3969 }
3970
3971 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
3972
3973 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3974 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3975 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3976 //   - TMA descriptor 描述 global memory 中矩阵的 shape/stride/dtype,
3977 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3978 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3979 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3980 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3981 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3982 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3983 //
3984 // * gmem_prob_stride + 1 的含义:
3985 //   语法层面 — 数组名退化 + 指针算术:
3986 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3987 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3988 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3989 //   语义层面 — 跳过隐式的最内维 stride:
3990 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3991 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3992 //     固定等于 sizeof(element_type), 不需要显式传入。
3993 //     对于 tensorRank=2 的 FP16 矩阵:
3994 //       dim 0 (内维, K): stride = sizeof(half) ← 隐式, API 不需要
3995 //       dim 1 (外维, M): stride = sizeof(half)*K ← 需要传给 API
3996 //     gmem_prob_stride 数组的布局是内维在前:
3997 //       [0] = sizeof(half) ← 内维, 不传
3998 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3999 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
4000 //
4001 // * smem_box_stride 为什么不需要 +1?
4002 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
4003 //   最内维 stride 也必须显式传入 (不隐式)。
4004 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
4005 //   都是连续存放的, 维度间没有 padding/stride。
4006 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
4007 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
4008 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
4009 //   两个 stride 值, 不需要跳过任何一个。
4010 //
4011 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
4012
4013 template <int BlockMajorSize, int BlockMinorSize>
4014 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
4015                                               half *gmem_ptr,
4016                                               int blocks_height,
4017                                               int blocks_width) {
4018     void *gmem_address = (void *)gmem_ptr;
4019     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
4020                                     (uint64_t)BlockMajorSize * blocks_height,
4021                                     1, 1, 1};
4022     uint64_t gmem_prob_stride[5] = {
4023         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
4024     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
4025                                    uint32_t(BlockMajorSize), 1, 1, 1};
4026     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
4027     CUresult result = cuTensorMapEncodeTiled(
4028         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
4029         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
4030         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
4031         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);

```

```

4032 if (result != CUDA_SUCCESS)
4033     printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
4034 }
4035
4036 // BlockMajorSize = major dim (outer), BlockMinorSize = minor dim (inner, contiguous).
4037 // For row-major [Major, Minor] matrices.
4038 // Default <128, 64> matches HGEMM A/B^T and FA Q; FA K/V use <64, 64>.
4039 template <int BlockMajorSize = 128, int BlockMinorSize = 64>
4040 __host__ __static inline CUTensorMap *allocate_and_create_tensor_map(
4041     half *src, int blocks_height, int blocks_width) {
4042     CUTensorMap *tma_map_d;
4043     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
4044     CUTensorMap tma_map_host;
4045     create_tensor_map<BlockMajorSize, BlockMinorSize>(&tma_map_host, src,
4046                                                         blocks_height,
4047                                                         blocks_width);
4048     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
4049                cudaMemcpyHostToDevice);
4050     return tma_map_d;
4051 }
4052 #endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */
4053
4054 // =====
4055 // Phase 8: FlashAttention-2 + MMA m16n8k16)
4056 // =====
4057 // 面试题点 (FlashAttention-2算法) :
4058 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
4059 //    但 HBM 带宽是瓶颈 → FlashAttention-2用 tiling + online softmax 避免写回
4060 // 2. FA 三板斧:
4061 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
4062 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
4063 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
4064 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
4065 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
4066 //    - 优点: 减少 warp 间通信和 shuffle
4067 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
4068 //    for each K,V tile:
4069 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
4070 //        m_new = max(m_old, row_max(S_cur * scale))
4071 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
4072 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
4073 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
4074 //        O_final = O_new / l_final
4075 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
4076 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
4077 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
4078 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
4079 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
4080 //      都天然同构”的通用结论
4081 //
4082 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
4083 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
4084 // Grid: ((N + 63) / 64, B * H, 1), Br=64
4085 // Block: (128, 1, 1), kNumThreads=kWarpSize*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
4086 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
4087
4088 // =====
4089 // FlashAttention-2 Split-Q Kernel (完整实现)
4090 // =====
4091 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
4092 //
4093 // Tile 设计 (以 kHeadDim=64 为例) :
4094 // Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*{4,8}*1 = {64,128}

```

```

4095 // Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
4096 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
4097 //
4098 // 执行流程:
4099 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
4100 // 2) 外循环: 沿 K seqLen 分块迭代 (Tc = seqLen/Bc)
4101 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
4102 // 4) 3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
4103 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
4104 // → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
4105 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
4106 // → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
4107 // 7) 3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
4108 // 8) 最终 rescale: O_final = (1/l_final) * O_final
4109 // 9) Epilogue: warp shuffle + 128-bit collective store
4110 //
4111 // ★ Pad 与手动 XOR swizzle 的 profile 结论 (SM120, B=1,H=32,N=4096,D=64) :
4112 // - XOR 能消除目标 ldmatrix lane pattern 的 bank conflict, 但收益是局部的;
4113 // XOR 本身及 `2` 的计算不是主要瓶颈, nvcc 已将该式化简为 bit operations.
4114 // - compact Q/K/V XOR 改变了 cp.async 的 shared destination pattern: LDGSTS
4115 // wavefronts 从 pad 的 30.15M 增至 68.16M (2.26x), long-scoreboard、
4116 // LG-throttle、MIO-throttle 分别约为 pad 的 3.25x、2.74x、1.60x。
4117 // - compact XOR 虽节省约 5 KiB smem, 但没有提高此 kernel 的 occupancy; 最终
4118 // 约 120.0 TFLOPS, 显著低于 Q/K/V kPad=8 的 166.6 TFLOPS。
4119 // - 所以当前 kernel 默认对 Q/K/V 都用 kPad=8。保留 swizzle 路径是为了学习和消融: 评价
4120 // shared layout 必须同时观察 ldmatrix reads 与 cp.async/LDGSTS writes, 不能只看
4121 // bank-conflict counter, 也不能只优化 XOR 地址算术。
4122 // ---- 寄存器填充辅助函数 (FlashAttention 用, 前移以供 TMA_MMA_WS kernel 使用) ----
4123 template <typename T, int M, const int N, const int K = 2>
4124 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
4125 #pragma unroll
4126     for (int i = 0; i < M; ++i)
4127 #pragma unroll
4128         for (int j = 0; j < N; ++j)
4129 #pragma unroll
4130             for (int k = 0; k < K; ++k)
4131                 R[i][j][k] = val;
4132 }
4133
4134 template <typename T, int M, const int N = 2>
4135 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
4136 #pragma unroll
4137     for (int i = 0; i < M; ++i)
4138 #pragma unroll
4139         for (int j = 0; j < N; ++j)
4140             R[i][j] = val;
4141 }
4142
4143 template <
4144     const int kHeadDim,           // head dim: 32, 64, 128
4145     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
4146     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
4147     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
4148     const int kMmaAccF32,         // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
4149     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
4150     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
4151     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
4152     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
4153     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
4154     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
4155     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
4156     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
4157     const int kStagesK,           // pipeline stages for K: >= 1; NO stages required for Q/V

```

```

4158     const int kPadQ,           // Q row padding; 0 selects compact XOR swizzle
4159     const int kPadK,           // K row padding; 0 selects compact XOR swizzle
4160     const int kPadV>          // V row padding; 0 selects compact XOR swizzle
4161 __global__ void __launch_bounds__(kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK)
4162 flash_attn_mma_stages_split_q(
4163     half *Q, half *K, half *V, half *0, int N, int H) {
4164     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
4165     static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
4166     static_assert(kPadQ >= 0 && kPadK >= 0 && kPadV >= 0,
4167         "Q/K/V padding must be non-negative");
4168     constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
4169     constexpr bool kSwizzleQ = kPadQ == 0;
4170     constexpr bool kSwizzleK = kPadK == 0;
4171     constexpr bool kSwizzleV = kPadV == 0;
4172     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 16*8*1=128
4173     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 8*1*8=64
4174     constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 32*8*1=256
4175     const int Tc = (N + Bc - 1) / Bc;
4176     // 原始实现默认 seqLen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
4177     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
4178     const float scale = 1.0f / sqrtf((float)kHeadDim);
4179
4180     // Block indexing
4181     const int Nb_id = blockIdx.y / H; // batch id
4182     const int Nh_id = blockIdx.y % H; // head id
4183     const int Q_tile_id = blockIdx.x; // tile id along Q's M dim (Br)
4184     const int tid = threadIdx.x;
4185     const int warp_id = tid / kWarpSize;
4186     const int lane_id = tid % kWarpSize;
4187     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
4188     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
4189
4190     // Global memory base offsets for this (batch, head)
4191     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
4192     const int Q_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
4193     const int K_gmem_offset = Q_gmem_offset;
4194     const int V_gmem_offset = Q_gmem_offset;
4195     const int O_gmem_offset = Q_gmem_offset;
4196
4197     // Thread-to-smem mapping for cooperative load
4198     int load_smem_Q_Br = tid / (kNumThreads / Br);
4199     int load_smem_Q_d = (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
4200     int load_smem_K_Bc = tid / (kNumThreads / Bc);
4201     int load_smem_K_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
4202     int load_smem_V_Bc = tid / (kNumThreads / Bc);
4203     int load_smem_V_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
4204
4205     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
4206     if (load_gmem_Q_Br >= N)
4207         return;
4208
4209     // Shared memory layout
4210     // Q/K/V independently use padded row-major when kPad* > 0 and compact XOR
4211     // swizzle when kPad* == 0. Padding and XOR are never combined per operand.
4212     // The swizzled physical layout is [col / 16][row][16]; swizzle<16>() selects
4213     // the 0/8 phase inside the final 16-half tile.
4214     extern __shared__ half smem[];
4215     constexpr int Q_tile_size = Br * (kHeadDim + kPadQ);
4216     constexpr int K_tile_size = Bc * (kHeadDim + kPadK);
4217     constexpr int kSmemStrideQ = kHeadDim + kPadQ;
4218     constexpr int kSmemStrideK = kHeadDim + kPadK;
4219     constexpr int kSmemStrideV = kHeadDim + kPadV;
4220     half *Q_tile_smem = smem;

```

```

4221 half *K_tile_smem = Q_tile_smem + Q_tile_size;
4222 half *V_tile_smem = K_tile_smem + kStagesK * K_tile_size;
4223
4224 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
4225 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
4226 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
4227
4228 // Online Softmax persistent state
4229 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
4230 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
4231 float lane_block_row_max_old[kValTileSeqLenQ][2];
4232 float lane_block_row_sum_old[kValTileSeqLenQ][2];
4233 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
4234 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
4235
4236 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
4237 uint32_t R_K[kValTileSeqLenK][2]; // K regs
4238 uint32_t R_V[kValTileHeadDimV][2]; // V regs
4239 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
4240 // f16 acc: 每 tile 2 个 uint32 (4 half) ; f32 acc: 每 tile 4 个 uint32 (4 f32) 。
4241 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
4242 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs]; // S=Q@K^T / P=softmax(S)
4243 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O for current tile
4244 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O accumulator (final output)
4245
4246 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
4247 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);
4248
4249 // =====
4250 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
4251 // =====
4252 {
4253     int load_gmem_Q_addr =
4254         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
4255     if constexpr (kSwizzleQ) {
4256 #pragma unroll
4257         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4258             int col = load_smem_Q_d + i;
4259             uint32_t ptr = smem_Q_base_ptr +
4260                 ((col / kMmaAtomK) * Br * kMmaAtomK +
4261                  load_smem_Q_Br * kMmaAtomK +
4262                  swizzle<kMmaAtomK>(load_smem_Q_Br, col % kMmaAtomK)) *
4263                 sizeof(half);
4264             CP_ASYNC_CG(ptr, &Q[load_gmem_Q_addr + i], 16);
4265         }
4266     } else {
4267         uint32_t load_smem_Q_ptr =
4268             smem_Q_base_ptr +
4269             (load_smem_Q_Br * kSmemStrideQ + load_smem_Q_d) * sizeof(half);
4270 #pragma unroll
4271         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4272             CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
4273         }
4274     }
4275     CP_ASYNC_COMMIT_GROUP();
4276 }
4277
4278 // =====
4279 // Step 2: 预加载前 (kStagesK-1) 个 K tile (多 stage pipeline 预热)
4280 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
4281 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
4282 // =====
4283 if constexpr (kStagesK > 1) {

```

```

4284 #pragma unroll
4285 for (int stage = 0; stage < (kStagesK - 1); ++stage) {
4286     int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
4287     int load_gmem_K_addr =
4288         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4289     if constexpr (kSwizzleK) {
4290 #pragma unroll
4291         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4292             int col = load_smem_K_d + i;
4293             uint32_t ptr = smem_K_base_ptr +
4294                 (stage * K_tile_size +
4295                  (col / kMmaAtomK) * Bc * kMmaAtomK +
4296                  load_smem_K_Bc * kMmaAtomK +
4297                  swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4298                 sizeof(half);
4299             CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4300         }
4301     } else {
4302         uint32_t load_smem_K_ptr =
4303             smem_K_base_ptr +
4304             (stage * K_tile_size + load_smem_K_Bc * kSmemStrideK +
4305              load_smem_K_d) *
4306             sizeof(half);
4307 #pragma unroll
4308         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4309             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4310         }
4311     }
4312     CP_ASYNC_COMMIT_GROUP();
4313 }
4314 CP_ASYNC_WAIT_GROUP(kStagesK - 2);
4315 __syncthreads();
4316 }
4317
4318 // =====
4319 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
4320 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
4321 // =====
4322 #pragma unroll 1
4323 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
4324     int smem_sel = tile_K_seqlen % kStagesK;
4325     int smem_sel_next = (tile_K_seqlen + (kStagesK - 1)) % kStagesK;
4326
4327     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
4328     // kStagesK>1: V 加载当前 tile, K 预取下一 tile (pipeline)
4329     // kStagesK=1: V 和 K 都加载当前 tile (smem_sel==smem_sel_next==0),
4330     // 无 pipeline 预取, 每轮 QK 前必须 wait 当前 K 就绪。
4331     // Load current V tile (no pipeline for V — one stage is enough)
4332     {
4333         int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
4334         int load_gmem_V_addr =
4335             V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
4336         if constexpr (kSwizzleV) {
4337 #pragma unroll
4338             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4339                 int col = load_smem_V_d + i;
4340                 uint32_t ptr = smem_V_base_ptr +
4341                     ((col / kMmaAtomK) * Bc * kMmaAtomK +
4342                      load_smem_V_Bc * kMmaAtomK +
4343                      swizzle<kMmaAtomK>(load_smem_V_Bc, col % kMmaAtomK)) *
4344                     sizeof(half);
4345                 CP_ASYNC_CG(ptr, &V[load_gmem_V_addr + i], 16);
4346             }

```

```

4347     } else {
4348         uint32_t load_smem_V_ptr =
4349             smem_V_base_ptr +
4350             (load_smem_V_Bc * kSmemStrideV + load_smem_V_d) * sizeof(half);
4351 #pragma unroll
4352         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4353             CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
4354         }
4355     }
4356     CP_ASYNC_COMMIT_GROUP();
4357 }
4358
4359 if constexpr (kStagesK > 1) {
4360     // Prefetch next K tile (pipelined)
4361     if ((tile_K_seqlen + 1) < Tc) {
4362         int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
4363         int load_gmem_K_addr =
4364             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4365         if constexpr (kSwizzleK) {
4366 #pragma unroll
4367             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4368                 int col = load_smem_K_d + i;
4369                 uint32_t ptr = smem_K_base_ptr +
4370                     (smem_sel_next * K_tile_size +
4371                      (col / kMmaAtomK) * Bc * kMmaAtomK +
4372                      load_smem_K_Bc * kMmaAtomK +
4373                      swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4374                     sizeof(half);
4375                 CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4376             }
4377         } else {
4378             uint32_t load_smem_K_ptr =
4379                 smem_K_base_ptr +
4380                 (smem_sel_next * K_tile_size +
4381                  load_smem_K_Bc * kSmemStrideK +
4382                  load_smem_K_d) *
4383                 sizeof(half);
4384 #pragma unroll
4385             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4386                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4387             }
4388         }
4389         CP_ASYNC_COMMIT_GROUP();
4390     }
4391 } else {
4392     // kStagesK==1: 加载当前 K tile (smem_sel_next == smem_sel == 0) 。
4393     // Step 2 预加载循环 (kStagesK-1=0) 不执行, 所以每轮 3a 必须加载当前 K。
4394     {
4395         int load_gmem_K_Bc = tile_K_seqlen * Bc + load_smem_K_Bc;
4396         int load_gmem_K_addr =
4397             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4398         if constexpr (kSwizzleK) {
4399 #pragma unroll
4400             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4401                 int col = load_smem_K_d + i;
4402                 uint32_t ptr = smem_K_base_ptr +
4403                     (smem_sel * K_tile_size +
4404                      (col / kMmaAtomK) * Bc * kMmaAtomK +
4405                      load_smem_K_Bc * kMmaAtomK +
4406                      swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4407                     sizeof(half);
4408                 CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4409             }

```

```

4410     } else {
4411         uint32_t load_smem_K_ptr =
4412             smem_K_base_ptr +
4413             (smem_sel * K_tile_size +
4414              load_smem_K_Bc * kSmemStrideK +
4415              load_smem_K_d) *
4416             sizeof(half);
4417 #pragma unroll
4418         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4419             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4420         }
4421     }
4422     CP_ASYNC_COMMIT_GROUP();
4423 }
4424 }
4425
4426 // 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环
4427 // kStagesK==1: 3a 刚提交当前 K 的 cp.async, 必须 wait 后才能 ldmatrix.
4428 // kStagesK>1: K 已在上一轮预取并 wait (Step 2 或 3e 末尾的 wait), 无需再 wait.
4429 if constexpr (kStagesK == 1) {
4430     CP_ASYNC_WAIT_GROUP(0);
4431     __syncthreads();
4432 }
4433 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
4434 #pragma unroll
4435 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
4436     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
4437     #pragma unroll
4438     for (int i = 0; i < kValTileSeqLenQ; ++i) {
4439         int warp_smem_Q_Br =
4440             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
4441         int lane_smem_Q_Br =
4442             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
4443         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
4444         uint32_t lane_smem_Q_ptr;
4445         if constexpr (kSwizzleQ) {
4446             lane_smem_Q_ptr = smem_Q_base_ptr +
4447                 ((lane_smem_Q_d / kMmaAtomK) * Br * kMmaAtomK +
4448                  lane_smem_Q_Br * kMmaAtomK +
4449                  swizzle<kMmaAtomK>(lane_smem_Q_Br,
4450                                   lane_smem_Q_d % kMmaAtomK)) *
4451                 sizeof(half);
4452         } else {
4453             lane_smem_Q_ptr = smem_Q_base_ptr +
4454                 (lane_smem_Q_Br * kSmemStrideQ + lane_smem_Q_d) * sizeof(half);
4455         }
4456         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4457                    lane_smem_Q_ptr);
4458     }
4459
4460     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
4461     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
4462     #pragma unroll
4463     for (int j = 0; j < kValTileSeqLenK; ++j) {
4464         int warp_smem_K_Bc =
4465             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
4466         int lane_smem_K_Bc =
4467             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
4468         int lane_smem_K_d =
4469             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
4470         uint32_t lane_smem_K_ptr;
4471         if constexpr (kSwizzleK) {
4472             lane_smem_K_ptr = smem_K_base_ptr +

```

```

4473         (smem_sel * K_tile_size +
4474         (lane_smem_K_d / kMmaAtomK) * Bc * kMmaAtomK +
4475         lane_smem_K_Bc * kMmaAtomK +
4476         swizzle<kMmaAtomK>(lane_smem_K_Bc,
4477         lane_smem_K_d % kMmaAtomK)) *
4478         sizeof(half);
4479     } else {
4480         lane_smem_K_ptr = smem_K_base_ptr +
4481         (smem_sel * K_tile_size + lane_smem_K_Bc * kSmemStrideK +
4482         lane_smem_K_d) *
4483         sizeof(half);
4484     }
4485     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
4486 }
4487
4488 // MMA: S[tile] += Q[tile] @ K^T[tile]
4489 #pragma unroll
4490 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4491     #pragma unroll
4492     for (int j = 0; j < kValTileSeqLenK; ++j) {
4493         if constexpr (kMmaAccF32) {
4494             MMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
4495             R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4496             R_K[j][0], R_K[j][1],
4497             R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
4498         } else {
4499             MMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
4500             R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
4501             R_S[i][j][1]);
4502         }
4503     }
4504 }
4505 } // end loop over d
4506 __syncthreads();
4507
4508 // =====
4509 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
4510 // =====
4511 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
4512 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
4513 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
4514 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
4515 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
4516 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
4517 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
4518 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
4519 // kValTileSeqLenK tiles.
4520
4521 float lane_row_max_new[kValTileSeqLenQ][2];
4522 float lane_row_sum_new[kValTileSeqLenQ][2];
4523 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
4524 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
4525
4526 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
4527 #pragma unroll
4528 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4529     #pragma unroll
4530     for (int j = 0; j < kValTileSeqLenK; ++j) {
4531         float tmp_max_0, tmp_max_1;
4532         if constexpr (kMmaAccF32) {
4533             float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
4534             tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
4535             tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;

```

```

4536     } else {
4537         float2 t_reg_S_0 =
4538             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
4539         float2 t_reg_S_1 =
4540             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
4541         tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
4542         tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
4543     }
4544     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
4545     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
4546 }
4547 // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
4548 // {0,4,8,...,28} hold valid data)
4549 lane_row_max_new[i][0] =
4550     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
4551 lane_row_max_new[i][1] =
4552     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
4553 }
4554
4555 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
4556 // 面试关键点: 这里将 P 写回 R_S 寄存器!
4557 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
4558 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
4559 #pragma unroll
4560 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4561     // m_new = max(m_old, m_cur)
4562     float block_row_max_new_0 =
4563         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
4564     float block_row_max_new_1 =
4565         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
4566
4567     #pragma unroll
4568     for (int j = 0; j < kValTileSeqLenK; ++j) {
4569         if constexpr (kMmaAccF32) {
4570             float *t_fptr_S = reinterpret_cast<float*>(&R_S[i][j][0]);
4571             half *t_hptr_S = reinterpret_cast<half*>(&R_S[i][j][0]);
4572             t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
4573             t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
4574             t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
4575             t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
4576             lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
4577             lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
4578             t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
4579             t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
4580             t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
4581             t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
4582         } else {
4583             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
4584             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
4585             t_reg_S_0.x =
4586                 __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
4587             t_reg_S_0.y =
4588                 __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
4589             t_reg_S_1.x =
4590                 __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
4591             t_reg_S_1.y =
4592                 __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
4593             lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
4594             lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
4595             HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
4596             HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
4597         }
4598     }
4599 }

```

```

4599 // Warp-level reduce sum (kWarpWidth = 4, same as max)
4600 lane_row_sum_new[i][0] =
4601     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4602 lane_row_sum_new[i][1] =
4603     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4604 }
4605 __syncthreads();
4606
4607 // =====
4608 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = 0[Br,d]
4609 // =====
4610 // Wait for V to be ready before computing P@V
4611 if constexpr (kStagesK > 1) {
4612     if ((tile_K_seqlen + 1) < Tc) {
4613         CP_ASYNC_WAIT_GROUP(1);
4614     } else {
4615         CP_ASYNC_WAIT_GROUP(0);
4616     }
4617 } else {
4618     CP_ASYNC_WAIT_GROUP(0);
4619 }
4620 __syncthreads();
4621
4622 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_0, 0);
4623
4624 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4625 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4626 #pragma unroll
4627 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4628     // ldmatrix.x2.trans: load V[Bc,d] with transposition
4629     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4630     // transposed ldmatrix
4631     #pragma unroll
4632     for (int j = 0; j < kValTileHeadDimV; ++j) {
4633         int warp_smem_V_d =
4634             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4635         int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4636         int lane_smem_V_d = warp_smem_V_d;
4637         uint32_t lane_smem_V_ptr;
4638         if constexpr (kSwizzleV) {
4639             lane_smem_V_ptr = smem_V_base_ptr +
4640                 ((lane_smem_V_d / kMmaAtomK) * Bc * kMmaAtomK +
4641                  lane_smem_V_Bc * kMmaAtomK +
4642                  swizzle<kMmaAtomK>(lane_smem_V_Bc,
4643                                   lane_smem_V_d % kMmaAtomK)) *
4644                 sizeof(half);
4645         } else {
4646             lane_smem_V_ptr = smem_V_base_ptr +
4647                 (lane_smem_V_Bc * kSmemStrideV + lane_smem_V_d) * sizeof(half);
4648         }
4649         LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4650     }
4651 }
4652
4653 // P matrix layout in R_S[i][j][2]:
4654 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4655 // | 64x64 | warp_KV 0 |
4656 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4657 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4658 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4659 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4660 // tile_V_Bc selects which 16-column slice of P to use:
4661 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0

```

```

4662 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
4663 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
4664 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
4665 // 对应的 MMA A fragment 布局可以这样记:
4666 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
4667 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
4668 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
4669 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
4670 // fragment 都无条件成立的通用结论。layout转换逻辑:
4671 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
4672 int w = tile_V_Bc * 2;
4673 #pragma unroll
4674 for (int i = 0; i < kValTileSeqLenP; ++i) {
4675 #pragma unroll
4676 for (int j = 0; j < kValTileHeadDimV; ++j) {
4677 if constexpr (kMmaAccF32) {
4678 HMMA16816F32(R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3],
4679 R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
4680 R_V[j][0], R_V[j][1],
4681 R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3]);
4682 } else {
4683 HMMA16816(R_0[i][j][0], R_0[i][j][1],
4684 R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
4685 R_V[j][0], R_V[j][1],
4686 R_0[i][j][0], R_0[i][j][1]);
4687 }
4688 }
4689 }
4690 } // end for tile_V_Bc
4691 __syncthreads();
4692
4693 // =====
4694 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
4695 // =====
4696 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
4697 #pragma unroll
4698 for (int i = 0; i < kValTileSeqLenP; ++i) {
4699 float block_row_max_new_0 = lane_row_max_new[i][0];
4700 float block_row_max_new_1 = lane_row_max_new[i][1];
4701 float block_row_sum_new_0 = lane_row_sum_new[i][0];
4702 float block_row_sum_new_1 = lane_row_sum_new[i][1];
4703
4704 float block_row_max_old_0 = lane_block_row_max_old[i][0];
4705 float block_row_max_old_1 = lane_block_row_max_old[i][1];
4706
4707 block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
4708 block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
4709
4710 // Handle first iteration: m_old = -inf, need to use m_new directly
4711 block_row_max_old_0 =
4712 (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
4713 block_row_max_old_1 =
4714 (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
4715
4716 float rescale_o_factor_0 =
4717 __expf(block_row_max_old_0 - block_row_max_new_0);
4718 float rescale_o_factor_1 =
4719 __expf(block_row_max_old_1 - block_row_max_new_1);
4720
4721 // Rescale 0_old + Add P@V in one fused step
4722 #pragma unroll
4723 for (int j = 0; j < kValTileHeadDimV; ++j) {
4724 if constexpr (kMmaAccF32) {

```

```

4725     float *t_fptr_0 = reinterpret_cast<float *>(&R_0[i][j][0]);
4726     float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
4727     t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_0[0]);
4728     t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_0[1]);
4729     t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_0[2]);
4730     t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_0[3]);
4731 } else {
4732     float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
4733     float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
4734     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4735     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4736     t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
4737     t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
4738     t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
4739     t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
4740     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4741     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4742 }
4743 }
4744
4745 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
4746 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
4747 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
4748 lane_block_row_sum_old[i][0] = __fmaf_rn(
4749     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
4750 lane_block_row_sum_old[i][1] = __fmaf_rn(
4751     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
4752
4753 // Update m
4754 lane_block_row_max_old[i][0] = block_row_max_new_0;
4755 lane_block_row_max_old[i][1] = block_row_max_new_1;
4756 }
4757
4758 // Wait for next K tile to be ready in smem before next iteration
4759 if constexpr (kStagesK > 1) {
4760     if ((tile_K_seqlen + 1) < Tc) {
4761         CP_ASYNC_WAIT_GROUP(0);
4762     }
4763     __syncthreads();
4764 }
4765 } // end outer loop over K seqlen
4766 __syncthreads();
4767
4768 // =====
4769 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
4770 // =====
4771 #pragma unroll
4772 for (int i = 0; i < kValTileSeqLenP; ++i) {
4773     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4774     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4775 #pragma unroll
4776 for (int j = 0; j < kValTileHeadDimV; ++j) {
4777     if constexpr (kMmaAccF32) {
4778         float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
4779         half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
4780         t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
4781         t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
4782         t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
4783         t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
4784     } else {
4785         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4786         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4787         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;

```

```

4788     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4789     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4790     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
4791     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4792     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4793 }
4794 }
4795 }
4796
4797 // =====
4798 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4799 // =====
4800 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4801 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4802 #pragma unroll
4803 for (int i = 0; i < kValTileSeqLenP; ++i) {
4804     #pragma unroll
4805     for (int j = 0; j < kValTileHeadDimV; ++j) {
4806         uint32_t R_Z[2][4];
4807         R_Z[0][0] = R_D[i][j][0];
4808         R_Z[1][0] = R_D[i][j][1];
4809         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4810         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4811         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4812         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4813         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
4814         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
4815
4816         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4817         if (lane_id % 4 == 0) {
4818             int store_warp_regs_0_Br =
4819                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4820             int store_lane_gmem_0_Br =
4821                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4822             int store_warp_regs_0_d =
4823                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4824             int store_lane_gmem_0_d = store_warp_regs_0_d;
4825             int store_gmem_0_addr_0 = 0_gmem_offset +
4826                 (store_lane_gmem_0_Br + 0) * kHeadDim +
4827                 store_lane_gmem_0_d;
4828             int store_gmem_0_addr_1 = 0_gmem_offset +
4829                 (store_lane_gmem_0_Br + 8) * kHeadDim +
4830                 store_lane_gmem_0_d;
4831             // LDST128BITS = reinterpret_cast<float4*>
4832             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4833                 *reinterpret_cast<float4*>(&R_Z[0][0]);
4834             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4835                 *reinterpret_cast<float4*>(&R_Z[1][0]);
4836         }
4837     }
4838 }
4839 }
4840
4841 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
4842 // =====
4843 // Phase 8b: FlashAttention-2 TMA + MMA Warp Specialization (SM120)
4844 // =====
4845 // 面试要点 (FA TMA MMA WS vs cp.async FA) :
4846 // - Producer (WG0, 128T) 用 TMA 128B SWIZZLE 搬运 Q/K/V tile 到 smem
4847 // - Consumer (WG1, 256T) 用 ldmatrix + mma.sync.m16n8k16 做 Q@K^T / P@V
4848 // - 与 hgemm_tma_mma_ws_tn 的对比:
4849 // * HGEMM 只做一次 GEMM, K 维迭代; FA 做 QK^T 和 PV 两次 GEMM, KV seqlen 迭代
4850 // * HGEMM 的 A/B 都 staged; FA 中 Q 只 load 一次 (split-Q), K staged, V 单 buffer

```

```

4851 //
4852 // ★ split-Q 语义 (关键, 避免误解) :
4853 //   - grid = (seqlen/Br, B*H), 每个 block 处理一个 **block-level Q tile [Br, d]**
4854 //     (不是全局 Q)。对 Q 的 seqlen 做 block-level 并行。
4855 //   - 每个 block 遍历**完整的 KV seqlen** (Tc = seqlen/Bc 次外循环)。
4856 //   - 因此每个 block 的 Q tile 只需 **load 一次到 smem**, 在整个 KV 遍历中复用
4857 //     → 只需 full_Q barrier (无 empty_Q)。
4858 //
4859 // ★ Pipeline 同步协议 (arrive_count = 257 = 256 consumer + 1 producer) :
4860 //   - Q: full_Q — Producer 发 arrive_tx, Consumer 在主循环前 wait 一次
4861 //   - K: full_K[kStagesK] / empty_K[kStagesK] — 多 stage pipeline
4862 //   - V: full_V / empty_V — 单 buffer 串行 (QK 前发 TMA → QK 重叠 → PV 前等 → PV 后释放)
4863 //
4864 // ★ V ldmatrix.x2.trans 正确性 (已论证无风险) :
4865 //   TMA 128B SWIZZLE 是 1-1 映射, ldmatrix 用 swizzle<64>(row,col) 计算的物理地址
4866 //   = TMA 写入的物理地址 (smem 1024B 对齐保证 phase=0)。ldmatrix.x2.trans 的转置
4867 //   语义在寄存器层面工作, 与 smem 物理布局无关 → 必然正确。
4868 //
4869 // ★ Shared Memory 占用分析 (D=64/128, kStagesK=1/2/3/4, kStagesV=1/2) :
4870 //   smem = (Q[Br,D] + K[kStagesK,Bc,D] + V[kStagesV,Bc,D]) * sizeof(half)
4871 //         = D * (Br + kStagesK*Bc + kStagesV*Bc) * 2
4872 //         = D * (128 + (kStagesK+kStagesV)*64) * 2      (Br=128, Bc=64)
4873 //
4874 //   | kStagesK | kStagesV | D=64 bytes | D=128 bytes |
4875 //   |-----|-----|-----|-----|
4876 //   | 1      | 1      | 32 KB      | 64 KB      |
4877 //   | 2      | 1      | 40 KB      | 80 KB      |
4878 //   | 2      | 2      | 48 KB      | 96 KB      |
4879 //   | 3      | 1      | 48 KB      | 96 KB      |
4880 //   | 3      | 2      | 56 KB      | 112 KB     |
4881 //   | 4      | 1      | 56 KB      | 112 KB     |
4882 //
4883 //   SM120 (RTX PRO 5000) optin smem 上限 ~100KB:
4884 //   - D=64: S=1/2/3/4 全部可行 (32-56 KB)
4885 //   - D=128: S=1/2/3 可行 (64-96 KB); S=4 (112 KB) 超出 optin 上限, 会被
4886 //     check_smem_feasible 兜底判为 SMEM SKIP。D=128 推荐 S=2 (甜点: 80 KB,
4887 //     2 blocks/SM, pipeline 深度足够隐藏 TMA 延迟)。
4888 //
4889 // v1 限制: kHeadDim=64 or 128; seqlen % Br == 0 且 seqlen % Bc == 0 (不处理尾 tile)
4890 //
4891 // =====
4892 // FA TMA WS smem offset helper (D=64/128 通用)
4893 // =====
4894 // 背景: CU_TENSOR_MAP_SWIZZLE_128B 硬件要求 box innermost dim ≤ 128B = 64 half
4895 // (见 /usr/local/cuda/include/cuda.h 的 cuTensorMapEncodeTiled 文档)。因此 D=128
4896 // 不能用单个 TMA box=(128, Br) 覆盖整行。
4897 //
4898 // 方案: D=128 时 box 固定为 (64, Br), 沿 head_dim 方向连续发 kTmaChunks 次 TMA
4899 // (minor_coord = c*64), 写入 chunk-major smem 布局 [kTmaChunks, Br, 64]:
4900 //   - chunk c 的 smem 起始偏移 = c * Br * 64
4901 //   - chunk c 内部按 [Br, 64] row-major + SWIZZLE_128B 存放 (64 half = 128B 周期)
4902 //
4903 // 本 helper 计算 (row, col) 在该 chunk-major 布局下的 swizzled smem 元素偏移:
4904 //   chunk   = col / 64      ∈ [0, kTmaChunks)
4905 //   col_in  = col % 64      ∈ [0, 64)
4906 //   offset  = chunk * Br * 64 + row * 64 + swizzle<64>(row, col_in)
4907 //
4908 // D=64 退化: kTmaChunks=1, chunk=0, offset = row*64 + swizzle<64>(row, col)
4909 //           与原公式 (row*64 + swizzle<64>(row, col)) 完全一致 → 无回归。
4910 template <int kHeadDim, int Br>
4911 __device__ __forceinline__ int swizzle_fa(int row, int col) {
4912     static_assert(kHeadDim == 64 || kHeadDim == 128, "D=64 or 128 only");
4913     if constexpr (kHeadDim == 64) {

```

```

4914     return row * 64 + swizzle<64>(row, col);
4915 } else { // kHeadDim == 128
4916     const int chunk = col >> 6; // col / 64 ∈ {0, 1}
4917     const int col_in = col & 63; // col % 64
4918     return chunk * (Br * 64) + row * 64 + swizzle<64>(row, col_in);
4919 }
4920 }
4921
4922 template <
4923     const int kHeadDim, // head dim: v1 only 64
4924     const int kMmaAtomM, // 16 (MMA instruction M dim)
4925     const int kMmaAtomN, // 8 (MMA instruction N dim)
4926     const int kMmaAtomK, // 16 (MMA instruction K dim)
4927     const int kMmaAccF32, // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
4928     const int kMmaTileSeqLenQ, // MMA tiles along Q's M dim, 8 → Br=16*8=128
4929     const int kMmaTileSeqLenK, // MMA tiles along K's N dim, 1 → Bc basis=8
4930     const int kMmaTileSeqLenP, // MMA tiles for P@V M dim, == kMmaTileSeqLenQ
4931     const int kMmaTileHeadDimV, // MMA tiles for P@V N dim (head dim direction)
4932     const int kValTileSeqLenQ, // value tiles along Q's M, 1 → Br per warp=16
4933     const int kValTileSeqLenK, // value tiles along K's N, 8 → Bc_warp=8*8=64
4934     const int kValTileSeqLenP, // value tiles for P@V M dim, 1
4935     const int kValTileHeadDimV, // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
4936     const int kStagesK, // K pipeline depth (>=1)
4937     const int kStagesV, // V pipeline depth (>=1; 1=single buffer, >=2=pipelined)
4938     const int kNumThreads> // 384, 128 producer + 256 consumer
4939 __global__ void __launch_bounds__(kNumThreads, 1)
4940     flash_attn_tma_mma_ws_stages_split_q(
4941         half *Q, half *K, half *V, half *O, int N, int H,
4942         const CUTensorMap *__restrict__ tensorMapQ,
4943         const CUTensorMap *__restrict__ tensorMapK,
4944         const CUTensorMap *__restrict__ tensorMapV) {
4945     static_assert(kHeadDim == 64 || kHeadDim == 128,
4946         "v1 supports kHeadDim == 64 or 128");
4947     static_assert(kNumThreads == 384, "128 producer + 256 consumer");
4948     static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
4949     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
4950     static_assert(kStagesV >= 1, "kStagesV must be >= 1");
4951     constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
4952
4953     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 128
4954     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
4955     // BK for QK: kHeadDim=64, split into kHeadDim/kMmaAtomK=4 MMA K slices
4956     // BK for PV: Bc=64, split into Bc/kMmaAtomK=4 MMA K slices (P rows)
4957     constexpr int kConsumerThreads = 256; // 8 warps
4958     constexpr int kProducerThreads = 128; // 4 warps, only thread 0 issues TMA
4959     constexpr int kQTileBytes = Br * kHeadDim * sizeof(half); // 16 KB
4960     constexpr int kKTileBytes = Bc * kHeadDim * sizeof(half); // 8 KB
4961     constexpr int kVTileBytes = Bc * kHeadDim * sizeof(half); // 8 KB
4962     // TMA box innermost 固定 64 half (128B), 满足 CU_TENSOR_MAP_SWIZZLE_128B 的
4963     // box innermost ≤ 128B 硬约束。D=128 时沿 head_dim 发 kTmaChunks 次 TMA,
4964     // 写入 chunk-major smem 布局 [kTmaChunks, Br, 64]。
4965     constexpr int kTmaBoxMinor = 64;
4966     constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
4967
4968     // Block indexing — split-Q: blockIdx.x indexes Q tile along seqlen
4969     const int Nb_id = blockIdx.y / H;
4970     const int Nh_id = blockIdx.y % H;
4971     const int Q_tile_id = blockIdx.x;
4972     const int Tc = (N + Bc - 1) / Bc; // number of KV tiles along seqlen
4973     const float scale = 1.0f / sqrtf((float)kHeadDim);
4974
4975     // Per-head gmem base offset (self-attention: Q=K=V share layout)
4976     const int QKV_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;

```

```

4977 const int O_gmem_offset = QKV_gmem_offset;
4978 // TMA descriptor covers the entire [B*H*N, D] gmem as one 2D matrix; the
4979 // producer must offset major_coord by this head/batch base so each block
4980 // loads its own (batch, head) Q/K/V tile instead of always reading head 0.
4981 const int qkv_major_base = (Nb_id * H + Nh_id) * N;
4982
4983 // ---- Shared memory layout ----
4984 // TMA CU_TENSOR_MAP_SWIZZLE_128B requires 1024B-aligned smem base so the
4985 // hardware swizzle phase starts at zero. Consumer swizzle<64>() assumes
4986 // zero phase (no base_offset compensation like WGMMMA descriptor).
4987 // Q: [Br, kHeadDim] = [128, 64] = 16 KB (1024B aligned ✓)
4988 // K: [kStagesK, Bc, kHeadDim] = [kStagesK, 64, 64] (8 KB per stage)
4989 // V: [kStagesV, Bc, kHeadDim] = [kStagesV, 64, 64] (8 KB per stage)
4990 extern __shared__ __align__(1024) uint8_t smem_fa_tma_ws[];
4991 half *Q_smem = reinterpret_cast<half*>(smem_fa_tma_ws);
4992 half *K_smem = Q_smem + Br * kHeadDim;
4993 half *V_smem = K_smem + kStagesK * Bc * kHeadDim;
4994
4995 const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
4996 const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem);
4997 const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem);
4998
4999 // ---- Barriers ----
5000 // arrive_count = kConsumerThreads + 1 = 257 (256 consumer arrives + 1 producer arrive_tx)
5001 #pragma nv_diag_suppress static_var_with_dynamic_init
5002 __shared__ cuda::barrier<cuda::thread_scope_block> full_Q;
5003 __shared__ cuda::barrier<cuda::thread_scope_block> full_K[kStagesK];
5004 __shared__ cuda::barrier<cuda::thread_scope_block> empty_K[kStagesK];
5005 __shared__ cuda::barrier<cuda::thread_scope_block> full_V[kStagesV];
5006 __shared__ cuda::barrier<cuda::thread_scope_block> empty_V[kStagesV];
5007 #pragma nv_diag_default static_var_with_dynamic_init
5008
5009 // Thread partition: first kProducerThreads (0~127) = Producer WG,
5010 // remaining kConsumerThreads (128~383) = Consumer WG.
5011 // WARN: Must use kProducerThreads (not kConsumerThreads) as the divisor, otherwise
5012 // the split is wrong: 384/256=1.5 would give Producer 256 threads and
5013 // Consumer only 128, but barriers expect 256 consumer arrives → deadlock.
5014 const int wg_idx = (threadIdx.x < kProducerThreads) ? 0 : 1;
5015 const int wg_tid = (threadIdx.x < kProducerThreads) ? threadIdx.x :
5016                 (threadIdx.x - kProducerThreads);
5017
5018 if (threadIdx.x == 0) {
5019     init(&full_Q, kConsumerThreads + 1);
5020     for (int s = 0; s < kStagesV; ++s) {
5021         init(&full_V[s], kConsumerThreads + 1);
5022         init(&empty_V[s], kConsumerThreads + 1);
5023     }
5024     for (int s = 0; s < kStagesK; ++s) {
5025         init(&full_K[s], kConsumerThreads + 1);
5026         init(&empty_K[s], kConsumerThreads + 1);
5027     }
5028     tma_fence_proxy_async_shared_cta();
5029 }
5030 __syncthreads();
5031
5032 // =====
5033 // Producer Warpgroup (WG0, threadIdx.x 0~127)
5034 // Only wg_tid == 0 issues TMA. Split-Q: Q tile loaded once, K staged, V single buffer.
5035 // D=128 时每个 tile 沿 head_dim 发 kTmaChunks 次 TMA (box innermost=64 half) ,
5036 // 共享同一 barrier: N 次 cp.async.bulk.tensor 各 reduce chunk_bytes 到 tx count,
5037 // 1 次 mbarrier_arrive_expect_tx 声明总字节 + producer thread arrive.
5038 // =====
5039 if (wg_idx == 0) {

```

```

5040 NOTES_V2_REG_DEALLOC(40);
5041 if (wg_tid == 0) {
5042     // Step P0: Load block-level Q tile [Br, d] once. D=128 时发 kTmaChunks 次。
5043     // minor_coord = c * 64, major_coord = qkv_major_base + Q_tile_id * Br
5044     // smem dst = Q_smem + c * Br * 64 (chunk-major 布局)
5045     for (int c = 0; c < kTmaChunks; ++c) {
5046         tma_load_2d(Q_smem + c * Br * kTmaBoxMinor, tensorMapQ,
5047             c * kTmaBoxMinor, qkv_major_base + Q_tile_id * Br, full_Q);
5048     }
5049     tma_arrive_expect_tx(full_Q, kQTileBytes);
5050
5051     // Step P1: Prefetch first (kStagesK-1) K tiles.
5052     // K tile coords: minor = c*64, major = qkv_major_base + s*Bc
5053     for (int s = 0; s < kStagesK - 1; ++s) {
5054         empty_K[s].wait(empty_K[s].arrive());
5055         for (int c = 0; c < kTmaChunks; ++c) {
5056             tma_load_2d(K_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5057                 tensorMapK, c * kTmaBoxMinor,
5058                 qkv_major_base + s * Bc, full_K[s]);
5059         }
5060         tma_arrive_expect_tx(full_K[s], kKTileBytes);
5061     }
5062
5063     // Step P1b: Prefetch first (kStagesV-1) V tiles (only if kStagesV > 1).
5064     // kStagesV=1: no prefetch; V[k] loaded in P2 overlaps only QK gemm.
5065     // kStagesV>=2: V prefetched here; in P2 V[k+kStagesV-1] overlaps
5066     // QK+softmax+PV of current iter → true V pipeline.
5067     if constexpr (kStagesV > 1) {
5068         for (int s = 0; s < kStagesV - 1; ++s) {
5069             empty_V[s].wait(empty_V[s].arrive());
5070             for (int c = 0; c < kTmaChunks; ++c) {
5071                 tma_load_2d(V_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5072                     tensorMapV, c * kTmaBoxMinor,
5073                     qkv_major_base + s * Bc, full_V[s]);
5074             }
5075             tma_arrive_expect_tx(full_V[s], kVTileBytes);
5076         }
5077     }
5078
5079     // Step P2: Main loop over KV seqlen tiles
5080     for (int k = 0; k < Tc; ++k) {
5081         [[maybe_unused]] const int stage = k % kStagesK;
5082
5083         // Issue V[k+kStagesV-1] TMA to stage_next_v (pipelined).
5084         // kStagesV=1: loads V[k] to stage 0, overlaps with QK of same iter.
5085         // kStagesV>=2: loads V[k+kStagesV-1], overlaps with QK+softmax+PV.
5086         {
5087             const int v_tile = k + kStagesV - 1;
5088             if (v_tile < Tc) {
5089                 const int stage_next_v = v_tile % kStagesV;
5090                 empty_V[stage_next_v].wait(empty_V[stage_next_v].arrive());
5091                 for (int c = 0; c < kTmaChunks; ++c) {
5092                     tma_load_2d(V_smem + stage_next_v * Bc * kHeadDim +
5093                         c * Bc * kTmaBoxMinor,
5094                         tensorMapV, c * kTmaBoxMinor,
5095                         qkv_major_base + v_tile * Bc,
5096                         full_V[stage_next_v]);
5097                 }
5098                 tma_arrive_expect_tx(full_V[stage_next_v], kVTileBytes);
5099             }
5100         }
5101
5102         // Prefetch K[k+kStagesK-1] (pipelined).

```

```

5103     if (k + kStagesK - 1 < Tc) {
5104         const int stage_next = (k + kStagesK - 1) % kStagesK;
5105         empty_K[stage_next].wait(empty_K[stage_next].arrive());
5106         for (int c = 0; c < kTmaChunks; ++c) {
5107             tma_load_2d(K_smem + stage_next * Bc * kHeadDim +
5108                 c * Bc * kTmaBoxMinor,
5109                 tensorMapK, c * kTmaBoxMinor,
5110                 qkv_major_base + (k + kStagesK - 1) * Bc,
5111                 full_K[stage_next]);
5112         }
5113         tma_arrive_expect_tx(full_K[stage_next], kKTileBytes);
5114     }
5115 }
5116 }
5117 }
5118 // =====
5119 // Consumer Warpgroup (WG1, threadIdx.x 128~383, 256 threads = 8 warps)
5120 // All 256 threads participate in ldmatrix + mma.sync.
5121 // =====
5122 else {
5123     const int warp_id = wg_tid / kWarpSize; // 0~7
5124     const int lane_id = wg_tid % kWarpSize; // 0~31
5125     // Split-Q warp layout: 8 warps form 8x1 grid (warp_QP = 0~7, warp_KV = 0)
5126     // Br = 16 * 8 * 1 = 128 → 8 warps each handle 16 rows of Q
5127     const int warp_QP = warp_id;
5128     const int warp_KV = 0;
5129
5130     // Step C0: init — mark all K stages and V stages as "empty" (consumable by producer)
5131     // Consumer register budget per Triton flash_attn_v2 maxnreg strategy on
5132     // Blackwell warp_specialize: D=128 -> 168, otherwise -> 80.
5133     if constexpr (kHeadDim == 128) {
5134         NOTES_V2_REG_ALLOC(168);
5135     } else {
5136         NOTES_V2_REG_ALLOC(80);
5137     }
5138
5139     for (int s = 0; s < kStagesK; ++s) {
5140         [[maybe_unused]] auto token = empty_K[s].arrive();
5141     }
5142     for (int s = 0; s < kStagesV; ++s) {
5143         [[maybe_unused]] auto token = empty_V[s].arrive();
5144     }
5145
5146     // Wait for Q tile ready once (split-Q: Q smem is reused across all KV iters)
5147     full_Q.wait(full_Q.arrive());
5148
5149     // Online softmax persistent state (per-warp, per-row-pair)
5150     float lane_block_row_max_old[kValTileSeqLenQ][2];
5151     float lane_block_row_sum_old[kValTileSeqLenQ][2];
5152     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
5153     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
5154
5155     uint32_t R_Q[kValTileSeqLenQ][4];
5156     uint32_t R_K[kValTileSeqLenK][2];
5157     uint32_t R_V[kValTileHeadDimV][2];
5158     uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs];
5159     uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
5160     uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
5161
5162     fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);
5163
5164     // ---- Main loop over KV seqlen ----
5165     for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {

```

```

5166     const int stage = tile_K_seqlen % kStagesK;
5167     const int stage_v = tile_K_seqlen % kStagesV;
5168
5169     // Wait for K[stage] ready
5170     full_K[stage].wait(full_K[stage].arrive());
5171     tma_fence_proxy_async_shared_cta();
5172
5173     // ---- 3b: Q @ K^T = S[Br, Bc] ----
5174     // Q smem reused (split-Q). K from stage `stage`.
5175     // smem addr (TMA 128B swizzle, swizzle<64>)
5176     fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
5177 #pragma unroll
5178     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
5179         // ldmatrix.x4: load Q m16k16 fragment (Q row-major, non-transpose)
5180 #pragma unroll
5181         for (int i = 0; i < kValTileSeqLenQ; ++i) {
5182             const int warp_smem_Q_Br =
5183                 warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
5184             const int lane_smem_Q_Br = warp_smem_Q_Br + lane_id % 16;
5185             const int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8;
5186             const uint32_t lane_smem_Q_ptr =
5187                 smem_Q_base_ptr +
5188                 swizzle_fa<kHeadDim, Br>(lane_smem_Q_Br, lane_smem_Q_d) *
5189                     sizeof(half);
5190             LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
5191                 lane_smem_Q_ptr);
5192         }
5193
5194         // ldmatrix.x2: load K k16n8 fragment (K row-major = K^T col-major)
5195 #pragma unroll
5196         for (int j = 0; j < kValTileSeqLenK; ++j) {
5197             const int warp_smem_K_Bc =
5198                 warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
5199             const int lane_smem_K_Bc = warp_smem_K_Bc + lane_id % 8;
5200             const int lane_smem_K_d =
5201                 tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8;
5202             const uint32_t lane_smem_K_ptr =
5203                 smem_K_base_ptr +
5204                 (stage * Bc * kHeadDim +
5205                     swizzle_fa<kHeadDim, Bc>(lane_smem_K_Bc, lane_smem_K_d)) *
5206                     sizeof(half);
5207             LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
5208         }
5209
5210 #pragma unroll
5211         for (int i = 0; i < kValTileSeqLenQ; ++i) {
5212 #pragma unroll
5213             for (int j = 0; j < kValTileSeqLenK; ++j) {
5214                 if constexpr (kMmaAccF32) {
5215                     HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
5216                         R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
5217                         R_K[j][0], R_K[j][1],
5218                         R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
5219                 } else {
5220                     HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
5221                         R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
5222                         R_S[i][j][0], R_S[i][j][1]);
5223                 }
5224             }
5225         }
5226     }
5227
5228     // Release K[stage] for producer reuse as early as possible: QK^T GEMM has

```

```

5229 // consumed all K smem data; the subsequent softmax (3c) and PV GEMM (3d)
5230 // only touch registers (R_S, R_0) and V smem, so K[stage] is free to be
5231 // overwritten by the producer's next TMA prefetch.
5232 {
5233     [[maybe_unused]] auto token = empty_K[stage].arrive();
5234 }
5235
5236 // ---- 3c: Online Safe Softmax — row max + exp + sum, P back to R_S ----
5237 // Softmax only touches R_S (registers), so V TMA latency can be overlapped
5238 // with this compute. Defer full_V.wait() until just before P@V (3d).
5239 float lane_row_max_new[kValTileSeqLenQ][2];
5240 float lane_row_sum_new[kValTileSeqLenQ][2];
5241 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
5242 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
5243
5244 // Pass 1: row-wise max across kValTileSeqLenK tiles (warp_reduce_max<4>)
5245 #pragma unroll
5246 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5247     #pragma unroll
5248     for (int j = 0; j < kValTileSeqLenK; ++j) {
5249         float tmp_max_0, tmp_max_1;
5250         if constexpr (kMmaAccF32) {
5251             float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
5252             tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
5253             tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
5254         } else {
5255             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
5256             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
5257             tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
5258             tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
5259         }
5260         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
5261         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
5262     }
5263     lane_row_max_new[i][0] =
5264         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
5265     lane_row_max_new[i][1] =
5266         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
5267 }
5268
5269 // Pass 2: P = exp(S*scale - m_new), store back to R_S, accumulate row sums
5270 #pragma unroll
5271 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5272     float block_row_max_new_0 =
5273         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
5274     float block_row_max_new_1 =
5275         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
5276
5277     #pragma unroll
5278     for (int j = 0; j < kValTileSeqLenK; ++j) {
5279         if constexpr (kMmaAccF32) {
5280             float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
5281             half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
5282             t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
5283             t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
5284             t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
5285             t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
5286             lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
5287             lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
5288             t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
5289             t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
5290             t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
5291             t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);

```

```

5292     } else {
5293         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
5294         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
5295         t_reg_S_0.x = __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
5296         t_reg_S_0.y = __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
5297         t_reg_S_1.x = __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
5298         t_reg_S_1.y = __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
5299         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
5300         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
5301         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
5302         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
5303     }
5304 }
5305 lane_row_sum_new[i][0] =
5306     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
5307 lane_row_sum_new[i][1] =
5308     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
5309 }
5310
5311 // ---- 3d: P @ V = 0[Br, d] ----
5312 // Wait for V[stage_v] ready just before P@V.
5313 // kStagesV=1: V was loaded in same iter, TMA overlaps QK+softmax.
5314 // kStagesV>=2: V was loaded in previous iter, TMA overlaps QK+softmax+PV.
5315 full_V[stage_v].wait(full_V[stage_v].arrive());
5316 tma_fence_proxy_async_shared_cta();
5317
5318 // ldmatrix.x2.trans: V row-major → col-major B fragment for NN matmul
5319 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_0, 0);
5320 #pragma unroll
5321 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
5322     #pragma unroll
5323     for (int j = 0; j < kValTileHeadDimV; ++j) {
5324         const int warp_smem_V_d =
5325             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
5326         const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
5327         const int lane_smem_V_d = warp_smem_V_d;
5328         const uint32_t lane_smem_V_ptr =
5329             smem_V_base_ptr +
5330             (stage_v * Bc * kHeadDim +
5331              swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d)) *
5332             sizeof(half);
5333         LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
5334     }
5335
5336     // P fragment (R_S) directly feeds A of P@V MMA (layout reuse trick)
5337     const int w = tile_V_Bc * 2;
5338     #pragma unroll
5339     for (int i = 0; i < kValTileSeqLenP; ++i) {
5340         #pragma unroll
5341         for (int j = 0; j < kValTileHeadDimV; ++j) {
5342             if constexpr (kMmaAccF32) {
5343                 HMMA16816F32(R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3],
5344                             R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
5345                             R_V[j][0], R_V[j][1],
5346                             R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3]);
5347             } else {
5348                 HMMA16816(R_0[i][j][0], R_0[i][j][1],
5349                           R_S[i][w][0], R_S[i][w][1],
5350                           R_S[i][w + 1][0], R_S[i][w + 1][1],
5351                           R_V[j][0], R_V[j][1],
5352                           R_0[i][j][0], R_0[i][j][1]);
5353             }
5354         }
5355     }

```

```

5355     }
5356 }
5357
5358 // Release V[stage_v] for producer reuse: PV GEMM has consumed all
5359 // V smem data; the subsequent online rescaling (3e) only touches registers
5360 // (R_0, R_D, R_S, lane_block_*), so V[stage_v] is free to be overwritten.
5361 {
5362     [[maybe_unused]] auto token = empty_V[stage_v].arrive();
5363 }
5364
5365 // ---- 3e: Online rescaling —  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$  ----
5366 #pragma unroll
5367 for (int i = 0; i < kValTileSeqLenP; ++i) {
5368     float block_row_max_new_0 = lane_row_max_new[i][0];
5369     float block_row_max_new_1 = lane_row_max_new[i][1];
5370     float block_row_sum_new_0 = lane_row_sum_new[i][0];
5371     float block_row_sum_new_1 = lane_row_sum_new[i][1];
5372     float block_row_max_old_0 = lane_block_row_max_old[i][0];
5373     float block_row_max_old_1 = lane_block_row_max_old[i][1];
5374
5375     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
5376     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
5377
5378     // First iteration: m_old = -inf, use m_new directly
5379     block_row_max_old_0 = (tile_K_seqlen > 0 ? block_row_max_old_0
5380                                     : block_row_max_new_0);
5381     block_row_max_old_1 = (tile_K_seqlen > 0 ? block_row_max_old_1
5382                                     : block_row_max_new_1);
5383
5384     float rescale_o_factor_0 =
5385         __expf(block_row_max_old_0 - block_row_max_new_0);
5386     float rescale_o_factor_1 =
5387         __expf(block_row_max_old_1 - block_row_max_new_1);
5388
5389 #pragma unroll
5390 for (int j = 0; j < kValTileHeadDimV; ++j) {
5391     if constexpr (kMmaAccF32) {
5392         float *t_fptr_0 = reinterpret_cast<float *>(&R_0[i][j][0]);
5393         float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
5394         t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_0[0]);
5395         t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_0[1]);
5396         t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_0[2]);
5397         t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_0[3]);
5398     } else {
5399         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
5400         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
5401         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
5402         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
5403         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
5404         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
5405         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
5406         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
5407         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
5408         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
5409     }
5410 }
5411
5412 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
5413 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
5414 lane_block_row_sum_old[i][0] =
5415     __fmaf_rn(rescale_o_factor_0, block_row_sum_old_0,
5416             block_row_sum_new_0);
5417 lane_block_row_sum_old[i][1] =

```

```

5418     __fmaf_rn(rescale_o_factor_1, block_row_sum_old_1,
5419               block_row_sum_new_1);
5420     lane_block_row_max_old[i][0] = block_row_max_new_0;
5421     lane_block_row_max_old[i][1] = block_row_max_new_1;
5422 }
5423 }
5424
5425 // ---- Step 4: Final rescale — O_final = (1/l_final) * O_final ----
5426 #pragma unroll
5427 for (int i = 0; i < kValTileSeqLenP; ++i) {
5428     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
5429     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
5430 #pragma unroll
5431 for (int j = 0; j < kValTileHeadDimV; ++j) {
5432     if constexpr (kMmaAccF32) {
5433         float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
5434         half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
5435         t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
5436         t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
5437         t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
5438         t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
5439     } else {
5440         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
5441         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
5442         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
5443         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
5444         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
5445         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
5446         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
5447         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
5448     }
5449 }
5450 }
5451
5452 // ---- Step 5: Epilogue — warp shuffle + 128-bit collective store ----
5453 #pragma unroll
5454 for (int i = 0; i < kValTileSeqLenP; ++i) {
5455 #pragma unroll
5456 for (int j = 0; j < kValTileHeadDimV; ++j) {
5457     uint32_t R_Z[2][4];
5458     R_Z[0][0] = R_D[i][j][0];
5459     R_Z[1][0] = R_D[i][j][1];
5460     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
5461     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
5462     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
5463     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
5464     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
5465     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
5466
5467     if (lane_id % 4 == 0) {
5468         const int store_warp_regs_0_Br =
5469             warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
5470         const int store_lane_gmem_0_Br =
5471             Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
5472         const int store_warp_regs_0_d =
5473             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
5474         const int store_lane_gmem_0_d = store_warp_regs_0_d;
5475         const int store_gmem_0_addr_0 =
5476             0_gmem_offset + store_lane_gmem_0_Br * kHeadDim +
5477             store_lane_gmem_0_d;
5478         const int store_gmem_0_addr_1 =
5479             0_gmem_offset + (store_lane_gmem_0_Br + 8) * kHeadDim +
5480             store_lane_gmem_0_d;

```

```

5481     *reinterpret_cast<float4 *>(&[store_gmem_0_addr_0]) =
5482     *reinterpret_cast<float4 *>(&R_Z[0][0]);
5483     *reinterpret_cast<float4 *>(&[store_gmem_0_addr_1]) =
5484     *reinterpret_cast<float4 *>(&R_Z[1][0]);
5485 }
5486 }
5487 }
5488 }
5489 }
5490
5491 // =====
5492 // Phase 8c: FlashAttention-3-style TMA + MMA Warp Specialization (SM120)
5493 //   Dual 128-thread consumer warpgroups processing disjoint KV tiles,
5494 //   with per-WG independent K pipeline (kStagesK) and single V buffer (kStagesV=1).
5495 //
5496 // 与 flash_attn_tma_mma_ws_stages_split_q (Phase 8b) 的核心区别:
5497 //   - 8b: 1 producer WG (128T) + 1 consumer WG (256T, 8 warps), 全 KV 遍历
5498 //   - 8c: 1 producer WG (128T) + 2 consumer WGs (各128T, 4 warps), KV tile 奇偶拆分
5499 //
5500 // 线程布局 (384 threads):
5501 //   WG0 [0,127]:   TMA producer (仅 thread 0 发 TMA)
5502 //   WG1 [128,255]: consumer_id=0, 处理偶数 KV tile (0,2,4,...)
5503 //   WG2 [256,383]: consumer_id=1, 处理奇数 KV tile (1,3,5,...)
5504 //
5505 // per-WG 独立 stage 语义 (关键改进):
5506 // -----
5507 // 每个 consumer WG 拥有独立的 K stage slots (kStagesK 个) 和 1 个 V buffer。
5508 // K/V smem 按 consumer_id 分 bank: K_smem[cid][stage], V_smem[cid]。
5509 // Barrier 也按 consumer_id 隔离: full_K[cid][s], empty_K[cid][s], full_V[cid], empty_V[cid]。
5510 // 两个 WG 绝不争抢同一 smem slot 或 barrier, producer 通过 cid = tile&1 路由 TMA。
5511 //
5512 // 这样 stage 语义从 "跨 WG 共享的 2 slot (每 WG 实际 depth=1)" 变为
5513 // "每 WG 独立拥有 kStagesK 个 K slot (每 WG 真正 depth=kStagesK)"。
5514 //
5515 // Pipeline 时序 (Sk=1, Sv=1):
5516 //   K[tile] TMA 被 softmax+PV+rescale 隐藏 (Sk=1 复用同 slot, 用完即释放);
5517 //   V[tile] TMA 被 rescale+QK[next]+softmax[next] 隐藏 (Sv=1 同理)。
5518 //
5519 // Pipeline 时序 (Sk=2, Sv=1, D=64 only):
5520 //   K[tile+2] 在 QK[tile] 之前就 prefetch 到另一个 slot, 被整个 tile compute 隐藏;
5521 //   V[tile] TMA 同 Sk=1, 被 QK+softmax 隐藏。
5522 //
5523 // 为什么是奇偶 tile 拆分 (而非前/后半区各自一半)?
5524 // -----
5525 // 关键约束: producer 按 global tile 顺序 0,1,2,3,... 连续 TMA。
5526 // 奇偶拆分让 cid = tile & 1 静态路由: tile 0,2,4,... -> WG1; tile 1,3,5,... -> WG2。
5527 // 每 WG 的下一个 tile (tile+2) 正好是 producer 当前/下一轮要加载的 tile,
5528 // 保证 producer/consumer 流水无停顿。
5529 //
5530 // 若改为前/后半区拆分: 过渡区两个 WG 可能同时需要同一 cid 的 slot,
5531 // 且 WG2 跳跃访问远端 tile 时数据尚未被 producer TMA 到 smem。
5532 //
5533 // Partial state merge (Split-KV / FlashDecoding-style reduction):
5534 // -----
5535 // 每个 WG 各自处理一半 KV tile 序列, 独立维护 "未归一化" 的 online-softmax
5536 // 中间状态 (m, l, 0acc), 合并后等价于全 KV 遍历。
5537 //
5538 // 稳定归并公式 (避免 exp 溢出):
5539 //   m      = max(m_0, m_1)           // 全局 row max
5540 //   alpha  = exp(m_0 - m) (<=1)       // WG1 rescale factor
5541 //   beta   = exp(m_1 - m) (<=1)       // WG2 rescale factor
5542 //   l      = alpha * l_0 + beta * l_1
5543 //   0acc   = alpha * 0acc_0 + beta * 0acc_1

```

```

5544 // 0 = 0acc / l // 最终归一化
5545 //
5546 // Tc==1 退化: WG2 无 tile, m_l=-inf, l_l=0, 0acc_1=0;
5547 // beta = exp(-inf) = 0, 退化为 WG1 结果。
5548 //
5549 // 限制 (首版):
5550 // - kStagesV == 1 (V 只需 1 buffer, TMA 被 QK+softmax 隐藏)
5551 // - kStagesK >= 1 (K pipeline depth; D=128 在 SM120 只支持 Sk=1)
5552 // - D=128 时 Sk=1 (Sk=2 需 112KB > 101KB optin 上限)
5553 // - Br=64, Bc=64, D=64/128, aligned seqlen (N % Br == 0 && N % Bc == 0)
5554 // - 仅 self-attention, 无 causal/varlen/GQA
5555 // =====
5556 template <
5557     const int kHeadDim,
5558     const int kMmaAtomM, const int kMmaAtomN, const int kMmaAtomK,
5559     const int kMmaAccF32,
5560     const int kMmaTileSeqLenQ, const int kMmaTileSeqLenK,
5561     const int kMmaTileSeqLenP, const int kMmaTileHeadDimV,
5562     const int kValTileSeqLenQ, const int kValTileSeqLenK,
5563     const int kValTileSeqLenP, const int kValTileHeadDimV,
5564     const int kStagesK, const int kStagesV,
5565     const int kNumThreads>
5566 __global__ void __launch_bounds__(kNumThreads, 1)
5567     flash_attn_3_tma_ws_stages_split_q(
5568         half *Q, half *K, half *V, half *O, int N, int H,
5569         const UTensorMap *__restrict__ tensorMapQ,
5570         const UTensorMap *__restrict__ tensorMapK,
5571         const UTensorMap *__restrict__ tensorMapV) {
5572     static_assert(kHeadDim == 64 || kHeadDim == 128,
5573         "v1 supports kHeadDim == 64 or 128");
5574     static_assert(kNumThreads == 384, "128 producer + 256 consumer (2 WGs)");
5575     static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
5576     static_assert(kStagesV == 1, "per-WG V pipeline depth must be 1");
5577     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
5578     constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
5579
5580     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
5581     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
5582     constexpr int kConsumerThreadsPerWG = 128;
5583     constexpr int kProducerThreads = 128;
5584     constexpr int kNumConsumerWGs = 2;
5585     constexpr int kQTileBytes = Br * kHeadDim * sizeof(half);
5586     constexpr int kKTileBytes = Bc * kHeadDim * sizeof(half);
5587     constexpr int kVTileBytes = Bc * kHeadDim * sizeof(half);
5588     constexpr int kTmaBoxMinor = 64;
5589     constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
5590
5591     const int Nb_id = blockIdx.y / H;
5592     const int Nh_id = blockIdx.y % H;
5593     const int Q_tile_id = blockIdx.x;
5594     const int Tc = (N + Bc - 1) / Bc;
5595     const float scale = 1.0f / sqrtf((float)kHeadDim);
5596
5597     const int QKV_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
5598     const int O_gmem_offset = QKV_gmem_offset;
5599     const int qkv_major_base = (Nb_id * H + Nh_id) * N;
5600
5601     // ---- Shared memory layout (per-WG independent K/V banks) ----
5602     // [Q_shared: Br*D]
5603     // [K[cid=0][s=0..Sk-1]: Sk*Bc*D] [K[cid=1][s=0..Sk-1]]
5604     // [V[cid=0]: Bc*D] [V[cid=1]: Bc*D]
5605     extern __shared__ __align__(1024) uint8_t smem_fa3_tma_ws[];
5606     half *Q_smem = reinterpret_cast<half *>(smem_fa3_tma_ws);

```

```

5607     half *K_smem_base = Q_smem + Br * kHeadDim;
5608     // K: [numConsumerWGs][kStagesK][Bc*kHeadDim], linearized
5609     //   K_smem_base + cid * (kStagesK * Bc * kHeadDim) + stg * (Bc * kHeadDim)
5610     half *V_smem_base = K_smem_base + kNumConsumerWGs * kStagesK * Bc * kHeadDim;
5611     // V: [numConsumerWGs][Bc*kHeadDim]
5612     //   V_smem_base + cid * (Bc * kHeadDim)
5613
5614     const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
5615     const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem_base);
5616     const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem_base);
5617
5618     #pragma nv_diag_suppress static_var_with_dynamic_init
5619     __shared__ cuda::barrier<cuda::thread_scope_block> full_Q;
5620     // per-WG K barriers: [cid][stage]
5621     __shared__ cuda::barrier<cuda::thread_scope_block> full_K[kNumConsumerWGs][kStagesK];
5622     __shared__ cuda::barrier<cuda::thread_scope_block> empty_K[kNumConsumerWGs][kStagesK];
5623     // per-WG V barriers: [cid] (kStagesV=1)
5624     __shared__ cuda::barrier<cuda::thread_scope_block> full_V[kNumConsumerWGs];
5625     __shared__ cuda::barrier<cuda::thread_scope_block> empty_V[kNumConsumerWGs];
5626     #pragma nv_diag_default static_var_with_dynamic_init
5627
5628     const bool is_producer = threadIdx.x < kProducerThreads;
5629     const int consumer_id = is_producer ? 0
5630         : (threadIdx.x - kProducerThreads) / kConsumerThreadsPerWG;
5631     const int wg_tid = is_producer ? threadIdx.x
5632         : (threadIdx.x - kProducerThreads) % kConsumerThreadsPerWG;
5633
5634     if (threadIdx.x == 0) {
5635         init(&full_Q, 256 + 1); // 2 consumer WGs * 128 + 1 producer
5636         for (int cid = 0; cid < kNumConsumerWGs; ++cid) {
5637             for (int s = 0; s < kStagesK; ++s) {
5638                 init(&full_K[cid][s], kConsumerThreadsPerWG + 1);
5639                 init(&empty_K[cid][s], kConsumerThreadsPerWG + 1);
5640             }
5641             init(&full_V[cid], kConsumerThreadsPerWG + 1);
5642             init(&empty_V[cid], kConsumerThreadsPerWG + 1);
5643         }
5644         tma_fence_proxy_async_shared_cta();
5645     }
5646     __syncthreads();
5647
5648     // Helper: K_smem ptr for (cid, stg)
5649     // K layout: K_smem_base + cid*(Sk*Bc*D) + stg*(Bc*D)
5650     // D=128 chunk-major: each tile's 2 chunks at stg*Bc*D + c*Bc*64
5651
5652     // =====
5653     // Producer Warpgroup (WG0, threadIdx.x 0~127)
5654     // Single global tile loop over all KV tiles (0..Tc-1).
5655     // cid = tile & 1 routes TMA to the owning consumer WG's smem bank.
5656     //
5657     // Per-tile load order: V first, then K (prefetch next).
5658     //   - V[tile] TMA: blocked until consumer releases V[cid] (after PV).
5659     //     But K[tile] signal already sent in warmup/prev iter, so consumer
5660     //     can do QK[tile] while producer waits for V release.
5661     //   - K[tile+2] prefetch (Sk>=2): blocked until consumer releases K[cid][stg_next]
5662     //     (after QK). QK is first compute step, always done before producer reaches here.
5663     // =====
5664     if (is_producer) {
5665         NOTES_V2_REG_DEALLOC(40);
5666         if (wg_tid == 0) {
5667             // P0: Load Q tile once
5668             for (int c = 0; c < kTmaChunks; ++c) {
5669                 tma_load_2d(Q_smem + c * Br * kTmaBoxMinor, tensorMapQ,

```

```

5670         c * kTmaBoxMinor, qkv_major_base + Q_tile_id * Br, full_Q);
5671     }
5672     tma_arrive_expect_tx(full_Q, kQTileBytes);
5673
5674     // P1: Warmup - prefetch first K tile for each consumer WG (Sk-1 tiles)
5675     // tile 0 -> cid=0, tile 1 -> cid=1
5676     if constexpr (kStagesK > 1) {
5677         for (int cid = 0; cid < kNumConsumerWGs; ++cid) {
5678             const int tile0 = cid; // first tile for this WG
5679             if (tile0 < Tc) {
5680                 const int stg = 0; // first slot
5681                 empty_K[cid][stg].wait(empty_K[cid][stg].arrive());
5682                 for (int c = 0; c < kTmaChunks; ++c) {
5683                     tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
5684                               stg * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5685                               tensorMapK, c * kTmaBoxMinor,
5686                               qkv_major_base + tile0 * Bc, full_K[cid][stg]);
5687                 }
5688                 tma_arrive_expect_tx(full_K[cid][stg], kKTileBytes);
5689             }
5690         }
5691     }
5692
5693     // P2: Main loop over all global KV tiles
5694     // K 优先于 V: K 的 release 依赖 QK (tile 第一步 compute, 早完成),
5695     // V 的 release 依赖 PV (tile 最后一步 compute, 晚完成)。
5696     // 先发 K 让 consumer 尽早开始 QK, V 的 TMA 在 K 之后发出,
5697     // 被 QK+softmax 的 compute 时间隐藏。
5698     for (int tile = 0; tile < Tc; ++tile) {
5699         const int cid = tile & 1;
5700
5701         // Step 1: Load/prefetch K first (release depends on QK, which is early)
5702         if constexpr (kStagesK == 1) {
5703             // Sk=1: load K[tile] to the single slot (reuse after consumer QK release)
5704             empty_K[cid][0].wait(empty_K[cid][0].arrive());
5705             for (int c = 0; c < kTmaChunks; ++c) {
5706                 tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
5707                           c * Bc * kTmaBoxMinor,
5708                           tensorMapK, c * kTmaBoxMinor,
5709                           qkv_major_base + tile * Bc, full_K[cid][0]);
5710             }
5711             tma_arrive_expect_tx(full_K[cid][0], kKTileBytes);
5712         } else {
5713             // Sk>=2: prefetch K[tile+2] to next slot (true ping-pong pipeline!)
5714             const int local_tile = (tile - cid) / 2;
5715             const int stg_next = (local_tile + 1) % kStagesK;
5716             const int next_tile = tile + 2;
5717             if (next_tile < Tc) {
5718                 empty_K[cid][stg_next].wait(empty_K[cid][stg_next].arrive());
5719                 for (int c = 0; c < kTmaChunks; ++c) {
5720                     tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
5721                               stg_next * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5722                               tensorMapK, c * kTmaBoxMinor,
5723                               qkv_major_base + next_tile * Bc,
5724                               full_K[cid][stg_next]);
5725                 }
5726                 tma_arrive_expect_tx(full_K[cid][stg_next], kKTileBytes);
5727             }
5728         }
5729
5730         // Step 2: Load V[tile] (release depends on PV, which is late)
5731         empty_V[cid].wait(empty_V[cid].arrive());
5732         for (int c = 0; c < kTmaChunks; ++c) {

```

```

5733         tma_load_2d(V_smem_base + cid * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
5734                     tensorMapV, c * kTmaBoxMinor,
5735                     qkv_major_base + tile * Bc, full_V[cid]);
5736     }
5737     tma_arrive_expect_tx(full_V[cid], kVTileBytes);
5738 }
5739 }
5740 }
5741 // =====
5742 // Consumer Warpgroups (WG1 consumer_id=0, WG2 consumer_id=1)
5743 // Each WG processes every other KV tile (consumer_id, consumer_id+2, ...).
5744 // K stage index: local_iter % kStagesK (Sk=1: always 0; Sk=2: 0,1,0,1,...)
5745 // V stage index: always consumer_id (only 1 V buffer per WG)
5746 // =====
5747 else {
5748     const int warp_id = wg_tid / kWarpSize; // 0~3
5749     const int lane_id = wg_tid % kWarpSize; // 0~31
5750     const int warp_QP = warp_id;
5751     const int warp_KV = 0;
5752
5753     if constexpr (kHeadDim == 128) {
5754         NOTES_V2_REG_ALLOC(168);
5755     } else {
5756         NOTES_V2_REG_ALLOC(80);
5757     }
5758
5759     // C0: init - mark OWN K/V slots as empty (consumer arrives, producer is 129th)
5760     for (int s = 0; s < kStagesK; ++s) {
5761         [[maybe_unused]] auto tk = empty_K[consumer_id][s].arrive();
5762     }
5763     {
5764         [[maybe_unused]] auto tv = empty_V[consumer_id].arrive();
5765     }
5766
5767     // Wait for Q tile (shared by both WGs)
5768     full_Q.wait(full_Q.arrive());
5769
5770     // Partial online-softmax state (per-WG, independent)
5771     float lane_block_row_max_old[kValTileSeqLenQ][2];
5772     float lane_block_row_sum_old[kValTileSeqLenQ][2];
5773     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
5774     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
5775
5776     uint32_t R_Q[kValTileSeqLenQ][4];
5777     uint32_t R_K[kValTileSeqLenK][2];
5778     uint32_t R_V[kValTileHeadDimV][2];
5779     uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs];
5780     uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
5781     uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
5782     fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);
5783
5784     // K smem offset for this WG: base + cid*(Sk*Bc*D)
5785     // stg within: local_iter % kStagesK
5786     // V smem offset for this WG: V_smem_base + cid*(Bc*D)
5787     const uint32_t smem_K_wg_base = smem_K_base_ptr +
5788         consumer_id * (kStagesK * Bc * kHeadDim) * sizeof(half);
5789     const uint32_t smem_V_wg_base = smem_V_base_ptr +
5790         consumer_id * (Bc * kHeadDim) * sizeof(half);
5791
5792     int local_iter = 0;
5793     for (int tile = consumer_id; tile < Tc; tile += 2) {
5794         const int k_stg = local_iter % kStagesK;
5795

```

```

5796 // K smem offset for this stage: wg_base + stg*(Bc*D)
5797 const uint32_t smem_K_stage_ptr = smem_K_wg_base +
5798     k_stg * Bc * kHeadDim * sizeof(half);
5799
5800 // Wait for K[consumer_id][k_stg] ready
5801 full_K[consumer_id][k_stg].wait(full_K[consumer_id][k_stg].arrive());
5802 tma_fence_proxy_async_shared_cta();
5803
5804 // ---- Q @ K^T = S[Br, Bc] ----
5805 // K early release: 在最后一个 tile_K_d 的 ldmatrix_K 完成后立即 release K,
5806 // 不等整个 QK MMA 循环结束。K 数据已全部读入 R_K 寄存器, smem 可被
5807 // producer 复用。这对 Sk=1 尤其重要: producer 可更早开始下一个 K TMA。
5808 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
5809 #pragma unroll
5810 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
5811 #pragma unroll
5812     for (int i = 0; i < kValTileSeqLenQ; ++i) {
5813         const int warp_smem_Q_Br =
5814             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
5815         const int lane_smem_Q_Br = warp_smem_Q_Br + lane_id % 16;
5816         const int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8;
5817         const uint32_t lane_smem_Q_ptr =
5818             smem_Q_base_ptr +
5819             swizzle_fa<kHeadDim, Br>(lane_smem_Q_Br, lane_smem_Q_d) *
5820                 sizeof(half);
5821         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
5822             lane_smem_Q_ptr);
5823     }
5824
5825 #pragma unroll
5826     for (int j = 0; j < kValTileSeqLenK; ++j) {
5827         const int warp_smem_K_Bc =
5828             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
5829         const int lane_smem_K_Bc = warp_smem_K_Bc + lane_id % 8;
5830         const int lane_smem_K_d =
5831             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8;
5832         // K smem: stage_ptr + swizzle_fa offset (no stage*Bc*D since stage_ptr already includes it)
5833         const uint32_t lane_smem_K_ptr =
5834             smem_K_stage_ptr +
5835             swizzle_fa<kHeadDim, Bc>(lane_smem_K_Bc, lane_smem_K_d) *
5836                 sizeof(half);
5837         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
5838     }
5839
5840 // Early release K after last ldmatrix_K: K data now fully in registers
5841 if (tile_K_d == (kHeadDim / kMmaAtomK) - 1) {
5842     [[maybe_unused]] auto token = empty_K[consumer_id][k_stg].arrive();
5843 }
5844
5845 #pragma unroll
5846 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5847 #pragma unroll
5848     for (int j = 0; j < kValTileSeqLenK; ++j) {
5849         if constexpr (kMmaAccF32) {
5850             HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
5851                 R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
5852                 R_K[j][0], R_K[j][1],
5853                 R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
5854         } else {
5855             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
5856                 R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
5857                 R_S[i][j][0], R_S[i][j][1]);
5858         }

```

```

5859     }
5860 }
5861 }
5862
5863 // K already released above (after last ldmatrix_K)
5864
5865 // ---- Online Safe Softmax ----
5866 // This compute overlaps with producer's V[tile] TMA (which waits for
5867 // empty_V from previous PV, or is already in-flight)
5868 float lane_row_max_new[kValTileSeqLenQ][2];
5869 float lane_row_sum_new[kValTileSeqLenQ][2];
5870 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
5871 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
5872
5873 #pragma unroll
5874 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5875 #pragma unroll
5876 for (int j = 0; j < kValTileSeqLenK; ++j) {
5877     float tmp_max_0, tmp_max_1;
5878     if constexpr (kMmaAccF32) {
5879         float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
5880         tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
5881         tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
5882     } else {
5883         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
5884         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
5885         tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
5886         tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
5887     }
5888     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
5889     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
5890 }
5891 lane_row_max_new[i][0] =
5892     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
5893 lane_row_max_new[i][1] =
5894     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
5895 }
5896
5897 #pragma unroll
5898 for (int i = 0; i < kValTileSeqLenQ; ++i) {
5899     float block_row_max_new_0 =
5900         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
5901     float block_row_max_new_1 =
5902         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
5903
5904 #pragma unroll
5905 for (int j = 0; j < kValTileSeqLenK; ++j) {
5906     if constexpr (kMmaAccF32) {
5907         float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
5908         half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
5909         t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
5910         t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
5911         t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
5912         t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
5913         lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
5914         lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
5915         t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
5916         t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
5917         t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
5918         t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
5919     } else {
5920         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
5921         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));

```

```

5922         t_reg_S_0.x = __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
5923         t_reg_S_0.y = __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
5924         t_reg_S_1.x = __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
5925         t_reg_S_1.y = __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
5926         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
5927         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
5928         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
5929         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
5930     }
5931 }
5932 lane_row_sum_new[i][0] =
5933     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
5934 lane_row_sum_new[i][1] =
5935     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
5936 }
5937
5938 // ---- P @ V = 0[Br, d] ----
5939 // Wait for V[consumer_id] ready
5940 full_V[consumer_id].wait(full_V[consumer_id].arrive());
5941 tma_fence_proxy_async_shared_cta();
5942
5943 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_0, 0);
5944 #pragma unroll
5945 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
5946 #pragma unroll
5947     for (int j = 0; j < kValTileHeadDimV; ++j) {
5948         const int warp_smem_V_d =
5949             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
5950         const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
5951         const int lane_smem_V_d = warp_smem_V_d;
5952         // V smem: V_wg_base + swizzle_fa offset
5953         const uint32_t lane_smem_V_ptr =
5954             smem_V_wg_base +
5955             swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d) *
5956                 sizeof(half);
5957         LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
5958     }
5959
5960     // Early release V after last ldmatrix_V: V data now fully in registers.
5961     // This lets producer start next V TMA during the remaining PV MMA +
5962     // online rescaling, giving more overlap window than releasing after PV.
5963     if (tile_V_Bc == (Bc / kMmaAtomK) - 1) {
5964         [[maybe_unused]] auto token = empty_V[consumer_id].arrive();
5965     }
5966
5967     const int w = tile_V_Bc * 2;
5968 #pragma unroll
5969     for (int i = 0; i < kValTileSeqLenP; ++i) {
5970 #pragma unroll
5971         for (int j = 0; j < kValTileHeadDimV; ++j) {
5972             if constexpr (kMmaAccF32) {
5973                 HMMA16816F32(R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3],
5974                     R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
5975                     R_V[j][0], R_V[j][1],
5976                     R_0[i][j][0], R_0[i][j][1], R_0[i][j][2], R_0[i][j][3]);
5977             } else {
5978                 HMMA16816(R_0[i][j][0], R_0[i][j][1],
5979                     R_S[i][w][0], R_S[i][w][1],
5980                     R_S[i][w + 1][0], R_S[i][w + 1][1],
5981                     R_V[j][0], R_V[j][1],
5982                     R_0[i][j][0], R_0[i][j][1]);
5983             }
5984         }

```

```

5985     }
5986 }
5987
5988 // V already released above (after last ldmatrix_V)
5989
5990 // ---- Online rescaling -  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$  ----
5991 #pragma unroll
5992 for (int i = 0; i < kValTileSeqLenP; ++i) {
5993     float block_row_max_new_0 = lane_row_max_new[i][0];
5994     float block_row_max_new_1 = lane_row_max_new[i][1];
5995     float block_row_sum_new_0 = lane_row_sum_new[i][0];
5996     float block_row_sum_new_1 = lane_row_sum_new[i][1];
5997     float block_row_max_old_0 = lane_block_row_max_old[i][0];
5998     float block_row_max_old_1 = lane_block_row_max_old[i][1];
5999
6000     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
6001     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
6002
6003     // First iteration per WG:  $m_{\text{old}} = -\text{inf}$ , use  $m_{\text{new}}$  directly
6004     block_row_max_old_0 = (local_iter > 0 ? block_row_max_old_0
6005                                     : block_row_max_new_0);
6006     block_row_max_old_1 = (local_iter > 0 ? block_row_max_old_1
6007                                     : block_row_max_new_1);
6008
6009     float rescale_o_factor_0 =
6010         __expf(block_row_max_old_0 - block_row_max_new_0);
6011     float rescale_o_factor_1 =
6012         __expf(block_row_max_old_1 - block_row_max_new_1);
6013
6014 #pragma unroll
6015     for (int j = 0; j < kValTileHeadDimV; ++j) {
6016         if constexpr (kMmaAccF32) {
6017             float *t_fptr_0 = reinterpret_cast<float*>(&R_0[i][j][0]);
6018             float *t_fptr_D = reinterpret_cast<float*>(&R_D[i][j][0]);
6019             t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_0[0]);
6020             t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_0[1]);
6021             t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_0[2]);
6022             t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_0[3]);
6023         } else {
6024             float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
6025             float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
6026             float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
6027             float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
6028             t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
6029             t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
6030             t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
6031             t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
6032             HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
6033             HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
6034         }
6035     }
6036
6037     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
6038     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
6039     lane_block_row_sum_old[i][0] =
6040         __fmaf_rn(rescale_o_factor_0, block_row_sum_old_0,
6041                 block_row_sum_new_0);
6042     lane_block_row_sum_old[i][1] =
6043         __fmaf_rn(rescale_o_factor_1, block_row_sum_old_1,
6044                 block_row_sum_new_1);
6045     lane_block_row_max_old[i][0] = block_row_max_new_0;
6046     lane_block_row_max_old[i][1] = block_row_max_new_1;
6047 }

```

```

6048     ++local_iter;
6049 }
6050
6051 // =====
6052 // Final merge of two WG partial states (split-KV reduction)
6053 //
6054 // 安全 alias dynamic smem 的前提:
6055 // 1) producer 已退出 mainloop (所有 TMA 已 arrive_expect_tx);
6056 // 2) 两个 consumer 都已 release 自己的 K/V stage (QK 后 release K,
6057 //    PV 后 release V), 且 mainloop 结束后不再访问任何 K/V smem;
6058 // 3) Q_smem 在 mainloop 内只读, 此后也不再访问。
6059 // 故此时整个 dynamic smem (Q/K/V 区) 生命周期结束, 可安全重命名为
6060 // merge scratch, 不增加 launch smem_bytes。第一次 __syncthreads
6061 // 保证所有 TMA/ldmatrix 完成后才开始写 scratch。
6062 // 完整的 split-KV merge 数学推导见 kernel 头部注释。
6063 // =====
6064 __syncthreads();
6065
6066 // Merge scratch 布局 (按 wg_tid 索引, 每 WG 128 thread):
6067 // [merge_rd]: 128 * kRdPerThread 个 uint32 <- WG2 未归一化 R_D fragment
6068 // [merge_ml]: 128 * kValTileSeqLenQ 个 float4 <- WG2 的 [m0,m1,l0,l1] interleaved
6069 // WG1 用自己的 R_D/m/l 作为 (0acc_0, m_0, l_0), 读取 WG2 写出的作为
6070 // (0acc_1, m_1, l_1), 按 stable merge 公式归并后由 WG1 写最终 0。
6071 // m/l interleaved 为 float4 布局, 写入/读取各一次 128-bit store/load。
6072 constexpr int kRdPerThread = kValTileSeqLenP * kValTileHeadDimV * kNRegs;
6073 constexpr int kRdPackPerThread = kRdPerThread / 4; // float4 个数 (kNRegs 始终偶数)
6074 constexpr int kMlPackPerThread = kValTileSeqLenQ; // 每 thread 的 float4 个数
6075 uint32_t *merge_rd = reinterpret_cast<uint32_t*>(smem_fa3_tma_ws);
6076 float4 *merge_rd_pack = reinterpret_cast<float4*>(merge_rd);
6077 float4 *merge_ml_pack = merge_rd_pack + 128 * kRdPackPerThread;
6078
6079 // WG2 写出未归一化 partial (R_D, m, l) 到 scratch, 按 wg_tid 索引。
6080 // R_D: 以 float4 (128-bit) 连续写入, kRdPackPerThread 次。
6081 // m/l: interleaved 为 float4 [m0, m1, l0, l1], 每次 128-bit store。
6082 // 按 wg_tid 索引保证 WG1 的同一 lane 读到对应的 WG2 partial:
6083 // R_D[i][j][k] fragment 对应的 Q rows 与 lane_block_row_max/sum[i][0/1]
6084 // 的两组 rows (rows 0-7 / rows 8-15) 严格对齐, 故 WG1/WG2 同一
6085 // wg_tid 的 partial 覆盖相同 Q 行, 可直接逐元素归并。
6086 // Tc==1 时 WG2 未进入循环, m=-inf/l=0/R_D=0, merge 自然退化。
6087 // 写完后第二次 __syncthreads 让 WG1 看到 WG2 的全部写入。
6088 if (consumer_id == 1) {
6089     // R_D: 128-bit stores
6090     float4 *rd_dst = merge_rd_pack + wg_tid * kRdPackPerThread;
6091 #pragma unroll
6092     for (int idx = 0; idx < kRdPackPerThread; ++idx)
6093         rd_dst[idx] = reinterpret_cast<float4*>(&R_D[0][0][0])[idx];
6094     // m/l interleaved: [m0, m1, l0, l1] per i, 128-bit store
6095 #pragma unroll
6096     for (int i = 0; i < kValTileSeqLenQ; ++i) {
6097         float4 ml;
6098         ml.x = lane_block_row_max_old[i][0];
6099         ml.y = lane_block_row_max_old[i][1];
6100         ml.z = lane_block_row_sum_old[i][0];
6101         ml.w = lane_block_row_sum_old[i][1];
6102         merge_ml_pack[wg_tid * kMlPackPerThread + i] = ml;
6103     }
6104 }
6105 __syncthreads();
6106
6107 // WG1 merges WG2's partial into its own, normalizes, and writes 0
6108 if (consumer_id == 0) {
6109     // 把 WG2 的未归一化 R_D (0acc_1) 载入 R_0 寄存器复用为临时 buffer。
6110     // R_D: 以 float4 (128-bit) 连续读取。

```

```

6111 // 后续 merge 用 R_D[i][j] 作为 0acc_0, R_0[i][j] 作为 0acc_1, 逐元素加权后
6112 // 结果写回 R_D, 再做最终 0=0acc/l 归一化。
6113 float4 *rd_src = merge_rd_pack + wg_tid * kRdPackPerThread;
6114 #pragma unroll
6115 for (int idx = 0; idx < kRdPackPerThread; ++idx)
6116     reinterpret_cast<float4 *>(&R_0[0][0][0])[idx] = rd_src[idx];
6117
6118 // 稳定归并 (实现层面, 完整数学见头部注释):
6119 // - m0/l0 = WG1 自己的 partial (寄存器), m1/l1 = WG2 的 partial (scratch)
6120 // - [i][0] 对应 Q rows [i*16 .. i*16+7], [i][1] 对应 rows [i*16+8 .. +15]
6121 // - alpha=exp(m0-m)<=1, beta=exp(m1-m)<=1: 用全局 max 做 pivot 避免 exp 溢出
6122 // - F32Acc: 直接对 float fragment 逐元素加权求和
6123 // - F16Acc: 把 packed half2 提升到 float2 计算, 再 pack 回 half2
6124 //   (避免 half 精度下 alpha/beta 加权累加的精度损失)
6125 // - Tc==1: m1=-inf -> beta=exp(-inf)=0, 0acc/0acc_1 项归零, 退化为 WG1 结果
6126 // m=max(m0,m1), a=exp(m0-m), b=exp(m1-m),
6127 // l=a*l0+b*l1, 0acc=a*0acc0+b*0acc1
6128 #pragma unroll
6129 for (int i = 0; i < kValTileSeqLenQ; ++i) {
6130     // 128-bit load: [m1_0, m1_1, l1_0, l1_1] interleaved
6131     float4 m1l = merge_ml_pack[wg_tid * kMlPackPerThread + i];
6132     float m1_0 = m1l.x;
6133     float m1_1 = m1l.y;
6134     float l1_0 = m1l.z;
6135     float l1_1 = m1l.w;
6136
6137     float m0_0 = lane_block_row_max_old[i][0];
6138     float m0_1 = lane_block_row_max_old[i][1];
6139     float l0_0 = lane_block_row_sum_old[i][0];
6140     float l0_1 = lane_block_row_sum_old[i][1];
6141
6142     float m_0 = fmaxf(m0_0, m1_0);
6143     float m_1 = fmaxf(m0_1, m1_1);
6144     float alpha_0 = __expf(m0_0 - m_0);
6145     float alpha_1 = __expf(m0_1 - m_1);
6146     float beta_0 = __expf(m1_0 - m_0);
6147     float beta_1 = __expf(m1_1 - m_1);
6148
6149     lane_block_row_max_old[i][0] = m_0;
6150     lane_block_row_max_old[i][1] = m_1;
6151     lane_block_row_sum_old[i][0] = alpha_0 * l0_0 + beta_0 * l1_0;
6152     lane_block_row_sum_old[i][1] = alpha_1 * l0_1 + beta_1 * l1_1;
6153
6154 #pragma unroll
6155 for (int j = 0; j < kValTileHeadDimV; ++j) {
6156     if constexpr (kMmaAccF32) {
6157         float *d0 = reinterpret_cast<float *>(&R_D[i][j][0]);
6158         float *d1 = reinterpret_cast<float *>(&R_0[i][j][0]);
6159         d0[0] = alpha_0 * d0[0] + beta_0 * d1[0];
6160         d0[1] = alpha_0 * d0[1] + beta_0 * d1[1];
6161         d0[2] = alpha_1 * d0[2] + beta_1 * d1[2];
6162         d0[3] = alpha_1 * d0[3] + beta_1 * d1[3];
6163     } else {
6164         float2 d0_0 = __half22float2(HALF2(R_D[i][j][0]));
6165         float2 d0_1 = __half22float2(HALF2(R_D[i][j][1]));
6166         float2 d1_0 = __half22float2(HALF2(R_0[i][j][0]));
6167         float2 d1_1 = __half22float2(HALF2(R_0[i][j][1]));
6168         d0_0.x = alpha_0 * d0_0.x + beta_0 * d1_0.x;
6169         d0_0.y = alpha_0 * d0_0.y + beta_0 * d1_0.y;
6170         d0_1.x = alpha_1 * d0_1.x + beta_1 * d1_1.x;
6171         d0_1.y = alpha_1 * d0_1.y + beta_1 * d1_1.y;
6172         HALF2(R_D[i][j][0]) = __float22half2_rn(d0_0);
6173         HALF2(R_D[i][j][1]) = __float22half2_rn(d0_1);

```

```

6174     }
6175 }
6176 }
6177
6178 // ---- Final rescale: 0 = 0acc / l ----
6179 #pragma unroll
6180 for (int i = 0; i < kValTileSeqLenP; ++i) {
6181     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
6182     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
6183 #pragma unroll
6184     for (int j = 0; j < kValTileHeadDimV; ++j) {
6185         if constexpr (kMmaAccF32) {
6186             float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
6187             half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
6188             t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
6189             t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
6190             t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
6191             t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
6192         } else {
6193             float2 t_reg_D_0 = __half2float2(HALF2(R_D[i][j][0]));
6194             float2 t_reg_D_1 = __half2float2(HALF2(R_D[i][j][1]));
6195             t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
6196             t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
6197             t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
6198             t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
6199             HALF2(R_D[i][j][0]) = __float2half2_rn(t_reg_D_0);
6200             HALF2(R_D[i][j][1]) = __float2half2_rn(t_reg_D_1);
6201         }
6202     }
6203 }
6204
6205 // ---- Epilogue: warp shuffle + 128-bit collective store ----
6206 #pragma unroll
6207 for (int i = 0; i < kValTileSeqLenP; ++i) {
6208 #pragma unroll
6209     for (int j = 0; j < kValTileHeadDimV; ++j) {
6210         uint32_t R_Z[2][4];
6211         R_Z[0][0] = R_D[i][j][0];
6212         R_Z[1][0] = R_D[i][j][1];
6213         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
6214         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
6215         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
6216         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
6217         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
6218         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
6219
6220         if (lane_id % 4 == 0) {
6221             const int store_warp_regs_0_Br =
6222                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
6223             const int store_lane_gmem_0_Br =
6224                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
6225             const int store_warp_regs_0_d =
6226                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
6227             const int store_lane_gmem_0_d = store_warp_regs_0_d;
6228             const int store_gmem_0_addr_0 =
6229                 0_gmem_offset + store_lane_gmem_0_Br * kHeadDim +
6230                 store_lane_gmem_0_d;
6231             const int store_gmem_0_addr_1 =
6232                 0_gmem_offset + (store_lane_gmem_0_Br + 8) * kHeadDim +
6233                 store_lane_gmem_0_d;
6234             *reinterpret_cast<float4 *>(&0[store_gmem_0_addr_0]) =
6235                 *reinterpret_cast<float4 *>(&R_Z[0][0]);
6236             *reinterpret_cast<float4 *>(&0[store_gmem_0_addr_1]) =

```

```

6237         *reinterpret_cast<float4*>(&R_Z[1][0]);
6238     }
6239 }
6240 }
6241 }
6242 }
6243 }
6244
6245 #if defined(NOTES_V2_ENABLE_CUTE)
6246 namespace fa_cute {
6247 using namespace cute;
6248
6249 // FlashAttn2CuTeTraits: FA2-style CuTe kernel 配置 (Br=128, Bc=64, 单 consumer WG)。
6250 // 与 FlashAttn3CuTeTraits 的核心区别:
6251 //   - Br=128 -> 8 warps along M (vs FA3 的 4), Q tile [128, D] (vs [64, D])
6252 //   - 单 consumer WG 全 KV 遍历, 无 split-KV merge
6253 //   - K/V tile 仍为 [64, D], SmemLayoutAtom/MmaAtom/SmemCopyAtom 与 FA3 一致
6254 //     (设计原因见 FlashAttn3CuTeTraits 头注释)
6255 //
6256 // 设计差异详述:
6257 // — SmemLayout (Q 更大, K/V 不变) —
6258 //   SmemLayoutQ = tile_to_shape(atom, (128, D)): Q tile [128, D] (Br=128),
6259 //   是 FA3 Q tile [64, D] 的 2 倍行数。8x8 atom 按 128 行重复, swizzle pattern 不变。
6260 //   SmemLayoutKV = tile_to_shape(atom, (64, D)): K/V tile [64, D] (Bc=64),
6261 //   与 FA3 的 SmemLayoutQKV 完全一致。
6262 //   SmemLayoutVt: 同 FA3, composition 叠加 col-major (D, 64) 得 V^T 转置视图。
6263 //
6264 // — MMA (8 warps, 单 WG 覆盖 Br=128) —
6265 //   TiledMma = TiledMMA(atom, EURepeat<8,1,1>, Tile<128,16,16>):
6266 //   - EURepeat<8,1,1>: 8 warp 沿 M 重复 MMA atom
6267 //   -> 8 warps × 32 threads = 256 threads = 1 consumer WG (FA3 为 128 threads)
6268 //   - Tile<128,16,16>: 逻辑 MMA tile
6269 //   M=128 = 8 warps × 16 rows = Br (单 WG 覆盖 Br=128, FA3 为 64)
6270 //   N=16, K=16: 与 FA3 一致
6271 //   MmaAtom / SmemCopyAtom / SmemCopyAtomTransposed 与 FA3 完全相同,
6272 //   不再重复说明 (见 FlashAttn3CuTeTraits 头注释)。
6273 // 复用 fa_cute 的 gemm_ss / gemm_rs / convert_layout_acc_* / convert_type。
6274 template <int kHeadDim>
6275 struct FlashAttn2CuTeTraits {
6276     static_assert(kHeadDim == 64 || kHeadDim == 128);
6277
6278     using Element = cutlass::half_t;
6279     using SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>;
6280     // Q tile: [Br=128, D], K/V tile: [Bc=64, D]
6281     using SmemLayoutQ = decltype(tile_to_shape(
6282         SmemLayoutAtom{}, Shape<_128, Int<kHeadDim>>{}));
6283     using SmemLayoutKV = decltype(tile_to_shape(
6284         SmemLayoutAtom{}, Shape<_64, Int<kHeadDim>>{}));
6285     using SmemLayoutVt = decltype(composition(
6286         SmemLayoutKV{},
6287         make_layout(Shape<Int<kHeadDim>, _64>{}, GenRowMajor{})));
6288
6289     // 8 warps along M (Br=128 = 8*16), ValTile N=2 (16 cols = 2*8)
6290     using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;
6291     using TiledMma = TiledMMA<
6292         MmaAtom, Layout<Shape<_8, _1, _1>>,
6293         Tile<_128, _16, _16>>;
6294     using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
6295     using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
6296 };
6297
6298 // FlashAttn3CuTeTraits: FA3-style CuTe kernel 编译期配置 (Br=Bc=64, 双 consumer WG)。
6299 // 所有 layout/atom/tiler 都是编译期类型, kernel body 只接收实例化后的类型。

```

```

6300 //
6301 // — SmemLayout 设计 (与 TMA 128B swizzle 匹配, 保证 ldmatrix 无 bank conflict) —
6302 //   SmemLayoutAtom = GMMMA::Layout_K_SW128_Atom<Element>:
6303 //     - GMMMA 128B swizzle atom: (8, 8) layout + Swizzle<3,4,3>
6304 //     - 一个 atom 覆盖 8 行 × 8 half = 128B, 恰好一个 swizzle 周期
6305 //     - 与 TMA CU_TENSOR_MAP_SWIZZLE_128B 写入 pattern 完全一致:
6306 //       producer 用 TMA 按 128B 粒度写入 swizzle 地址, consumer 用 cute::copy
6307 //       读取时由同款 swizzle layout 推导物理地址, 读写两侧 swizzle 一致 -> 无 bank conflict
6308 //   SmemLayoutQKV = tile_to_shape(atom, (64, D)):
6309 //     - Q/K/V tile 都是 [Br=Bc=64, D] (FA3 中 Br=Bc=64)
6310 //     - tile_to_shape 把 8×8 atom 按 (64, D) 重复, 保持 swizzle pattern
6311 //   SmemLayoutVt = composition(QKV, col-major (D, 64)):
6312 //     - V 转置视图: P@V 时 V 作为 MMA 的 B 矩阵, mma.sync.row.col 要求
6313 //       B 为 col-major. V 物理存储是 row-major [64, D], 通过 composition
6314 //       叠加 col-major (D, 64) 逻辑视图, 得到 V^T[D, 64] 访问接口。
6315 //     - composition 不搬数据, 只改 layout 解读, 物理 swizzle 不变。
6316 //
6317 // — MMA 设计 (Ampere m16n8k16, f32 累加保证 softmax 精度) —
6318 //   MmaAtom = SM80_16x8x16_F32F16F16F32_TN:
6319 //     - f16 输入 + f32 累加。FA 的 softmax 对精度敏感 (exp/sum),
6320 //       f16 累加会导致 row sum 溢出/下溢, 必须用 f32。
6321 //     - TN 布局: A(row-major) × B(col-major), 与 smem 物理布局天然匹配。
6322 //   TiledMma = TiledMMA(atom, EURepeat<4,1,1>, Tile<64,16,16>):
6323 //     - EURepeat<4,1,1>: 4 warp 沿 M 重复 MMA atom
6324 //       -> 4 warps × 32 threads = 128 threads = 1 consumer WG
6325 //     - Tile<64,16,16>: 逻辑 MMA tile
6326 //       M=64 = 4 warps × 16 rows = Br (单 WG 覆盖 Br=64)
6327 //       N=16 = 2 × kMmaN(8) -> 一次覆盖 16 列
6328 //       K=16 = kMmaK (单次 MMA K slice)
6329 //
6330 // — SmemCopy 设计 (S->R ldmatrix, 与 smem swizzle layout 配合) —
6331 //   SmemCopyAtom = SM75_U32x4_LDSM_N:
6332 //     - 对应 ldmatrix.sync.aligned.x4.m8n8.shared.b16 (非转置)
6333 //     - 一次加载 4 个 8×8 half 片段到 4 条 uint32 寄存器
6334 //     - 用于 Q 和 K: Q/K 在 smem 是 row-major, ldmatrix.x4 非转置加载
6335 //       得到 col-major fragment, 匹配 mma.row.col 的 A/B 输入约定
6336 //   SmemCopyAtomTransposed = SM75_U16x8_LDSM_T:
6337 //     - 对应 ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 (转置)
6338 //     - 用于 V: V 在 smem 是 row-major [Bc, D], P@V 需把 V 当 col-major
6339 //       B 矩阵。ldmatrix.x2.trans 转置加载为 col-major fragment, 直接匹配
6340 //       mma.row.col 的 B 输入, 无需软件转置 V smem。
6341 //     - 用 x2 (非 x4): V 的 K 维 (Bc=64) 按 kMmaK=16 切片, 每片 16×8
6342 //       = 1 个 ldmatrix.x2 (2 个 8×8 fragment)
6343 template <int kHeadDim>
6344 struct FlashAttn3CuTeTraits {
6345     static_assert(kHeadDim == 64 || kHeadDim == 128);
6346
6347     using Element = cutlass::half_t;
6348     using SmemLayoutAtom = GMMMA::Layout_K_SW128_Atom<Element>;
6349     using SmemLayoutQKV = decltype(tile_to_shape(
6350         SmemLayoutAtom{}, Shape<_64, Int<kHeadDim>>{}));
6351     using SmemLayoutVt = decltype(composition(
6352         SmemLayoutQKV{},
6353         make_layout(Shape<Int<kHeadDim>, _64>{}, GenRowMajor{})));
6354
6355     using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;
6356     using TiledMma = TiledMMA<
6357         MmaAtom, Layout<Shape<_4, _1, _1>>,
6358         Tile<_64, _16, _16>>;
6359     using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
6360     using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
6361 };
6362

```

```

6363 // convert_layout_acc_rowcol: 把 MMA accumulator 的 (MMA=4, MMA_M, MMA_N)
6364 // layout 转换为 ((2, MMA_M), (2, MMA_N)) 的 (nrow, ncol) 二维布局。
6365 //
6366 // 为什么需要转换? FA 的 online softmax 需要按"行"操作 score 矩阵 S:
6367 //   - 求 row max、exp(S - max)、row sum 都要求同一行的元素聚集在同一线程。
6368 //   - 但 m16n8k16 的 MMA C fragment 的 4 个寄存器按 (MMA=4, MMA_M, MMA_N)
6369 //     排布, lane 0/4/8/.. 持有同一行片段但散布在 MMA mode 0/1。
6370 //   - logical_divide(_, Shape<_2>{}) 把 MMA=4 拆成 (2, 2):
6371 //     (2, 2, MMA_M, MMA_N) -> 重组为 ((2, MMA_M), (2, MMA_N))
6372 //     外层 (2, MMA_M) = nrow: 前 2 个 MMA mode × MMA_M 组成"行块"
6373 //     内层 (2, MMA_N) = ncol: 后 2 个 MMA mode × MMA_N 组成"列块"
6374 //   - 转换后 scores(r, c) 的 r 索引行、c 索引列, online softmax 可直接遍历
6375 //     size<1>(scores) 做行内 reduce max/sum, 再用 __shfl_xor_sync 在 4-lane
6376 //     子组内归约 (因为 m16n8k16 下每 4 个 lane 共享同一行的不同列片段)。
6377 //
6378 // 出处: flash-attention/csrc/flash_attn/src/utils.h:188
6379 //       flash-attention/csrc/flash_attn/src/softmax.h:139 (调用点)
6380 //       用于 tCrS (QK score) 和 tCr0 (PV accumulator) 的 rowcol 视图。
6381 template <typename Layout>
6382 CUTE_DEVICE auto convert_layout_acc_rowcol(Layout acc_layout) {
6383     auto divided = logical_divide(acc_layout, Shape<_2>{});
6384     return make_layout(
6385         make_layout(get<0, 1>(divided), get<1>(divided)),
6386         make_layout(get<0, 0>(divided), get<2>(divided)));
6387 }
6388
6389 // convert_layout_acc_Aregs: 把 MMA accumulator (QK 的 score S) 转换为
6390 // P@V 所需的 A-register (左矩阵 P) layout, 使 softmax 后的 P 能直接喂给
6391 // 下一个 MMA 做 P@V, 无需额外数据搬运。
6392 //
6393 // 为什么需要转换? QK 和 PV 用同一个 TiledMma, 但 C fragment 和 A fragment
6394 // 的寄存器排布不同:
6395 //   - QK:  $S = Q @ K^T$ , S 是 C fragment, layout = (MMA=4, MMA_M, MMA_N)
6396 //   - PV:  $O = P @ V$ , P 是 A fragment, layout = (MMA, MMA_M, MMA_K)
6397 //   - m16n8k16 的 MMA=4 对应 4 个寄存器, C 的 4 个按 (2行, 2列) 排布,
6398 //     A 的 4 个按 (2行, 2K-slice) 排布, 直接用 C layout 做 A 会导致
6399 //     寄存器对齐错误。
6400 //   - logical_divide(_, Shape<X, X, _2>{}) 把 MMA_N 拆成 (2, MMA_N/2):
6401 //     (MMA=4, MMA_M, (2, MMA_N/2)) -> 重组为 ((4, 2), MMA_M, MMA_N/2)
6402 //     外层 (4, 2) = 新的 MMA mode, 把 C 的列方向 2 个 half 配对到
6403 //     A 的 K 方向连续 2 个元素, 匹配 m16n8k16 的 K=16 内部 2-element 步长。
6404 //   - 转换后 tCrPv 可直接作为 gemm_rs 的 tCrA 参数, P 的数据仍在原寄存器中,
6405 //     只是逻辑坐标重解释, 零拷贝。
6406 //
6407 // ★ 这就是 FA "寄存器复用" 的核心技巧: softmax 后的 P 不写回 smem 再
6408 //   ldmatrix, 而是原地重解释为 A fragment, 省一次 smem 往返。
6409 //   但注意这依赖 m16n8k16 的特定 fragment 约定, 不是通用性质。
6410 //
6411 // 出处: flash-attention/csrc/flash_attn/src/utils.h:200
6412 //       flash-attention/csrc/flash_attn/src/flash_fwd_kernel.h:365 (调用点)
6413 //       注释 "Reshape rP from (MMA=4, MMA_M, MMA_N) to ((4, 2), MMA_M, MMA_N / 2)"
6414 template <typename TiledMma, typename Layout>
6415 CUTE_DEVICE auto convert_layout_acc_Aregs(Layout acc_layout) {
6416     using X = Underscore;
6417     auto divided = logical_divide(acc_layout, Shape<X, X, _2>{});
6418     return make_layout(
6419         make_layout(get<0>(divided), get<2, 0>(divided)),
6420         get<1>(divided), get<2, 1>(divided));
6421 }
6422
6423 // convert_type: 在寄存器层面做 dtype 转换, 不搬数据只改元素类型解释。
6424 //
6425 // 为什么需要? MMA accumulator 是 f32 (F32F16F16F32_TN), 但下游消费需要 f16:

```

```

6426 // - tCrP = convert_type<Element>(tCrS): softmax 后的 score S (f32) 转 P (f16),
6427 // 供 P@V 的 A fragment (mma 要求 f16 输入)
6428 // - tCr0_half = convert_type<Element>(tCr0): 最终 0 (f32 acc) 转 f16, 供 store
6429 //
6430 // 实现: NumericArrayConverter 一次性批量转换整个 fragment (kElements 个元素),
6431 // 返回的 tensor 共享原 layout, 只是 value_type 从 From 变为 To。
6432 // 零拷贝, 纯寄存器内转换 (make_rmem_ptr 标记为 register memory)。
6433 //
6434 // 出处: flash-attention/csrc/flash_attn/src/epilogue/epilogue.hpp (同类实现)
6435 template <typename To, typename Engine, typename Layout>
6436 CUTE_DEVICE auto convert_type(Tensor<Engine, Layout> const &tensor) {
6437     using From = typename Engine::value_type;
6438     constexpr int kElements = decltype(size(tensor))::value;
6439     cutlass::NumericArrayConverter<To, From, kElements> convert;
6440     auto fragment = convert(
6441         *reinterpret_cast<cutlass::Array<From, kElements> const *>(tensor.data()));
6442     return make_tensor(make_rmem_ptr<To>(&fragment), tensor.layout());
6443 }
6444
6445 // gemm_ss: Shared-Shared GEMM, A 和 B 都从 smem 加载到寄存器再做 MMA。
6446 // 用于 FA 的 Q@K^T 步骤: Q 和 K 都在 smem 中 (Q 由 TMA/cp.async 预加载,
6447 // K 由 TMA 加载到当前 stage), 需要 ldmatrix 同时搬运 A(Q) 和 B(K)。
6448 //
6449 // 流程 (software-pipelined ldmatrix + mma):
6450 // 1. retile_D: 把 A/B 的 register fragment 按 TiledCopy 的 source 视图重排,
6451 // 使 copy() 能正确写入后续 MMA 消费的寄存器位置。
6452 // 2. 预加载 tile_k=0: copy A[0] 和 B[0] 到寄存器。
6453 // 3. for tile_k = 0..MMA_K-1:
6454 //     a. 若 tile_k+1 存在, 预加载 A[tile_k+1] 和 B[tile_k+1] (与当前 MMA 重叠)
6455 //     b. gemm(mma, A[tile_k], B[tile_k], acc) - 发射 mma.sync 指令
6456 // 每个 K slice 的 ldmatrix 与前一个 slice 的 MMA 重叠执行, 隐藏 smem->reg 延迟。
6457 //
6458 // 参数:
6459 // acc: (MMA, MMA_M, MMA_N) 累加器, 输入输出
6460 // fragment_a/fragment_b: A/B 寄存器 fragment, 会被 retile 后复用
6461 // shared_a/shared_b: smem 中的 A/B tile (带 swizzle layout)
6462 // tiled_mma: TiledMMA 类型实例
6463 // tiled_copy_a/b: A/B 的 S->R TiledCopy (make_tiled_copy_A/B 生成)
6464 // thread_copy_a/b: A/B 的线程级 copy slice
6465 //
6466 // 出处: flash-attention/csrc/flash_attn/src/utils.h:166 (gemm_rs 的对照)
6467 // FlashMLA/csrc/sm90/helpers.h:97 (gemm_ss 的同类实现)
6468 // 本 notes 的 QK GEMM 调用: fa_cute::gemm_ss(tCrS, tCrQ, tCrK, ...)
6469 template <typename TensorC, typename TensorA, typename TensorB,
6470         typename TensorSA, typename TensorSB, typename TiledMma,
6471         typename TiledCopyA, typename TiledCopyB,
6472         typename ThreadCopyA, typename ThreadCopyB>
6473 CUTE_DEVICE void gemm_ss(
6474     TensorC &acc, TensorA &fragment_a, TensorB &fragment_b,
6475     TensorSA const &shared_a, TensorSB const &shared_b,
6476     TiledMma tiled_mma, TiledCopyA tiled_copy_a, TiledCopyB tiled_copy_b,
6477     ThreadCopyA thread_copy_a, ThreadCopyB thread_copy_b) {
6478     auto copy_view_a = thread_copy_a.retile_D(fragment_a);
6479     auto copy_view_b = thread_copy_b.retile_D(fragment_b);
6480     copy(tiled_copy_a, shared_a(_, _, _0{}), copy_view_a(_, _, _0{}));
6481     copy(tiled_copy_b, shared_b(_, _, _0{}), copy_view_b(_, _, _0{}));
6482 #pragma unroll
6483     for (int tile_k = 0; tile_k < size<2>(fragment_a); ++tile_k) {
6484         if (tile_k + 1 < size<2>(fragment_a)) {
6485             copy(tiled_copy_a, shared_a(_, _, tile_k + 1),
6486                 copy_view_a(_, _, tile_k + 1));
6487             copy(tiled_copy_b, shared_b(_, _, tile_k + 1),
6488                 copy_view_b(_, _, tile_k + 1));

```

```

6489     }
6490     gemm(tiled_mma, fragment_a(_, _, tile_k),
6491         fragment_b(_, _, tile_k), acc);
6492 }
6493 }
6494
6495 // gemm_rs: Register-Shared GEMM, A 在寄存器中 (已是 fragment), B 从 smem 加载。
6496 // 用于 FA 的 P@V 步骤: P 已经在寄存器中 (softmax 后的 tCrS, 经 convert_layout_acc_Aregs
6497 // 重解释为 A fragment), V 在 smem 中 (TMA 加载), 只需 ldmatrix 搬运 B(V)。
6498 //
6499 // 与 gemm_ss 的区别: A 不从 smem 加载 (P 已在寄存器), 省 A 的 ldmatrix,
6500 // 这就是 FA "寄存器复用" 的性能优势: softmax 后的 P 不回写 smem, 直接做 A。
6501 //
6502 // 流程 (与 gemm_ss 对称, 但只 pipelined B):
6503 //   1. retile_D: B 的 register fragment 按 TiledCopy source 视图重排。
6504 //   2. 预加载 tile_k=0: copy B[0] 到寄存器。
6505 //   3. for tile_k = 0..MMA_K-1:
6506 //       a. 若 tile_k+1 存在, 预加载 B[tile_k+1]
6507 //       b. gemm(mma, A[tile_k], B[tile_k], acc) - A 是 P (寄存器), B 是 V (smem)
6508 //
6509 // 参数:
6510 //   acc: (MMA, MMA_M, MMA_N) 累加器, 输出 0
6511 //   fragment_a: P 寄存器 fragment (softmax 后的 score, 已 Aregs 转换)
6512 //   fragment_b: V 寄存器 fragment (会被 retile 后复用)
6513 //   shared_b: smem 中的 V tile (带 transposed swizzle layout)
6514 //
6515 // 出处: flash-attention/csrc/flash_attn/src/utils.h:166
6516 //       flash-attention/csrc/flash_attn/src/flash_fwd_kernel.h:367 (调用点)
6517 //       本 notes 的 PV GEMM 调用: fa_cute::gemm_rs(tCr0, tCrPv, tCrV, ...)
6518 template <typename TensorC, typename TensorA, typename TensorB,
6519           typename TensorSB, typename TiledMma, typename TiledCopyB,
6520           typename ThreadCopyB>
6521 CUTE_DEVICE void gemm_rs(
6522     TensorC &acc, TensorA &fragment_a, TensorB &fragment_b,
6523     TensorSB const &shared_b, TiledMma tiled_mma,
6524     TiledCopyB tiled_copy_b, ThreadCopyB thread_copy_b) {
6525     auto copy_view_b = thread_copy_b.retile_D(fragment_b);
6526     copy(tiled_copy_b, shared_b(_, _, _0{}), copy_view_b(_, _, _0{}));
6527 #pragma unroll
6528     for (int tile_k = 0; tile_k < size<2>(fragment_a); ++tile_k) {
6529         if (tile_k + 1 < size<2>(fragment_a)) {
6530             copy(tiled_copy_b, shared_b(_, _, tile_k + 1),
6531                 copy_view_b(_, _, tile_k + 1));
6532         }
6533         gemm(tiled_mma, fragment_a(_, _, tile_k),
6534             fragment_b(_, _, tile_k), acc);
6535     }
6536 }
6537
6538 } // namespace fa_cute
6539
6540 // =====
6541 // FA2 CuTe TMA MMA WS (1 Consumer WG) (single consumer, Br=128, Bc=64)
6542 // =====
6543 // 与 flash_attn_3_tma_mma_ws_split_q_cute (FA3-style, dual consumer, Br=64) 的区别:
6544 //   - 单 consumer WG (256T = 8 warps) 全 KV 遍历, 无 split-KV merge
6545 //   - Br=128 -> TiledMMA EURepeat_8 / Tile<_128,_16,_16> (8c 为 _4 / Tile<_64,_16,_16>)
6546 //   - Q smem tile 为 [128, D] (8c 为 [64, D]), K/V tile 仍为 [64, D]
6547 //   - kStagesK 和 kStagesV 均可配置 (>=1), 对齐 8b flash_attn_tma_mma_ws_stages_split_q 语义
6548 //   - producer 顺序对齐 8b: V-first (load V[tile+Sv-1]) 然后 K-after (prefetch K[tile+Sk-1])
6549 //   - mainloop 结束后直接 normalize (0=0acc/l) + store, 无 merge scratch
6550 //
6551 // Barrier 协议 (单 consumer, 256 threads):

```

```

6552 // q_full: TmaBarrier init 1 (producer arrive_and_expect_tx, consumer wait)
6553 // k_full[Sk]: TmaBarrier init 1; k_empty[Sk]: CtaBarrier init 256 (consumer arrive, producer wait)
6554 // v_full[Sv]: TmaBarrier init 1; v_empty[Sv]: CtaBarrier init 256
6555 // phase: consumer wait k_full[stage] at (tile/Sk)&1; producer wait k_empty at (k_tile/Sk)&1
6556 // consumer wait v_full[stage_v] at (tile/Sv)&1; producer wait v_empty at (v_tile/Sv)&1
6557 //
6558 // 复用 fa_cute::gemm_ss / gemm_rs / convert_layout_acc_rowcol /
6559 // convert_layout_acc_Aregs / convert_type (通用 helper, 不依赖 Br)
6560 template <int kHeadDim, typename TmaQ, typename TmaK, typename TmaV,
6561         int kStagesK = 1, int kStagesV = 1>
6562 __global__ void __launch_bounds__(384, 1)
6563 flash_attn_tma_mma_ws_split_q_cute(
6564     CUTLASS_GRID_CONSTANT TmaQ const tma_q,
6565     CUTLASS_GRID_CONSTANT TmaK const tma_k,
6566     CUTLASS_GRID_CONSTANT TmaV const tma_v,
6567     cutlass::half_t *output, int rows, int seqlen) {
6568     // rows: B * H * seqlen, seqlen: N, kHeadDim: D
6569     using namespace cute;
6570     using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
6571     using Element = typename Traits::Element;
6572     using SmemLayoutQ = typename Traits::SmemLayoutQ;
6573     using SmemLayoutKV = typename Traits::SmemLayoutKV;
6574     using TmaBarrier = cutlass::arch::ClusterTransactionBarrier;
6575     using CtaBarrier = cutlass::arch::ClusterBarrier;
6576     constexpr int kBr = 128;
6577     constexpr int kBc = 64;
6578     constexpr int kConsumerThreads = 256;
6579     constexpr int kProducerThreads = 128;
6580     constexpr int kQTileElements = cosize(SmemLayoutQ{});
6581     constexpr int kKVTileElements = cosize(SmemLayoutKV{});
6582
6583     extern __shared__ __align__(1024) Element shm[];
6584     auto sQ = make_tensor(make_smem_ptr(shm), SmemLayoutQ{});
6585     Element *k_base = shm + kQTileElements;
6586     Element *v_base = k_base + kStagesK * kKVTileElements;
6587
6588     __shared__ uint64_t q_full;
6589     __shared__ uint64_t k_full[kStagesK];
6590     __shared__ uint64_t k_empty[kStagesK];
6591     __shared__ uint64_t v_full[kStagesV];
6592     __shared__ uint64_t v_empty[kStagesV];
6593
6594     const bool is_producer = threadIdx.x < kProducerThreads;
6595     const int wg_tid = is_producer ? threadIdx.x
6596         : threadIdx.x - kProducerThreads;
6597
6598     if (threadIdx.x == 0) {
6599         TmaBarrier::init(&q_full, 1);
6600         for (int s = 0; s < kStagesK; ++s) {
6601             TmaBarrier::init(&k_full[s], 1);
6602             CtaBarrier::init(&k_empty[s], kConsumerThreads);
6603         }
6604         for (int s = 0; s < kStagesV; ++s) {
6605             TmaBarrier::init(&v_full[s], 1);
6606             CtaBarrier::init(&v_empty[s], kConsumerThreads);
6607         }
6608     }
6609     __syncthreads();
6610
6611     int q_tile = blockIdx.y * (seqlen / kBr) + blockIdx.x;
6612     int kv_tiles = seqlen / kBc;
6613
6614     // =====

```

```

6615 // Producer Warpgroup (WG0, threadIdx.x 0~127)
6616 // Only wg_tid == 0 issues TMA. Load Q once, then V-first K-after KV loop.
6617 // P1: prefetch K[0..Sk-2] + V[0..Sv-2] (if Sk/Sv > 1).
6618 // P2: V[tile+Sv-1] then K[tile+Sk-1] (prefetch ahead Sk-1/Sv-1, 对齐 8b).
6619 // =====
6620 if (is_producer) {
6621     if (wg_tid == 0) {
6622         auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6623         auto gQ = local_tile(
6624             mQ, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
6625         auto q_slice = tma_q.get_slice(_0{});
6626         auto tQgQ = q_slice.partition_S(gQ);
6627         auto tQsQ = q_slice.partition_D(sQ);
6628         TmaBarrier::arrive_and_expect_tx(
6629             &q_full, sizeof(Element) * size(sQ));
6630         copy(tma_q.with(q_full), tQgQ, tQsQ);
6631
6632         auto mK = tma_k.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6633         auto mV = tma_v.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6634         auto k_slice = tma_k.get_slice(_0{});
6635         auto v_slice = tma_v.get_slice(_0{});
6636
6637         // * 易错点 (CuTe TMA multi-head): K/V local_tile 的 coord 是相对于 TMA
6638         // tensor row 维的全局 tile 索引 (TMA tensor shape = [B*H*N, D]), 不是
6639         // per-head tile 索引. 多 head 场景必须加 `blockIdx.y * kv_tiles` 偏移,
6640         // 否则所有 head 都读 head 0 的 K/V -> correctness test (H=1) 正常但
6641         // bench (H>1) 误差飙升 2 个量纲. Q 的 q_tile 已含 blockIdx.y 无需修复.
6642         // 参考: /memories/repo/leetcuda-tma-fa-multihead.md
6643         // P1: Prefetch first (kStagesK-1) K tiles
6644         for (int s = 0; s < kStagesK - 1; ++s) {
6645             if (s < kv_tiles) {
6646                 auto sK = make_tensor(
6647                     make_smem_ptr(k_base + s * kKVTileElements), SmemLayoutKV{});
6648                 CtaBarrier::wait(&k_empty[s], 0);
6649                 auto gK = local_tile(
6650                     mK, Shape<_64, Int<kHeadDim>>{},
6651                     make_coord(blockIdx.y * kv_tiles + s, _0{}));
6652                 auto tKgK = k_slice.partition_S(gK);
6653                 auto tKsK = k_slice.partition_D(sK);
6654                 TmaBarrier::arrive_and_expect_tx(
6655                     &k_full[s], sizeof(Element) * size(sK));
6656                 copy(tma_k.with(k_full[s]), tKgK, tKsK);
6657             }
6658         }
6659
6660         // P1b: Prefetch first (kStagesV-1) V tiles
6661         for (int s = 0; s < kStagesV - 1; ++s) {
6662             if (s < kv_tiles) {
6663                 auto sV = make_tensor(
6664                     make_smem_ptr(v_base + s * kKVTileElements), SmemLayoutKV{});
6665                 CtaBarrier::wait(&v_empty[s], 0);
6666                 auto gV = local_tile(
6667                     mV, Shape<_64, Int<kHeadDim>>{},
6668                     make_coord(blockIdx.y * kv_tiles + s, _0{}));
6669                 auto tVgV = v_slice.partition_S(gV);
6670                 auto tVsV = v_slice.partition_D(sV);
6671                 TmaBarrier::arrive_and_expect_tx(
6672                     &v_full[s], sizeof(Element) * size(sV));
6673                 copy(tma_v.with(v_full[s]), tVgV, tVsV);
6674             }
6675         }
6676
6677         // P2: Main loop - V-first then K-after (对齐 8b producer 顺序)

```

```

6678 for (int tile = 0; tile < kv_tiles; ++tile) {
6679     // V: load V[tile+Sv-1] (prefetch ahead Sv-1)
6680     {
6681         int v_tile = tile + kStagesV - 1;
6682         if (v_tile < kv_tiles) {
6683             int stage_v = v_tile % kStagesV;
6684             int phase_v = (v_tile / kStagesV) & 1;
6685             auto sV = make_tensor(
6686                 make_smem_ptr(v_base + stage_v * kKVTileElements),
6687                 SmemLayoutKV{});
6688             CtaBarrier::wait(&v_empty[stage_v], phase_v);
6689             auto gV = local_tile(
6690                 mV, Shape<_64, Int<kHeadDim>>{},
6691                 make_coord(blockIdx.y * kv_tiles + v_tile, _0{}));
6692             auto tVgV = v_slice.partition_S(gV);
6693             auto tVsV = v_slice.partition_D(sV);
6694             TmaBarrier::arrive_and_expect_tx(
6695                 &v_full[stage_v], sizeof(Element) * size(sV));
6696             copy(tma_v.with(v_full[stage_v]), tVgV, tVsV);
6697         }
6698     }
6699     // K: prefetch K[tile+Sk-1] (prefetch ahead Sk-1)
6700     {
6701         int k_tile = tile + kStagesK - 1;
6702         if (k_tile < kv_tiles) {
6703             int stage_k = k_tile % kStagesK;
6704             int phase_k = (k_tile / kStagesK) & 1;
6705             auto sK = make_tensor(
6706                 make_smem_ptr(k_base + stage_k * kKVTileElements),
6707                 SmemLayoutKV{});
6708             CtaBarrier::wait(&k_empty[stage_k], phase_k);
6709             auto gK = local_tile(
6710                 mK, Shape<_64, Int<kHeadDim>>{},
6711                 make_coord(blockIdx.y * kv_tiles + k_tile, _0{}));
6712             auto tKgK = k_slice.partition_S(gK);
6713             auto tKsK = k_slice.partition_D(sK);
6714             TmaBarrier::arrive_and_expect_tx(
6715                 &k_full[stage_k], sizeof(Element) * size(sK));
6716             copy(tma_k.with(k_full[stage_k]), tKgK, tKsK);
6717         }
6718     }
6719 }
6720 }
6721 }
6722 // =====
6723 // Consumer Warpgroup (WG1, threadIdx.x 128~383, 256 threads = 8 warps)
6724 // Single consumer: full KV traversal, no split-KV merge.
6725 // Flow: wait K -> QK -> release K -> softmax -> wait V -> PV -> release V.
6726 // =====
6727 else {
6728     TmaBarrier::wait(&q_full, 0);
6729
6730     // V layout 从 stage 0 推导 (所有 stage layout 相同)
6731     auto sV0 = make_tensor(make_smem_ptr(v_base), SmemLayoutKV{});
6732     auto sVt0 = make_tensor(sV0.data(), typename Traits::SmemLayoutVt{});
6733     // * get_nonswizzle_portion 只用于 partition_fragment_B 推导寄存器侧 B
6734     // fragment layout (每线程持有哪些 V 元素), 寄存器布局与 smem swizzle 无关。
6735     // V 的 smem bank conflict 由 TMA 写入 (SmemLayoutKV 带 swizzle) +
6736     // ldmatrix 读取 (主循环内 partition_S(sVt_stg) 带 swizzle) 共同解决。
6737     // 这里不能用带 swizzle 的 sVt0 做 partition_fragment_B, 否则 CuTe 推导
6738     // ldmatrix.trans 的线程-数据映射时会与 swizzle composition 冲突。
6739     // 参考: flash-attention/csrc/flash_attn/src/kernel_traits.h
6740     // SmemLayoutVtransposedNoSwizzle 的同样用法。

```

```

6741 auto sVt0_ns = make_tensor(
6742     sV0.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));
6743
6744 typename Traits::TiledMma tiled_mma;
6745 auto thr_mma = tiled_mma.get_thread_slice(wg_tid);
6746 auto tCrQ = thr_mma.partition_fragment_A(sQ);
6747 auto tCrV_layout = thr_mma.partition_fragment_B(sVt0_ns).layout();
6748 auto tCr0 = partition_fragment_C(
6749     tiled_mma, Shape<_128, Int<kHeadDim>>{});
6750 clear(tCr0);
6751
6752 auto s2r_copy_q = make_tiled_copy_A(
6753     typename Traits::SmemCopyAtom{}, tiled_mma);
6754 auto s2r_thr_q = s2r_copy_q.get_thread_slice(wg_tid);
6755 auto tQsQ_s2r = s2r_thr_q.partition_S(sQ);
6756 auto s2r_copy_k = make_tiled_copy_B(
6757     typename Traits::SmemCopyAtom{}, tiled_mma);
6758 auto s2r_thr_k = s2r_copy_k.get_thread_slice(wg_tid);
6759 auto s2r_copy_v = make_tiled_copy_B(
6760     typename Traits::SmemCopyAtomTransposed{}, tiled_mma);
6761 auto s2r_thr_v = s2r_copy_v.get_thread_slice(wg_tid);
6762
6763 auto tCr0_rc = make_tensor(
6764     tCr0.data(),
6765     fa_cute::convert_layout_acc_rowcol(tCr0.layout()));
6766 constexpr int kRows = decltype(size<0>(tCr0_rc))::value;
6767 float row_max[kRows], row_sum[kRows];
6768 #pragma unroll
6769 for (int r = 0; r < kRows; ++r) {
6770     row_max[r] = -INFINITY;
6771     row_sum[r] = 0.0f;
6772 }
6773
6774 // Init: mark all K/V slots as empty (256 consumer arrivals each)
6775 for (int s = 0; s < kStagesK; ++s)
6776     CtaBarrier::arrive(&k_empty[s]);
6777 for (int s = 0; s < kStagesV; ++s)
6778     CtaBarrier::arrive(&v_empty[s]);
6779
6780 float scale = rsqrtf(static_cast<float>(kHeadDim));
6781 for (int tile = 0; tile < kv_tiles; ++tile) {
6782     int k_stg = tile % kStagesK;
6783     int k_phase = (tile / kStagesK) & 1;
6784     int v_stg = tile % kStagesV;
6785     int v_phase = (tile / kStagesV) & 1;
6786
6787     // Per-stage K smem and S2R partition
6788     auto sK_stg = make_tensor(
6789         make_smem_ptr(k_base + k_stg * kKVTileElements), SmemLayoutKV{});
6790     auto tCrK = thr_mma.partition_fragment_B(sK_stg);
6791     CUTE_STATIC_ASSERT_V(size(tCrK) == size(tCrV_layout));
6792     auto tCrV = make_tensor(tCrK.data(), tCrV_layout);
6793     auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);
6794
6795     // 1) Wait K, QK gemm, release K early
6796     TmaBarrier::wait(&k_full[k_stg], k_phase);
6797     auto tCrS = partition_fragment_C(tiled_mma, Shape<_128, _64>{});
6798     clear(tCrS);
6799     fa_cute::gemm_ss(
6800         tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
6801         tiled_mma, s2r_copy_q, s2r_copy_k, s2r_thr_q, s2r_thr_k);
6802     { CtaBarrier::arrive(&k_empty[k_stg]); }
6803

```

```

6804 // 2) Online softmax (overlaps with producer's V TMA)
6805 auto scores = make_tensor(
6806     tCrS.data(),
6807     fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
6808 #pragma unroll
6809 for (int r = 0; r < kRows; ++r) {
6810     float tile_max = -INFINITY;
6811     #pragma unroll
6812     for (int c = 0; c < size<1>(scores); ++c)
6813         tile_max = fmaxf(tile_max, scores(r, c) * scale);
6814     tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
6815     tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
6816     float nxt = fmaxf(row_max[r], tile_max);
6817     float rs = __expf(row_max[r] - nxt);
6818     #pragma unroll
6819     for (int c = 0; c < size<1>(tCr0_rc); ++c)
6820         tCr0_rc(r, c) *= rs;
6821     float ts = 0.0f;
6822     #pragma unroll
6823     for (int c = 0; c < size<1>(scores); ++c) {
6824         float p = __expf(scores(r, c) * scale - nxt);
6825         scores(r, c) = p;
6826         ts += p;
6827     }
6828     ts += __shfl_xor_sync(0xffffffff, ts, 1);
6829     ts += __shfl_xor_sync(0xffffffff, ts, 2);
6830     row_sum[r] = row_sum[r] * rs + ts;
6831     row_max[r] = nxt;
6832 }
6833
6834 // 3) Wait V, PV gemm, release V early
6835 // * sVt_stg 带 swizzle: partition_S 推导 smem 源地址映射, copy() 会
6836 // 正确应用 swizzle 物理地址, 与 TMA 写入 (SmemLayoutKV 带 swizzle) 匹配,
6837 // 解决 V 的 smem bank conflict。这是 V bank-conflict-free 的关键路径。
6838 auto sVt_stg = make_tensor(
6839     make_smem_ptr(v_base + v_stg * kKVTileElements), SmemLayoutKV{});
6840 auto sVt_stg = make_tensor(sVt_stg.data(), typename Traits::SmemLayoutVt{});
6841 auto tVsVt_s2r = s2r_thr_v.partition_S(sVt_stg);
6842
6843 TmaBarrier::wait(&v_full[v_stg], v_phase);
6844 auto tCrP = fa_cute::convert_type<Element>(tCrS);
6845 auto tCrPv = make_tensor(
6846     tCrP.data(),
6847     fa_cute::convert_layout_acc_Aregs<typename Traits::TiledMma>(
6848         tCrP.layout()));
6849 fa_cute::gemm_rs(
6850     tCr0, tCrPv, tCrV, tVsVt_s2r,
6851     tiled_mma, s2r_copy_v, s2r_thr_v);
6852 { CtaBarrier::arrive(&v_empty[v_stg]); }
6853 }
6854
6855 // ---- Final normalize: 0 = 0acc / 1 (single consumer, no merge) ----
6856 #pragma unroll
6857 for (int r = 0; r < kRows; ++r) {
6858     float inv_sum = 1.0f / row_sum[r];
6859     #pragma unroll
6860     for (int c = 0; c < size<1>(tCr0_rc); ++c)
6861         tCr0_rc(r, c) *= inv_sum;
6862 }
6863
6864 auto tCr0_half = fa_cute::convert_type<Element>(tCr0);
6865 auto m0 = make_tensor(
6866     make_gmem_ptr(output),

```

```

6867     make_shape(rows, Int<kHeadDim>{}),
6868     make_stride(Int<kHeadDim>{}, _1{}));
6869     auto g0 = local_tile(
6870         m0, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
6871     auto tCg0 = thr_mma.partition_C(g0);
6872     copy(tCr0_half, tCg0);
6873 }
6874 }
6875
6876 template <int kHeadDim, typename TmaQ, typename TmaK, typename TmaV,
6877         int kStagesK = 1>
6878 __global__ void __launch_bounds__(384, 1)
6879 flash_attn_3_tma_mma_ws_split_q_cute(
6880     CUTLASS_GRID_CONSTANT TmaQ const tma_q,
6881     CUTLASS_GRID_CONSTANT TmaK const tma_k,
6882     CUTLASS_GRID_CONSTANT TmaV const tma_v,
6883     cutlass::half_t *output, int rows, int seqlen) {
6884     // rows: B * H * seqlen, seqlen: N, kHeadDim: D
6885     using namespace cute;
6886     using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
6887     using Element = typename Traits::Element;
6888     using SmemLayout = typename Traits::SmemLayoutQKV;
6889     using TmaBarrier = cutlass::arch::ClusterTransactionBarrier;
6890     using CtaBarrier = cutlass::arch::ClusterBarrier;
6891     constexpr int kTile = 64;
6892     constexpr int kNumConsumers = 2;
6893     constexpr int kConsumerThreads = 128;
6894     constexpr int kProducerThreads = 128;
6895     constexpr int kTileElements = cosize(SmemLayout{});
6896
6897     extern __shared__ __align__(1024) Element shm[];
6898     auto sQ = make_tensor(make_smem_ptr(shm), SmemLayout{});
6899     Element *k_base = shm + kTileElements;
6900     Element *v_base = k_base + kNumConsumers * kStagesK * kTileElements;
6901
6902     __shared__ uint64_t q_full;
6903     __shared__ uint64_t k_full[kNumConsumers][kStagesK];
6904     __shared__ uint64_t k_empty[kNumConsumers][kStagesK];
6905     __shared__ uint64_t v_full[kNumConsumers];
6906     __shared__ uint64_t v_empty[kNumConsumers];
6907
6908     const bool is_producer = threadIdx.x < kProducerThreads;
6909     const int consumer_id = is_producer ? 0
6910         : (threadIdx.x - kProducerThreads) / kConsumerThreads;
6911     const int wg_tid = is_producer ? threadIdx.x
6912         : (threadIdx.x - kProducerThreads) % kConsumerThreads;
6913
6914     if (threadIdx.x == 0) {
6915         TmaBarrier::init(&q_full, 1);
6916         for (int cid = 0; cid < kNumConsumers; ++cid) {
6917             for (int s = 0; s < kStagesK; ++s) {
6918                 TmaBarrier::init(&k_full[cid][s], 1);
6919                 CtaBarrier::init(&k_empty[cid][s], kConsumerThreads);
6920             }
6921             TmaBarrier::init(&v_full[cid], 1);
6922             CtaBarrier::init(&v_empty[cid], kConsumerThreads);
6923         }
6924     }
6925     __syncthreads();
6926
6927     int q_tile = blockIdx.y * (seqlen / kTile) + blockIdx.x;
6928     int kv_tiles = seqlen / kTile;
6929

```

```

6930 // =====
6931 // Producer Warpgroup (WG0, threadIdx.x 0~127)
6932 // Only wg_tid == 0 issues TMA. Load Q once, then K-first KV loop.
6933 // Sk=1: load K[tile] + V[tile] per iteration.
6934 // Sk>=2: warmup prefetch K[cid][0], then V[tile] + prefetch K[tile+2].
6935 // =====
6936 if (is_producer) {
6937     if (wg_tid == 0) {
6938         auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6939         auto gQ = local_tile(
6940             mQ, Shape<_64, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
6941         auto q_slice = tma_q.get_slice(_0{});
6942         auto tQgQ = q_slice.partition_S(gQ);
6943         auto tQsQ = q_slice.partition_D(sQ);
6944         TmaBarrier::arrive_and_expect_tx(
6945             &q_full, sizeof(Element) * size(sQ));
6946         copy(tma_q.with(q_full), tQgQ, tQsQ);
6947
6948         auto mK = tma_k.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6949         auto mV = tma_v.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6950         auto k_slice = tma_k.get_slice(_0{});
6951         auto v_slice = tma_v.get_slice(_0{});
6952
6953         // Warmup: prefetch first K tile per consumer (Sk>=2 only)
6954         if constexpr (kStagesK > 1) {
6955             for (int cid = 0; cid < kNumConsumers; ++cid) {
6956                 if (cid < kv_tiles) {
6957                     auto sK = make_tensor(
6958                         make_smem_ptr(k_base + cid * kStagesK * kTileElements),
6959                         SmemLayout{});
6960                     CtaBarrier::wait(&k_empty[cid][0], 0);
6961                     auto gK = local_tile(
6962                         mK, Shape<_64, Int<kHeadDim>>{},
6963                         make_coord(blockIdx.y * kv_tiles + cid, _0{}));
6964                     auto tKgK = k_slice.partition_S(gK);
6965                     auto tKsK = k_slice.partition_D(sK);
6966                     TmaBarrier::arrive_and_expect_tx(
6967                         &k_full[cid][0], sizeof(Element) * size(sK));
6968                     copy(tma_k.with(k_full[cid][0]), tKgK, tKsK);
6969                 }
6970             }
6971         }
6972
6973         for (int tile = 0; tile < kv_tiles; ++tile) {
6974             int cid = tile & 1;
6975             int local = (tile - cid) / kNumConsumers;
6976             int kv_tile = blockIdx.y * kv_tiles + tile;
6977
6978             // Step 1: K — load current (Sk=1) or prefetch next (Sk>=2)
6979             if constexpr (kStagesK == 1) {
6980                 auto sK = make_tensor(
6981                     make_smem_ptr(k_base + cid * kStagesK * kTileElements),
6982                     SmemLayout{});
6983                 CtaBarrier::wait(&k_empty[cid][0], local & 1);
6984                 auto gK = local_tile(
6985                     mK, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_tile, _0{}));
6986                 auto tKgK = k_slice.partition_S(gK);
6987                 auto tKsK = k_slice.partition_D(sK);
6988                 TmaBarrier::arrive_and_expect_tx(
6989                     &k_full[cid][0], sizeof(Element) * size(sK));
6990                 copy(tma_k.with(k_full[cid][0]), tKgK, tKsK);
6991             } else {
6992                 int stg_next = (local + 1) % kStagesK;

```

```

6993     int next_tile = tile + kNumConsumers;
6994     if (next_tile < kv_tiles) {
6995         int next_local = local + 1;
6996         int next_phase = (next_local / kStagesK) & 1;
6997         auto sK = make_tensor(
6998             make_smem_ptr(k_base +
6999                 (cid * kStagesK + stg_next) * kTileElements),
7000             SmemLayout{});
7001         CtaBarrier::wait(&k_empty[cid][stg_next], next_phase);
7002         auto gK = local_tile(
7003             mK, Shape<_64, Int<kHeadDim>>{},
7004             make_coord(blockIdx.y * kv_tiles + next_tile, _0{}));
7005         auto tKgK = k_slice.partition_S(gK);
7006         auto tKsK = k_slice.partition_D(sK);
7007         TmaBarrier::arrive_and_expect_tx(
7008             &k_full[cid][stg_next], sizeof(Element) * size(sK));
7009         copy(tma_k.with(k_full[cid][stg_next]), tKgK, tKsK);
7010     }
7011 }
7012
7013 // Step 2: V — single buffer, release depends on PV (late step).
7014 // V TMA latency is hidden by consumer QK+softmax compute.
7015 auto sV = make_tensor(
7016     make_smem_ptr(v_base + cid * kTileElements), SmemLayout{});
7017 CtaBarrier::wait(&v_empty[cid], local & 1);
7018 auto gV = local_tile(
7019     mV, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_tile, _0{}));
7020 auto tVgV = v_slice.partition_S(gV);
7021 auto tVsV = v_slice.partition_D(sV);
7022 TmaBarrier::arrive_and_expect_tx(
7023     &v_full[cid], sizeof(Element) * size(sV));
7024 copy(tma_v.with(v_full[cid]), tVgV, tVsV);
7025 }
7026 }
7027 }
7028 // =====
7029 // Consumer Warpgroups (WG1 consumer_id=0, WG2 consumer_id=1)
7030 // Each WG processes every other KV tile independently.
7031 // K staged (kStagesK depth), V single buffer (always 1).
7032 // Flow: wait K → QK → release K → softmax → wait V → PV → release V → rescale.
7033 // =====
7034 else {
7035     TmaBarrier::wait(&q_full, 0);
7036
7037     auto sV = make_tensor(
7038         make_smem_ptr(v_base + consumer_id * kTileElements), SmemLayout{});
7039     auto sVt = make_tensor(sV.data(), typename Traits::SmemLayoutVt{});
7040     auto sVt_ns = make_tensor(
7041         sV.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));
7042
7043     typename Traits::TiledMma tiled_mma;
7044     auto thr_mma = tiled_mma.get_thread_slice(wg_tid);
7045     auto tCrQ = thr_mma.partition_fragment_A(sQ);
7046     auto tCrV_layout = thr_mma.partition_fragment_B(sVt_ns).layout();
7047     auto tCr0 = partition_fragment_C(
7048         tiled_mma, Shape<_64, Int<kHeadDim>>{});
7049     clear(tCr0);
7050
7051     auto s2r_copy_q = make_tiled_copy_A(
7052         typename Traits::SmemCopyAtom{}, tiled_mma);
7053     auto s2r_thr_q = s2r_copy_q.get_thread_slice(wg_tid);
7054     auto tQsQ_s2r = s2r_thr_q.partition_S(sQ);
7055     auto s2r_copy_k = make_tiled_copy_B(

```

```

7056     typename Traits::SmemCopyAtom{}, tiled_mma);
7057     auto s2r_thr_k = s2r_copy_k.get_thread_slice(wg_tid);
7058     auto s2r_copy_v = make_tiled_copy_B(
7059         typename Traits::SmemCopyAtomTransposed{}, tiled_mma);
7060     auto s2r_thr_v = s2r_copy_v.get_thread_slice(wg_tid);
7061     auto tVsVt_s2r = s2r_thr_v.partition_S(sVt);
7062
7063     auto tCr0_rc = make_tensor(
7064         tCr0.data(),
7065         fa_cute::convert_layout_acc_rowcol(tCr0.layout()));
7066     constexpr int kRows = decltype(size<0>(tCr0_rc)).value;
7067     float row_max[kRows], row_sum[kRows];
7068     #pragma unroll
7069     for (int r = 0; r < kRows; ++r) {
7070         row_max[r] = -INFINITY;
7071         row_sum[r] = 0.0f;
7072     }
7073
7074     // Init: mark own K/V slots as empty
7075     for (int s = 0; s < kStagesK; ++s)
7076         CtaBarrier::arrive(&k_empty[consumer_id][s]);
7077     CtaBarrier::arrive(&v_empty[consumer_id]);
7078
7079     float scale = rsqrtf(static_cast<float>(kHeadDim));
7080     int local_iter = 0;
7081     for (int tile = consumer_id; tile < kv_tiles;
7082         tile += kNumConsumers, ++local_iter) {
7083         int k_stg = local_iter % kStagesK;
7084         int k_phase = (local_iter / kStagesK) & 1;
7085
7086         // Per-stage K smem and S2R partition
7087         auto sK_stg = make_tensor(
7088             make_smem_ptr(k_base +
7089                 (consumer_id * kStagesK + k_stg) * kTileElements),
7090             SmemLayout{});
7091         auto tCrK = thr_mma.partition_fragment_B(sK_stg);
7092         CUTE_STATIC_ASSERT_V(size(tCrK) == size(tCrV_layout));
7093         auto tCrV = make_tensor(tCrK.data(), tCrV_layout);
7094         auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);
7095
7096         // 1) Wait K, QK gemm, release K early
7097         TmaBarrier::wait(&k_full[consumer_id][k_stg], k_phase);
7098         auto tCrS = partition_fragment_C(tiled_mma, Shape<_64, _64>{});
7099         clear(tCrS);
7100         fa_cute::gemm_ss(
7101             tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
7102             tiled_mma, s2r_copy_q, s2r_copy_k, s2r_thr_q, s2r_thr_k);
7103         { CtaBarrier::arrive(&k_empty[consumer_id][k_stg]); }
7104
7105         // 2) Online softmax (overlaps with producer's V TMA)
7106         auto scores = make_tensor(
7107             tCrS.data(),
7108             fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
7109         #pragma unroll
7110         for (int r = 0; r < kRows; ++r) {
7111             float tile_max = -INFINITY;
7112             #pragma unroll
7113             for (int c = 0; c < size<1>(scores); ++c)
7114                 tile_max = fmaxf(tile_max, scores(r, c) * scale);
7115             tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
7116             tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
7117             float nxt = fmaxf(row_max[r], tile_max);
7118             float rs = __expf(row_max[r] - nxt);

```

```

7119 #pragma unroll
7120     for (int c = 0; c < size<1>(tCr0_rc); ++c)
7121         tCr0_rc(r, c) *= rs;
7122     float ts = 0.0f;
7123 #pragma unroll
7124     for (int c = 0; c < size<1>(scores); ++c) {
7125         float p = __expf(scores(r, c) * scale - nxt);
7126         scores(r, c) = p;
7127         ts += p;
7128     }
7129     ts += __shfl_xor_sync(0xffffffff, ts, 1);
7130     ts += __shfl_xor_sync(0xffffffff, ts, 2);
7131     row_sum[r] = row_sum[r] * rs + ts;
7132     row_max[r] = nxt;
7133 }
7134
7135 // 3) Wait V, PV gemm, release V early
7136 TmaBarrier::wait(&v_full[consumer_id], local_iter & 1);
7137 auto tCrP = fa_cute::convert_type<Element>(tCrS);
7138 auto tCrPv = make_tensor(
7139     tCrP.data(),
7140     fa_cute::convert_layout_acc_Aregs<typename Traits::TiledMma>(
7141         tCrP.layout()));
7142 fa_cute::gemm_rs(
7143     tCr0, tCrPv, tCrV, tVsVt_s2r,
7144     tiled_mma, s2r_copy_v, s2r_thr_v);
7145 { CtaBarrier::arrive(&v_empty[consumer_id]); }
7146 }
7147
7148 // Split-KV merge
7149 __syncthreads();
7150 float *merge_o = reinterpret_cast<float *>(shm);
7151 float4 *merge_stats = reinterpret_cast<float4 *>(
7152     merge_o + 128 * size(tCr0));
7153
7154 if (consumer_id == 1) {
7155 #pragma unroll
7156     for (int idx = 0; idx < size(tCr0); ++idx)
7157         merge_o[idx * 128 + wg_tid] = tCr0(idx);
7158     merge_stats[wg_tid] = make_float4(
7159         row_max[0], row_max[1], row_sum[0], row_sum[1]);
7160 }
7161 __syncthreads();
7162
7163 if (consumer_id == 0) {
7164     float4 st = merge_stats[wg_tid];
7165     float o_max[kRows] = {st.x, st.y};
7166     float o_sum[kRows] = {st.z, st.w};
7167 #pragma unroll
7168     for (int r = 0; r < kRows; ++r) {
7169         float mg = fmaxf(row_max[r], o_max[r]);
7170         float lhs = __expf(row_max[r] - mg);
7171         float rhs = __expf(o_max[r] - mg);
7172         float ms = lhs * row_sum[r] + rhs * o_sum[r];
7173 #pragma unroll
7174         for (int c = 0; c < size<1>(tCr0_rc); ++c) {
7175             int idx = tCr0_rc.layout()(make_coord(r, c));
7176             float ov = merge_o[idx * 128 + wg_tid];
7177             tCr0_rc(r, c) = (lhs * tCr0_rc(r, c) + rhs * ov) / ms;
7178         }
7179     }
7180
7181     auto tCr0_half = fa_cute::convert_type<Element>(tCr0);

```

```

7182     auto m0 = make_tensor(
7183         make_gmem_ptr(output),
7184         make_shape(rows, Int<kHeadDim>{}),
7185         make_stride(Int<kHeadDim>{}, _1{}));
7186     auto g0 = local_tile(
7187         m0, Shape<_64, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
7188     auto tCg0 = thr_mma.partition_C(g0);
7189     copy(tCr0_half, tCg0);
7190 }
7191 }
7192 }
7193
7194
7195 template <int kHeadDim, typename TmaQ>
7196 __global__ void flash_attn_3_cute_tma_copy_smoke(
7197     CUTLASS_GRID_CONSTANT TmaQ const tma_q,
7198     cutlass::half_t *output, int rows) {
7199     using namespace cute;
7200     using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
7201     using Element = typename Traits::Element;
7202     using SmemLayout = typename Traits::SmemLayoutQKV;
7203     using TransactionBarrier = cutlass::arch::ClusterTransactionBarrier;
7204
7205     extern __shared__ __align__(1024) Element shared_storage[];
7206     auto sQ = make_tensor(make_smem_ptr(shared_storage), SmemLayout{});
7207     __shared__ uint64_t tma_barrier;
7208
7209     if (threadIdx.x == 0) {
7210         TransactionBarrier::init(&tma_barrier, 1);
7211     }
7212     __syncthreads();
7213
7214     auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
7215     auto gQ = local_tile(
7216         mQ, Shape<_64, Int<kHeadDim>>{},
7217         make_coord(blockIdx.x, _0{}));
7218     auto cta_tma_q = tma_q.get_slice(_0{});
7219     auto tQgQ = cta_tma_q.partition_S(gQ);
7220     auto tQsQ = cta_tma_q.partition_D(sQ);
7221
7222     if (threadIdx.x == 0) {
7223         TransactionBarrier::arrive_and_expect_tx(
7224             &tma_barrier, sizeof(half) * size(sQ));
7225         copy(tma_q.with(tma_barrier), tQgQ, tQsQ);
7226     }
7227     TransactionBarrier::wait(&tma_barrier, 0);
7228
7229     for (int linear = threadIdx.x; linear < size(sQ); linear += blockDim.x) {
7230         int row = linear / kHeadDim;
7231         int col = linear % kHeadDim;
7232         output[(blockIdx.x * 64 + row) * kHeadDim + col] = sQ(row, col);
7233     }
7234 }
7235 #endif
7236 #endif // END NOTES_V2_ENABLE_TMA_MMA_WS
7237
7238 // =====
7239 // 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
7240 // =====
7241
7242 // Bench / test mode globals (placed early so all functions can reference them)
7243 static bool g_debug = false;
7244 static bool g_bench_hgemm = false;

```

```

7245 static bool g_bench_hgemm_all = false;
7246 static bool g_bench_fa = false;
7247 static bool g_bench_fa3_cute_only = false;
7248 static bool g_bench_all = false;
7249 static bool g_fa_skip_check = false;
7250 static bool g_swizzle_eq_check = false;
7251 enum class FALayout {
7252     All, Pad, SwizzleQ, SwizzleK, SwizzleV, SwizzleQK, SwizzleQV, SwizzleKV, Swizzle
7253 };
7254 static FALayout g_fa_layout = FALayout::Pad;
7255 static int g_bench_M = 4096, g_bench_N = 4096, g_bench_K = 4096;
7256 static int g_bench_B = 1, g_bench_H = 48, g_bench_Nfa = 8192, g_bench_D = 128;
7257 static int g_warmup = 2, g_repeat = 3;
7258 static float g_fa_f16_max_tflops = 0.0f;
7259 static float g_fa_f32_max_tflops = 0.0f;
7260 static bool g_verbose = false;
7261
7262 // Decide whether to print a FA TFLOPS line. When --verbose/--debug is off,
7263 // only print when the current TFLOPS exceeds the running max for its
7264 // accumulator category (f16/f32). Correctness failures always print so they
7265 // are never silently dropped (callers gate FAIL paths themselves).
7266 static bool should_print_fa_tflops(int acc_f32, float tflops) {
7267     if (g_verbose || g_debug) return true;
7268     float &max_tflops = acc_f32 ? g_fa_f32_max_tflops : g_fa_f16_max_tflops;
7269     if (tflops > max_tflops) {
7270         max_tflops = tflops;
7271         return true;
7272     }
7273     return false;
7274 }
7275
7276 static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
7277     int device = 0;
7278     int max_smem = 0;
7279     cudaFuncAttributes attrs{};
7280     cudaGetDevice(&device);
7281     cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
7282     cudaFuncGetAttributes(&attrs, kernel_func);
7283     return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
7284 }
7285
7286 static inline void check(cudaError_t err, const char *msg) {
7287     if (err != cudaSuccess) {
7288         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
7289         exit(EXIT_FAILURE);
7290     }
7291 }
7292
7293 #if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7294 template <int kHeadDim>
7295 static void test_flash_attn_3_cute_tma_copy_smoke() {
7296     using namespace cute;
7297     using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
7298     using SmemLayout = typename Traits::SmemLayoutQKV;
7299     constexpr int kRows = 128;
7300     constexpr int kCount = kRows * kHeadDim;
7301
7302     half *h_input = (half *)malloc(kCount * sizeof(half));
7303     half *h_output = (half *)malloc(kCount * sizeof(half));
7304     for (int idx = 0; idx < kCount; ++idx) {
7305         h_input[idx] = __float2half((idx % 251) * 0.00390625f);
7306     }
7307

```

```

7308     half *d_input;
7309     half *d_output;
7310     check(cudaMalloc(&d_input, kCount * sizeof(half)), "cute tma smoke alloc input");
7311     check(cudaMalloc(&d_output, kCount * sizeof(half)), "cute tma smoke alloc output");
7312     check(cudaMemcpy(d_input, h_input, kCount * sizeof(half), cudaMemcpyHostToDevice),
7313           "cute tma smoke H2D");
7314
7315     auto mQ = make_tensor(
7316         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_input)),
7317         make_shape(kRows, Int<kHeadDim>{}),
7318         make_stride(Int<kHeadDim>{}, _1{}));
7319     auto tma_q = make_tma_copy(
7320         SM90_TMA_LOAD{}, mQ, SmemLayout{},
7321         Shape<_64, Int<kHeadDim>>{}, _1{});
7322     auto kernel = flash_attn_3_cute_tma_copy_smoke<kHeadDim, decltype(tma_q)>;
7323     constexpr int kSmemBytes = cosize(SmemLayout{}) * sizeof(cutlass::half_t);
7324     check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
7325                               kSmemBytes),
7326           "cute tma smoke set smem");
7327     kernel<<<2, 128, kSmemBytes>>>(<
7328         tma_q, reinterpret_cast<cutlass::half_t *>(d_output), kRows);
7329     check(cudaGetLastError(), "cute tma smoke launch");
7330     check(cudaDeviceSynchronize(), "cute tma smoke sync");
7331     check(cudaMemcpy(h_output, d_output, kCount * sizeof(half), cudaMemcpyDeviceToHost),
7332           "cute tma smoke D2H");
7333
7334     float max_err = 0.0f;
7335     for (int idx = 0; idx < kCount; ++idx) {
7336         max_err = max(max_err, fabsf(__half2float(h_input[idx]) -
7337                                     __half2float(h_output[idx])));
7338     }
7339     char label[64];
7340     snprintf(label, sizeof(label), "CuTe TMA copy smoke (D=%d)", kHeadDim);
7341     printf("| %-56s | %.3e |\n", label, max_err);
7342
7343     free(h_input);
7344     free(h_output);
7345     cudaFree(d_input);
7346     cudaFree(d_output);
7347 }
7348
7349 template <int kHeadDim>
7350 static void test_flash_attn_3_tma_mma_ws_split_q_cute() {
7351     using namespace cute;
7352     using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
7353     using SmemLayout = typename Traits::SmemLayoutQKV;
7354     constexpr int kSeqLen = 128;
7355     constexpr int kCount = kSeqLen * kHeadDim;
7356
7357     half *h_q = (half *)malloc(kCount * sizeof(half));
7358     half *h_k = (half *)malloc(kCount * sizeof(half));
7359     half *h_v = (half *)malloc(kCount * sizeof(half));
7360     half *h_o = (half *)malloc(kCount * sizeof(half));
7361     float *ref_o = (float *)malloc(kCount * sizeof(float));
7362     srand(42 + kHeadDim);
7363     for (int idx = 0; idx < kCount; ++idx) {
7364         h_q[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7365         h_k[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7366         h_v[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7367     }
7368
7369     float scale = 1.0f / sqrtf((float)kHeadDim);
7370     for (int row = 0; row < kSeqLen; ++row) {

```

```

7371 float scores[kSeqlen];
7372 float row_max = -INFINITY;
7373 for (int col = 0; col < kSeqlen; ++col) {
7374     float score = 0.0f;
7375     for (int dim = 0; dim < kHeadDim; ++dim) {
7376         score += __half2float(h_q[row * kHeadDim + dim]) *
7377             __half2float(h_k[col * kHeadDim + dim]);
7378     }
7379     scores[col] = score * scale;
7380     row_max = max(row_max, scores[col]);
7381 }
7382 float row_sum = 0.0f;
7383 for (int col = 0; col < kSeqlen; ++col) {
7384     scores[col] = expf(scores[col] - row_max);
7385     row_sum += scores[col];
7386 }
7387 for (int dim = 0; dim < kHeadDim; ++dim) {
7388     float output = 0.0f;
7389     for (int col = 0; col < kSeqlen; ++col) {
7390         output += scores[col] * __half2float(h_v[col * kHeadDim + dim]);
7391     }
7392     ref_o[row * kHeadDim + dim] = output / row_sum;
7393 }
7394 }
7395
7396 half *d_q;
7397 half *d_k;
7398 half *d_v;
7399 half *d_o;
7400 check(cudaMalloc(&d_q, kCount * sizeof(half)), "cute fa3 alloc Q");
7401 check(cudaMalloc(&d_k, kCount * sizeof(half)), "cute fa3 alloc K");
7402 check(cudaMalloc(&d_v, kCount * sizeof(half)), "cute fa3 alloc V");
7403 check(cudaMalloc(&d_o, kCount * sizeof(half)), "cute fa3 alloc O");
7404 check(cudaMemcpy(d_q, h_q, kCount * sizeof(half), cudaMemcpyHostToDevice),
7405     "cute fa3 H2D Q");
7406 check(cudaMemcpy(d_k, h_k, kCount * sizeof(half), cudaMemcpyHostToDevice),
7407     "cute fa3 H2D K");
7408 check(cudaMemcpy(d_v, h_v, kCount * sizeof(half), cudaMemcpyHostToDevice),
7409     "cute fa3 H2D V");
7410
7411 auto make_tma = [=](half *pointer) {
7412     auto tensor = make_tensor(
7413         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
7414         make_shape(kSeqlen, Int<kHeadDim>{}),
7415         make_stride(Int<kHeadDim>{}, _1{}));
7416     return make_tma_copy(
7417         SM90_TMA_LOAD{}, tensor, SmemLayout{},
7418         Shape<_64, Int<kHeadDim>>{}, _1{});
7419 };
7420 auto tma_q = make_tma(d_q);
7421 auto tma_k = make_tma(d_k);
7422 auto tma_v = make_tma(d_v);
7423 auto kernel = flash_attn_3_tma_mma_ws_split_q_cute<
7424     kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v)>;
7425 auto acc_o = partition_fragment_C(
7426     typename Traits::TiledMma{}, Shape<_64, Int<kHeadDim>>{});
7427 constexpr int kNumConsumers = 2;
7428 constexpr int kStagesK = 1;
7429 constexpr int kTiles = 1 + kNumConsumers * kStagesK + kNumConsumers;
7430 constexpr int kTilesBytes =
7431     kTiles * cosize(SmemLayout{}) * sizeof(cutlass::half_t);
7432 int merge_bytes = 128 * size(acc_o) * sizeof(float) + 128 * sizeof(float4);
7433 int smem_bytes = max(kTilesBytes, merge_bytes);

```

```

7434 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
7435                             smem_bytes),
7436         "cute fa3 set smem");
7437 kernel<<<dim3(2, 1), 384, smem_bytes>>>(
7438     tma_q, tma_k, tma_v,
7439     reinterpret_cast<cutlass::half_t*>(d_o), kSeqlen, kSeqlen);
7440 check(cudaGetLastError(), "cute fa3 launch");
7441 check(cudaDeviceSynchronize(), "cute fa3 sync");
7442 check(cudaMemcpy(h_o, d_o, kCount * sizeof(half), cudaMemcpyDeviceToHost),
7443         "cute fa3 D2H");
7444
7445 float max_err = 0.0f;
7446 for (int idx = 0; idx < kCount; ++idx) {
7447     max_err = max(max_err, fabsf(__half2float(h_o[idx]) - ref_o[idx]));
7448 }
7449 char label[80];
7450 snprintf(label, sizeof(label), "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=1, Sv=1, F32Acc, D=%d)", kHeadDim);
7451 printf("| %-56s | %.3e |\n", label, max_err);
7452
7453 free(h_q);
7454 free(h_k);
7455 free(h_v);
7456 free(h_o);
7457 free(ref_o);
7458 cudaFree(d_q);
7459 cudaFree(d_k);
7460 cudaFree(d_v);
7461 cudaFree(d_o);
7462 }
7463 #endif
7464
7465 #if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7466 // FA2-style CuTe test: single consumer, Br=128, no merge.
7467 // kSeqlen=256 -> 2 Q tiles, covers multi-Q-tile path.
7468 template <int kHeadDim>
7469 static void test_flash_attn_tma_mma_ws_split_q_cute() {
7470     using namespace cute;
7471     using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
7472     using SmemLayoutQ = typename Traits::SmemLayoutQ;
7473     using SmemLayoutKV = typename Traits::SmemLayoutKV;
7474     constexpr int kBr = 128;
7475     constexpr int kSeqlen = 256;
7476     constexpr int kCount = kSeqlen * kHeadDim;
7477
7478     half *h_q = (half *)malloc(kCount * sizeof(half));
7479     half *h_k = (half *)malloc(kCount * sizeof(half));
7480     half *h_v = (half *)malloc(kCount * sizeof(half));
7481     half *h_o = (half *)malloc(kCount * sizeof(half));
7482     float *ref_o = (float *)malloc(kCount * sizeof(float));
7483     srand(42 + kHeadDim);
7484     for (int idx = 0; idx < kCount; ++idx) {
7485         h_q[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7486         h_k[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7487         h_v[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
7488     }
7489
7490     float scale = 1.0f / sqrtf((float)kHeadDim);
7491     for (int row = 0; row < kSeqlen; ++row) {
7492         float scores[kSeqlen];
7493         float row_max = -INFINITY;
7494         for (int col = 0; col < kSeqlen; ++col) {
7495             float score = 0.0f;

```

```

7496     for (int dim = 0; dim < kHeadDim; ++dim) {
7497         score += __half2float(h_q[row * kHeadDim + dim]) *
7498             __half2float(h_k[col * kHeadDim + dim]);
7499     }
7500     scores[col] = score * scale;
7501     row_max = max(row_max, scores[col]);
7502 }
7503 float row_sum = 0.0f;
7504 for (int col = 0; col < kSeqlen; ++col) {
7505     scores[col] = expf(scores[col] - row_max);
7506     row_sum += scores[col];
7507 }
7508 for (int dim = 0; dim < kHeadDim; ++dim) {
7509     float output = 0.0f;
7510     for (int col = 0; col < kSeqlen; ++col) {
7511         output += scores[col] * __half2float(h_v[col * kHeadDim + dim]);
7512     }
7513     ref_o[row * kHeadDim + dim] = output / row_sum;
7514 }
7515 }
7516
7517 half *d_q;
7518 half *d_k;
7519 half *d_v;
7520 half *d_o;
7521 check(cudaMalloc(&d_q, kCount * sizeof(half)), "cute fa2 alloc Q");
7522 check(cudaMalloc(&d_k, kCount * sizeof(half)), "cute fa2 alloc K");
7523 check(cudaMalloc(&d_v, kCount * sizeof(half)), "cute fa2 alloc V");
7524 check(cudaMalloc(&d_o, kCount * sizeof(half)), "cute fa2 alloc O");
7525 check(cudaMemcpy(d_q, h_q, kCount * sizeof(half), cudaMemcpyHostToDevice),
7526     "cute fa2 H2D Q");
7527 check(cudaMemcpy(d_k, h_k, kCount * sizeof(half), cudaMemcpyHostToDevice),
7528     "cute fa2 H2D K");
7529 check(cudaMemcpy(d_v, h_v, kCount * sizeof(half), cudaMemcpyHostToDevice),
7530     "cute fa2 H2D V");
7531
7532 // Q tile: [128, D], K/V tile: [64, D] (both use GMMMA::Layout_K_SW128_Atom)
7533 auto make_tma_q = [=]() {
7534     auto tensor = make_tensor(
7535         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_q)),
7536         make_shape(kSeqlen, Int<kHeadDim>{}),
7537         make_stride(Int<kHeadDim>{}, _1{}));
7538     return make_tma_copy(
7539         SM90_TMA_LOAD{}, tensor, SmemLayoutQ{},
7540         Shape<_128, Int<kHeadDim>>{}, _1{});
7541 };
7542 auto make_tma_kv = [=](half *pointer) {
7543     auto tensor = make_tensor(
7544         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
7545         make_shape(kSeqlen, Int<kHeadDim>{}),
7546         make_stride(Int<kHeadDim>{}, _1{}));
7547     return make_tma_copy(
7548         SM90_TMA_LOAD{}, tensor, SmemLayoutKV{},
7549         Shape<_64, Int<kHeadDim>>{}, _1{});
7550 };
7551 auto tma_q = make_tma_q();
7552 auto tma_k = make_tma_kv(d_k);
7553 auto tma_v = make_tma_kv(d_v);
7554 auto kernel = flash_attn_tma_mma_ws_split_q_cute<
7555     kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v)>;
7556 // smem = Q[128*D] + K[Sk*64*D] + V[Sv*64*D] (Sk=1, Sv=1 default)
7557 constexpr int kSmemBytes =
7558     (cosize(SmemLayoutQ{}) +

```

```

7559     1 * cosize(SmemLayoutKV{}}) +
7560     1 * cosize(SmemLayoutKV{}})) * sizeof(cutlass::half_t);
7561     check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
7562                                kSmemBytes),
7563            "cute fa2 set smem");
7564     kernel<<<dim3(kSeqlen / kBr, 1), 384, kSmemBytes>>>(
7565         tma_q, tma_k, tma_v,
7566         reinterpret_cast<cutlass::half_t *>(d_o), kSeqlen, kSeqlen);
7567     check(cudaGetLastError(), "cute fa2 launch");
7568     check(cudaDeviceSynchronize(), "cute fa2 sync");
7569     check(cudaMemcpy(h_o, d_o, kCount * sizeof(half), cudaMemcpyDeviceToHost),
7570            "cute fa2 D2H");
7571
7572     float max_err = 0.0f;
7573     for (int idx = 0; idx < kCount; ++idx) {
7574         max_err = max(max_err, fabsf(__half2float(h_o[idx]) - ref_o[idx]));
7575     }
7576     char label[80];
7577     snprintf(label, sizeof(label), "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=1, Sv=1, F32Acc, D=%d)", kHeadDim);
7578     printf("| %-56s | %.3e |\n", label, max_err);
7579
7580     free(h_q);
7581     free(h_k);
7582     free(h_v);
7583     free(h_o);
7584     free(ref_o);
7585     cudaFree(d_q);
7586     cudaFree(d_k);
7587     cudaFree(d_v);
7588     cudaFree(d_o);
7589 }
7590 #endif
7591
7592 static void test_block_reduce(int N) {
7593
7594     srand(42);
7595     float *h_a = (float *)malloc((size_t)N * sizeof(float));
7596     for (int i = 0; i < N; i++)
7597         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7598
7599     // CPU reference: sum of all elements
7600     double ref = 0.0;
7601     for (int i = 0; i < N; i++) ref += (double)h_a[i];
7602
7603     float *d_a, *d_y;
7604     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
7605     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
7606
7607     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
7608     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
7609
7610     dim3 block(128);
7611     dim3 grid((N + 127) / 128);
7612     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
7613     check(cudaGetLastError(), "blockreduce launch");
7614     check(cudaDeviceSynchronize(), "blockreduce sync");
7615
7616     float result;
7617     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
7618
7619     float err = fabsf(result - (float)ref);
7620     printf("| %-56s | %.3e |\n", "BlockReduce", err);

```

```

7621     free(h_a);
7622     cudaFree(d_a); cudaFree(d_y);
7623 }
7624
7625
7626
7627 static void test_dot(int N) {
7628
7629     srand(42);
7630     float *h_a = (float *)malloc((size_t)N * sizeof(float));
7631     float *h_b = (float *)malloc((size_t)N * sizeof(float));
7632     for (int i = 0; i < N; i++) {
7633         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7634         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7635     }
7636
7637     // CPU reference
7638     double ref = 0.0;
7639     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
7640
7641     float *d_a, *d_b, *d_y;
7642     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
7643     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
7644     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
7645
7646     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
7647     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
7648     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
7649
7650     dim3 block(128);
7651     dim3 grid((N + 127) / 128);
7652     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
7653     check(cudaGetLastError(), "dot launch");
7654     check(cudaDeviceSynchronize(), "dot sync");
7655
7656     float result;
7657     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
7658
7659     float err = fabsf(result - (float)ref);
7660     printf("| %-56s | %.3e |\n", "Dot", err);
7661
7662     // ---- Dot Vec4 ----
7663     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
7664     dim3 block_v4(32);
7665     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
7666     check(cudaGetLastError(), "dot_vec4 launch");
7667     check(cudaDeviceSynchronize(), "dot_vec4 sync");
7668
7669     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
7670     float err_v4 = fabsf(result - (float)ref);
7671     printf("| %-56s | %.3e |\n", "Dot-Vec4", err_v4);
7672
7673     free(h_a); free(h_b);
7674     cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
7675 }
7676
7677
7678 static void test_relu(int N) {
7679     srand(42);
7680     float *h_x = (float *)malloc((size_t)N * sizeof(float));
7681     float *h_y = (float *)malloc((size_t)N * sizeof(float));
7682     for (int i = 0; i < N; i++)
7683         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

7684
7685 float *d_x, *d_y;
7686 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
7687 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
7688 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
7689
7690 for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
7691 dim3 block256(256);
7692 dim3 grid256((N + 255) / 256);
7693 relu<<<grid256, block256>>>(d_x, d_y, N);
7694 check(cudaGetLastError(), "relu launch");
7695 check(cudaDeviceSynchronize(), "relu sync");
7696 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
7697 float max_err = 0.0f;
7698 for (int i = 0; i < N; i++) {
7699     float expected = fmaxf(0.0f, h_x[i]);
7700     float err = fabsf(h_y[i] - expected);
7701     if (err > max_err) max_err = err;
7702 }
7703 printf("| %-56s | %.3e |\n", "ReLU", max_err);
7704
7705 dim3 block64(64);
7706 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
7707 check(cudaGetLastError(), "relu_vec4 launch");
7708 check(cudaDeviceSynchronize(), "relu_vec4 sync");
7709 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
7710 max_err = 0.0f;
7711 for (int i = 0; i < N; i++) {
7712     float expected = fmaxf(0.0f, h_x[i]);
7713     float err = fabsf(h_y[i] - expected);
7714     if (err > max_err) max_err = err;
7715 }
7716 printf("| %-56s | %.3e |\n", "ReLU-Vec4", max_err);
7717
7718 free(h_x); free(h_y);
7719 cudaFree(d_x); cudaFree(d_y);
7720 }
7721
7722
7723 static void test_elementwise(int N) {
7724     srand(42);
7725     float *h_a = (float *)malloc((size_t)N * sizeof(float));
7726     float *h_b = (float *)malloc((size_t)N * sizeof(float));
7727     for (int i = 0; i < N; i++) {
7728         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7729         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7730     }
7731
7732     float *d_a, *d_b, *d_c;
7733     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
7734     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
7735     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
7736     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
7737     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
7738
7739     dim3 block256(256);
7740     dim3 grid256((N + 255) / 256);
7741     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
7742     check(cudaGetLastError(), "eadd launch");
7743     check(cudaDeviceSynchronize(), "eadd sync");
7744     float *h_c = (float *)malloc((size_t)N * sizeof(float));
7745     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
7746     float max_err = 0.0f;

```

```

7747     for (int i = 0; i < N; i++) {
7748         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
7749         if (err > max_err) max_err = err;
7750     }
7751     printf("| %-56s | %.3e |\n", "ElemwiseAdd", max_err);
7752
7753     dim3 block64(64);
7754     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
7755     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
7756     check(cudaGetLastError(), "eadd_vec4 launch");
7757     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
7758     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
7759     max_err = 0.0f;
7760     for (int i = 0; i < N; i++) {
7761         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
7762         if (err > max_err) max_err = err;
7763     }
7764     printf("| %-56s | %.3e |\n", "ElemwiseAdd-Vec4", max_err);
7765
7766     free(h_a); free(h_b); free(h_c);
7767     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
7768 }
7769
7770
7771 static void test_histogram(int N) {
7772     srand(42);
7773     int BINS = 16;
7774     int *h_hist = (int *)calloc(BINS, sizeof(int));
7775     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
7776     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
7777     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
7778     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
7779
7780     int *d_idx, *d_hist;
7781     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
7782     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
7783     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
7784     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
7785
7786     dim3 block256(256);
7787     dim3 grid256((N + 255) / 256);
7788     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
7789     check(cudaGetLastError(), "histogram launch");
7790     check(cudaDeviceSynchronize(), "histogram sync");
7791     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
7792
7793     float max_err = 0.0f;
7794     for (int i = 0; i < BINS; i++) {
7795         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
7796         if (err > max_err) max_err = err;
7797     }
7798     printf("| %-56s | %.3e |\n", "Histogram", max_err);
7799
7800     free(h_hist); free(h_hist_ref); free(h_idx);
7801     cudaFree(d_idx); cudaFree(d_hist);
7802 }
7803
7804
7805 static void test_merge_attn_states(int num_tokens, int num_heads,
7806                                   int head_size) {
7807
7808     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
7809     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);

```

```

7810
7811 float *h_prefix_out = (float *)malloc(out_size);
7812 float *h_suffix_out = (float *)malloc(out_size);
7813 float *h_prefix_lse = (float *)malloc(lse_size);
7814 float *h_suffix_lse = (float *)malloc(lse_size);
7815 float *h_output_ref = (float *)malloc(out_size);
7816
7817 srand(42);
7818 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
7819     h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7820     h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
7821 }
7822 for (int i = 0; i < num_heads * num_tokens; i++) {
7823     h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
7824     h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
7825 }
7826
7827 // CPU reference: 与 kernel 完全相同的浮点计算
7828 for (int t = 0; t < num_tokens; t++) {
7829     for (int h = 0; h < num_heads; h++) {
7830         float p_lse = h_prefix_lse[h * num_tokens + t];
7831         float s_lse = h_suffix_lse[h * num_tokens + t];
7832         p_lse = isinf(p_lse) ? -INFINITY : p_lse;
7833         s_lse = isinf(s_lse) ? -INFINITY : s_lse;
7834
7835         float max_lse = fmaxf(p_lse, s_lse);
7836         p_lse -= max_lse;
7837         s_lse -= max_lse;
7838         float p_se = expf(p_lse);
7839         float s_se = expf(s_lse);
7840         float p_scale = p_se / (p_se + s_se);
7841         float s_scale = s_se / (p_se + s_se);
7842
7843         int head_off = t * num_heads * head_size + h * head_size;
7844         for (int d = 0; d < head_size; d++) {
7845             h_output_ref[head_off + d] =
7846                 h_prefix_out[head_off + d] * p_scale +
7847                 h_suffix_out[head_off + d] * s_scale;
7848         }
7849     }
7850 }
7851
7852 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
7853 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
7854 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
7855 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
7856 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
7857 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
7858
7859 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
7860                 cudaMemcpyHostToDevice),
7861        "merge_attn H2D p_out");
7862 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
7863                 cudaMemcpyHostToDevice),
7864        "merge_attn H2D s_out");
7865 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
7866                 cudaMemcpyHostToDevice),
7867        "merge_attn H2D p_lse");
7868 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
7869                 cudaMemcpyHostToDevice),
7870        "merge_attn H2D s_lse");
7871
7872 int threads_per_head = head_size / 4;

```

```

7873 int total_threads = num_tokens * num_heads * threads_per_head;
7874 dim3 block(128);
7875 dim3 grid((total_threads + 127) / 128);
7876 merge_attn_states<<<grid, block>>>{
7877     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
7878     num_tokens, num_heads, head_size);
7879 check(cudaGetLastError(), "merge_attn launch");
7880 check(cudaDeviceSynchronize(), "merge_attn sync");
7881
7882 float *h_output = (float *)malloc(out_size);
7883 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
7884     "merge_attn D2H");
7885
7886 float max_err = 0.0f;
7887 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
7888     float err = fabsf(h_output[i] - h_output_ref[i]);
7889     if (err > max_err) max_err = err;
7890 }
7891 printf("| %-56s | %.3e |\n", "MergeAttnStates", max_err);
7892
7893 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
7894 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
7895 h_suffix_lse[0] = 0.0f;
7896 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
7897     cudaMemcpyHostToDevice),
7898     "merge_attn H2D inf lse");
7899 merge_attn_states<<<grid, block>>>{
7900     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
7901     num_tokens, num_heads, head_size);
7902 check(cudaGetLastError(), "merge_attn inf launch");
7903 check(cudaDeviceSynchronize(), "merge_attn inf sync");
7904 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
7905     "merge_attn inf D2H");
7906
7907 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
7908 float inf_err = 0.0f;
7909 for (int d = 0; d < head_size; d++) {
7910     float err = fabsf(h_output[d] - h_suffix_out[d]);
7911     if (err > inf_err) inf_err = err;
7912 }
7913 printf("| %-56s | %.3e |\n", "MergeAttnStates-inf", inf_err);
7914
7915 free(h_prefix_out);
7916 free(h_suffix_out);
7917 free(h_prefix_lse);
7918 free(h_suffix_lse);
7919 free(h_output_ref);
7920 free(h_output);
7921 cudaFree(d_prefix_out);
7922 cudaFree(d_suffix_out);
7923 cudaFree(d_prefix_lse);
7924 cudaFree(d_suffix_lse);
7925 cudaFree(d_output);
7926 }
7927
7928
7929 static void test_softmax(int N) {
7930     // Use N = blockDim.x so one block processes all elements as one token.
7931     constexpr int kNumThreads = 256;
7932     if (N != kNumThreads) {
7933         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
7934         return;
7935     }

```

```

7936 srand(42);
7937 float *h_x = (float *)malloc((size_t)N * sizeof(float));
7938 float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
7939 for (int i = 0; i < N; i++)
7940     h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
7941
7942 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
7943 float max_val = -FLT_MAX;
7944 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
7945 double sum_exp = 0.0;
7946 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
7947 for (int i = 0; i < N; i++)
7948     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
7949
7950 float *d_x, *d_y;
7951 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
7952 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
7953
7954 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
7955
7956 dim3 block(256);
7957 dim3 grid(1); // one token covering all N elements
7958 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
7959 check(cudaGetLastError(), "softmax launch");
7960 check(cudaDeviceSynchronize(), "softmax sync");
7961
7962 float *h_y = (float *)malloc((size_t)N * sizeof(float));
7963 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
7964
7965 float max_err = 0.0f;
7966 for (int i = 0; i < N; i++) {
7967     float err = fabsf(h_y[i] - h_y_ref[i]);
7968     if (err > max_err) max_err = err;
7969 }
7970 printf("| %-56s | %.3e |\n", "OnlineSafeSoftmax", max_err);
7971
7972 // ---- Safe Softmax ----
7973 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
7974 check(cudaGetLastError(), "safe_softmax launch");
7975 check(cudaDeviceSynchronize(), "safe_softmax sync");
7976 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
7977 max_err = 0.0f;
7978 for (int i = 0; i < N; i++) {
7979     float err = fabsf(h_y[i] - h_y_ref[i]);
7980     if (err > max_err) max_err = err;
7981 }
7982 printf("| %-56s | %.3e |\n", "SafeSoftmax", max_err);
7983
7984 // ---- Naive Softmax ----
7985 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
7986 check(cudaGetLastError(), "naive_softmax launch");
7987 check(cudaDeviceSynchronize(), "naive_softmax sync");
7988 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
7989 max_err = 0.0f;
7990 for (int i = 0; i < N; i++) {
7991     float err = fabsf(h_y[i] - h_y_ref[i]);
7992     if (err > max_err) max_err = err;
7993 }
7994 printf("| %-56s | %.3e |\n", "NaiveSoftmax", max_err);
7995
7996 free(h_x); free(h_y); free(h_y_ref);
7997 cudaFree(d_x); cudaFree(d_y);

```

```

7999 }
8000
8001
8002 static void test_rms_norm(int N, int K) {
8003
8004     srand(42);
8005     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
8006     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
8007     for (int i = 0; i < N * K; i++)
8008         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8009     float g = 1.5f; // gain
8010
8011     // CPU reference: y = (x / rms(x)) * g
8012     float epsilon = 1e-5f;
8013     for (int n = 0; n < N; n++) {
8014         double sum_sq = 0.0;
8015         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
8016         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
8017         for (int k = 0; k < K; k++)
8018             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
8019     }
8020
8021     float *d_x, *d_y;
8022     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
8023     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
8024     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
8025
8026     dim3 block(128);
8027     dim3 grid(N);
8028     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
8029     check(cudaGetLastError(), "rmsnorm launch");
8030     check(cudaDeviceSynchronize(), "rmsnorm sync");
8031
8032     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
8033     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
8034
8035     float max_err = 0.0f;
8036     for (int i = 0; i < N * K; i++) {
8037         float err = fabsf(h_y[i] - h_y_ref[i]);
8038         if (err > max_err) max_err = err;
8039     }
8040     printf("| %-56s | %.3e |\n", "RMSNorm", max_err);
8041
8042     // ---- RMS Norm Vec4 ----
8043     dim3 block_rv4(32);
8044     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
8045     check(cudaGetLastError(), "rmsnorm_vec4 launch");
8046     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
8047     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
8048     max_err = 0.0f;
8049     for (int i = 0; i < N * K; i++) {
8050         float err = fabsf(h_y[i] - h_y_ref[i]);
8051         if (err > max_err) max_err = err;
8052     }
8053     printf("| %-56s | %.3e |\n", "RMSNorm-Vec4", max_err);
8054
8055     free(h_x); free(h_y); free(h_y_ref);
8056     cudaFree(d_x); cudaFree(d_y);
8057 }
8058
8059
8060 static void test_layer_norm(int N, int K) {
8061

```

```

8062 srand(42);
8063 float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
8064 float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
8065 for (int i = 0; i < N * K; i++)
8066     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8067 float g = 1.5f, b = 0.3f; // gain and bias
8068
8069 // CPU reference: y = ((x - mean) / std) * g + b
8070 float epsilon = 1e-5f;
8071 for (int n = 0; n < N; n++) {
8072     double sum = 0.0;
8073     for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
8074     float mean = (float)sum / (float)K;
8075     double sum_sq = 0.0;
8076     for (int k = 0; k < K; k++) {
8077         float diff = h_x[n * K + k] - mean;
8078         sum_sq += (double)diff * (double)diff;
8079     }
8080     float std = sqrtf((float)sum_sq / (float)K + epsilon);
8081     for (int k = 0; k < K; k++)
8082         h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
8083 }
8084
8085 float *d_x, *d_y;
8086 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
8087 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
8088 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
8089
8090 dim3 block(128);
8091 dim3 grid(N);
8092 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
8093 check(cudaGetLastError(), "layernorm launch");
8094 check(cudaDeviceSynchronize(), "layernorm sync");
8095
8096 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
8097 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
8098
8099 float max_err = 0.0f;
8100 for (int i = 0; i < N * K; i++) {
8101     float err = fabsf(h_y[i] - h_y_ref[i]);
8102     if (err > max_err) max_err = err;
8103 }
8104 printf("| %-56s | %.3e |\n", "LayerNorm", max_err);
8105
8106 // ---- Layer Norm Vec4 ----
8107 dim3 block_lv4(32);
8108 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
8109 check(cudaGetLastError(), "layernorm_vec4 launch");
8110 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
8111 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
8112 max_err = 0.0f;
8113 for (int i = 0; i < N * K; i++) {
8114     float err = fabsf(h_y[i] - h_y_ref[i]);
8115     if (err > max_err) max_err = err;
8116 }
8117 printf("| %-56s | %.3e |\n", "LayerNorm-Vec4", max_err);
8118
8119 free(h_x); free(h_y); free(h_y_ref);
8120 cudaFree(d_x); cudaFree(d_y);
8121 }
8122
8123
8124 static void test_rope(int seq_len, int N) {

```

```

8125
8126 int total_pairs = seq_len * N;
8127 int total_elems = total_pairs * 2;
8128 size_t size = (size_t)total_elems * sizeof(float);
8129
8130 float *h_x = (float *)malloc(size);
8131 float *h_y_ref = (float *)malloc(size);
8132
8133 srand(42);
8134 for (int i = 0; i < total_elems; i++)
8135     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8136
8137 // CPU reference: 2D rotation for each pair
8138 for (int idx = 0; idx < total_pairs; idx++) {
8139     int token_pos = idx / N;
8140     int token_idx = idx % N;
8141     float x1 = h_x[idx * 2];
8142     float x2 = h_x[idx * 2 + 1];
8143     float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
8144     float angle = (float)token_pos * theta;
8145     float cos_v = cosf(angle);
8146     float sin_v = sinf(angle);
8147     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
8148     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
8149 }
8150
8151 float *d_x, *d_y;
8152 check(cudaMalloc(&d_x, size), "rope alloc X");
8153 check(cudaMalloc(&d_y, size), "rope alloc Y");
8154 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
8155
8156 dim3 block(256);
8157 dim3 grid((total_pairs + 255) / 256);
8158 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
8159 check(cudaGetLastError(), "rope launch");
8160 check(cudaDeviceSynchronize(), "rope sync");
8161
8162 float *h_y = (float *)malloc(size);
8163 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
8164
8165 float max_err = 0.0f;
8166 for (int i = 0; i < total_elems; i++) {
8167     float err = fabsf(h_y[i] - h_y_ref[i]);
8168     if (err > max_err) max_err = err;
8169 }
8170 printf("| %-56s | %.3e |\n", "RoPE", max_err);
8171
8172 free(h_x);
8173 free(h_y);
8174 free(h_y_ref);
8175 cudaFree(d_x);
8176 cudaFree(d_y);
8177 }
8178
8179
8180 static void test_mat_transpose(int row, int col) {
8181
8182     size_t size_in = (size_t)row * col * sizeof(float);
8183     size_t size_out = (size_t)col * row * sizeof(float);
8184
8185     float *h_x = (float *)malloc(size_in);
8186     float *h_y_ref = (float *)malloc(size_out);
8187

```

```

8188 srand(42);
8189 for (int i = 0; i < row * col; i++)
8190     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8191
8192 // CPU reference: y[j][i] = x[i][j] (row-major)
8193 for (int i = 0; i < row; i++)
8194     for (int j = 0; j < col; j++)
8195         h_y_ref[j * row + i] = h_x[i * col + j];
8196
8197 float *d_x, *d_y;
8198 check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
8199 check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
8200 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
8201
8202 dim3 block(16, 16);
8203 dim3 grid((col + 15) / 16, (row + 15) / 16);
8204 mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
8205 check(cudaGetLastError(), "mattrans launch");
8206 check(cudaDeviceSynchronize(), "mattrans sync");
8207
8208 float *h_y = (float *)malloc(size_out);
8209 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
8210
8211 float max_err = 0.0f;
8212 for (int i = 0; i < col * row; i++) {
8213     float err = fabsf(h_y[i] - h_y_ref[i]);
8214     if (err > max_err) max_err = err;
8215 }
8216 printf("| %-56s | %.3e |\n", "MatTranspose", max_err);
8217
8218 free(h_x);
8219 free(h_y);
8220 free(h_y_ref);
8221 cudaFree(d_x);
8222 cudaFree(d_y);
8223 }
8224
8225
8226 static void test_mat_transpose_padded(int row, int col) {
8227
8228     size_t size_in = (size_t)row * col * sizeof(float);
8229     size_t size_out = (size_t)col * row * sizeof(float);
8230
8231     float *h_x = (float *)malloc(size_in);
8232     float *h_y_ref = (float *)malloc(size_out);
8233
8234     srand(42);
8235     for (int i = 0; i < row * col; i++)
8236         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8237
8238     // CPU reference: y[j][i] = x[i][j] (row-major)
8239     for (int i = 0; i < row; i++)
8240         for (int j = 0; j < col; j++)
8241             h_y_ref[j * row + i] = h_x[i * col + j];
8242
8243     float *d_x, *d_y;
8244     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
8245     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
8246     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
8247
8248     dim3 block(16, 16);
8249     dim3 grid((col + 15) / 16, (row + 63) / 64);
8250     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);

```

```

8251 check(cudaGetLastError(), "mattrans_padded launch");
8252 check(cudaDeviceSynchronize(), "mattrans_padded sync");
8253
8254 float *h_y = (float *)malloc(size_out);
8255 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
8256
8257 float max_err = 0.0f;
8258 for (int i = 0; i < col * row; i++) {
8259     float err = fabsf(h_y[i] - h_y_ref[i]);
8260     if (err > max_err) max_err = err;
8261 }
8262 printf("| %-56s | %.3e |\n", "MatTransposePadded", max_err);
8263
8264 free(h_x);
8265 free(h_y);
8266 free(h_y_ref);
8267 cudaFree(d_x);
8268 cudaFree(d_y);
8269 }
8270
8271
8272 static void test_sgemv(int M, int K) {
8273
8274     srand(42);
8275     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
8276     float *h_x = (float *)malloc((size_t)K * sizeof(float));
8277     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
8278     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8279     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8280
8281     // CPU reference: y[m] = sum_k A[m,k] * x[k]
8282     for (int m = 0; m < M; m++) {
8283         double sum = 0.0;
8284         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
8285         h_y_ref[m] = (float)sum;
8286     }
8287
8288     float *d_a, *d_x, *d_y;
8289     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
8290     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
8291     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
8292
8293     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
8294     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
8295
8296     dim3 block(32, 4);
8297     dim3 grid((M + 3) / 4);
8298     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
8299     check(cudaGetLastError(), "sgemv launch");
8300     check(cudaDeviceSynchronize(), "sgemv sync");
8301
8302     float *h_y = (float *)malloc((size_t)M * sizeof(float));
8303     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
8304
8305     float max_err = 0.0f;
8306     for (int m = 0; m < M; m++) {
8307         float err = fabsf(h_y[m] - h_y_ref[m]);
8308         if (err > max_err) max_err = err;
8309     }
8310     printf("| %-56s | %.3e |\n", "SGEMV-K128", max_err);
8311
8312     // ---- SGEMV K32 ----
8313     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");

```

```

8314 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
8315 check(cudaGetLastError(), "sgemv_k32 launch");
8316 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
8317
8318 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
8319
8320 max_err = 0.0f;
8321 for (int m = 0; m < M; m++) {
8322     float err = fabsf(h_y[m] - h_y_ref[m]);
8323     if (err > max_err) max_err = err;
8324 }
8325 printf("| %-56s | %.3e |\n", "SGEMV-K32", max_err);
8326
8327 // ---- SGEMV K16 ----
8328 free(h_a); free(h_x); free(h_y); free(h_y_ref);
8329 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
8330
8331 int K16 = 16;
8332 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
8333 h_x = (float *)malloc((size_t)K16 * sizeof(float));
8334 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
8335 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8336 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8337
8338 for (int m = 0; m < M; m++) {
8339     double sum = 0.0;
8340     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
8341     h_y_ref[m] = (float)sum;
8342 }
8343
8344 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
8345 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
8346 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
8347
8348 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
8349 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
8350
8351 dim3 grid_k16((M + 7) / 8);
8352 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
8353 check(cudaGetLastError(), "sgemv_k16 launch");
8354 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
8355
8356 h_y = (float *)malloc((size_t)M * sizeof(float));
8357 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
8358
8359 max_err = 0.0f;
8360 for (int m = 0; m < M; m++) {
8361     float err = fabsf(h_y[m] - h_y_ref[m]);
8362     if (err > max_err) max_err = err;
8363 }
8364 printf("| %-56s | %.3e |\n", "SGEMV-K16", max_err);
8365
8366 free(h_a); free(h_x); free(h_y); free(h_y_ref);
8367 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
8368 }
8369
8370
8371 static void test_sgemm(int M, int N, int K) {
8372
8373     size_t size_a = (size_t)M * K * sizeof(float);
8374     size_t size_b = (size_t)K * N * sizeof(float);
8375     size_t size_c = (size_t)M * N * sizeof(float);
8376

```

```

8377 float *h_a = (float *)malloc(size_a);
8378 float *h_b = (float *)malloc(size_b);
8379 float *h_c_ref = (float *)malloc(size_c);
8380
8381 srand(42);
8382 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8383 for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8384
8385 float *d_a, *d_b, *d_c;
8386 check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
8387 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
8388 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
8389
8390 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
8391 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
8392
8393 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
8394 cublasHandle_t handle;
8395 cublasCreate(&handle);
8396 float alpha = 1.0f, beta = 0.0f;
8397 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
8398             &alpha, d_b, N, d_a, K, &beta, d_c, N);
8399 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
8400
8401 // Kernel
8402 cudaEvent_t start, stop;
8403 cudaEventCreate(&start);
8404 cudaEventCreate(&stop);
8405
8406 dim3 block(32, 32);
8407 dim3 grid((N + 31) / 32, (M + 31) / 32);
8408 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
8409 check(cudaGetLastError(), "sgemm launch");
8410 check(cudaDeviceSynchronize(), "sgemm sync");
8411
8412 float *h_c = (float *)malloc(size_c);
8413 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
8414
8415 // Verify
8416 float max_err = 0.0f;
8417 for (int i = 0; i < M * N; i++) {
8418     float err = fabsf(h_c[i] - h_c_ref[i]);
8419     if (err > max_err) max_err = err;
8420 }
8421 printf("| %-56s | %.3e |\n", "SGEMM", max_err);
8422
8423 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
8424
8425 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
8426 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
8427 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
8428 check(cudaGetLastError(), "sgemm_vec4 launch");
8429 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
8430
8431 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
8432
8433 max_err = 0.0f;
8434 for (int i = 0; i < M * N; i++) {
8435     float err = fabsf(h_c[i] - h_c_ref[i]);
8436     if (err > max_err) max_err = err;
8437 }
8438 printf("| %-56s | %.3e |\n", "SGEMM-Vec4", max_err);
8439

```

```

8440 free(h_a); free(h_b); free(h_c); free(h_c_ref);
8441 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
8442 cublasDestroy(handle);
8443 cudaEventDestroy(start);
8444 cudaEventDestroy(stop);
8445 }
8446
8447
8448 static void test_hgemm_mma(int M, int N, int K) {
8449
8450     size_t size_a = (size_t)M * K * sizeof(half);
8451     size_t size_b = (size_t)K * N * sizeof(half);
8452     size_t size_c = (size_t)M * N * sizeof(half);
8453
8454     half *h_a = (half *)malloc(size_a);
8455     half *h_b = (half *)malloc(size_b);
8456     half *h_c_ref = (half *)malloc(size_c);
8457
8458     srand(42);
8459     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8460     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8461
8462     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
8463     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
8464     size_t size_b_t = (size_t)N * K * sizeof(half);
8465     half *h_b_t = (half *)malloc(size_b_t);
8466     for (int n = 0; n < N; n++)
8467         for (int k = 0; k < K; k++)
8468             h_b_t[n * K + k] = h_b[k * N + n];
8469
8470     half *d_a, *d_b, *d_b_t, *d_c;
8471     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
8472     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
8473     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
8474     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
8475
8476     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
8477     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
8478     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
8479
8480     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
8481     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
8482     cublasHandle_t handle;
8483     cublasCreate(&handle);
8484     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
8485     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
8486                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
8487                 &beta_h, d_c, CUDA_R_16F, N,
8488                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
8489     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
8490
8491     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
8492     cudaEvent_t start, stop;
8493     cudaEventCreate(&start);
8494     cudaEventCreate(&stop);
8495
8496     constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
8497     size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
8498     dim3 block(256);
8499     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
8500     hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
8501     check(cudaGetLastError(), "hgemm launch");
8502     check(cudaDeviceSynchronize(), "hgemm sync");

```

```

8503
8504     half *h_c = (half *)malloc(size_c);
8505     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
8506
8507     // Verify
8508     float max_err = 0.0f;
8509     for (int i = 0; i < M * N; i++) {
8510         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
8511         if (err > max_err) max_err = err;
8512     }
8513     printf("| %-56s | %.3e |\n", "HGEMM MMA", max_err);
8514
8515     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
8516     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
8517     cublasDestroy(handle);
8518     cudaEventDestroy(start);
8519     cudaEventDestroy(stop);
8520 }
8521
8522
8523 static void test_hgemm_swizzle(int M, int N, int K) {
8524     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
8525     // + Register Double Buffering (kValTileK=4, BK=64)
8526     // TN layout: C[M×N] = A[M×K] × BT[N×K]
8527     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
8528     // (kValTileK=4, kStages=2, BK=64)
8529     // smem: kStages × (BM×BK + BN×BK) halves
8530
8531     size_t size_a = (size_t)M * K * sizeof(half);
8532     size_t size_b = (size_t)K * N * sizeof(half);
8533     size_t size_c = (size_t)M * N * sizeof(half);
8534
8535     half *h_a = (half *)malloc(size_a);
8536     half *h_b = (half *)malloc(size_b);
8537     half *h_c_ref = (half *)malloc(size_c);
8538
8539     srand(42);
8540     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8541     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8542
8543     // Kernel expects BT [N×K] row-major layout (TN layout convention).
8544     size_t size_b_t = (size_t)N * K * sizeof(half);
8545     half *h_b_t = (half *)malloc(size_b_t);
8546     for (int n = 0; n < N; n++)
8547         for (int k = 0; k < K; k++)
8548             h_b_t[n * K + k] = h_b[k * N + n];
8549
8550     half *d_a, *d_b, *d_b_t, *d_c;
8551     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
8552     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
8553     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
8554     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
8555
8556     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
8557     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
8558     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
8559
8560     // cuBLAS FP16 reference
8561     cublasHandle_t handle;
8562     cublasCreate(&handle);
8563     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
8564     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
8565         &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,

```

```

8566         &beta_h, d_c, CUDA_R_16F, N,
8567         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
8568 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
8569
8570 // MMA swizzle kernel (default params: kStages=2, kValTileK=4, BK=64)
8571 constexpr int BM = 128, BN = 128, BK = 64, K_STAGE_S = 2;
8572 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
8573 cudaFuncSetAttribute(
8574     (const void *)hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, K_STAGE_S, 0>,
8575     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
8576 dim3 block(256);
8577 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
8578 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
8579 check(cudaGetLastError(), "hgemm_swizzle launch");
8580 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
8581
8582 half *h_c = (half *)malloc(size_c);
8583 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
8584
8585 // Verify
8586 float max_err = 0.0f;
8587 for (int i = 0; i < M * N; i++) {
8588     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
8589     if (err > max_err) max_err = err;
8590 }
8591 printf("| %-56s | %.3e |\n", "HGMEM Swizzle + Reg2x", max_err);
8592
8593 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
8594 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
8595 cublasDestroy(handle);
8596 }
8597
8598
8599 #if defined(NOTES_V2_ENABLE_CUTE)
8600 static void test_hgemm_cute(int M, int N, int K) {
8601     // HGMEM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
8602     // TN layout: C[MxN] = A[MxK] × B^T[NxK]
8603     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
8604     // Tile: BM=128, BN=256, BK=32, 128 threads/block
8605
8606     size_t size_a = (size_t)M * K * sizeof(half);
8607     size_t size_b = (size_t)K * N * sizeof(half);
8608     size_t size_c = (size_t)M * N * sizeof(half);
8609
8610     half *h_a = (half *)malloc(size_a);
8611     half *h_b = (half *)malloc(size_b);
8612     half *h_c_ref = (half *)malloc(size_c);
8613
8614     srand(42);
8615     for (int i = 0; i < M * K; i++)
8616         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8617     for (int i = 0; i < K * N; i++)
8618         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8619
8620     // CuTe kernel expects B^T [NxK] row-major (TN layout).
8621     size_t size_b_t = (size_t)N * K * sizeof(half);
8622     half *h_b_t = (half *)malloc(size_b_t);
8623     for (int n = 0; n < N; n++)
8624         for (int k = 0; k < K; k++)
8625             h_b_t[n * K + k] = h_b[k * N + n];
8626
8627     half *d_a, *d_b, *d_b_t, *d_c;
8628     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");

```

```

8629 check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
8630 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
8631 check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
8632
8633 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
8634 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
8635 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
8636
8637 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
8638 cublasHandle_t handle;
8639 cublasCreate(&handle);
8640 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
8641 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
8642             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
8643             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
8644 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
8645
8646 // CuTe kernel (Stages=2, TN layout)
8647 launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
8648 check(cudaGetLastError(), "hgemm_cute launch");
8649 check(cudaDeviceSynchronize(), "hgemm_cute sync");
8650
8651 half *h_c = (half *)malloc(size_c);
8652 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
8653
8654 // Verify
8655 float max_err = 0.0f;
8656 for (int i = 0; i < M * N; i++) {
8657     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
8658     if (err > max_err) max_err = err;
8659 }
8660 printf("| %-56s | %.3e |\n", "HGEMM CuTe Swizzle + Reg2x",
8661       max_err);
8662
8663 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
8664 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
8665 cublasDestroy(handle);
8666 }
8667 #endif /* NOTES_V2_ENABLE_CUTE */
8668
8669
8670 #if defined(NOTES_V2_ENABLE_WGMMMA)
8671 static void test_hgemm_wgmma(int M, int N, int K) {
8672     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
8673     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
8674     // Kernel: hgemm_wgmma_stages_tn with default template params
8675
8676     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
8677
8678     // M, K must be divisible by tile dims
8679     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
8680         printf("| %-56s | %-9s |\n", "HGEMM WGMMMA", "SKIP");
8681         return;
8682     }
8683
8684     size_t size_a = (size_t)M * K * sizeof(half);
8685     size_t size_b = (size_t)K * N * sizeof(half);
8686     size_t size_c = (size_t)M * N * sizeof(half);
8687
8688     half *h_a = (half *)malloc(size_a);
8689     half *h_b = (half *)malloc(size_b);
8690     half *h_c_ref = (half *)malloc(size_c);
8691

```

```

8692 srand(42);
8693 for (int i = 0; i < M * K; i++)
8694     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8695 for (int i = 0; i < K * N; i++)
8696     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8697
8698 // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
8699 size_t size_b_t = (size_t)N * K * sizeof(half);
8700 half *h_b_t = (half *)malloc(size_b_t);
8701 for (int n = 0; n < N; n++)
8702     for (int k = 0; k < K; k++)
8703         h_b_t[n * K + k] = h_b[k * N + n];
8704
8705 half *d_a, *d_b, *d_b_t, *d_c;
8706 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
8707 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
8708 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
8709 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
8710
8711 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
8712 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
8713        "wgmma H2D B (cuBLAS)");
8714 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
8715        "wgmma H2D B_t (kernel)");
8716
8717 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
8718 cublasHandle_t handle;
8719 cublasCreate(&handle);
8720 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
8721 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
8722              CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
8723              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
8724 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
8725        "wgmma D2H ref");
8726
8727 // Create TMA tensor maps for A and B^T
8728 // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
8729 // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
8730 CUTensorMap *tma_a =
8731     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
8732 CUTensorMap *tma_b =
8733     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
8734
8735 // Launch WGMMMA kernel
8736 // kBlockSwizzle=false → 2D grid, no swizzle
8737 size_t smem_bytes =
8738     kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
8739 cudaFuncSetAttribute(
8740     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
8741     false>,
8742     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
8743
8744 dim3 block(kNumThreads);
8745 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
8746 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
8747     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
8748 check(cudaGetLastError(), "wgmma launch");
8749 check(cudaDeviceSynchronize(), "wgmma sync");
8750
8751 half *h_c = (half *)malloc(size_c);
8752 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
8753
8754 // Verify

```

```

8755 float max_err = 0.0f;
8756 for (int i = 0; i < M * N; i++) {
8757     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
8758     if (err > max_err)
8759         max_err = err;
8760 }
8761 printf("| %-56s | %.3e |\n", "HGEMM TMA WGMMA WS (3-stage)", max_err);
8762
8763 free(h_a);
8764 free(h_b);
8765 free(h_b_t);
8766 free(h_c);
8767 free(h_c_ref);
8768 cudaFree(d_a);
8769 cudaFree(d_b);
8770 cudaFree(d_b_t);
8771 cudaFree(d_c);
8772 cudaFree(tma_a);
8773 cudaFree(tma_b);
8774 cublasDestroy(handle);
8775 }
8776 #endif /* NOTES_V2_ENABLE_WGMMA */
8777
8778 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
8779 template <int kStages, int kBlockSwizzle = 0>
8780 static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
8781                                     half *d_b_t, half *d_c,
8782                                     CUTensorMap *tma_a, CUTensorMap *tma_b) {
8783     constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
8784     constexpr int kMmaTileM = 2, kMmaTileN = 2;
8785     constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
8786     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
8787     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
8788     constexpr int BK = kMmaK * kValTileK;
8789     constexpr int kNumThreads = 256;
8790     constexpr size_t payload_bytes =
8791         kStages * (BM * BK + BN * BK) * sizeof(half);
8792     constexpr size_t smem_bytes = payload_bytes;
8793     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
8794                             const CUTensorMap *);
8795     Kernel kernel = hgemm_tma_mma_ws_tn<
8796         kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
8797         kValTileK, kStages, kNumThreads, kBlockSwizzle>;
8798
8799     int device = 0;
8800     int max_smem = 0;
8801     cudaFuncAttributes attributes{};
8802     check(cudaGetDevice(&device), "tma_mma_ws get device");
8803     check(cudaDeviceGetAttribute(&max_smem,
8804                                 cudaDevAttrMaxSharedMemoryPerBlockOptin,
8805                                 device),
8806           "tma_mma_ws max shared memory");
8807     check(cudaFuncGetAttributes(&attributes, kernel),
8808           "tma_mma_ws function attributes");
8809     if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
8810         if (g_debug)
8811             printf("| %-56s | %-9s |\n",
8812                   kStages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
8813                               : "HGEMM TMA MMA WS (S=3, SW=0)",
8814                   "SMEM SKIP");
8815         return false;
8816     }
8817 }

```

```

8818 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
8819                               smem_bytes),
8820        "tma_mma_ws set dynamic shared memory");
8821 dim3 block(kNumThreads);
8822 constexpr int kSwizzleN = 16;
8823 const int n_tiles = N / BN;
8824 const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
8825 const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
8826 dim3 grid(grid_x, M / BM, grid_z);
8827 kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
8828 check(cudaGetLastError(), "tma_mma_ws launch");
8829 check(cudaDeviceSynchronize(), "tma_mma_ws sync");
8830 return true;
8831 }
8832
8833 static void test_hgemm_tma_mma_ws(int M, int N, int K) {
8834     constexpr int BM = 128, BN = 128, BK = 64;
8835     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
8836         printf("| %-5s | %-9s |\n", "HGEMM TMA MMA WS", "SKIP");
8837         return;
8838     }
8839
8840     const size_t size_a = size_t(M) * K * sizeof(half);
8841     const size_t size_b = size_t(K) * N * sizeof(half);
8842     const size_t size_b_t = size_t(N) * K * sizeof(half);
8843     const size_t size_c = size_t(M) * N * sizeof(half);
8844     half *h_a = static_cast<half *>(malloc(size_a));
8845     half *h_b = static_cast<half *>(malloc(size_b));
8846     half *h_b_t = static_cast<half *>(malloc(size_b_t));
8847     half *h_c = static_cast<half *>(malloc(size_c));
8848     half *h_c_ref = static_cast<half *>(malloc(size_c));
8849     srand(42);
8850     for (int i = 0; i < M * K; ++i)
8851         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
8852     for (int i = 0; i < K * N; ++i)
8853         h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
8854     for (int n = 0; n < N; ++n)
8855         for (int k = 0; k < K; ++k)
8856             h_b_t[n * K + k] = h_b[k * N + n];
8857
8858     half *d_a, *d_b, *d_b_t, *d_c;
8859     check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
8860     check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
8861     check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
8862     check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
8863     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
8864     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
8865     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
8866           "tma_mma_ws H2D B_t");
8867
8868     cublasHandle_t handle;
8869     cublasCreate(&handle);
8870     half alpha = __float2half(1.0f), beta = __float2half(0.0f);
8871     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
8872                  CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
8873                  CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
8874     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
8875           "tma_mma_ws D2H reference");
8876
8877     CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
8878     CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
8879     for (int block_swizzle : {0, 1}) {
8880         for (int stages : {2, 3}) {

```

```

8881     const bool launched = stages == 2
8882         ? (block_swizzle
8883             ? launch_hgemmm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
8884                 tma_a, tma_b)
8885             : launch_hgemmm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
8886                 tma_a, tma_b))
8887         : (block_swizzle
8888             ? launch_hgemmm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
8889                 tma_a, tma_b)
8890             : launch_hgemmm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
8891                 tma_a, tma_b));
8892     if (!launched)
8893         continue;
8894     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
8895         "tma_mma_ws D2H");
8896     float max_err = 0.0f;
8897     for (int i = 0; i < M * N; ++i)
8898         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
8899             __half2float(h_c_ref[i])));
8900     printf("| %-56s | %.3e |\n",
8901         block_swizzle
8902             ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=1)"
8903                 : "HGEMM TMA MMA WS (S=3, SW=1)")
8904             : (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
8905                 : "HGEMM TMA MMA WS (S=3, SW=0)"),
8906         max_err);
8907 }
8908 }
8909
8910 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
8911 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
8912 cudaFree(tma_a); cudaFree(tma_b);
8913 cublasDestroy(handle);
8914 }
8915 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
8916
8917
8918 static void test_flash_attn(int seqlen, int head_dim) {
8919     // FlashAttention-2 with split-Q, MMA m16n8k16
8920     int B = 1, H = 8;
8921
8922     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
8923
8924     srand(42);
8925     half *h_q = (half *)malloc(sz);
8926     half *h_k = (half *)malloc(sz);
8927     half *h_v = (half *)malloc(sz);
8928     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
8929         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8930         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8931         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
8932     }
8933
8934     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
8935     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
8936     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
8937     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
8938     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
8939     int count = B * H * seqlen * head_dim;
8940     for (int i = 0; i < count; i++) {
8941         ref_q[i] = __half2float(h_q[i]);
8942         ref_k[i] = __half2float(h_k[i]);
8943         ref_v[i] = __half2float(h_v[i]);

```

```

8944 }
8945
8946 float scale = 1.0f / sqrtf((float)head_dim);
8947 for (int bi = 0; bi < B * H; bi++) {
8948     for (int qi = 0; qi < seqlen; qi++) {
8949         // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
8950         float smax = -INFINITY;
8951         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
8952         for (int kj = 0; kj < seqlen; kj++) {
8953             float s = 0.0f;
8954             for (int d = 0; d < head_dim; d++)
8955                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
8956                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
8957             S[kj] = s * scale;
8958             if (S[kj] > smax) smax = S[kj];
8959         }
8960         // softmax
8961         double sum_exp = 0.0;
8962         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
8963         float inv_sum = 1.0f / (float)sum_exp;
8964         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
8965         for (int d = 0; d < head_dim; d++) {
8966             double o_acc = 0.0;
8967             for (int kj = 0; kj < seqlen; kj++)
8968                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
8969                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
8970             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
8971         }
8972         free(S);
8973     }
8974 }
8975
8976 half *d_q, *d_k, *d_v, *d_o;
8977 check(cudaMalloc(&d_q, sz), "fa alloc Q");
8978 check(cudaMalloc(&d_k, sz), "fa alloc K");
8979 check(cudaMalloc(&d_v, sz), "fa alloc V");
8980 check(cudaMalloc(&d_o, sz), "fa alloc O");
8981 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
8982 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
8983 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
8984
8985 // Template params for kHeadDim=64, kStagesK=2
8986 constexpr int kHeadDim = 64;
8987 constexpr int kStagesK = 2;
8988 constexpr int kPadQ = 8;
8989 constexpr int kPadK = 8;
8990 constexpr int kPadV = 8;
8991 constexpr int kMmaAtomM = 16;
8992 constexpr int kMmaAtomN = 8;
8993 constexpr int kMmaAtomK = 16;
8994 constexpr int kMmaTileSeqLenQ = 8;
8995 constexpr int kMmaTileSeqLenK = 1;
8996 constexpr int kMmaTileSeqLenP = 8;
8997 constexpr int kMmaTileHeadDimV = 1;
8998 constexpr int kValTileSeqLenQ = 1;
8999 constexpr int kValTileSeqLenK = 8;
9000 constexpr int kValTileSeqLenP = 1;
9001 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
9002
9003 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
9004 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
9005 if (seqlen < Br) return; // kernel requires seqlen >= tile size
9006 size_t smem_bytes =

```

```

9007     (Br * (kHeadDim + kPadQ) +
9008     kStagesK * Bc * (kHeadDim + kPadK) +
9009     Bc * (kHeadDim + kPadV)) * sizeof(half);
9010
9011 dim3 block(256);
9012 dim3 grid((seqlen + Br - 1) / Br, B * H);
9013
9014 // Test both accumulator variants: f16 (kMmaAccF32=0) and f32 (kMmaAccF32=1)
9015 for (int acc = 0; acc <= 1; ++acc) {
9016     const int kMmaAcc = acc;
9017     half *h_o = (half *)malloc(sz);
9018
9019     if (kMmaAcc == 0) {
9020         using FAKernel = void (*)(half *, half *, half *, half *, int, int);
9021         FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
9022             kMmaAtomK, 0, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
9023             kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
9024             kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
9025         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
9026         fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
9027     } else {
9028         using FAKernel = void (*)(half *, half *, half *, half *, int, int);
9029         FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
9030             kMmaAtomK, 1, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
9031             kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
9032             kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
9033         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
9034         fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
9035     }
9036     check(cudaGetLastError(), "fa launch");
9037     check(cudaDeviceSynchronize(), "fa sync");
9038
9039     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
9040
9041     float max_err = 0.0f;
9042     for (int i = 0; i < count; i++) {
9043         float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
9044         if (err > max_err) max_err = err;
9045     }
9046     const char *acc_label = kMmaAcc ? "F32Acc" : "F16Acc";
9047     char label[64];
9048     snprintf(label, sizeof(label), "FA2 (kStagesK=2, Pad, %s)", acc_label);
9049     printf("| %-56s | %.3e |\n", label,
9050         max_err);
9051     free(h_o);
9052 }
9053
9054 free(h_q); free(h_k); free(h_v);
9055 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
9056 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
9057 }
9058
9059
9060 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
9061 // Test for flash_attn_tma_mma_ws_stages_split_q (SM120, D=64/128)
9062 // Reference: cuDNN SDPA (half) if available, else CPU FP32 (float) fallback.
9063 template <int kHeadDim>
9064 static void test_flash_attn_tma_mma_ws_impl(int seqlen, int head_dim) {
9065     int B = 1, H = 8;
9066     constexpr int kStagesK = 2;
9067     constexpr int kStagesV = 1;
9068     constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
9069     constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1;

```

```

9070 constexpr int kMmaTileSeqLenP = 8, kMmaTileHeadDimV = 1;
9071 constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
9072 constexpr int kValTileSeqLenP = 1;
9073 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
9074 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 128
9075 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
9076 constexpr int kNumThreads = 384;
9077
9078 if (seqlen % Br != 0 || seqlen % Bc != 0 || seqlen < Br) {
9079     printf("| %-56s | %-9s |\n",
9080         "FA2 TMA MMA WS (1 Consumer WG) (unaligned)", "SKIP");
9081     return;
9082 }
9083
9084 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
9085 srand(42);
9086 half *h_q = (half *)malloc(sz);
9087 half *h_k = (half *)malloc(sz);
9088 half *h_v = (half *)malloc(sz);
9089 for (int i = 0; i < B * H * seqlen * head_dim; ++i) {
9090     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9091     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9092     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9093 }
9094
9095 half *d_q, *d_k, *d_v, *d_o;
9096 check(cudaMalloc(&d_q, sz), "fa_tma_ws alloc Q");
9097 check(cudaMalloc(&d_k, sz), "fa_tma_ws alloc K");
9098 check(cudaMalloc(&d_v, sz), "fa_tma_ws alloc V");
9099 check(cudaMalloc(&d_o, sz), "fa_tma_ws alloc O");
9100 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D Q");
9101 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D K");
9102 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D V");
9103
9104 // Reference output: either cuDNN (half) or CPU (float)
9105 half *h_o_ref = nullptr;
9106 float *ref_o = nullptr;
9107 int count = B * H * seqlen * head_dim;
9108
9109 #if defined(NOTES_V2_ENABLE_CUDNN)
9110 {
9111     // Try cuDNN SDPA first; fall back to CPU if unsupported on this SM
9112     bool cudnn_ok = false;
9113     half *d_o_ref;
9114     check(cudaMalloc(&d_o_ref, sz), "fa_tma_ws alloc O_ref");
9115     {
9116         cudnnHandle_t cudnn_handle;
9117         cudnnCreate(&cudnn_handle);
9118         auto graph = std::make_shared<fe::graph::Graph>();
9119         graph->set_io_data_type(fe::DataType_t::HALF)
9120             .set_intermediate_data_type(fe::DataType_t::FLOAT)
9121             .set_compute_data_type(fe::DataType_t::FLOAT);
9122
9123         auto Q = graph->tensor(fe::graph::Tensor_attributes()
9124             .set_uid(1).set_dim({B, H, seqlen, head_dim})
9125             .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
9126         auto K = graph->tensor(fe::graph::Tensor_attributes()
9127             .set_uid(2).set_dim({B, H, seqlen, head_dim})
9128             .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
9129         auto V = graph->tensor(fe::graph::Tensor_attributes()
9130             .set_uid(3).set_dim({B, H, seqlen, head_dim})
9131             .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
9132

```

```

9133     auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
9134         fe::graph::SDPA_attributes()
9135             .set_name("sdpa_ref")
9136             .set_attn_scale(1.0f / sqrtf((float)head_dim)));
9137
9138     O_sdpa->set_output(true).set_uid(4)
9139         .set_dim({B, H, seqlen, head_dim})
9140         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
9141
9142     auto build_status = graph->build(cudnn_handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
9143     if (build_status.is_good()) {
9144         std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
9145             {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
9146         int64_t ws_size = 0;
9147         if (graph->get_workspace_size(ws_size).is_good()) {
9148             int8_t *d_ws = nullptr;
9149             if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "fa_tma_ws workspace");
9150             if (graph->execute(cudnn_handle, vp, d_ws).is_good()) {
9151                 check(cudaDeviceSynchronize(), "fa_tma_ws cudnn sync");
9152                 cudnn_ok = true;
9153             }
9154             if (d_ws) cudaFree(d_ws);
9155         }
9156     }
9157     cudnnDestroy(cudnn_handle);
9158 }
9159
9160 if (cudnn_ok) {
9161     h_o_ref = (half *)malloc(sz);
9162     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H ref");
9163 } else {
9164     fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
9165 }
9166 cudaFree(d_o_ref);
9167 }
9168 #endif
9169
9170 // CPU reference fallback
9171 if (!h_o_ref) {
9172     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
9173     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
9174     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
9175     ref_o = (float *)malloc(sz * 4 / sizeof(half));
9176     for (int i = 0; i < count; ++i) {
9177         ref_q[i] = __half2float(h_q[i]);
9178         ref_k[i] = __half2float(h_k[i]);
9179         ref_v[i] = __half2float(h_v[i]);
9180     }
9181     float scale = 1.0f / sqrtf((float)head_dim);
9182     for (int bi = 0; bi < B * H; ++bi) {
9183         for (int qi = 0; qi < seqlen; ++qi) {
9184             float smax = -INFINITY;
9185             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
9186             for (int kj = 0; kj < seqlen; ++kj) {
9187                 float s = 0.0f;
9188                 for (int d = 0; d < head_dim; ++d)
9189                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
9190                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
9191                 S[kj] = s * scale;
9192                 if (S[kj] > smax) smax = S[kj];
9193             }
9194             double sum_exp = 0.0;
9195             for (int kj = 0; kj < seqlen; ++kj)

```

```

9196         sum_exp += (double)expf(S[kj] - smax);
9197         float inv_sum = 1.0f / (float)sum_exp;
9198         for (int d = 0; d < head_dim; ++d) {
9199             double o_acc = 0.0;
9200             for (int kj = 0; kj < seqlen; ++kj)
9201                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
9202                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
9203             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
9204         }
9205         free(S);
9206     }
9207 }
9208 free(ref_q);
9209 free(ref_k);
9210 free(ref_v);
9211 }
9212
9213 // TMA descriptors: box innermost 固定 64 half (128B), 满足
9214 // CU_TENSOR_MAP_SWIZZLE_128B 对 box innermost ≤ 128B 的硬约束。
9215 // D=64: blocks_width=1, 单次 TMA 覆盖整行。
9216 // D=128: blocks_width=2, kernel producer 沿 head_dim 连续发 2 次 TMA,
9217 //         minor_coord = 0/64, 写入 chunk-major smem 布局 [2, Br, 64]。
9218 // Q/K/V gmem 是 [B, H, seqlen, head_dim] row-major, 作为 2D
9219 // [B*H*seqlen, head_dim] 矩阵描述。blocks_height = B*H*seqlen/tile_major。
9220 // kernel 用 (Nb_id*H+Nh_id)*N 偏移 major_coord。
9221 constexpr int kTmaBoxMinor = 64; // box innermost = 64 half = 128B
9222 constexpr int kTmaChunksQ = kHeadDim / kTmaBoxMinor;
9223 constexpr int kTmaChunksKV = kHeadDim / kTmaBoxMinor;
9224 CUTensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
9225     d_q, B * H * seqlen / Br, kTmaChunksQ);
9226 CUTensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
9227     d_k, B * H * seqlen / Bc, kTmaChunksKV);
9228 CUTensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
9229     d_v, B * H * seqlen / Bc, kTmaChunksKV);
9230
9231 using FAKernel = void (*)(half *, half *, half *, half *, int, int,
9232     const CUTensorMap *, const CUTensorMap *,
9233     const CUTensorMap *);
9234 FAKernel fa_k = flash_attn_tma_mma_ws_stages_split_q<
9235     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
9236     kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
9237     kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
9238     kNumThreads>;
9239
9240 // smem = Q[Br*d] + K[kStagesK*Bc*d] + V[kStagesV*Bc*d]
9241 size_t smem_bytes = (Br * kHeadDim + kStagesK * Bc * kHeadDim +
9242     kStagesV * Bc * kHeadDim) * sizeof(half);
9243 int device = 0, max_smem = 0;
9244 cudaFuncAttributes attributes{};
9245 cudaGetDevice(&device);
9246 cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin,
9247     device);
9248 cudaFuncGetAttributes(&attributes, fa_k);
9249 bool smem_ok =
9250     (smem_bytes + attributes.sharedSizeBytes <= (size_t)max_smem);
9251 if (!smem_ok) {
9252     if (g_debug)
9253         printf("| %-56s | %-9s |\n",
9254             "FA2 TMA MMA WS (1 Consumer WG) (SMEM SKIP)", "SMEM SKIP");
9255 } else {
9256     dim3 block(kNumThreads);
9257     dim3 grid((seqlen + Br - 1) / Br, B * H);
9258     // Test both accumulator variants

```

```

9259 for (int acc = 0; acc <= 1; ++acc) {
9260     const int kMmaAcc = acc;
9261     half *h_o = (half *)malloc(sz);
9262
9263     if (kMmaAcc == 0) {
9264         using FAK = void (*)(half *, half *, half *, half *, int, int,
9265                               const CTensorMap *, const CTensorMap *,
9266                               const CTensorMap *);
9267         FAK fk = flash_attn_tma_mma_ws_stages_split_q<
9268             kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
9269             kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
9270             kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK,
9271             kStagesV, kNumThreads>;
9272         cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
9273                               smem_bytes);
9274         fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
9275                                           tma_k, tma_v);
9276     } else {
9277         using FAK = void (*)(half *, half *, half *, half *, int, int,
9278                               const CTensorMap *, const CTensorMap *,
9279                               const CTensorMap *);
9280         FAK fk = flash_attn_tma_mma_ws_stages_split_q<
9281             kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 1, kMmaTileSeqLenQ,
9282             kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
9283             kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK,
9284             kStagesV, kNumThreads>;
9285         cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
9286                               smem_bytes);
9287         fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
9288                                           tma_k, tma_v);
9289     }
9290     check(cudaGetLastError(), "fa_tma_ws launch");
9291     check(cudaDeviceSynchronize(), "fa_tma_ws sync");
9292
9293     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H");
9294     float max_err = 0.0f;
9295     bool checked = h_o_ref || ref_o;
9296     if (checked) {
9297         for (int i = 0; i < count; ++i) {
9298             float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
9299             float err = fabsf(__half2float(h_o[i]) - ref_val);
9300             if (err > max_err) max_err = err;
9301         }
9302     }
9303     const char *acc_label = kMmaAcc ? "F32Acc" : "F16Acc";
9304     char label[64];
9305     snprintf(label, sizeof(label),
9306              "FA2 TMA MMA WS (1 Consumer WG) (%s)",
9307              acc_label);
9308     printf("| %-56s | %.3e |\n",
9309           label, max_err);
9310     free(h_o);
9311 }
9312 }
9313
9314 free(h_q); free(h_k); free(h_v);
9315 free(h_o_ref); free(ref_o);
9316 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
9317 cudaFree(tma_q); cudaFree(tma_k); cudaFree(tma_v);
9318 }
9319
9320 // D=64/128 dispatch wrapper
9321 static void test_flash_attn_tma_mma_ws(int seqlen, int head_dim) {

```

```

9322 if (head_dim == 64) {
9323     test_flash_attn_tma_mma_ws_impl<64>(seqlen, head_dim);
9324 } else if (head_dim == 128) {
9325     test_flash_attn_tma_mma_ws_impl<128>(seqlen, head_dim);
9326 } else {
9327     printf("| %-56s | %-9s |\n",
9328         "FA2 TMA MMA WS (1 Consumer WG) (D!=64/128)", "SKIP");
9329 }
9330 }
9331
9332 // FA3-style dual-consumer correctness test (Br=64, Sk=Sv=2)
9333 // Tests both F16Acc and F32Acc against cuDNN SDPA reference.
9334 #if defined(NOTES_V2_ENABLE_CUDNN)
9335 namespace fe = cudnn_frontend;
9336 static float bench_cudnn_sdpa_tflops(half *d_q, half *d_k, half *d_v,
9337                                     half *d_o_ref, int B, int H, int seqlen,
9338                                     int head_dim, fe::DataType_t compute_type);
9339 #endif
9340 template <int kHeadDim, int kStagesK>
9341 static void test_flash_attn_3_tma_ws_impl(int seqlen, int head_dim) {
9342     int B = 1, H = 8;
9343     constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
9344     constexpr int kMmaTileSeqLenQ = 4, kMmaTileSeqLenK = 1;
9345     constexpr int kMmaTileSeqLenP = 4, kMmaTileHeadDimV = 1;
9346     constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
9347     constexpr int kValTileSeqLenP = 1;
9348     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
9349     constexpr int kStagesV = 1; // per-WG V: always 1 buffer
9350     constexpr int kNumConsumerWGs = 2;
9351     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
9352     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
9353     constexpr int kNumThreads = 384;
9354
9355     if (seqlen % Br != 0 || seqlen % Bc != 0 || seqlen < Br) {
9356         printf("| %-56s | %-9s |\n",
9357             "FA3 TMA MMA WS (2 Consumer WG) (unaligned)", "SKIP");
9358         return;
9359     }
9360
9361     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
9362     srand(42);
9363     half *h_q = (half *)malloc(sz);
9364     half *h_k = (half *)malloc(sz);
9365     half *h_v = (half *)malloc(sz);
9366     for (int i = 0; i < B * H * seqlen * head_dim; ++i) {
9367         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9368         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9369         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9370     }
9371
9372     half *d_q, *d_k, *d_v, *d_o;
9373     check(cudaMalloc(&d_q, sz), "fa3 alloc Q");
9374     check(cudaMalloc(&d_k, sz), "fa3 alloc K");
9375     check(cudaMalloc(&d_v, sz), "fa3 alloc V");
9376     check(cudaMalloc(&d_o, sz), "fa3 alloc O");
9377     check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa3 H2D Q");
9378     check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa3 H2D K");
9379     check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa3 H2D V");
9380
9381     // Reference: cuDNN SDPA (half output)
9382     half *h_o_ref = nullptr;
9383     int count = B * H * seqlen * head_dim;
9384     #if defined(NOTES_V2_ENABLE_CUDNN)

```

```

9385 {
9386     half *d_o_ref;
9387     check(cudaMalloc(&d_o_ref, sz), "fa3 alloc 0_ref");
9388     bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref, B, H, seqlen, head_dim,
9389                             fe::DataType_t::FLOAT);
9390     h_o_ref = (half *)malloc(sz);
9391     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "fa3 ref D2H");
9392     cudaFree(d_o_ref);
9393 }
9394 #endif
9395
9396 // TMA descriptors (Br=64 for Q, Bc=64 for K/V)
9397 constexpr int kTmaBoxMinor = 64;
9398 constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
9399 CUTensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
9400     d_q, B * H * seqlen / Br, kTmaChunks);
9401 CUTensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
9402     d_k, B * H * seqlen / Bc, kTmaChunks);
9403 CUTensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
9404     d_v, B * H * seqlen / Bc, kTmaChunks);
9405
9406 using FAKernel = void (*)(half *, half *, half *, half *, int, int,
9407                             const CUTensorMap *, const CUTensorMap *,
9408                             const CUTensorMap *);
9409
9410 // smem = Q + K[2WGs * Sk * Bc*D] + V[2WGs * 1 * Bc*D]
9411 size_t smem_bytes = (Br * kHeadDim +
9412                     kNumConsumerWGs * kStagesK * Bc * kHeadDim +
9413                     kNumConsumerWGs * kStagesV * Bc * kHeadDim) * sizeof(half);
9414 dim3 block(kNumThreads);
9415 dim3 grid((seqlen + Br - 1) / Br, B * H);
9416
9417 for (int acc = 0; acc <= 1; ++acc) {
9418     half *h_o = (half *)malloc(sz);
9419     FAKernel fk;
9420     if (acc == 0) {
9421         fk = flash_attn_3_tma_ws_stages_split_q<
9422             kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
9423             kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
9424             kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
9425             kNumThreads>;
9426     } else {
9427         fk = flash_attn_3_tma_ws_stages_split_q<
9428             kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 1, kMmaTileSeqLenQ,
9429             kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
9430             kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
9431             kNumThreads>;
9432     }
9433     bool smem_ok = check_smem_feasible((const void *)fk, smem_bytes);
9434     if (!smem_ok) {
9435         const char *acc_label = acc ? "F32Acc" : "F16Acc";
9436         char label[64];
9437         snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (%s)",
9438                 acc_label);
9439         if (g_debug)
9440             printf("| %-56s | %-9s |\n", label, "SMEM too large");
9441         free(h_o);
9442         continue;
9443     }
9444     cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
9445                         smem_bytes);
9446     fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
9447                                     tma_k, tma_v);

```

```

9448     check(cudaGetLastError(), "fa3 launch");
9449     check(cudaDeviceSynchronize(), "fa3 sync");
9450
9451     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa3 D2H");
9452     float max_err = 0.0f;
9453     bool checked = h_o_ref != nullptr;
9454     if (checked) {
9455         for (int i = 0; i < count; ++i) {
9456             float err = fabsf(__half2float(h_o[i]) - __half2float(h_o_ref[i]));
9457             if (err > max_err) max_err = err;
9458         }
9459     }
9460     const char *acc_label = acc ? "F32Acc" : "F16Acc";
9461     char label[64];
9462     snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (%s)",
9463              acc_label);
9464     printf("| %-56s | %.3e |\n", label, max_err);
9465     free(h_o);
9466 }
9467
9468 free(h_q); free(h_k); free(h_v); free(h_o_ref);
9469 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
9470 cudaFree(tma_q); cudaFree(tma_k); cudaFree(tma_v);
9471 }
9472
9473 static void test_flash_attn_3_tma_ws(int seqlen, int head_dim) {
9474     // Try stages up to 4; test_flash_attn_3_tma_ws_impl internally checks
9475     // cudaDevAttrMaxSharedMemoryPerBlockOptin and skips oversized configs.
9476     if (head_dim == 64) {
9477         test_flash_attn_3_tma_ws_impl<64, 1>(seqlen, head_dim);
9478         test_flash_attn_3_tma_ws_impl<64, 2>(seqlen, head_dim);
9479         test_flash_attn_3_tma_ws_impl<64, 3>(seqlen, head_dim);
9480         test_flash_attn_3_tma_ws_impl<64, 4>(seqlen, head_dim);
9481     } else if (head_dim == 128) {
9482         test_flash_attn_3_tma_ws_impl<128, 1>(seqlen, head_dim);
9483         test_flash_attn_3_tma_ws_impl<128, 2>(seqlen, head_dim);
9484         test_flash_attn_3_tma_ws_impl<128, 3>(seqlen, head_dim);
9485         test_flash_attn_3_tma_ws_impl<128, 4>(seqlen, head_dim);
9486     } else {
9487         printf("| %-56s | %-9s |\n",
9488                "FA3 TMA MMA WS (2 Consumer WG) (D!=64/128)", "SKIP");
9489     }
9490 }
9491 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
9492
9493 static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
9494     double flops = 2.0 * M * N * K;
9495     return (float)(flops / (double)time_ms / 1e9);
9496 }
9497
9498 static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
9499     double flops = 4.0 * B * H * N * N * D;
9500     return (float)(flops / (double)time_ms / 1e9);
9501 }
9502
9503 static float bench_cublas_hgemm_tflops(cublasHandle_t handle, int M, int N, int K,
9504                                         half *d_a, half *d_b, half *d_c) {
9505     half alpha = __float2half(1.0f), beta = __float2half(0.0f);
9506     cudaStream_t stream;
9507     cudaStreamCreate(&stream);
9508     cublasSetStream(handle, stream);
9509     // warmup
9510     for (int w = 0; w < g_warmup; ++w)

```

```

9511     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
9512                 d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta,
9513                 d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
9514     cudaStreamSynchronize(stream);
9515     // timed repeat
9516     cudaEvent_t start, stop;
9517     cudaEventCreate(&start);
9518     cudaEventCreate(&stop);
9519     cudaEventRecord(start, stream);
9520     for (int r = 0; r < g_repeat; ++r)
9521         cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
9522                     d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta,
9523                     d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
9524     cudaEventRecord(stop, stream);
9525     cudaEventSynchronize(stop);
9526     float time_ms = 0;
9527     cudaEventElapsedTime(&time_ms, start, stop);
9528     time_ms /= g_repeat;
9529     cudaEventDestroy(start);
9530     cudaEventDestroy(stop);
9531     cublasSetStream(handle, nullptr);
9532     cudaStreamDestroy(stream);
9533     return bench_hgemm_tflops(M, N, K, time_ms);
9534 }
9535
9536 // =====
9537 // Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
9538 // =====
9539 template <int kStages, int kBlockSwizzle>
9540 static bool launch_timed_hgemm_mma(
9541     half *d_a, half *d_b_t, half *d_c, half *h_c,
9542     half *h_c_ref, int M, int N, int K, size_t size_c,
9543     cudaEvent_t start, cudaEvent_t stop, float &max_err,
9544     float &time_ms
9545 ) {
9546     constexpr int BM = 128, BN = 128, BK = 16;
9547     using Kernel = void (*)(half *, half *, half *, int, int, int);
9548     Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
9549     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
9550     if (!check_smem_feasible((const void *)k, smem)) return false;
9551     dim3 block(256);
9552     const int tiles_n = (N + BN - 1) / BN;
9553     constexpr int kSwizzleN = 16;
9554     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
9555     int gz = kBlockSwizzle ? kSwizzleN : 1;
9556     dim3 grid(gx, (M + BM - 1) / BM, gz);
9557     for (int w = 0; w < g_warmup; w++)
9558         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
9559     cudaDeviceSynchronize();
9560     cudaEventRecord(start);
9561     for (int r = 0; r < g_repeat; r++)
9562         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
9563     cudaEventRecord(stop);
9564     cudaEventSynchronize(stop);
9565     cudaEventElapsedTime(&time_ms, start, stop);
9566     time_ms /= g_repeat;
9567     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
9568     max_err = 0.0f;
9569     for (int i = 0; i < M * N; i++) {
9570         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
9571         if (err > max_err) max_err = err;
9572     }
9573     return true;

```

```

9574 }
9575
9576 static void bench_hgemm_mma(int M, int N, int K) {
9577     size_t size_a = (size_t)M * K * sizeof(half);
9578     size_t size_b = (size_t)K * N * sizeof(half);
9579     size_t size_c = (size_t)M * N * sizeof(half);
9580     half *h_a = (half *)malloc(size_a);
9581     half *h_b = (half *)malloc(size_b);
9582     half *h_c_ref = (half *)malloc(size_c);
9583     srand(42);
9584     for (int i = 0; i < M * K; i++)
9585         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9586     for (int i = 0; i < K * N; i++)
9587         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9588     size_t size_b_t = (size_t)N * K * sizeof(half);
9589     half *h_b_t = (half *)malloc(size_b_t);
9590     for (int n = 0; n < N; n++)
9591         for (int k = 0; k < K; k++)
9592             h_b_t[n * K + k] = h_b[k * N + n];
9593     half *d_a, *d_b, *d_b_t, *d_c;
9594     check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
9595     check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
9596     check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
9597     check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
9598     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
9599     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");
9600     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
9601     cublasHandle_t handle;
9602     cublasCreate(&handle);
9603     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
9604     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
9605                 d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
9606                 CUBLAS_GEMM_DEFAULT);
9607     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
9608     half *h_c = (half *)malloc(size_c);
9609     float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
9610     cudaEvent_t start, stop;
9611     cudaEventCreate(&start);
9612     cudaEventCreate(&stop);
9613     for (int stages : {2, 3}) {
9614         for (int swizzle : {0, 1}) {
9615             float max_err = 0, time_ms = 0;
9616             bool ok = false;
9617             if (stages == 2)
9618                 ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
9619                     K, size_c, start, stop, max_err, time_ms)
9620                     : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
9621                     K, size_c, start, stop, max_err, time_ms);
9622             else
9623                 ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
9624                     K, size_c, start, stop, max_err, time_ms)
9625                     : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
9626                     K, size_c, start, stop, max_err, time_ms);
9627             char label[64];
9628             snprintf(label, sizeof(label), "HGEMM MMA (S=%d, SW=%d)", stages, swizzle);
9629             if (!ok) {
9630                 if (g_debug)
9631                     printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
9632             } else {
9633                 float tflops = bench_hgemm_tflops(M, N, K, time_ms);
9634                 char tflops_str[32];
9635                 snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
cublas_tflops);

```

```

9636     printf("| %-56s | %.3e | %-19s |\n", label, max_err,
9637         tflops_str);
9638 }
9639 }
9640 }
9641 cudaEventDestroy(start);
9642 cudaEventDestroy(stop);
9643 free(h_a);
9644 free(h_b);
9645 free(h_b_t);
9646 free(h_c);
9647 free(h_c_ref);
9648 cudaFree(d_a);
9649 cudaFree(d_b);
9650 cudaFree(d_b_t);
9651 cudaFree(d_c);
9652 cublasDestroy(handle);
9653 }
9654
9655 // =====
9656 // Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=4, BK=64)
9657 // =====
9658 template <int kStages, int kBlockSwizzle>
9659 static bool launch_timed_hgemm_swizzle(
9660     half *d_a, half *d_b_t, half *d_c, half *h_c,
9661     half *h_c_ref, int M, int N, int K, size_t size_c,
9662     cudaEvent_t start, cudaEvent_t stop, float &max_err,
9663     float &time_ms
9664 ) {
9665     constexpr int BM = 128, BN = 128, BK = 64;
9666     using Kernel = void (*)(half *, half *, half *, int, int, int);
9667     Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, kStages, kBlockSwizzle>;
9668     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
9669     if (!check_smem_feasible((const void *)k, smem)) return false;
9670     cudaFuncSetAttribute((const void *)k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
9671     dim3 block(256);
9672     const int tiles_n = (N + BN - 1) / BN;
9673     constexpr int kSwizzleN = 16;
9674     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
9675     int gz = kBlockSwizzle ? kSwizzleN : 1;
9676     dim3 grid(gx, (M + BM - 1) / BM, gz);
9677     for (int w = 0; w < g_warmup; w++)
9678         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
9679     cudaDeviceSynchronize();
9680     cudaEventRecord(start);
9681     for (int r = 0; r < g_repeat; r++)
9682         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
9683     cudaEventRecord(stop);
9684     cudaEventSynchronize(stop);
9685     cudaEventElapsedTime(&time_ms, start, stop);
9686     time_ms /= g_repeat;
9687     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
9688     max_err = 0.0f;
9689     for (int i = 0; i < M * N; i++) {
9690         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
9691         if (err > max_err) max_err = err;
9692     }
9693     return true;
9694 }
9695
9696 static void bench_hgemm_swizzle(int M, int N, int K) {
9697     size_t size_a = (size_t)M * K * sizeof(half);
9698     size_t size_b = (size_t)K * N * sizeof(half);

```

```

9699 size_t size_c = (size_t)M * N * sizeof(half);
9700 half *h_a = (half *)malloc(size_a);
9701 half *h_b = (half *)malloc(size_b);
9702 half *h_c_ref = (half *)malloc(size_c);
9703 srand(42);
9704 for (int i = 0; i < M * K; i++)
9705     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9706 for (int i = 0; i < K * N; i++)
9707     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9708 size_t size_b_t = (size_t)N * K * sizeof(half);
9709 half *h_b_t = (half *)malloc(size_b_t);
9710 for (int n = 0; n < N; n++)
9711     for (int k = 0; k < K; k++)
9712         h_b_t[n * K + k] = h_b[k * N + n];
9713 half *d_a, *d_b, *d_b_t, *d_c;
9714 check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
9715 check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
9716 check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
9717 check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
9718 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
9719 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
9720 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
9721 cublasHandle_t handle;
9722 cublasCreate(&handle);
9723 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
9724 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
9725     d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
9726     CUBLAS_GEMM_DEFAULT);
9727 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
9728 half *h_c = (half *)malloc(size_c);
9729 float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
9730 cudaEvent_t start, stop;
9731 cudaEventCreate(&start);
9732 cudaEventCreate(&stop);
9733 for (int stages : {2, 3}) {
9734     for (int swizzle : {0, 1}) {
9735         float max_err = 0, time_ms = 0;
9736         bool ok = false;
9737         if (stages == 2)
9738             ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
9739                 N, K, size_c, start, stop, max_err, time_ms)
9740                 : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
9741                 N, K, size_c, start, stop, max_err, time_ms);
9742         else
9743             ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
9744                 N, K, size_c, start, stop, max_err, time_ms)
9745                 : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
9746                 N, K, size_c, start, stop, max_err, time_ms);
9747         char label[64];
9748         snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, SW=%d)", stages, swizzle);
9749         if (!ok) {
9750             if (g_debug)
9751                 printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
9752         } else {
9753             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
9754             char tflops_str[32];
9755             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
9756                 cublas_tflops);
9757             printf("| %-56s | %.3e | %-19s |\n", label, max_err,
9758                 tflops_str);
9759         }
9760     }
9761 }

```

```

9761 cudaEventDestroy(start);
9762 cudaEventDestroy(stop);
9763 free(h_a);
9764 free(h_b);
9765 free(h_b_t);
9766 free(h_c);
9767 free(h_c_ref);
9768 cudaFree(d_a);
9769 cudaFree(d_b);
9770 cudaFree(d_b_t);
9771 cudaFree(d_c);
9772 cublasDestroy(handle);
9773 }
9774
9775 #if defined(NOTES_V2_ENABLE_CUTE)
9776 // =====
9777 // Bench: HGEMM CuTe
9778 // =====
9779 static void bench_hgemm_cute(int M, int N, int K) {
9780     size_t size_a = (size_t)M * K * sizeof(half);
9781     size_t size_b = (size_t)K * N * sizeof(half);
9782     size_t size_c = (size_t)M * N * sizeof(half);
9783     half *h_a = (half *)malloc(size_a);
9784     half *h_b = (half *)malloc(size_b);
9785     half *h_c_ref = (half *)malloc(size_c);
9786     srand(42);
9787     for (int i = 0; i < M * K; i++)
9788         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9789     for (int i = 0; i < K * N; i++)
9790         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9791     size_t size_b_t = (size_t)N * K * sizeof(half);
9792     half *h_b_t = (half *)malloc(size_b_t);
9793     for (int n = 0; n < N; n++)
9794         for (int k = 0; k < K; k++)
9795             h_b_t[n * K + k] = h_b[k * N + n];
9796     half *d_a, *d_b, *d_b_t, *d_c;
9797     check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
9798     check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
9799     check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
9800     check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
9801     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
9802     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");
9803     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
9804     cublasHandle_t handle;
9805     cublasCreate(&handle);
9806     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
9807     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
9808         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
9809         CUBLAS_GEMM_DEFAULT);
9810     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
9811     half *h_c = (half *)malloc(size_c);
9812     float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
9813     cudaEvent_t start, stop;
9814     cudaEventCreate(&start);
9815     cudaEventCreate(&stop);
9816     for (int stages : {2, 3}) {
9817         for (int swizzle : {0, 1}) {
9818             char label[64];
9819             snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, SW=%d)", stages, swizzle);
9820             bool ok = false;
9821             float time_ms = 0, max_err = 0;
9822             if (stages == 2) {
9823                 auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 2, 1>

```

```

9824         : launch_hgemm_mma_stages_tn_cute<half, 2, 0>;
9825     for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
9826     cudaDeviceSynchronize();
9827     cudaEventRecord(start);
9828     for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
9829     cudaEventRecord(stop);
9830     cudaEventSynchronize(stop);
9831     cudaEventElapsedTime(&time_ms, start, stop);
9832     time_ms /= g_repeat;
9833     ok = true;
9834 } else {
9835     auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 3, 1>
9836                   : launch_hgemm_mma_stages_tn_cute<half, 3, 0>;
9837     for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
9838     cudaDeviceSynchronize();
9839     cudaEventRecord(start);
9840     for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
9841     cudaEventRecord(stop);
9842     cudaEventSynchronize(stop);
9843     cudaEventElapsedTime(&time_ms, start, stop);
9844     time_ms /= g_repeat;
9845     ok = true;
9846 }
9847 if (ok) {
9848     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
9849     max_err = 0.0f;
9850     for (int i = 0; i < M * N; i++) {
9851         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
9852         if (err > max_err) max_err = err;
9853     }
9854     float tflops = bench_hgemm_tflops(M, N, K, time_ms);
9855     char tflops_str[32];
9856     snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
cublas_tflops);
9857     printf("| %-56s | %.3e | %-19s |\n", label, max_err,
tflops_str);
9858 } else {
9859     printf("| %-56s | %-9s | %-19s |\n", label, "LAUNCH ERR", "None");
9860 }
9861 }
9862 }
9863 }
9864 cudaEventDestroy(start);
9865 cudaEventDestroy(stop);
9866 free(h_a);
9867 free(h_b);
9868 free(h_b_t);
9869 free(h_c);
9870 free(h_c_ref);
9871 cudaFree(d_a);
9872 cudaFree(d_b);
9873 cudaFree(d_b_t);
9874 cudaFree(d_c);
9875 cublasDestroy(handle);
9876 }
9877 #endif
9878
9879 #if defined(NOTES_V2_ENABLE_WGMMA)
9880 // =====
9881 // Bench: HGEMM WGMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
9882 // =====
9883 template <int kStages, int kBlockSwizzle>
9884 static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
CUtensorMap *tma_a, CUtensorMap *tma_b,

```

```

9886         int M, int N, int K, size_t size_c,
9887         cudaEvent_t start, cudaEvent_t stop,
9888         float &max_err, float &time_ms) {
9889     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
9890     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
9891                             const CUTensorMap *);
9892     Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
9893               kBlockSwizzle>;
9894     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
9895     if (!check_smem_feasible((const void *)k, smem)) return false;
9896     cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
9897     dim3 block(kNumThreads);
9898     const int tiles_n = (N + BN - 1) / BN;
9899     constexpr int kSwizzleN = 16;
9900     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
9901     int gz = kBlockSwizzle ? kSwizzleN : 1;
9902     dim3 grid(gx, (M + BM - 1) / BM, gz);
9903     for (int w = 0; w < g_warmup; w++)
9904         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
9905     cudaDeviceSynchronize();
9906     cudaEventRecord(start);
9907     for (int r = 0; r < g_repeat; r++)
9908         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
9909     cudaEventRecord(stop);
9910     cudaEventSynchronize(stop);
9911     cudaEventElapsedTime(&time_ms, start, stop);
9912     time_ms /= g_repeat;
9913     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
9914     max_err = 0.0f;
9915     for (int i = 0; i < M * N; i++) {
9916         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
9917         if (err > max_err) max_err = err;
9918     }
9919     return true;
9920 }
9921
9922 static void bench_hgemm_wgmma(int M, int N, int K) {
9923     constexpr int BM = 128, BN = 128, BK = 64;
9924     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
9925         printf("| %-56s | %-9s | %-19s |\n", "HGEMM WGMMMA (unaligned)", "SKIP", "None");
9926         return;
9927     }
9928     size_t size_a = (size_t)M * K * sizeof(half);
9929     size_t size_b = (size_t)K * N * sizeof(half);
9930     size_t size_c = (size_t)M * N * sizeof(half);
9931     half *h_a = (half *)malloc(size_a);
9932     half *h_b = (half *)malloc(size_b);
9933     half *h_c_ref = (half *)malloc(size_c);
9934     srand(42);
9935     for (int i = 0; i < M * K; i++)
9936         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9937     for (int i = 0; i < K * N; i++)
9938         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9939     size_t size_b_t = (size_t)N * K * sizeof(half);
9940     half *h_b_t = (half *)malloc(size_b_t);
9941     for (int n = 0; n < N; n++)
9942         for (int k = 0; k < K; k++)
9943             h_b_t[n * K + k] = h_b[k * N + n];
9944     half *d_a, *d_b, *d_b_t, *d_c;
9945     check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
9946     check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
9947     check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");
9948     check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");

```

```

9949 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
9950 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
9951 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
9952 cublasHandle_t handle;
9953 cublasCreate(&handle);
9954 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
9955 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
9956             d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
9957             CUBLAS_GEMM_DEFAULT);
9958 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
9959 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
9960 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
9961 half *h_c = (half *)malloc(size_c);
9962 float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
9963 cudaEvent_t start, stop;
9964 cudaEventCreate(&start);
9965 cudaEventCreate(&stop);
9966 for (int stages : {1, 2, 3}) {
9967     for (int swizzle : {0, 1}) {
9968         float max_err = 0, time_ms = 0;
9969         bool ok = false;
9970         if (stages == 2)
9971             ok = swizzle
9972                 ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
9973                 K, size_c, start, stop, max_err, time_ms)
9974                 : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
9975                 size_c, start, stop, max_err, time_ms);
9976         else
9977             ok = swizzle
9978                 ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
9979                 K, size_c, start, stop, max_err, time_ms)
9980                 : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
9981                 size_c, start, stop, max_err, time_ms);
9982         char label[64];
9983         snprintf(label, sizeof(label), "HGEMM TMA WGMMMA WS (S=%d, SW=%d)", stages, swizzle);
9984         if (!ok) {
9985             if (g_debug)
9986                 printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
9987         } else {
9988             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
9989             char tflops_str[32];
9990             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
9991             cublas_tflops);
9992             printf("| %-56s | %.3e | %-19s |\n", label, max_err,
9993             tflops_str);
9994         }
9995     }
9996 }
9997 cudaEventDestroy(start);
9998 cudaEventDestroy(stop);
9999 free(h_a);
10000 free(h_b);
10001 free(h_b_t);
10002 free(h_c);
10003 free(h_c_ref);
10004 cudaFree(d_a);
10005 cudaFree(d_b);
10006 cudaFree(d_b_t);
10007 cudaFree(d_c);
10008 cudaFree(tma_a);
10009 cudaFree(tma_b);
10010 cublasDestroy(handle);
}

```

```

10011 #endif
10012
10013 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
10014 // =====
10015 // Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
10016 // =====
10017 template <int kStages, int kBlockSwizzle>
10018 static bool bench_launch_tma_mma_ws(
10019     int M, int N, int K, half *d_a, half *d_b_t,
10020     half *d_c, half *h_c, half *h_c_ref, size_t size_c,
10021     CUTensorMap *tma_a, CUTensorMap *tma_b,
10022     cudaEvent_t start, cudaEvent_t stop,
10023     float &max_err, float &time_ms) {
10024     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
10025     constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
10026     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
10027                             const CUTensorMap *);
10028     Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
10029                             kNumThreads, kBlockSwizzle>;
10030     if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
10031     cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
10032                         payload_bytes);
10033     dim3 block(kNumThreads);
10034     constexpr int kSwizzleN = 16;
10035     const int n_tiles = N / BN;
10036     const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
10037     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
10038     dim3 grid(grid_x, M / BM, grid_z);
10039     for (int w = 0; w < g_warmup; w++)
10040         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
10041     cudaDeviceSynchronize();
10042     cudaEventRecord(start);
10043     for (int r = 0; r < g_repeat; r++)
10044         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
10045     cudaEventRecord(stop);
10046     cudaEventSynchronize(stop);
10047     cudaEventElapsedTime(&time_ms, start, stop);
10048     time_ms /= g_repeat;
10049     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
10050     max_err = 0.0f;
10051     for (int i = 0; i < M * N; ++i)
10052         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
10053     return true;
10054 }
10055
10056 static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
10057     constexpr int BM = 128, BN = 128, BK = 64;
10058     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
10059         printf("| %-56s | %-9s | %-19s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP", "None");
10060         return;
10061     }
10062     const size_t size_a = size_t(M) * K * sizeof(half);
10063     const size_t size_b = size_t(K) * N * sizeof(half);
10064     const size_t size_b_t = size_t(N) * K * sizeof(half);
10065     const size_t size_c = size_t(M) * N * sizeof(half);
10066     half *h_a = static_cast<half *>(malloc(size_a));
10067     half *h_b = static_cast<half *>(malloc(size_b));
10068     half *h_b_t = static_cast<half *>(malloc(size_b_t));
10069     half *h_c = static_cast<half *>(malloc(size_c));
10070     half *h_c_ref = static_cast<half *>(malloc(size_c));
10071     srand(42);
10072     for (int i = 0; i < M * K; ++i)
10073         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);

```

```

10074 for (int i = 0; i < K * N; ++i)
10075     h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
10076 for (int n = 0; n < N; ++n)
10077     for (int k = 0; k < K; ++k)
10078         h_b_t[n * K + k] = h_b[k * N + n];
10079 half *d_a, *d_b, *d_b_t, *d_c;
10080 check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
10081 check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
10082 check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
10083 check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
10084 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
10085 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
10086 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
10087     "bench tma mma ws H2D B_t");
10088 cublasHandle_t handle;
10089 cublasCreate(&handle);
10090 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
10091 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
10092     d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
10093     CUBLAS_GEMM_DEFAULT);
10094 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
10095     "bench tma mma ws D2H reference");
10096 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
10097 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
10098 float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
10099 cudaEvent_t start, stop;
10100 cudaEventCreate(&start);
10101 cudaEventCreate(&stop);
10102 for (int stages : {1, 2, 3}) {
10103     for (int swizzle : {0, 1}) {
10104         float max_err = 0, time_ms = 0;
10105         bool ok = false;
10106         if (stages == 2)
10107             ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
10108                 h_c_ref, size_c, tma_a, tma_b,
10109                 start, stop, max_err, time_ms)
10110             : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
10111                 h_c_ref, size_c, tma_a, tma_b,
10112                 start, stop, max_err, time_ms);
10113         else
10114             ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
10115                 h_c_ref, size_c, tma_a, tma_b,
10116                 start, stop, max_err, time_ms)
10117             : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
10118                 h_c_ref, size_c, tma_a, tma_b,
10119                 start, stop, max_err, time_ms);
10120         char label[64];
10121         snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, SW=%d)", stages, swizzle);
10122         if (!ok) {
10123             if (g_debug)
10124                 printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
10125         } else {
10126             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
10127             char tflops_str[32];
10128             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cublas_tflops, tflops /
10129                 cublas_tflops);
10129             printf("| %-56s | %.3e | %-19s |\n", label, max_err,
10130                 tflops_str);
10131         }
10132     }
10133 }
10134 cudaEventDestroy(start);
10135 cudaEventDestroy(stop);

```

```

10136 free(h_a);
10137 free(h_b);
10138 free(h_b_t);
10139 free(h_c);
10140 free(h_c_ref);
10141 cudaFree(d_a);
10142 cudaFree(d_b);
10143 cudaFree(d_b_t);
10144 cudaFree(d_c);
10145 cudaFree(tma_a);
10146 cudaFree(tma_b);
10147 cublasDestroy(handle);
10148 }
10149 #endif
10150
10151 // =====
10152 // Bench: FlashAttention-2 Split-Q (template on kHeadDim for dispatch)
10153 // =====
10154 template <int kHeadDim, int kStagesK = 2, int kPadQ = 8, int kPadK = 8,
10155         int kPadV = 8, int kMmaAccF32 = 0>
10156 static void bench_fa_launch(int B, int H, int seqlen, int head_dim,
10157         half *h_o_ref, float *ref_o, half *d_q, half *d_k, half *d_v, half *d_o,
10158         float cudnn_tflops_f16) {
10159     static_assert(kHeadDim == 64 || kHeadDim == 128, "Only D=64 and D=128 are supported");
10160     constexpr bool kSwizzleQ = kPadQ == 0;
10161     constexpr bool kSwizzleK = kPadK == 0;
10162     constexpr bool kSwizzleV = kPadV == 0;
10163     constexpr int kMmaAtomM = 16;
10164     constexpr int kMmaAtomN = 8;
10165     constexpr int kMmaAtomK = 16;
10166     constexpr int kMmaTileSeqLenQ = 8;
10167     constexpr int kMmaTileSeqLenK = 1;
10168     constexpr int kMmaTileSeqLenP = 8;
10169     constexpr int kMmaTileHeadDimV = 1;
10170     constexpr int kValTileSeqLenQ = 1;
10171     constexpr int kValTileSeqLenK = 8;
10172     constexpr int kValTileSeqLenP = 1;
10173     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
10174     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
10175     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
10176     constexpr const char *layout_name =
10177         kSwizzleQ && kSwizzleK && kSwizzleV ? "Swizzle" :
10178         kSwizzleQ && kSwizzleK ? "SwizzleQK" :
10179         kSwizzleQ && kSwizzleV ? "SwizzleQV" :
10180         kSwizzleK && kSwizzleV ? "SwizzleKV" :
10181         kSwizzleQ ? "SwizzleQ" :
10182         kSwizzleK ? "SwizzleK" :
10183         kSwizzleV ? "SwizzleV" : "Pad";
10184
10185     // Kernel requires seqlen >= Br (tile size); skip gracefully for short seqlen
10186     if (seqlen < Br) {
10187         char label[64];
10188         snprintf(label, sizeof(label), "FA2 (%s)",
10189                 layout_name);
10190         printf("| %-56s | %-9s | %-19s |\n", label,
10191                 "seqlen<Br", "None");
10192         return;
10193     }
10194     size_t smem_bytes =
10195         (Br * (kHeadDim + kPadQ) +
10196          kStagesK * Bc * (kHeadDim + kPadK) +
10197          Bc * (kHeadDim + kPadV)) * sizeof(half);
10198

```

```

10199 dim3 block(256);
10200 dim3 grid((seqlen + Br - 1) / Br, B * H);
10201
10202 cudaStream_t timing_stream;
10203 cudaStreamCreate(&timing_stream);
10204 cudaEvent_t start, stop;
10205 cudaEventCreate(&start);
10206 cudaEventCreate(&stop);
10207
10208 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
10209 FAKernel fa_k = flash_attn_mma_stages_split_q<
10210     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32,
10211     kMmaTileSeqLenQ, kMmaTileSeqLenK,
10212     kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
10213     kValTileSeqLenP, kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
10214 bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
10215 if (smem_ok) {
10216     cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
10217 }
10218
10219 // Warmup
10220 if (smem_ok) {
10221     for (int w = 0; w < g_warmup; w++)
10222         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
10223     check(cudaStreamSynchronize(timing_stream), "fa warmup sync");
10224 }
10225
10226 // Timed repeat
10227 cudaEventRecord(start, timing_stream);
10228 if (smem_ok) {
10229     for (int r = 0; r < g_repeat; r++)
10230         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
10231 }
10232 cudaEventRecord(stop, timing_stream);
10233 cudaEventSynchronize(stop);
10234
10235 float time_ms = 0;
10236 cudaEventElapsedTime(&time_ms, start, stop);
10237 time_ms /= g_repeat;
10238
10239 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
10240 half *h_o = (half *)malloc(sz);
10241 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");
10242
10243 int count = B * H * seqlen * head_dim;
10244 char label[64];
10245 snprintf(label, sizeof(label), "FA2 (S=%d, %s, %s)", kStagesK,
10246     layout_name, kMmaAccF32 ? "F32Acc" : "F16Acc");
10247 float max_err = 0.0f;
10248 bool checked = h_o_ref || ref_o;
10249 if (smem_ok && checked) {
10250     for (int i = 0; i < count; i++) {
10251         float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
10252         float err = fabsf(__half2float(h_o[i]) - ref_val);
10253         if (err > max_err) max_err = err;
10254     }
10255 }
10256 if (smem_ok && checked) {
10257     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10258     bool is_fail = max_err >= 5e-1f;
10259     if (is_fail || should_print_fa_tflops(kMmaAccF32, tflops)) {
10260         char tflops_str[32];
10261         if (cudnn_tflops_f16 > 0)

```

```

10262     snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cudnn_tflops_f16, tflops /
10263     cudnn_tflops_f16);
10264     else
10265         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
10266     printf("| %-56s | %.3e | %-19s |\n", label, max_err,
10267         tflops_str);
10268 }
10269 } else if (smem_ok) {
10270     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10271     if (should_print_fa_tflops(kMmaAccF32, tflops)) {
10272         char tflops_str[32];
10273         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
10274         printf("| %-56s | %-9s | %-19s |\n", label, "unchecked", tflops_str);
10275     }
10276 } else {
10277     if (g_debug)
10278         printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
10279 }
10280 cudaEventDestroy(start);
10281 cudaEventDestroy(stop);
10282 cudaStreamDestroy(timing_stream);
10283 free(h_o);
10284 }
10285
10286 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
10287 // Bench for flash_attn_tma_mma_ws_stages_split_q (SM120, D=64/128)
10288 template <int kHeadDim, int kStagesK, int kStagesV = 1, int kMmaAccF32 = 0>
10289 static void bench_fa_tma_mma_ws_launch(int B, int H, int seqlen, int head_dim,
10290     half *h_o_ref, float *ref_o,
10291     half *d_q, half *d_k, half *d_v,
10292     half *d_o, float cudnn_tflops_f16) {
10293     constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
10294     constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1;
10295     constexpr int kMmaTileSeqLenP = 8, kMmaTileHeadDimV = 1;
10296     constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
10297     constexpr int kValTileSeqLenP = 1;
10298     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
10299     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
10300     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
10301     constexpr int kNumThreads = 384;
10302
10303     if (seqlen < Br || seqlen % Br != 0 || seqlen % Bc != 0) {
10304         char label[64];
10305         snprintf(label, sizeof(label),
10306             "FA2 TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, unaligned)", kStagesK, kStagesV);
10307         printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
10308         return;
10309     }
10310
10311     // TMA descriptors: box innermost 固定 64 half (128B), 满足
10312     // CU_TENSOR_MAP_SWIZZLE_128B 对 box innermost ≤ 128B 的硬约束。
10313     // D=64: blocks_width=1, 单次 TMA 覆盖整行。
10314     // D=128: blocks_width=2, kernel producer 沿 head_dim 连续发 2 次 TMA,
10315     //         minor_coord = 0/64, 写入 chunk-major smem 布局 [2, Br, 64]。
10316     // Q/K/V gmem 是 [B, H, seqlen, head_dim] row-major, 作为 2D
10317     // [B*H*seqlen, head_dim] 矩阵描述。blocks_height = B*H*seqlen/tile_major。
10318     // kernel 用 (Nb_id*H+Nh_id)*N 偏移 major_coord。
10319     constexpr int kTmaBoxMinor = 64; // box innermost = 64 half = 128B
10320     constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
10321     CUtensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
10322         d_q, B * H * seqlen / Br, kTmaChunks);
10323     CUtensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(

```

```

10324     d_k, B * H * seqlen / Bc, kTmaChunks);
10325     CUTensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
10326         d_v, B * H * seqlen / Bc, kTmaChunks);
10327
10328     using FAKernel = void (*)(half *, half *, half *, half *, int, int,
10329                             const CUTensorMap *, const CUTensorMap *,
10330                             const CUTensorMap *);
10331     FAKernel fa_k = flash_attn_tma_mma_ws_stages_split_q<
10332         kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32, kMmaTileSeqLenQ,
10333         kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
10334         kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
10335         kNumThreads>;
10336
10337     size_t smem_bytes = (Br * kHeadDim + kStagesK * Bc * kHeadDim +
10338                         kStagesV * Bc * kHeadDim) * sizeof(half);
10339     bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
10340     if (smem_ok) {
10341         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize,
10342                             smem_bytes);
10343     }
10344
10345     dim3 block(kNumThreads);
10346     dim3 grid((seqlen + Br - 1) / Br, B * H);
10347
10348     cudaStream_t timing_stream;
10349     cudaStreamCreate(&timing_stream);
10350     cudaEvent_t start, stop;
10351     cudaEventCreate(&start);
10352     cudaEventCreate(&stop);
10353
10354     if (smem_ok) {
10355         for (int w = 0; w < g_warmup; ++w)
10356             fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
10357                                                             seqlen, H, tma_q, tma_k,
10358                                                             tma_v);
10359         check(cudaStreamSynchronize(timing_stream), "fa_tma_ws warmup sync");
10360     }
10361     cudaEventRecord(start, timing_stream);
10362     if (smem_ok) {
10363         for (int r = 0; r < g_repeat; ++r)
10364             fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
10365                                                             seqlen, H, tma_q, tma_k,
10366                                                             tma_v);
10367     }
10368     cudaEventRecord(stop, timing_stream);
10369     cudaEventSynchronize(stop);
10370
10371     float time_ms = 0;
10372     cudaEventElapsedTime(&time_ms, start, stop);
10373     time_ms /= g_repeat;
10374
10375     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
10376     half *h_o = (half *)malloc(sz);
10377     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H");
10378     int count = B * H * seqlen * head_dim;
10379
10380     char label[64];
10381     snprintf(label, sizeof(label), "FA2 TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, %s)",
10382             kStagesK, kStagesV, kMmaAccF32 ? "F32Acc" : "F16Acc");
10383     float max_err = 0.0f;
10384     bool checked = h_o_ref || ref_o;
10385     if (smem_ok && checked) {
10386         for (int i = 0; i < count; ++i) {

```

```

10387     float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
10388     float err = fabsf(__half2float(h_o[i]) - ref_val);
10389     if (err > max_err) max_err = err;
10390 }
10391 }
10392 if (smem_ok && checked) {
10393     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10394     bool is_fail = max_err >= 5e-1f;
10395     if (is_fail || should_print_fa_tflops(kMmaAccF32, tflops)) {
10396         char tflops_str[32];
10397         if (cudnn_tflops_f16 > 0)
10398             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cudnn_tflops_f16, tflops /
10399             cudnn_tflops_f16);
10400         else
10401             snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
10402         printf("| %-56s | %.3e | %-19s |\n", label, max_err,
10403             tflops_str);
10404     }
10405 } else if (smem_ok) {
10406     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10407     if (should_print_fa_tflops(kMmaAccF32, tflops)) {
10408         char tflops_str[32];
10409         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
10410         printf("| %-56s | %-9s | %-19s |\n", label, "unchecked",
10411             tflops_str);
10412     }
10413 } else {
10414     if (g_debug)
10415         printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large",
10416             "None");
10417 }
10418 cudaEventDestroy(start);
10419 cudaEventDestroy(stop);
10420 cudaStreamDestroy(timing_stream);
10421 free(h_o);
10422 cudaFree(tma_q);
10423 cudaFree(tma_k);
10424 cudaFree(tma_v);
10425 }
10426
10427 // D=64/128 dispatch wrapper for bench
10428 template <int kStagesK, int kStagesV = 1, int kMmaAccF32 = 0>
10429 static void bench_fa_tma_mma_ws_dispatch(int B, int H, int seqlen, int head_dim,
10430     half *h_o_ref, float *ref_o,
10431     half *d_q, half *d_k, half *d_v,
10432     half *d_o, float cudnn_tflops_f16) {
10433     if (head_dim == 64) {
10434         bench_fa_tma_mma_ws_launch<64, kStagesK, kStagesV, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10435             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10436     } else if (head_dim == 128) {
10437         bench_fa_tma_mma_ws_launch<128, kStagesK, kStagesV, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10438             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10439     } else {
10440         char label[64];
10441         snprintf(label, sizeof(label),
10442             "FA2 TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, D!=64/128)", kStagesK, kStagesV);
10443         printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
10444     }
10445 }
10446
10447 // FA3-style dual-consumer bench (Br=64, per-WG independent K pipeline, Sv=1)
10448 template <int kHeadDim, int kStagesK, int kMmaAccF32 = 0>

```

```

10449 static void bench_fa_3_tma_ws_launch(int B, int H, int seqlen, int head_dim,
10450                                     half *h_o_ref, float *ref_o,
10451                                     half *d_q, half *d_k, half *d_v,
10452                                     half *d_o, float cudnn_tflops_f16) {
10453     constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
10454     constexpr int kMmaTileSeqLenQ = 4, kMmaTileSeqLenK = 1;
10455     constexpr int kMmaTileSeqLenP = 4, kMmaTileHeadDimV = 1;
10456     constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
10457     constexpr int kValTileSeqLenP = 1;
10458     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
10459     constexpr int kStagesV = 1; // per-WG V: always 1 buffer
10460     constexpr int kNumConsumerWGs = 2;
10461     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
10462     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
10463     constexpr int kNumThreads = 384;
10464
10465     if (seqlen < Br || seqlen % Br != 0 || seqlen % Bc != 0) {
10466         char label[80];
10467         snprintf(label, sizeof(label),
10468                 "FA3 TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, unaligned)", kStagesK);
10469         printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
10470         return;
10471     }
10472
10473     constexpr int kTmaBoxMinor = 64;
10474     constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
10475     CUTensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
10476         d_q, B * H * seqlen / Br, kTmaChunks);
10477     CUTensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
10478         d_k, B * H * seqlen / Bc, kTmaChunks);
10479     CUTensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
10480         d_v, B * H * seqlen / Bc, kTmaChunks);
10481
10482     using FAKernel = void (*)(half *, half *, half *, half *, int, int,
10483                               const CUTensorMap *, const CUTensorMap *,
10484                               const CUTensorMap *);
10485     FAKernel fa_k = flash_attn_3_tma_ws_stages_split_q<
10486         kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32, kMmaTileSeqLenQ,
10487         kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
10488         kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
10489         kNumThreads>;
10490
10491     // smem = Q + K[2WGs * Sk * Bc*D] + V[2WGs * 1 * Bc*D]
10492     size_t smem_bytes = (Br * kHeadDim +
10493                         kNumConsumerWGs * kStagesK * Bc * kHeadDim +
10494                         kNumConsumerWGs * kStagesV * Bc * kHeadDim) * sizeof(half);
10495     bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
10496     if (smem_ok) {
10497         cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize,
10498                             smem_bytes);
10499     }
10500
10501     dim3 block(kNumThreads);
10502     dim3 grid((seqlen + Br - 1) / Br, B * H);
10503
10504     cudaStream_t timing_stream;
10505     cudaStreamCreate(&timing_stream);
10506     cudaEvent_t start, stop;
10507     cudaEventCreate(&start);
10508     cudaEventCreate(&stop);
10509
10510     if (smem_ok) {
10511         for (int w = 0; w < g_warmup; ++w)

```

```

10512     fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
10513                                                         seqlen, H, tma_q, tma_k,
10514                                                         tma_v);
10515     check(cudaStreamSynchronize(timing_stream), "fa3_tma_ws warmup sync");
10516 }
10517 cudaEventRecord(start, timing_stream);
10518 if (smem_ok) {
10519     for (int r = 0; r < g_repeat; ++r)
10520         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
10521                                                         seqlen, H, tma_q, tma_k,
10522                                                         tma_v);
10523 }
10524 cudaEventRecord(stop, timing_stream);
10525 cudaEventSynchronize(stop);
10526
10527 float time_ms = 0;
10528 cudaEventElapsedTime(&time_ms, start, stop);
10529 time_ms /= g_repeat;
10530
10531 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
10532 half *h_o = (half *)malloc(sz);
10533 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa3_tma_ws D2H");
10534 int count = B * H * seqlen * head_dim;
10535
10536 char label[80];
10537 snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, %s)",
10538          kStagesK, kMmaAccF32 ? "F32Acc" : "F16Acc");
10539 float max_err = 0.0f;
10540 bool checked = h_o_ref || ref_o;
10541 if (smem_ok && checked) {
10542     for (int i = 0; i < count; ++i) {
10543         float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
10544         float err = fabsf(__half2float(h_o[i]) - ref_val);
10545         if (err > max_err) max_err = err;
10546     }
10547 }
10548 if (smem_ok && checked) {
10549     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10550     bool is_fail = max_err >= 5e-1f;
10551     if (is_fail || should_print_fa_tflops(kMmaAccF32, tflops)) {
10552         char tflops_str[32];
10553         if (cudnn_tflops_f16 > 0)
10554             snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops, cudnn_tflops_f16, tflops /
10555             cudnn_tflops_f16);
10556         else
10557             snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
10558         printf("| %-56s | %.3e | %-19s |\n", label, max_err,
10559               tflops_str);
10560     }
10561 } else if (smem_ok) {
10562     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10563     if (should_print_fa_tflops(kMmaAccF32, tflops)) {
10564         char tflops_str[32];
10565         snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
10566         printf("| %-56s | %-9s | %-19s |\n", label, "unchecked",
10567               tflops_str);
10568     }
10569 } else {
10570     if (g_debug)
10571         printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large",
10572               "None");
10573 }

```

```

10574 cudaEventDestroy(start);
10575 cudaEventDestroy(stop);
10576 cudaStreamDestroy(timing_stream);
10577 free(h_o);
10578 cudaFree(tma_q);
10579 cudaFree(tma_k);
10580 cudaFree(tma_v);
10581 }
10582
10583 // D=64/128 dispatch wrapper for FA3-style bench
10584 template <int kMmaAccF32 = 0>
10585 static void bench_fa_3_tma_ws_dispatch(int B, int H, int seqlen, int head_dim,
10586                                     half *h_o_ref, float *ref_o,
10587                                     half *d_q, half *d_k, half *d_v,
10588                                     half *d_o, float cudnn_tflops_f16) {
10589     // Try stages up to 4; bench_fa_3_tma_ws_launch internally checks
10590     // cudaDevAttrMaxSharedMemoryPerBlockOptin and skips configs that
10591     // exceed the actual device limit (e.g. Hopper 224KB vs Blackwell 101KB).
10592     if (head_dim == 64) {
10593         bench_fa_3_tma_ws_launch<64, 1, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10594                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10595         bench_fa_3_tma_ws_launch<64, 2, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10596                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10597         bench_fa_3_tma_ws_launch<64, 3, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10598                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10599         bench_fa_3_tma_ws_launch<64, 4, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10600                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10601     } else if (head_dim == 128) {
10602         bench_fa_3_tma_ws_launch<128, 1, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10603                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10604         bench_fa_3_tma_ws_launch<128, 2, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10605                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10606         bench_fa_3_tma_ws_launch<128, 3, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10607                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10608         bench_fa_3_tma_ws_launch<128, 4, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
10609                                                     ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
10610     } else {
10611         char label[64];
10612         snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (D!=64/128)");
10613         printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
10614     }
10615 }
10616
10617 #if defined(NOTES_V2_ENABLE_CUTE)
10618 template <int kHeadDim, int kStagesK = 1>
10619 static void bench_fa_3_tma_mma_ws_cute_launch(
10620     int B, int H, int seqlen, half *h_o_ref, float *ref_o,
10621     half *d_q, half *d_k, half *d_v, half *d_o,
10622     float cudnn_tflops_f32) {
10623     using namespace cute;
10624     using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
10625     using SmemLayout = typename Traits::SmemLayoutQKV;
10626     constexpr int kNumConsumers = 2;
10627     if (seqlen < 64 || seqlen % 64 != 0) {
10628         char label[80];
10629         snprintf(label, sizeof(label),
10630                 "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, unaligned)", kStagesK);
10631         printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
10632         return;
10633     }
10634
10635     int rows = B * H * seqlen;
10636     auto make_tma = [=](half *pointer) {

```

```

10637     auto tensor = make_tensor(
10638         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
10639         make_shape(rows, Int<kHeadDim>{}),
10640         make_stride(Int<kHeadDim>{}, _1{}));
10641     return make_tma_copy(
10642         SM90_TMA_LOAD{}, tensor, SmemLayout{},
10643         Shape<_64, Int<kHeadDim>>{}, _1{});
10644 };
10645 auto tma_q = make_tma(d_q);
10646 auto tma_k = make_tma(d_k);
10647 auto tma_v = make_tma(d_v);
10648 auto kernel = flash_attn_3_tma_mma_ws_split_q_cute<
10649     kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v),
10650     kStagesK>;
10651 auto acc_o = partition_fragment_C(
10652     typename Traits::TiledMma{}, Shape<_64, Int<kHeadDim>>{});
10653 constexpr int kTiles = 1 + kNumConsumers * kStagesK + kNumConsumers;
10654 constexpr int kTilesBytes = kTiles * cosize(SmemLayout{}) * sizeof(cutlass::half_t);
10655 int merge_bytes = 128 * size(acc_o) * sizeof(float) + 128 * sizeof(float4);
10656 int smem_bytes = max(kTilesBytes, merge_bytes);
10657 bool smem_ok = check_smem_feasible((const void *)kernel, smem_bytes);
10658 if (!smem_ok) {
10659     char label[80];
10660     snprintf(label, sizeof(label),
10661         "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, SMEM)", kStagesK);
10662     if (g_debug)
10663         printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
10664     return;
10665 }
10666 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
10667     smem_bytes),
10668     "bench cute fa3 set smem");
10669
10670 dim3 grid(seqlen / 64, B * H);
10671 cudaStream_t stream;
10672 cudaEvent_t start;
10673 cudaEvent_t stop;
10674 check(cudaStreamCreate(&stream), "bench cute fa3 stream");
10675 check(cudaEventCreate(&start), "bench cute fa3 event start");
10676 check(cudaEventCreate(&stop), "bench cute fa3 event stop");
10677 for (int warmup = 0; warmup < g_warmup; ++warmup) {
10678     kernel<<<grid, 384, smem_bytes, stream>>>(
10679         tma_q, tma_k, tma_v,
10680         reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
10681 }
10682 check(cudaStreamSynchronize(stream), "bench cute fa3 warmup sync");
10683 check(cudaEventRecord(start, stream), "bench cute fa3 record start");
10684 for (int repeat = 0; repeat < g_repeat; ++repeat) {
10685     kernel<<<grid, 384, smem_bytes, stream>>>(
10686         tma_q, tma_k, tma_v,
10687         reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
10688 }
10689 check(cudaEventRecord(stop, stream), "bench cute fa3 record stop");
10690 check(cudaEventSynchronize(stop), "bench cute fa3 timing sync");
10691 float time_ms = 0.0f;
10692 check(cudaEventElapsedTime(&time_ms, start, stop), "bench cute fa3 elapsed");
10693 time_ms /= g_repeat;
10694
10695 size_t count = (size_t)rows * kHeadDim;
10696 half *h_o = (half *)malloc(count * sizeof(half));
10697 check(cudaMemcpy(h_o, d_o, count * sizeof(half), cudaMemcpyDeviceToHost),
10698     "bench cute fa3 D2H");
10699 float max_err = 0.0f;

```

```

10700 bool checked = h_o_ref || ref_o;
10701 if (checked) {
10702     for (size_t idx = 0; idx < count; ++idx) {
10703         float reference = h_o_ref ? __half2float(h_o_ref[idx]) : ref_o[idx];
10704         max_err = max(max_err, fabsf(__half2float(h_o[idx]) - reference));
10705     }
10706 }
10707 float tflops = bench_fa_tflops(B, H, seqlen, kHeadDim, time_ms);
10708 char label[80];
10709 snprintf(label, sizeof(label),
10710          "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, F32Acc)", kStagesK);
10711 bool is_fail = checked && max_err >= 5e-1f;
10712 if (is_fail || should_print_fa_tflops(1, tflops)) {
10713     char performance[32];
10714     if (cudnn_tflops_f32 > 0.0f) {
10715         snprintf(performance, sizeof(performance), "%.1f/%.1f (%.2fx)",
10716                  tflops, cudnn_tflops_f32, tflops / cudnn_tflops_f32);
10717     } else {
10718         snprintf(performance, sizeof(performance), "%.1f", tflops);
10719     }
10720     printf("| %-56s | %.3e | %-19s |\n", label, max_err, performance);
10721 }
10722
10723 free(h_o);
10724 cudaEventDestroy(start);
10725 cudaEventDestroy(stop);
10726 cudaStreamDestroy(stream);
10727 }
10728
10729 static void bench_fa_3_tma_mma_ws_cute_dispatch(
10730     int B, int H, int seqlen, int head_dim,
10731     half *h_o_ref, float *ref_o,
10732     half *d_q, half *d_k, half *d_v, half *d_o,
10733     float cudnn_tflops_f32) {
10734     if (head_dim == 64) {
10735         bench_fa_3_tma_mma_ws_cute_launch<64, 1>(B, H, seqlen, h_o_ref, ref_o,
10736            d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10737         bench_fa_3_tma_mma_ws_cute_launch<64, 2>(B, H, seqlen, h_o_ref, ref_o,
10738            d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10739         bench_fa_3_tma_mma_ws_cute_launch<64, 3>(B, H, seqlen, h_o_ref, ref_o,
10740            d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10741         bench_fa_3_tma_mma_ws_cute_launch<64, 4>(B, H, seqlen, h_o_ref, ref_o,
10742            d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10743     } else if (head_dim == 128) {
10744         bench_fa_3_tma_mma_ws_cute_launch<128, 1>(B, H, seqlen, h_o_ref, ref_o,
10745            d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10746         bench_fa_3_tma_mma_ws_cute_launch<128, 2>(B, H, seqlen, h_o_ref, ref_o,
10747            d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10748     }
10749 }
10750
10751 // FA2-style CuTe bench: single consumer, Br=128, no merge.
10752 template <int kHeadDim, int kStagesK, int kStagesV = 1>
10753 static void bench_fa_2_tma_mma_ws_cute_launch(
10754     int B, int H, int seqlen, half *h_o_ref, float *ref_o,
10755     half *d_q, half *d_k, half *d_v, half *d_o,
10756     float cudnn_tflops_f32) {
10757     using namespace cute;
10758     using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
10759     using SmemLayoutQ = typename Traits::SmemLayoutQ;
10760     using SmemLayoutKV = typename Traits::SmemLayoutKV;
10761     constexpr int kBr = 128;
10762     if (seqlen < kBr || seqlen % kBr != 0 || seqlen % 64 != 0) {

```

```

10763     char label[80];
10764     snprintf(label, sizeof(label),
10765              "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, unaligned)", kStagesK, kStagesV);
10766     printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
10767     return;
10768 }
10769
10770 int rows = B * H * seqLen;
10771 // Q tile: [128, D], K/V tile: [64, D]
10772 auto make_tma_q = [=]() {
10773     auto tensor = make_tensor(
10774         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_q)),
10775         make_shape(rows, Int<kHeadDim>{}),
10776         make_stride(Int<kHeadDim>{}, _1{}));
10777     return make_tma_copy(
10778         SM90_TMA_LOAD{}, tensor, SmemLayoutQ{},
10779         Shape<_128, Int<kHeadDim>>{}, _1{});
10780 };
10781 auto make_tma_kv = [=](half *pointer) {
10782     auto tensor = make_tensor(
10783         make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
10784         make_shape(rows, Int<kHeadDim>{}),
10785         make_stride(Int<kHeadDim>{}, _1{}));
10786     return make_tma_copy(
10787         SM90_TMA_LOAD{}, tensor, SmemLayoutKV{},
10788         Shape<_64, Int<kHeadDim>>{}, _1{});
10789 };
10790 auto tma_q = make_tma_q();
10791 auto tma_k = make_tma_kv(d_k);
10792 auto tma_v = make_tma_kv(d_v);
10793 auto kernel = flash_attn_tma_mma_ws_split_q_cute<
10794     kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v),
10795     kStagesK, kStagesV>;
10796 // smem = Q[128*D] + K[Sk*64*D] + V[Sv*64*D] (no merge scratch)
10797 int smem_bytes = (cosize(SmemLayoutQ{}) +
10798     kStagesK * cosize(SmemLayoutKV{}) +
10799     kStagesV * cosize(SmemLayoutKV{})) *
10800     sizeof(cutlass::half_t);
10801 bool smem_ok = check_smem_feasible((const void *)kernel, smem_bytes);
10802 if (!smem_ok) {
10803     char label[80];
10804     snprintf(label, sizeof(label),
10805              "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, SMEM)", kStagesK, kStagesV);
10806     if (g_debug)
10807         printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
10808     return;
10809 }
10810 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
10811     smem_bytes),
10812     "bench cute fa2 set smem");
10813
10814 dim3 grid(seqLen / kBr, B * H);
10815 cudaStream_t stream;
10816 cudaEvent_t start;
10817 cudaEvent_t stop;
10818 check(cudaStreamCreate(&stream), "bench cute fa2 stream");
10819 check(cudaEventCreate(&start), "bench cute fa2 event start");
10820 check(cudaEventCreate(&stop), "bench cute fa2 event stop");
10821 for (int warmup = 0; warmup < g_warmup; ++warmup) {
10822     kernel<<<grid, 384, smem_bytes, stream>>>(
10823         tma_q, tma_k, tma_v,
10824         reinterpret_cast<cutlass::half_t *>(d_o), rows, seqLen);
10825 }

```

```

10826 check(cudaStreamSynchronize(stream), "bench cute fa2 warmup sync");
10827 check(cudaEventRecord(start, stream), "bench cute fa2 record start");
10828 for (int repeat = 0; repeat < g_repeat; ++repeat) {
10829     kernel<<<grid, 384, smem_bytes, stream>>>(
10830         tma_q, tma_k, tma_v,
10831         reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
10832 }
10833 check(cudaEventRecord(stop, stream), "bench cute fa2 record stop");
10834 check(cudaEventSynchronize(stop), "bench cute fa2 timing sync");
10835 float time_ms = 0.0f;
10836 check(cudaEventElapsedTime(&time_ms, start, stop), "bench cute fa2 elapsed");
10837 time_ms /= g_repeat;
10838
10839 size_t count = (size_t)rows * kHeadDim;
10840 half *h_o = (half *)malloc(count * sizeof(half));
10841 check(cudaMemcpy(h_o, d_o, count * sizeof(half), cudaMemcpyDeviceToHost),
10842     "bench cute fa2 D2H");
10843 float max_err = 0.0f;
10844 bool checked = h_o_ref || ref_o;
10845 if (checked) {
10846     for (size_t idx = 0; idx < count; ++idx) {
10847         float reference = h_o_ref ? __half2float(h_o_ref[idx]) : ref_o[idx];
10848         max_err = max(max_err, fabsf(__half2float(h_o[idx]) - reference));
10849     }
10850 }
10851 float tflops = bench_fa_tflops(B, H, seqlen, kHeadDim, time_ms);
10852 char label[80];
10853 snprintf(label, sizeof(label),
10854     "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, F32Acc)", kStagesK, kStagesV);
10855 bool is_fail = checked && max_err >= 5e-1f;
10856 if (is_fail || should_print_fa_tflops(1, tflops)) {
10857     char performance[32];
10858     if (cudnn_tflops_f32 > 0.0f) {
10859         snprintf(performance, sizeof(performance), "%.1f/%.1f (%.2fx)",
10860             tflops, cudnn_tflops_f32, tflops / cudnn_tflops_f32);
10861     } else {
10862         snprintf(performance, sizeof(performance), "%.1f", tflops);
10863     }
10864     printf("| %-56s | %.3e | %-19s |\n", label, max_err, performance);
10865 }
10866
10867 free(h_o);
10868 cudaEventDestroy(start);
10869 cudaEventDestroy(stop);
10870 cudaStreamDestroy(stream);
10871 }
10872
10873 static void bench_fa_2_tma_mma_ws_cute_dispatch(
10874     int B, int H, int seqlen, int head_dim,
10875     half *h_o_ref, float *ref_o,
10876     half *d_q, half *d_k, half *d_v, half *d_o,
10877     float cudnn_tflops_f32) {
10878     if (head_dim == 64) {
10879         bench_fa_2_tma_mma_ws_cute_launch<64, 1, 1>(B, H, seqlen, h_o_ref, ref_o,
10880             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10881         bench_fa_2_tma_mma_ws_cute_launch<64, 2, 1>(B, H, seqlen, h_o_ref, ref_o,
10882             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10883         bench_fa_2_tma_mma_ws_cute_launch<64, 3, 1>(B, H, seqlen, h_o_ref, ref_o,
10884             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10885         bench_fa_2_tma_mma_ws_cute_launch<64, 4, 1>(B, H, seqlen, h_o_ref, ref_o,
10886             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10887         bench_fa_2_tma_mma_ws_cute_launch<64, 2, 2>(B, H, seqlen, h_o_ref, ref_o,
10888             d_q, d_k, d_v, d_o, cudnn_tflops_f32);

```

```

10889 } else if (head_dim == 128) {
10890     bench_fa_2_tma_mma_ws_cute_launch<128, 1, 1>(B, H, seqlen, h_o_ref, ref_o,
10891         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10892     bench_fa_2_tma_mma_ws_cute_launch<128, 2, 1>(B, H, seqlen, h_o_ref, ref_o,
10893         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10894     bench_fa_2_tma_mma_ws_cute_launch<128, 3, 1>(B, H, seqlen, h_o_ref, ref_o,
10895         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
10896 }
10897 }
10898 #endif
10899 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
10900
10901 #if defined(NOTES_V2_ENABLE_CUDNN)
10902 static float bench_cudnn_sdpa_tflops(half *d_q, half *d_k, half *d_v,
10903     half *d_o_ref, int B, int H, int seqlen,
10904     int head_dim, fe::DataType_t compute_type) {
10905     cudnnHandle_t handle;
10906     cudnnCreate(&handle);
10907     auto graph = std::make_shared<fe::graph::Graph>();
10908     graph->set_io_data_type(fe::DataType_t::HALF)
10909         .set_intermediate_data_type(fe::DataType_t::FLOAT)
10910         .set_compute_data_type(compute_type);
10911
10912     auto Q = graph->tensor(fe::graph::Tensor_attributes()
10913         .set_uid(1).set_dim({B, H, seqlen, head_dim})
10914         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
10915     auto K = graph->tensor(fe::graph::Tensor_attributes()
10916         .set_uid(2).set_dim({B, H, seqlen, head_dim})
10917         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
10918     auto V = graph->tensor(fe::graph::Tensor_attributes()
10919         .set_uid(3).set_dim({B, H, seqlen, head_dim})
10920         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
10921
10922     auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
10923         fe::graph::SDPA_attributes()
10924             .set_name("sdpa_ref")
10925             .set_attn_scale(1.0f / sqrtf((float)head_dim)));
10926
10927     O_sdpa->set_output(true).set_uid(4)
10928         .set_dim({B, H, seqlen, head_dim})
10929         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
10930
10931     auto build_status = graph->build(handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
10932     float tflops = -1.0f;
10933     if (build_status.is_good()) {
10934         std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
10935             {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
10936         int64_t ws_size = 0;
10937         if (graph->get_workspace_size(ws_size).is_good()) {
10938             int8_t *d_ws = nullptr;
10939             if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "bench cudnn ws");
10940             if (graph->execute(handle, vp, d_ws).is_good()) {
10941                 check(cudaDeviceSynchronize(), "bench cudnn sync");
10942                 for (int w = 1; w < g_warmup; ++w)
10943                     (void)graph->execute(handle, vp, d_ws);
10944                 cudaDeviceSynchronize();
10945                 cudaEvent_t ev_s, ev_e;
10946                 cudaEventCreate(&ev_s); cudaEventCreate(&ev_e);
10947                 cudaEventRecord(ev_s);
10948                 for (int r = 0; r < g_repeat; ++r)
10949                     (void)graph->execute(handle, vp, d_ws);
10950                 cudaEventRecord(ev_e);
10951                 cudaEventSynchronize(ev_e);

```

```

10952     float time_ms = 0;
10953     cudaEventElapsedTime(&time_ms, ev_s, ev_e);
10954     time_ms /= g_repeat;
10955     tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
10956     cudaEventDestroy(ev_s); cudaEventDestroy(ev_e);
10957 }
10958 if (d_ws) cudaFree(d_ws);
10959 }
10960 }
10961 cudnnDestroy(handle);
10962 return tflops;
10963 }
10964 #endif
10965
10966 static void bench_flash_attn(int B, int H, int N, int D) {
10967     int seqlen = N, head_dim = D;
10968
10969     if (head_dim != 64 && head_dim != 128) {
10970         printf("| %-56s | %-9s | %-19s |\n", "FlashAttention-2", "unsupported D", "None");
10971         return;
10972     }
10973
10974     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
10975
10976     srand(42);
10977     half *h_q = (half *)malloc(sz);
10978     half *h_k = (half *)malloc(sz);
10979     half *h_v = (half *)malloc(sz);
10980     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
10981         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
10982         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
10983         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
10984     }
10985
10986     // Device allocations — shared by kernel and reference
10987     half *d_q, *d_k, *d_v, *d_o;
10988     check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
10989     check(cudaMalloc(&d_k, sz), "bench fa alloc K");
10990     check(cudaMalloc(&d_v, sz), "bench fa alloc V");
10991     check(cudaMalloc(&d_o, sz), "bench fa alloc O");
10992     check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
10993     check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
10994     check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");
10995
10996     // Reference output: either cuDNN (half) or CPU (float)
10997     half *h_o_ref = nullptr;
10998     float *ref_o = nullptr;
10999     float cudnn_tflops_f16 = -1.0f, cudnn_tflops_f32 = -1.0f;
11000     int ref_count = B * H * seqlen * head_dim;
11001
11002     #if defined(NOTES_V2_ENABLE_CUDNN)
11003     if (!g_fa_skip_check) {
11004         half *d_o_ref;
11005         check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
11006         cudnn_tflops_f16 = bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref,
11007                                                     B, H, seqlen, head_dim,
11008                                                     fe::DataType_t::HALF);
11009         bool cudnn_ok = (cudnn_tflops_f16 > 0);
11010         if (cudnn_ok) {
11011             cudnn_tflops_f32 = bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref,
11012                                                         B, H, seqlen, head_dim,
11013                                                         fe::DataType_t::FLOAT);
11014             h_o_ref = (half *)malloc(sz);

```

```

11015     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
11016 } else {
11017     fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
11018 }
11019 cudaFree(d_o_ref);
11020 }
11021 #endif
11022
11023 // CPU reference fallback
11024 if (!g_fa_skip_check && !h_o_ref) {
11025     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
11026     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
11027     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
11028     ref_o = (float *)malloc(sz * 4 / sizeof(half));
11029     for (int i = 0; i < ref_count; i++) {
11030         ref_q[i] = __half2float(h_q[i]);
11031         ref_k[i] = __half2float(h_k[i]);
11032         ref_v[i] = __half2float(h_v[i]);
11033     }
11034
11035     float scale = 1.0f / sqrtf((float)head_dim);
11036     for (int bi = 0; bi < B * H; bi++) {
11037         for (int qi = 0; qi < seqlen; qi++) {
11038             float smax = -INFINITY;
11039             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
11040             for (int kj = 0; kj < seqlen; kj++) {
11041                 float s = 0.0f;
11042                 for (int d = 0; d < head_dim; d++)
11043                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
11044                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
11045                 S[kj] = s * scale;
11046                 if (S[kj] > smax) smax = S[kj];
11047             }
11048             double sum_exp = 0.0;
11049             for (int kj = 0; kj < seqlen; kj++)
11050                 sum_exp += (double)expf(S[kj] - smax);
11051             float inv_sum = 1.0f / (float)sum_exp;
11052             for (int d = 0; d < head_dim; d++) {
11053                 double o_acc = 0.0;
11054                 for (int kj = 0; kj < seqlen; kj++)
11055                     o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
11056                         ref_v[bi * seqlen * head_dim + kj * head_dim + d];
11057                 ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
11058             }
11059             free(S);
11060         }
11061     }
11062     free(ref_q);
11063     free(ref_k);
11064     free(ref_v);
11065 }
11066
11067 if (g_bench_fa3_cute_only) {
11068 #if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11069     bench_fa_3_tma_mma_ws_cute_dispatch(
11070         B, H, seqlen, head_dim, h_o_ref, ref_o,
11071         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11072 #endif
11073     free(h_q);
11074     free(h_k);
11075     free(h_v);
11076     free(h_o_ref);
11077     free(ref_o);

```

```

11078     cudaFree(d_q);
11079     cudaFree(d_k);
11080     cudaFree(d_v);
11081     cudaFree(d_o);
11082     return;
11083 }
11084
11085 if (head_dim == 64) {
11086     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
11087         cudaDeviceSynchronize();
11088         bench_fa_launch<64, 1, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11089             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11090         cudaDeviceSynchronize();
11091         bench_fa_launch<64, 2, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11092             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11093         cudaDeviceSynchronize();
11094         bench_fa_launch<64, 1, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11095             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11096         cudaDeviceSynchronize();
11097         bench_fa_launch<64, 2, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11098             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11099     }
11100     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
11101         cudaDeviceSynchronize();
11102         bench_fa_launch<64, 2, 0, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11103             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11104         cudaDeviceSynchronize();
11105         bench_fa_launch<64, 2, 0, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11106             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11107     }
11108     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
11109         cudaDeviceSynchronize();
11110         bench_fa_launch<64, 2, 8, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11111             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11112         cudaDeviceSynchronize();
11113         bench_fa_launch<64, 2, 8, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11114             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11115     }
11116     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
11117         cudaDeviceSynchronize();
11118         bench_fa_launch<64, 2, 8, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11119             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11120         cudaDeviceSynchronize();
11121         bench_fa_launch<64, 2, 8, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11122             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11123     }
11124     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
11125         cudaDeviceSynchronize();
11126         bench_fa_launch<64, 2, 0, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11127             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11128         cudaDeviceSynchronize();
11129         bench_fa_launch<64, 2, 0, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11130             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11131     }
11132     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
11133         cudaDeviceSynchronize();
11134         bench_fa_launch<64, 2, 0, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11135             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11136         cudaDeviceSynchronize();
11137         bench_fa_launch<64, 2, 0, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11138             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11139     }
11140     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {

```

```

11141     cudaDeviceSynchronize();
11142     bench_fa_launch<64, 2, 8, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11143                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11144     cudaDeviceSynchronize();
11145     bench_fa_launch<64, 2, 8, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11146                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11147 }
11148 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
11149     cudaDeviceSynchronize();
11150     bench_fa_launch<64, 2, 0, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11151                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11152     cudaDeviceSynchronize();
11153     bench_fa_launch<64, 2, 0, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11154                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11155 }
11156 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11157 {
11158     cudaDeviceSynchronize();
11159     bench_fa_tma_mma_ws_dispatch<1, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11160                                             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11161     cudaDeviceSynchronize();
11162     bench_fa_tma_mma_ws_dispatch<2, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11163                                             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11164     cudaDeviceSynchronize();
11165     bench_fa_tma_mma_ws_dispatch<3, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11166                                             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11167     cudaDeviceSynchronize();
11168     bench_fa_tma_mma_ws_dispatch<4, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11169                                             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11170     cudaDeviceSynchronize();
11171     bench_fa_tma_mma_ws_dispatch<2, 2, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11172                                             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11173     // F32Acc variants
11174     cudaDeviceSynchronize();
11175     bench_fa_tma_mma_ws_dispatch<1, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11176                                             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11177     cudaDeviceSynchronize();
11178     bench_fa_tma_mma_ws_dispatch<2, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11179                                             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11180     cudaDeviceSynchronize();
11181     bench_fa_tma_mma_ws_dispatch<3, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11182                                             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11183     cudaDeviceSynchronize();
11184     bench_fa_tma_mma_ws_dispatch<4, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11185                                             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11186     cudaDeviceSynchronize();
11187     bench_fa_tma_mma_ws_dispatch<2, 2, 1>(B, H, seqlen, head_dim, h_o_ref,
11188                                             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11189     // FA3-style dual-consumer
11190     cudaDeviceSynchronize();
11191     bench_fa_3_tma_ws_dispatch<0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11192                                     d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11193     cudaDeviceSynchronize();
11194     bench_fa_3_tma_ws_dispatch<1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11195                                     d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11196 #if defined(NOTES_V2_ENABLE_CUTE)
11197     cudaDeviceSynchronize();
11198     bench_fa_2_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
11199                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11200     cudaDeviceSynchronize();
11201     bench_fa_3_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
11202                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11203 #endif

```

```

11204     }
11205 #endif
11206 } else {
11207     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
11208         cudaDeviceSynchronize();
11209         bench_fa_launch<128, 1, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11210             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11211
11212         cudaDeviceSynchronize();
11213         bench_fa_launch<128, 2, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11214             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11215
11216         cudaDeviceSynchronize();
11217         bench_fa_launch<128, 1, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11218             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11219
11220         cudaDeviceSynchronize();
11221         bench_fa_launch<128, 2, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11222             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11223     }
11224     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
11225         cudaDeviceSynchronize();
11226         bench_fa_launch<128, 2, 0, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11227             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11228
11229         cudaDeviceSynchronize();
11230         bench_fa_launch<128, 2, 0, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11231             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11232     }
11233     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
11234         cudaDeviceSynchronize();
11235         bench_fa_launch<128, 2, 8, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11236             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11237
11238         cudaDeviceSynchronize();
11239         bench_fa_launch<128, 2, 8, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11240             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11241     }
11242     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
11243         cudaDeviceSynchronize();
11244         bench_fa_launch<128, 2, 8, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11245             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11246
11247         cudaDeviceSynchronize();
11248         bench_fa_launch<128, 2, 8, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11249             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11250     }
11251     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
11252         cudaDeviceSynchronize();
11253         bench_fa_launch<128, 2, 0, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11254             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11255
11256         cudaDeviceSynchronize();
11257         bench_fa_launch<128, 2, 0, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11258             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11259     }
11260     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
11261         cudaDeviceSynchronize();
11262         bench_fa_launch<128, 2, 0, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11263             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11264
11265         cudaDeviceSynchronize();
11266         bench_fa_launch<128, 2, 0, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,

```

```

11267         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11268     }
11269     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
11270         cudaDeviceSynchronize();
11271         bench_fa_launch<128, 2, 0, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11272             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11273         cudaDeviceSynchronize();
11274         bench_fa_launch<128, 2, 0, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11275             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11276     }
11277     #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11278     {
11279         cudaDeviceSynchronize();
11280         bench_fa_tma_mma_ws_dispatch<1, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11281             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11282         cudaDeviceSynchronize();
11283         bench_fa_tma_mma_ws_dispatch<2, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11284             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11285         cudaDeviceSynchronize();
11286         bench_fa_tma_mma_ws_dispatch<3, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11287             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11288         cudaDeviceSynchronize();
11289         bench_fa_tma_mma_ws_dispatch<2, 2, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11290             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11291         // F32Acc variants
11292         cudaDeviceSynchronize();
11293         bench_fa_tma_mma_ws_dispatch<1, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11294             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11295         cudaDeviceSynchronize();
11296         bench_fa_tma_mma_ws_dispatch<2, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11297             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11298         cudaDeviceSynchronize();
11299         bench_fa_tma_mma_ws_dispatch<3, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
11300             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11301         cudaDeviceSynchronize();
11302         bench_fa_tma_mma_ws_dispatch<2, 2, 1>(B, H, seqlen, head_dim, h_o_ref,
11303             ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11304         // FA3-style dual-consumer
11305         cudaDeviceSynchronize();
11306         bench_fa_3_tma_ws_dispatch<0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11307             d_q, d_k, d_v, d_o, cudnn_tflops_f16);
11308         cudaDeviceSynchronize();
11309         bench_fa_3_tma_ws_dispatch<1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
11310             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11311         #if defined(NOTES_V2_ENABLE_CUTE)
11312         cudaDeviceSynchronize();
11313         bench_fa_2_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
11314             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11315         cudaDeviceSynchronize();
11316         bench_fa_3_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
11317             d_q, d_k, d_v, d_o, cudnn_tflops_f32);
11318         #endif
11319     }
11320 #endif
11321 }
11322 cudaDeviceSynchronize();
11323
11324 free(h_q);
11325 free(h_k);
11326 free(h_v);
11327 free(h_o_ref);
11328 free(ref_o);
11329 cudaFree(d_q);

```

```

11330     cudaFree(d_k);
11331     cudaFree(d_v);
11332     cudaFree(d_o);
11333 }
11334
11335 // Host 端 v1/v2 等价性测试: 遍历 kColStride/i/j, 断言 swizzle_v1_impl == swizzle_v2_impl.
11336 // 用于在启用 NOTES_V2_ENABLE_SWIZZLE_V2 前后验证 v2 与 v1 bit-exact 等价.
11337 // kColStride=8 在 v2 内回退 v1, 故 8 也应恒等.
11338 template <int kColStride>
11339 static void swizzle_equiv_check_one(long &total, long &fail) {
11340     for (int i = 0; i < 256; ++i) {
11341         for (int j = 0; j < kColStride; ++j) {
11342             int v1 = swizzle_v1_impl<kColStride>(i, j);
11343             int v2 = swizzle_v2_impl<kColStride>(i, j);
11344             ++total;
11345             if (v1 != v2) {
11346                 ++fail;
11347                 if (fail <= 10)
11348                     printf(" MISMATCH cs=%d i=%d j=%d: v1=%d v2=%d\n", kColStride, i, j, v1, v2);
11349             }
11350         }
11351     }
11352 }
11353
11354 static void test_swizzle_equiv() {
11355     long total = 0, fail = 0;
11356     swizzle_equiv_check_one<8>(total, fail);
11357     swizzle_equiv_check_one<16>(total, fail);
11358     swizzle_equiv_check_one<32>(total, fail);
11359     swizzle_equiv_check_one<64>(total, fail);
11360     printf("| %-56s | %-9s |\n",
11361           "Swizzle v1/v2 equiv (host)", fail == 0 ? "ALL PASS" : "FAIL");
11362     printf(" total=%ld fail=%ld\n", total, fail);
11363 }
11364
11365 int main(int argc, char *argv[]) {
11366     #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11367         cuInit(0);
11368     #endif
11369
11370     for (int i = 1; i < argc; i++) {
11371         if (strcmp(argv[i], "--bench-hgemm") == 0) {
11372             g_bench_hgemm = true;
11373         } else if (strcmp(argv[i], "--bench") == 0) {
11374             g_bench_hgemm = true;
11375             g_bench_fa = true;
11376         } else if (strcmp(argv[i], "--bench-fa") == 0) {
11377             g_bench_fa = true;
11378         } else if (strcmp(argv[i], "--bench-fa3-cute") == 0) {
11379             g_bench_fa = true;
11380             g_bench_fa3_cute_only = true;
11381         } else if (strcmp(argv[i], "--bench-hgemm-all") == 0) {
11382             g_bench_hgemm_all = true;
11383         } else if (strcmp(argv[i], "--bench-fa-all") == 0) {
11384             g_bench_fa = true;
11385             g_fa_layout = FALayout::All;
11386         } else if (strcmp(argv[i], "--bench-all") == 0) {
11387             g_bench_all = true;
11388             g_fa_layout = FALayout::All;
11389         } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
11390             sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
11391         } else if (strcmp(argv[i], "--bhnd") == 0 && i + 1 < argc) {
11392             sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_H, &g_bench_Nfa, &g_bench_D);

```

```

11393 } else if (strcmp(argv[i], "--fa-layout") == 0 && i + 1 < argc) {
11394     const char *layout = argv[++i];
11395     if (strcmp(layout, "all") == 0)
11396         g_fa_layout = FALayout::All;
11397     else if (strcmp(layout, "pad") == 0)
11398         g_fa_layout = FALayout::Pad;
11399     else if (strcmp(layout, "swizzle-q") == 0)
11400         g_fa_layout = FALayout::SwizzleQ;
11401     else if (strcmp(layout, "swizzle-k") == 0)
11402         g_fa_layout = FALayout::SwizzleK;
11403     else if (strcmp(layout, "swizzle-v") == 0)
11404         g_fa_layout = FALayout::SwizzleV;
11405     else if (strcmp(layout, "swizzle-qk") == 0)
11406         g_fa_layout = FALayout::SwizzleQK;
11407     else if (strcmp(layout, "swizzle-qv") == 0)
11408         g_fa_layout = FALayout::SwizzleQV;
11409     else if (strcmp(layout, "swizzle-kv") == 0)
11410         g_fa_layout = FALayout::SwizzleKV;
11411     else if (strcmp(layout, "swizzle") == 0)
11412         g_fa_layout = FALayout::Swizzle;
11413     else {
11414         fprintf(stderr, "Unsupported FA layout: %s\n", layout);
11415         return EXIT_FAILURE;
11416     }
11417 } else if (strcmp(argv[i], "--fa-skip-check") == 0) {
11418     g_fa_skip_check = true;
11419 } else if (strcmp(argv[i], "--swizzle-eq-check") == 0) {
11420     g_swizzle_eq_check = true;
11421 } else if (strcmp(argv[i], "--debug") == 0) {
11422     g_debug = true;
11423 } else if (strcmp(argv[i], "--verbose") == 0) {
11424     g_verbose = true;
11425 } else if (strcmp(argv[i], "--warmup") == 0 && i + 1 < argc) {
11426     g_warmup = atoi(argv[++i]);
11427 } else if (strcmp(argv[i], "--repeat") == 0 && i + 1 < argc) {
11428     g_repeat = atoi(argv[++i]);
11429 }
11430 }
11431
11432 if (g_swizzle_eq_check) {
11433     printf("=== notes-v2.cu swizzle v1/v2 equivalence check ===\n");
11434     printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
11435     printf("|-----|-----|\n");
11436     test_swizzle_equiv();
11437     printf("=== Done ===\n");
11438     return 0;
11439 }
11440
11441 #if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11442 if (argc >= 2 && strcmp(argv[1], "--fa3-cute") == 0) {
11443     printf("=== CuTe FA3 TMA MMA WS correctness ===\n");
11444     printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
11445     printf("|-----|-----|\n");
11446     test_flash_attn_3_tma_mma_ws_split_q_cute<64>();
11447     test_flash_attn_3_tma_mma_ws_split_q_cute<128>();
11448     return 0;
11449 }
11450 if (argc >= 2 && strcmp(argv[1], "--fa3-cute-tma-smoke") == 0) {
11451     printf("=== CuTe TMA copy smoke ===\n");
11452     printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
11453     printf("|-----|-----|\n");
11454     test_flash_attn_3_cute_tma_copy_smoke<64>();
11455     test_flash_attn_3_cute_tma_copy_smoke<128>();

```

```

11456     return 0;
11457 }
11458 if (argc >= 2 && strcmp(argv[1], "--fa2-cute") == 0) {
11459     printf("=== CuTe FA2 TMA MMA WS correctness ===\n");
11460     printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
11461     printf("|-----|-----|\n");
11462     test_flash_attn_tma_mma_ws_split_q_cute<64>();
11463     test_flash_attn_tma_mma_ws_split_q_cute<128>();
11464     return 0;
11465 }
11466 #endif
11467
11468 if (g_bench_hgemm || g_bench_fa || g_bench_all) {
11469     printf("=== notes-v2.cu bench mode ===\n");
11470     printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
11471         g_bench_M, g_bench_N, g_bench_K,
11472         g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
11473     printf("| %-56s | %-9s | %-19s |\n", "Kernel", "Max Err", "TFLOPS/cu{BLAS,DNN}");
11474     printf("|-----|-----|-----|\n");
11475
11476     if (g_bench_hgemm || g_bench_hgemm_all || g_bench_all) {
11477 #if defined(NOTES_V2_ENABLE_CUTE)
11478         bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
11479 #endif
11480         if (g_bench_hgemm_all || g_bench_all) {
11481             bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
11482             bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
11483 #if defined(NOTES_V2_ENABLE_WGMMMA)
11484             bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
11485 #endif
11486 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11487             bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
11488 #endif
11489         }
11490     }
11491     if (g_bench_fa || g_bench_all)
11492         bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
11493
11494     printf("=== Bench done ===\n");
11495     return 0;
11496 }
11497
11498 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11499 if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
11500     int M = 128, N = 128, K = 64;
11501     if (argc > 4) {
11502         M = atoi(argv[2]);
11503         N = atoi(argv[3]);
11504         K = atoi(argv[4]);
11505     }
11506     printf("=== SM120 TMA MMA WS validation ===\n");
11507     printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
11508     printf("|-----|-----|\n");
11509     test_hgemm_tma_mma_ws(M, N, K);
11510     return 0;
11511 }
11512 #endif
11513 int M = 1024, N = 1024, K = 1024;
11514 if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
11515
11516 printf("=== notes-v2.cu verification harness ===\n");
11517 printf("| %-56s | %-9s |\n", "Kernel", "Max Err");

```

```

11518 printf("|-----|-----|\n");
11519
11520 test_block_reduce(N);
11521 test_dot(N);
11522 test_relu(1024);
11523 test_elementwise(1024);
11524 test_histogram(1024);
11525 test_merge_attn_states(512, 16, 128);
11526 test_softmax(256);
11527 test_rms_norm(8, 128);
11528 test_layer_norm(8, 128);
11529 test_rope(8, 128);
11530 test_mat_transpose(256, 256);
11531 test_mat_transpose_padded(256, 256);
11532 test_sgemv(256, 128);
11533 test_sgemm(M, N, K);
11534 test_hgemm_mma(M, N, K);
11535 test_hgemm_swizzle(M, N, K);
11536 #if (defined(NOTES_V2_ENABLE_CUTE))
11537     test_hgemm_cute(M, N, K);
11538 #endif
11539 #if (defined(NOTES_V2_ENABLE_WGMMA))
11540     test_hgemm_wgmma(M, N, K);
11541 #endif
11542 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11543     test_hgemm_tma_mma_ws(M, N, K);
11544 #endif
11545     test_flash_attn(1024, 64);
11546 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
11547     test_flash_attn_tma_mma_ws(1024, 64);
11548     test_flash_attn_tma_mma_ws(1024, 128);
11549     test_flash_attn_3_tma_ws(1024, 64);
11550     test_flash_attn_3_tma_ws(1024, 128);
11551 #endif
11552
11553 printf("=== All tests done ===\n");
11554 return 0;
11555 }
11556
11557 // =====
11558 // Quick build & run reference
11559 // =====
11560 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMA) :
11561 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
11562 // -I ../../third-party/cutlass/include \
11563 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
11564 //
11565 // # sm_89 单独编译 + 运行:
11566 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
11567 //
11568 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
11569 // 项目内置 third-party/cutlass) :
11570 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
11571 // -I ../../third-party/cutlass/include \
11572 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
11573 //
11574 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
11575 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
11576 // -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
11577 //
11578 // # sm_90a + CuTe + WGMMA:
11579 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
11580 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \

```

```

11581 // -I ../../third-party/cutlass/include \
11582 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
11583 //
11584 // # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
11585 // RTX PRO 5000 / RTX 5090):
11586 // nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
11587 // -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
11588 // ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
11589 //
11590 // # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
11591 // requires cudnn-frontend submodule: git submodule update --init):
11592 // nvcc -std=c++20 -O2 -arch=sm_120a \
11593 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
11594 // -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
11595 // -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
11596 // -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
11597 // # Default FA bench (kPadQ=kPadK=kPadV=8 only):
11598 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa --bhnd 1,48,4096,64
11599 // # All FA layout variants:
11600 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa-all --bhnd 1,48,4096,64
11601 // # FA TMA MMA WS bench (SM120, D=64 only, kStagesK=2/3):
11602 // ./notes_v2_cute_ws_sm120a.bin --bench-fa-tma-ws --bhnd 1,32,4096,64

```